

**LAPORAN AKHIR PRAKTIKUM
APLIKASI WEB DAN MOBILE
PERTEMUAN 1-15**



DISUSUN OLEH:

**Wianda Sukandari
23051430002**

**PROGRAM STUDI TEKNIK INDUSTRI
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE

LAPORAN AKHIR PRAKTIKUM APLIKASI WEB DAN MOBILE
PERTEMUAN 1-15
Wianda Sukandari
NIM.23051430002

Laporan ini Disusun guna memenuhi salah satu persyaratan
Menempuh mata kuliah aplikasi web dan mobile
Program studi Teknik industri
Fakultas Teknik UNY

Menyetujui/Mengesahkan:

Dosen Pengampu,

Mahasiswa,



Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT
NIP. 198103172005011001

Wianda Sukandari
NIM.23051430002

DAFTAR ISI

MODUL 1	5
PENGENALAN PEMROGRAMAN WEB & HTML DASAR	5
MODUL 2	24
CSS DASAR & STYLING	24
MODUL 3	53
LAYOUT CSS LANJUTAN (FLEXBOX & GRID) + BOOTSTRAP	53
Modul 4	79
Javascript Dasar (Variabel, Tipe Data, Operator, Control Flow)	79
MODUL 5	97
Function, Array, Object di JavaScript	97
MODUL 6	118
DOM Manipulation & Event Handling	118
MODUL 7	136
Form Validation & Local Storage	136
MODUL 8	177
Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)	177
MODUL 9	208
Pengenalan React.js (Component, JSX, Props)	208
MODUL 10	224
State, Hooks (useState, useEffect), dan Conditional Rendering	224
MODUL 11	242
React Routing & Multi-Page App + Integrasi API	242
MODUL 12	259
Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)	259
MODUL 14	278
Pengenalan React Native & Setup Expo	278
MODUL 15	303
Component, Navigation, dan Styling di React Native + Proyek Mobile	303

MODUL 1**PENGENALAN PEMROGRAMAN WEB & HTML DASAR****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 1)
Topik Utama	Pengenalan Pemrograman Web & HTML Dasar
Sub-Topik	Struktur halaman web, elemen HTML semantik, dan formulir dasar.
Durasi Praktikum	340 Menit
Tanggal Praktikum	14 February 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami peran penting web application dalam sistem informasi industri.
2	Mampu menjelaskan struktur dasar HTML dan cara kerjanya di browser.
3	Mampu membuat halaman web statis dengan elemen teks, gambar, dan tabel.
4	Mampu menyusun formulir input data sederhana (misal: data produksi).

C. ALAT DAN BAHAN

No	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Live Server Extension	64-bit	Lingkungan pengembangan
5	Git & GitHub	Versi terbaru	Manajemen versi & repository

D. DASAR TEORI

HTML itu kayak Bahasa dasar buat bikin halaman web. Ibaratnya kayak rangka rumah Dimana dia yang nentuin susunan bagian haaman, mulai dari judul, teks, table, sampai form input. Jadi walaupun tampilannya nanti bisa dibuat keren pakai CSS atau interaktif pakai JavaScript, semuanya tetap berdiri di atas struktur HTML dulu. Di Teknik Industri, ini kepeke banget misanya buat bikin form laporn produksi, checklist inspeksi, atau tampilan data sederhana.

Di HTML kita pakai yang namanya tag, yaitu kode pembuka dan penutup yang ngasih tahu browser suatu bagian itu fungsinya apa.

1. `<h1>` hingga `<h6>` : Judul (*heading*)
2. `<p>` : Paragraf teks
3. `<table>`, `<tr>`, `<th>` , `<td>` : Tabel (penting buat data di laporan)
4. `<form>`, `<input>`, `<label>` : Formulir input data
5. `<div>` : Kontaainer pembungkus (*block*)
6. `<span>` : Pembungkus teks (*inline*)

Walaupun kelihatannya sederhana, pemilihan tag yang tepat itu penting banget karena bikin kode lebih teratur dan gampang dikembangkan. Jadi belajar HTML bukan Cuma bikin web saja, tapi juga Latihan mikir terstruktur.

D.1 Konsep Utama

Konsep utamanya adalah memahami dasar pembuatan halaman web pakai HTML sebagai Bahasa markup untuk nyusun struktur tampilan. HTML berfungsi mengatur elemen kayak judul, teks, table, dan form biar informasinya rapi serta bisa dibaca browser. Tiap elemen dibentuk dengan tag yang punya fungsi beda beda.

Selain itu, modul ini juga menekankan penggunaan form sebagai sarana input data. Form bisa bikin pengguna masukin informasi melalui berbagai tipe input kayak teks, angka, pilihan, dan file, yang masing masing punya fungsi validasi berbeda. Secara keseluruhan, praktikum ini memberikan pemahaman dasar tentang bagaimana halaman web dibangun dari struktur hingga input data, yang menjadi fondasi penting sebelum mempelajari pengembangan web yang lebih lanjut.

D.2 Keterkaitan dengan Teknik Industri

Materi modul ini nyambung banget sama Teknik Industri karena isinya ngebahas cara memahami sistem kerja, menganalisis proses, dan bikin keputusan yang lebih efisien. Di dunia nyata, konsepnya dipakai misalnya buat ngukur produktivitas karyawan, nentuin jumlah tenaga kerja yang dibutuhkan, atau memperbaiki alur kerja di perusahaan supaya waktu, biaya, dan tenaga bisa dipakai seoptimal mungkin.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Kerangka HTML

Pertama bisa memencet Shift + 1 dan otomatis akan menghasilkan kerangka HTML seperti dibawah ini.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

Screenshot Hasil:

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Document</title>
7 </head>
8 <body>
9
10 </body>
11 </html>

```

Gambar E.1 – Kerangka HTML Dasar

E.2 Langkah 2 – Menyusun Profil Lini Produksi

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
  <title>Laporan Inspeksi Kualitas_Wianda
Sukandari_23051430002</title>
</head>
<body>
  <h1>Profil Mesin CNC-01</h1>
  <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
  <hr>

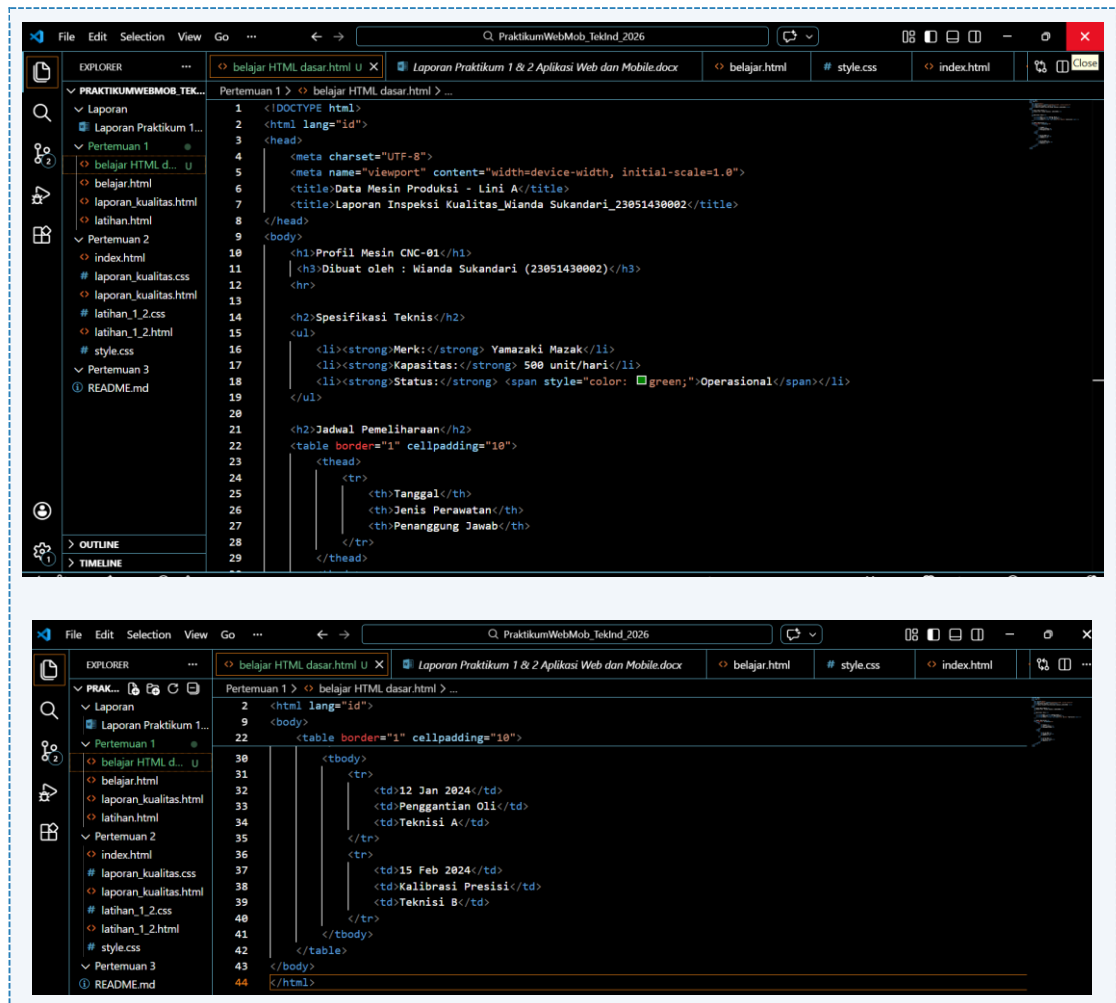
  <h2>Spesifikasi Teknis</h2>
  <ul>
    <li><strong>Merk:</strong> Yamazaki Mazak</li>
    <li><strong>Kapasitas:</strong> 500 unit/hari</li>
    <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
  </ul>

  <h2>Jadwal Pemeliharaan</h2>
  <table border="1" cellpadding="10">
    <thead>
      <tr>
        <th>Tanggal</th>

```

```
        <th>Jenis Perawatan</th>
        <th>Penanggung Jawab</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
    </tr>
    <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
    </tr>
</tbody>
</table>
</body>
</html>
```

Screenshot Hasil:



Gambar E.2 – Hasil Penyusunan Profil Lini Produksi

E.3 Langkah 3 – [Membuat Form Input Data Produksi]

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */

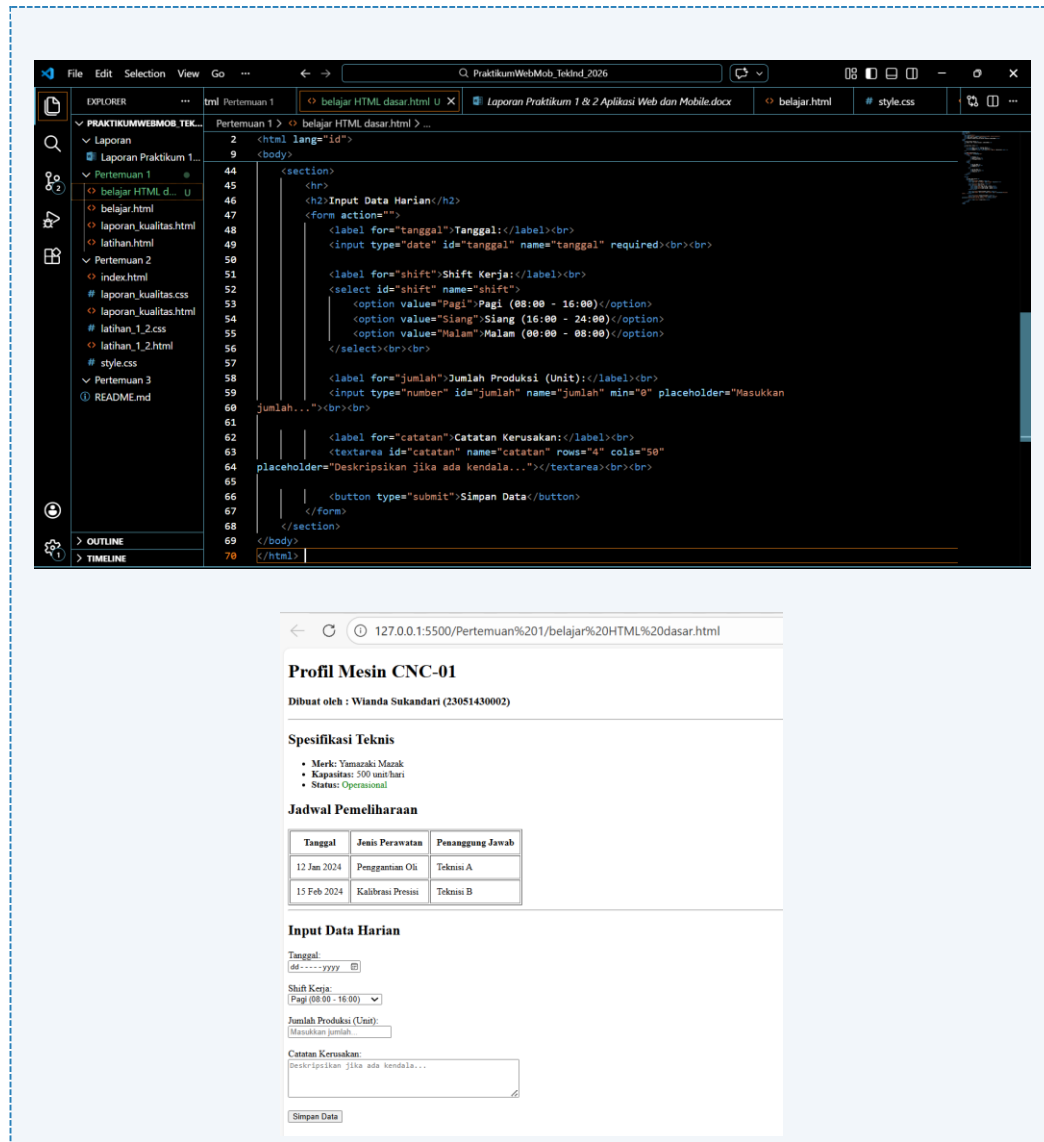
Input Data Harian
<section>
  <hr>
  <h2>Input Data Harian</h2>
  <form action="">
    <label for="tanggal">Tanggal:</label><br>
    <input type="date" id="tanggal" name="tanggal"
required><br><br>

    <label for="shift">Shift Kerja:</label><br>
    <select id="shift" name="shift">
      <option value="Pagi">Pagi (08:00 -
16:00)</option>

```

```
        <option value="Siang">Siang (16:00 -  
24:00)</option>  
        <option value="Malam">Malam (00:00 -  
08:00)</option>  
    </select><br><br>  
  
    <label for="jumlah">Jumlah Produksi  
(Unit) :</label><br>  
    <input type="number" id="jumlah" name="jumlah"  
min="0" placeholder="Masukkan  
jumlah..."><br><br>  
  
    <label for="catatan">Catatan  
Kerusakan:</label><br>  
    <textarea id="catatan" name="catatan" rows="4"  
cols="50"  
placeholder="Deskripsikan jika ada  
kendala..."></textarea><br><br>  
  
    <button type="submit">Simpan Data</button>  
</form>  
</section>  
</body>  
</html>
```

Screenshot Hasil:



Gambar E.3 – Hasil Pembuatan Form Input Data Produksi

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Penambahan Kolom Status

Pertanyaan / Instruksi Latihan:

Latihan 1 (Pengayaan Tabel): Tambahkan satu kolom baru di tabel "Jadwal Pemeliharaan" bernama "Status" dan isi datanya secara manual (Selesai/Pending).

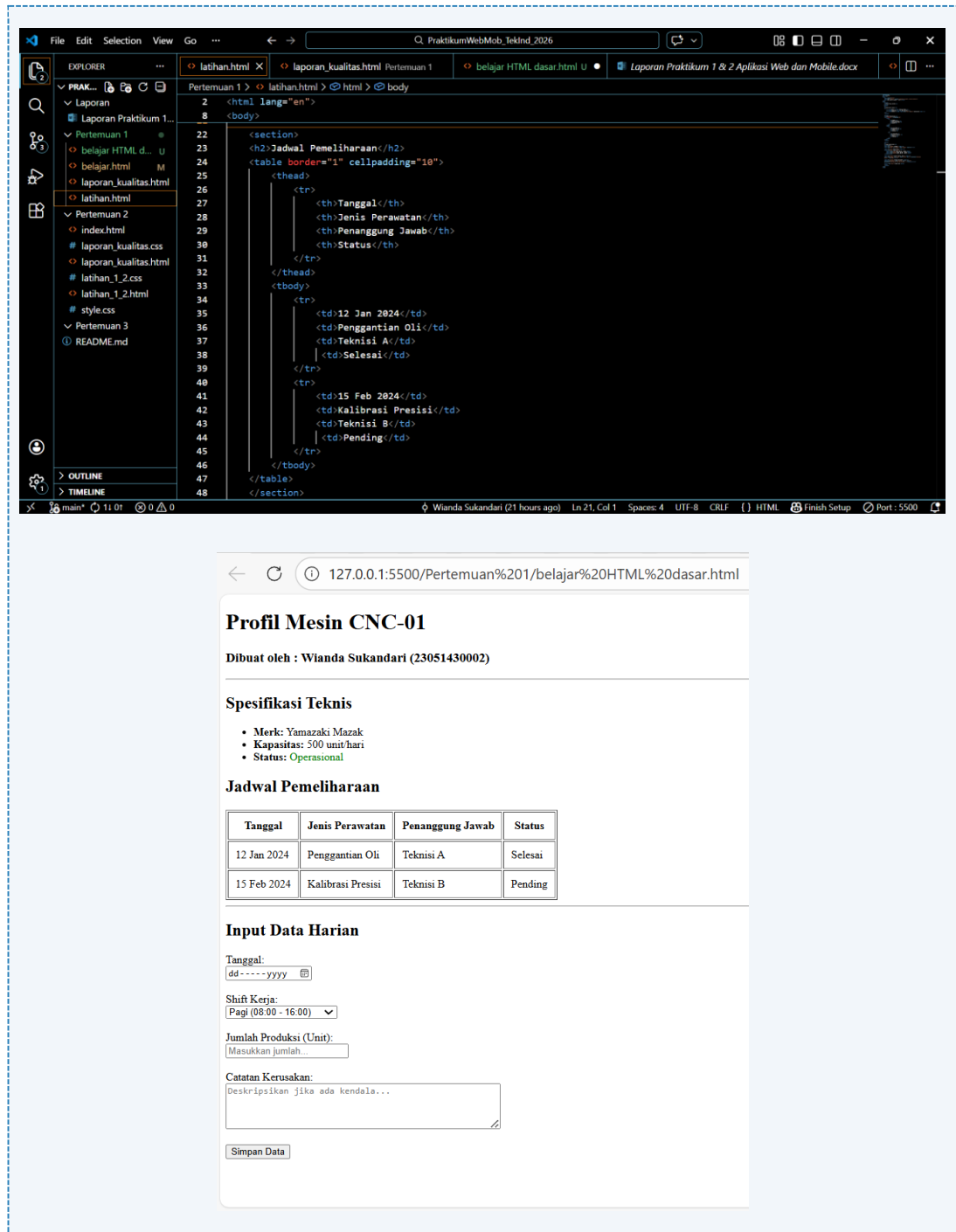
Jawaban / Kode Solusi:

Saya pakai kode dari Langkah 1 sampai 3 kemudian saya menambahkan `<th>Status</th>` pada bagian `<thead>` serta masing masing untuk `<tbody>` di

tanggal 12 Jan 2024 dan 15 Feb 2024 saya tambahkan <td>Selesai</td> dan <td>Pending</td>.

```
[<section>
  <h2>Jadwal Pemeliharaan</h2>
  <table border="1" cellpadding="10">
    <thead>
      <tr>
        <th>Tanggal</th>
        <th>Jenis Perawatan</th>
        <th>Penanggung Jawab</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td>Selesai</td>
      </tr>
      <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td>Pending</td>
      </tr>
    </tbody>
  </table>
</section>
```

Screenshot Hasil:



Gambar F.1 – Hasil Penambahan Kolom Status

F.2 Latihan 2 – Maksimal kapasitas mesin

Pertanyaan / Instruksi Latihan:

Latihan 2 (Validasi Dasar): Pada input "Jumlah Produksi", tambahkan atribut `max` dengan nilai 600 (maksimal kapasitas mesin).

Jawaban / Kode Solusi:

```
[<label for="jumlah">Jumlah Produksi (Unit):</label><br>
  <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan
  jumlah..." max="600"><br><br>
```

Screenshot Hasil:

The screenshot displays a web application interface for a CNC machine profile. The page title is 'Profil Mesin CNC-01'. Under the 'Spesifikasi Teknis' (Technical Specifications) section, the following details are listed:

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional

The 'Jadwal Pemeliharaan' (Maintenance Schedule) section contains a table with the following data:

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Below the table is the 'Input Data Harian' (Daily Data Input) form, which includes the following fields:

- Tanggal: (Date picker)
- Shift Kerja: (Dropdown menu showing 'Pagi (08:00 - 16:00)')
- Jumlah Produksi (Unit): (Input field with placeholder 'Masukkan')
- Catatan Kerusakan: (Text area with placeholder 'Deskripsikan jika ada kendala...')

Gambar F.2 – Hasil Maksimal kapasitas mesin

F.3 Tugas Proyek Mini – Laporan Inspeksi Kualitas

Deskripsi proyek mini:

Buatlah halaman HTML baru bernama `laporan\_kualitas.html`. Halaman ini berisi formulir "Laporan Inspeksi Kualitas" dengan field:

- ID Produk (Text)
- Nama Inspektur (Text)
- Jenis Cacat (Radio Button: Geometri, Permukaan, Fungsional)
- Jumlah Cacat (Number)
- Foto Produk (Input type file - placeholder saja)
- Tombol Submit

Penjelasan pendekatan/solusi yang digunakan:

Saya menyusun pakai struktur HTML dasar lalu langsung focus ke bagian `<form>` sebagai inti fitur. Setiap field saya pisahkan dan kasih komentar di kode (ini biar jelas aja fungsinya), misal bagian ID produk, nama inspektur, jenis cacat, dan seterusnya. Untuk pilihan jenis cacat itu saya pakai radio button dengan name yang sama supaya user Cuma bisa pilih 1 opsi.

Input jumlah cacat saya pakai tipe number biar otomatis cuma bisa diisi angka dan upload foto pakai `type="file"` sebagai *placeholder* sesuai instruksinya. Erakhir saya tambahkan tombol submit supaya form bisa dikirim. Saya pakai pendekatan ini karena paling sederhana, rapi, dan udah Menuhin semua requirement permintaan.

Kode Utama:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Laporan Inspeksi Kualitas</title>
  <title>Laporan Inspeksi Kualitas_Wianda
Sukandari_23051430002</title>
</head>
<body>

<h2>Form Laporan Inspeksi Kualitas</h2>
<h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>

<form>

  <!-- ID Produk -->
  <label>ID Produk:</label><br>
  <input type="text" name="id_produk"><br><br>
```

```
<!-- Nama Inspektur -->
<label>Nama Inspektur:</label><br>
<input type="text" name="inspektur"><br><br>

<!-- Jenis Cacat -->
<label>Jenis Cacat:</label><br>
<input type="radio" name="cacat" value="Geometri">
Geometri<br>
<input type="radio" name="cacat" value="Permukaan">
Permukaan<br>
<input type="radio" name="cacat" value="Fungsional">
Fungsional<br><br>

<!-- Jumlah Cacat -->
<label>Jumlah Cacat:</label><br>
<input type="number" name="jumlah_cacat"><br><br>

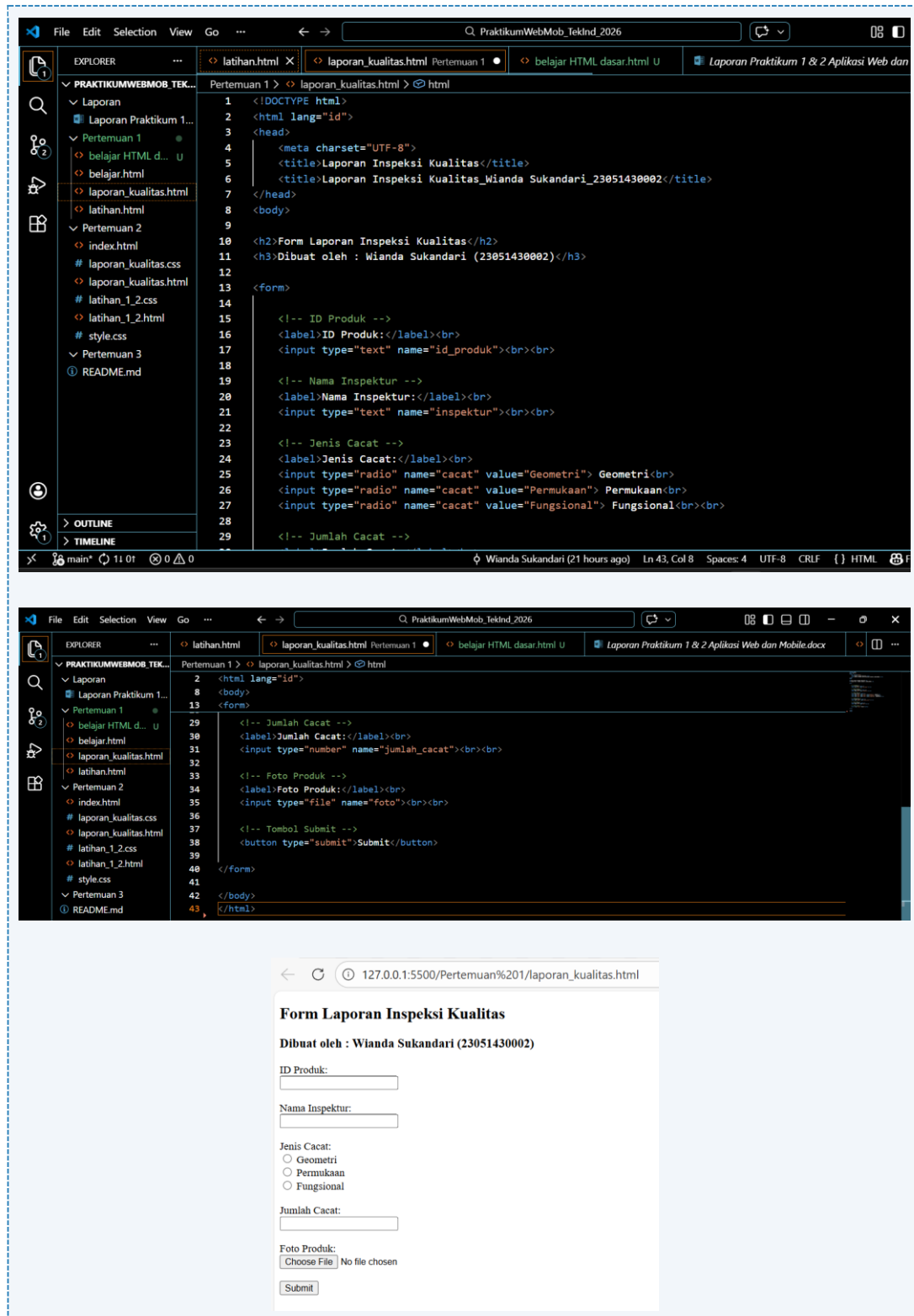
<!-- Foto Produk -->
<label>Foto Produk:</label><br>
<input type="file" name="foto"><br><br>

<!-- Tombol Submit -->
<button type="submit">Submit</button>

</form>

</body>
</html>
```

Screenshot Hasil:



Gambar F.3 – Hasil Laporan Inspeksi Kualitas

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

[Apakah hasil yang Anda peroleh sesuai dengan teori/konsep yang dipelajari? Jelaskan secara spesifik dengan mengacu pada hasil kode/output Anda.]

Hasil output yang saya buat sudah sesuai dengan konsep dasar HTML form yang dipelajari di modul. Secara struktur, halaman sudah mengikuti kerangka HTML standar yaitu diawali deklarasi `<!DOCTYPE html>`, lalu elemen `<html>`, `<head>`, dan `<body>`. Ini nunjukkan bahwa secara teori struktur dokumen sudah benar karena browser bisa baca dan nampilin halaman tanpa error. Judul halaman juga berhasil ditampilkan dengan tag `<title>` dan heading dengan `<h2>` serta `<h3>`, yang berarti penggunaan tag heading udah sesuai fungsi teorinya sebagai penanda hierarki teks.

Terus pada bagian form juga setiap field udah pakai tipe input yang sesuai dengan teori fungsi masing masing. Misalnya `type="text"` dipakai untuk ID Produk dan Nama Inspektur karena datanya berupa teks bebas, `type="radio"` dipakai untuk Jenis Cacat supaya user hanya bisa memilih 1 opsi, dan `type="number"` dipakai Jumlah Cacat biar input terbatas angka aja. Input `type="file"` juga sudah sesuai konsep karena berfungsi sebagai *placeholder* upload file meskipun belum ada proses *backend*. Intinya output saya sudah sesuai sama teori yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kesalahan penulisan atau typo	Memperbaiki dan pakai tag dengan benar
Tampilan kurang rapi	Pakai <code>
</code>
File tidak tampil di browser	Live server belum terinstall

G.3 Kaitan dengan Penerapan di Industri

[Bagaimana keterampilan/teknik yang dipelajari pada modul ini dapat diterapkan dalam konteks Teknik Industri nyata? Berikan contoh skenario konkret.]

Bikin HTML form seperti praktikum ini ada kaitannya langsung sama kebutuhan nyata di Teknik Industri, seperti system pencatatan data operasional. Misal di industry manufaktur, form “Laporan Inspeksi Kualitas” bisa dijadiin prototype awal system digital untuk pencatatan hasil inspeksi

produk di rantai produksi. Disbanding pencatatan manual di kertas, penggunaan form berbasis web bisa bikin input data lebih rapi, terstruktur, dan mudah disimpan ke database untuk analisis kualitas jangka panjangnya.

Konsep pemilihan tipe input yang tepat juga sangat relevan di industry. Contohnya penggunaan radio button untuk jenis cacat memastikan operator cuma milih 1 kkategori cacat jadi bisa ngurangin kesalahan input data. Input number untuk jumlah cacat bantu jaga validitas data karena system Cuma nerima angka. Ini cocok sama prinsip rekayasa system kerja di Teknik Industri yang menekankan penyamaan proses dan pengurangan human error dala pencatatan data produksi.

Kalua dari sisi nyatanya, ini bisa dikembangkan jadi system monitoring produksi berbasis web. Misalnya di pabrik, operator ngisi form inspeksi lewat tablet setiap selesai batch produksi. Data langsung masuk ke system pusat dan bisa dipantau supervisor secara real time buat lihat tren cacat, downtime mesin, atau performa shift kerja. Menurut saya HTML yang dipelajari dimodul ini adalah fondasi awal buat bangun system informasi industry yang lebih kompleks kayak dashboard produksi, system quality control digital, atau bahkan integrasi ke ERP.

Jadi, walau ini masih sederhana tapi konsep dibaliknya sangat aplikatif di industry. Bedanya Cuma skalanya (dipraktikum ini masih sebatas front-end statis sedangkan di industry akan ditambahkan database, backend, validasi lanjutan, dan keamanan system).

H. KESIMPULAN

1	Praktikum ini menunjukkan bahwa HTML adalah dasar pembuatan sistem input data digital yang relevan dengan kebutuhan industri.
2	Penulisan kode yang rapi terbukti mempermudah pembacaan dan pengembangan program.
3	Form input dengan berbagai tipe field berhasil dibuat sesuai kebutuhan data.
4	Data terstruktur seperti profil mesin dan tabel pemeliharaan dapat ditampilkan menggunakan elemen HTML yang sesuai.
5	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>HTML basics</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics
[3]	Rianto, I. (2025). Pemrograman web. <i>Penerbit Tahta Media</i> .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
 <u>Wianda Sukandari</u> NIM: 23051430002	 <u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 2

CSS DASAR & STYLING

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web Dan Mobile Manual Praktikum (Pertemuan 2)
Topik Utama	CSS Dasar & Styling
Sub-Topik	Pengenalan CSS, Selectors, Box Model, dan Styling Form.
Durasi Praktikum	340 Menit
Tanggal Praktikum	20 Feb. 26
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami cara kerja CSS untuk mempercantik antarmuka pengguna (UI).
2	Mampu memanipulasi warna, font, dan ukuran elemen.
3	Menguasai konsep Box Model (Margin, Border, Padding) untuk tata letak.
4	Menerapkan CSS pada formulir agar terlihat profesional dan mudah digunakan.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Live Server Extension	64-bit	Lingkungan pengembangan
5	Git & GitHub	Versi terbaru	Manajemen versi & repository

D. DASAR TEORI

Dasar teori focus ke konsep dasar User Interface (UI) dan pemakaian CSS dalam pengembangan web. UI penting karena tampilan sistem yang jelas, rapi, dan mudah dibaca (bantu pengguna memahami fungsi aplikasi dengan cepat dan mengurangi resiko kesalahan penggunaan). Kalau di system digital atau industry, tampilan yang buruk seperti teks kecil, warna tidak kontras, atau letak berantakan bisa menurunkan efisiensi kerja pengguna.

CSS dipakai buat ngatur tampilan HTML supaya lebih rapi dan konsisten. Ada tiga cara pakainya, yaitu inline, internal, dan external. Inline langsung di tag tapi biasanya kurang disarankan karena bikin kode susah dibaca. Internal ditulis di bagian head dan cocok buat halaman sederhana. Kalau external pakai file CSS terpisah, ini yang paling sering dipakai karena lebih rapi, mudah diatur, dan bisa dipakai ulang di banyak halaman.

Ada juga konsep box model di CSS, di mana setiap elemen dianggap seperti kotak. Di dalamnya ada content (isi), padding (jarak isi ke tepi dalam), border (garis tepi), dan margin (jarak luar antar elemen). Dengan ngerti konsep ini, kita bisa ngatur posisi dan jarak elemen dengan lebih presisi, jadi layout web tetap rapi walaupun dibuka di ukuran layar atau zoom yang berbeda.

D.1 Konsep Utama

Konsep utama modul ini membahas cara pakai CSS buat ngatur tampilan halaman web supaya lebih rapi, konsisten, dan nyaman dilihat. Intinya kita belajar gimana styling bisa bikin form, teks, tombol, dan layout jadi lebih jelas dan gampang dipakai, bukan cuma asal tampil. Di modul ini juga ditekankan pentingnya pakai CSS external biar kode lebih terstruktur dan mudah diedit kalau mau ubah desain. Selain itu, dipelajari juga konsep box model yang bantu kita ngatur jarak antar elemen supaya tampilan tetap proporsional walaupun ukuran layar atau zoom berubah.

D.2 Keterkaitan dengan Teknik Industri

Materi modul ini relevan banget sama Teknik Industri karena di dunia industri banyak sistem digital yang dipakai operator, misalnya dashboard monitoring produksi atau form input data kualitas. Kalau tampilan sistemnya rapi, jelas, dan responsif, operator bisa kerja lebih cepat dan risiko salah input juga berkurang. Contohnya pas bikin form inspeksi kualitas produksi, CSS dipakai buat ngatur layout, ukuran tombol, dan warna supaya operator gampang baca dan langsung ngerti fungsi tiap bagian tanpa bingung.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat File CSS Eksternal

Buat file baru di folder pertemuan 2 dengan nama 'index.html' dan 'style.css'. pada bagian file 'index.html' diisi sama seperti 'index.html' dipertemuan 1. Kemudian hubungkan kedua file tersebut ('index.html' dan 'style.css') dengan menambahkan kode `<link rel="stylesheet" href="style.css">` didalam '`<head>`'.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Data Mesin Produksi - Lini A</title>
  <title>Laporan Inspeksi Kualitas_Wianda
Sukandari_23051430002</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Profil Mesin CNC-01</h1>
    <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
```

```

</header>
<section>
  <hr>
  <h2>Spesifikasi Teknis</h2>
  <ul>
    <li><strong>Merk:</strong> Yamazaki Mazak</li>
    <li><strong>Kapasitas:</strong> 500 unit/hari</li>
    <li><strong>Status:</strong> <span style="color:
green;">Operasional</span></li>
  </ul>
</section>

<section>
  <h2>Jadwal Pemeliharaan</h2>
  <table border="1" cellpadding="10">
    <thead>
      <tr>
        <th>Tanggal</th>
        <th>Jenis Perawatan</th>
        <th>Penanggung Jawab</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td>Selesai</td>
      </tr>
      <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td>Pending</td>
      </tr>
    </tbody>
  </table>
</section>

<section>
  <hr>
  <h2>Input Data Harian</h2>
  <form action="">
    <label for="tanggal">Tanggal:</label><br>
    <input type="date" id="tanggal" name="tanggal"
required><br><br>

    <label for="shift">Shift Kerja:</label><br>
    <select id="shift" name="shift">

```

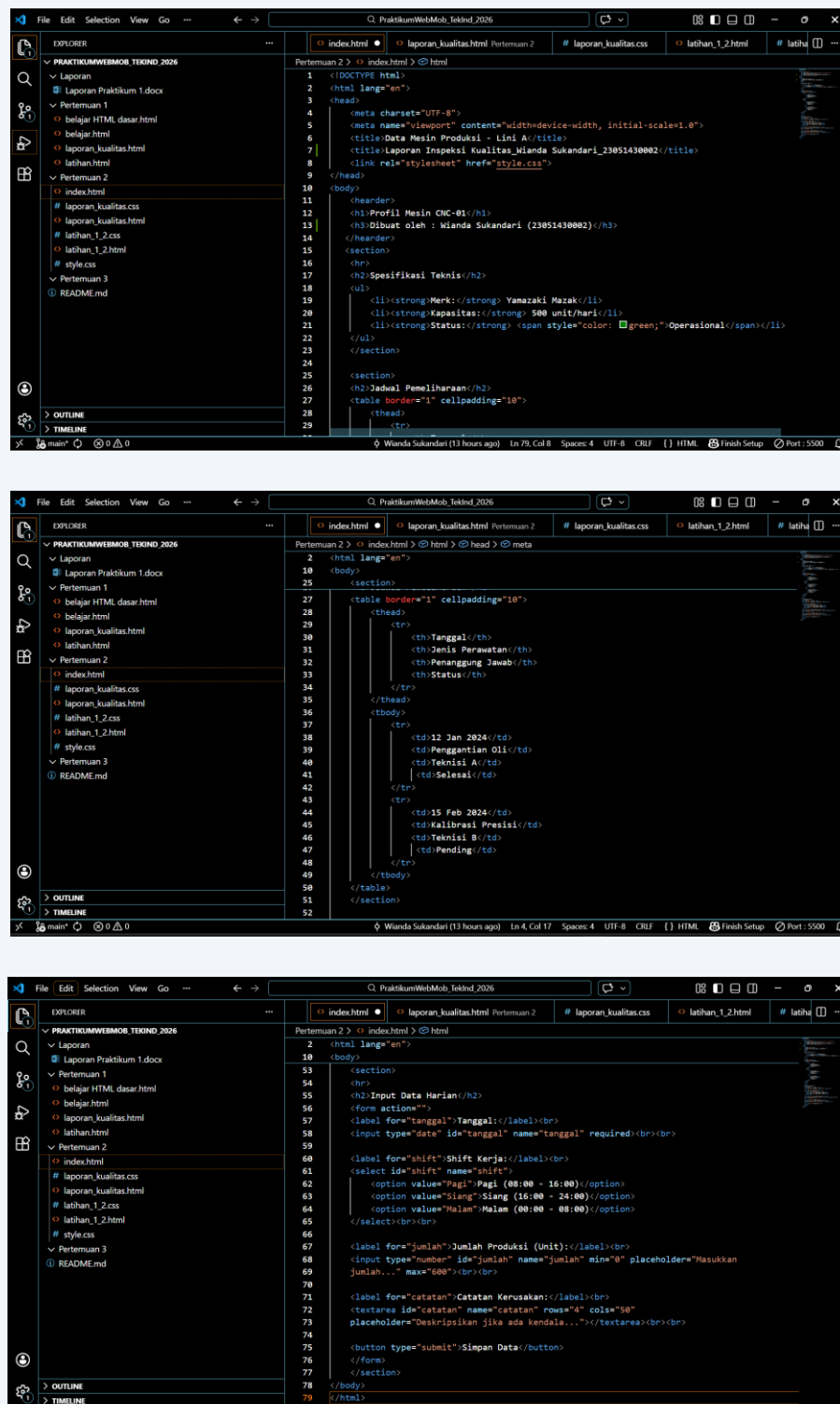
```
        <option value="Pagi">Pagi (08:00 - 16:00)</option>
        <option value="Siang">Siang (16:00 - 24:00)</option>
        <option value="Malam">Malam (00:00 - 08:00)</option>
    </select><br><br>

    <label for="jumlah">Jumlah Produksi (Unit):</label><br>
    <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan
    jumlah..." max="600"><br><br>

    <label for="catatan">Catatan Kerusakan:</label><br>
    <textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
    kendala..."></textarea><br><br>

    <button type="submit">Simpan Data</button>
</form>
</section>
</body>
</html>
```


Screenshot Hasil:



Gambar E.1 – Hasil Membuat File CSS Eksternal di VS CODE

E.2 Langkah 2 – Styling Body dan Tipografi

Input kode dibawah ini ke file 'style.css'

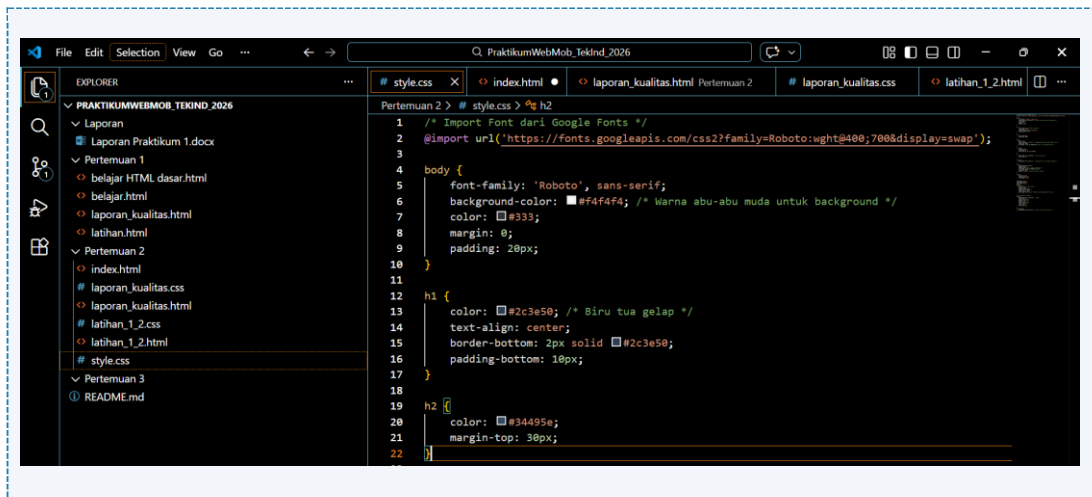
```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
/* Import Font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap')

body {
    font-family: 'Roboto', sans-serif;
    background-color: #f4f4f4; /* Warna abu-abu muda untuk background */
    color: #333;
    margin: 0;
    padding: 20px;
}

h1 {
    color: #2c3e50; /* Biru tua gelap */
    text-align: center;
    border-bottom: 2px solid #2c3e50;
    padding-bottom: 10px;
}

h2 {
    color: #34495e;
    margin-top: 30px;
}
```

Screenshot Hasil:



Gambar E.2 – Styling Body dan Tipografi

E.3 Langkah 3 – Styling Tabel Data

Tambahkan kode dibawah ini untuk melanjutkan kode sebelumnya di 'style.css' untuk membuat table biar mudah dibaca.

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
table {
    width: 100%;
    border-collapse: collapse; /* Menggabungkan border ganda
menjadi satu */
    margin-top: 20px;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1); /* Efek bayangan
halus */
}

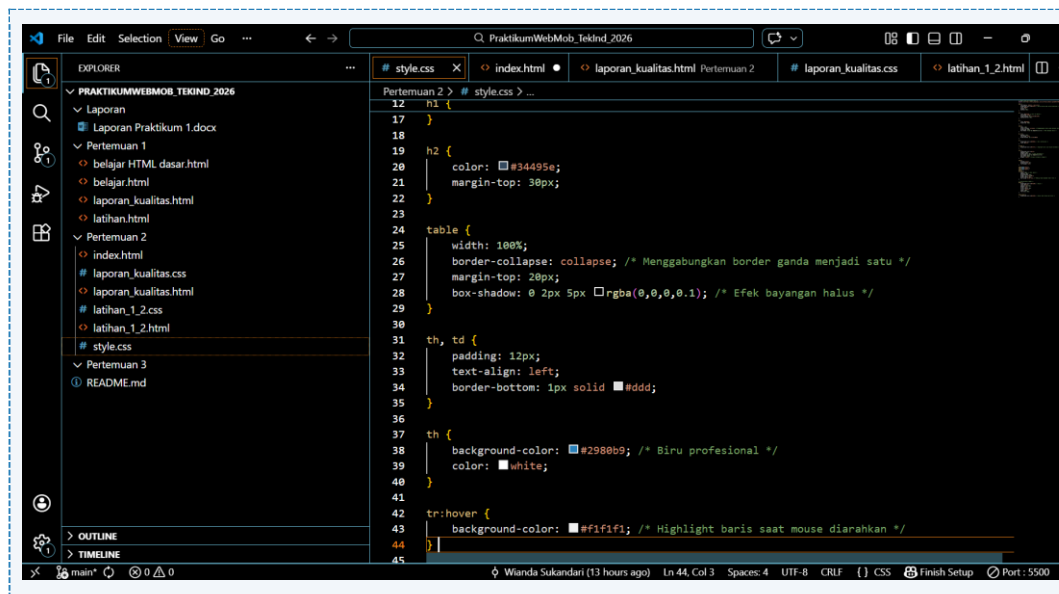
th, td {
    padding: 12px;
    text-align: left;
    border-bottom: 1px solid #ddd;
}

th {
    background-color: #2980b9; /* Biru profesional */
    color: white;
}

tr:hover {
    background-color: #f1f1f1; /* Highlight baris saat mouse
diarahkan */
}

```

Screenshot Hasil:



Gambar E.3 – Hasil Styling Tabel Data di VS CODE

E.4 Langkah 4 – Styling Formulir (Fokus pada UX)

Menambahkan input field supaya formulir mudah diisi dan tampilannya lebih lebar dan jelas.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
form {
  background-color: white;
  padding: 20px;
  border-radius: 8px; /* Sudut melengkung */
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  max-width: 600px;
  margin: 0 auto; /* Formulir berada di tengah */
}

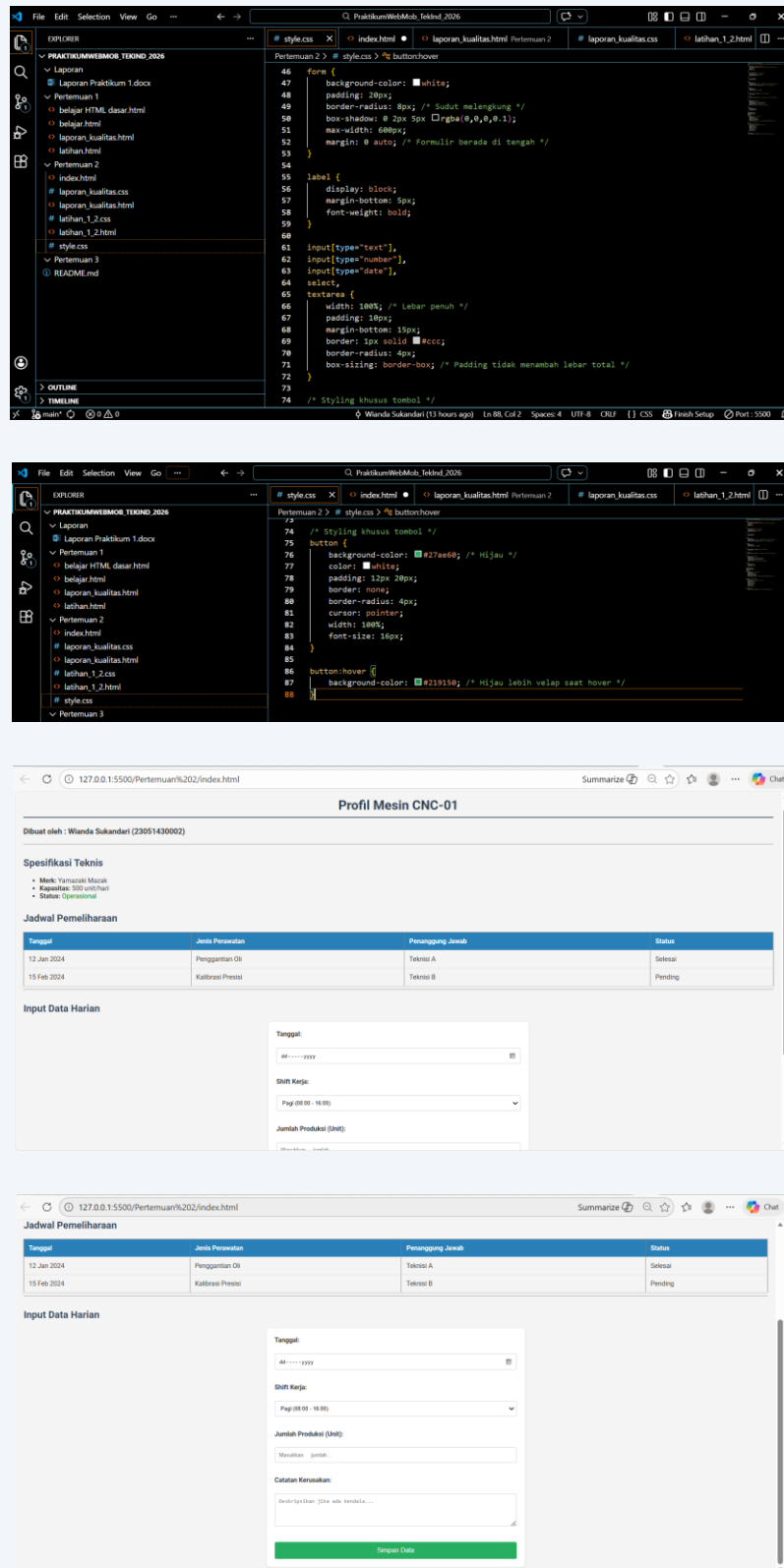
label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
  width: 100%; /* Lebar penuh */
  padding: 10px;
  margin-bottom: 15px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box; /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
  background-color: #27ae60; /* Hijau */
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  width: 100%;
  font-size: 16px;
}

button:hover {
  background-color: #219150; /* Hijau lebih gelap saat hover */
}
```

Screenshot Hasil:



Gambar E.3 – Hasil Styling Tabel Data di VS CODE

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Class Selector

Pertanyaan / Instruksi Latihan:

Latihan 1 (Class Selector): Berikan kelas `status-ok` pada teks status operasional di HTML, dan berikan warna hijau tebal di CSS. Berikan kelas `status-down` dan berikan warna merah.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <link rel="stylesheet" href="latihan_1_2.css">
  <title>Data Mesin Produksi - Lini A</title>
  <title>Laporan Inspeksi Kualitas_Wianda
Sukandari_23051430002</title>
</head>

<body>
  <h1>Profil Mesin CNC-01</h1>
  <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
  <hr>

  <h2>Spesifikasi Teknis</h2>
  <ul>
    <li><strong>Merk:</strong> Yamazaki Mazak</li>
    <li><strong>Kapasitas:</strong> 500 unit/hari</li>
    <!-- Penambahan class status ok di text operasional
-->
    <li><strong>Status:</strong> <span class="status-
ok">Operasional</span></li>
  </ul>
  <ul>
    <li><strong>Merk:</strong> Mori Seiki</li>
    <li><strong>Kapasitas:</strong> 300 unit/hari</li>
    <!-- Penambahan class status down di text tidak
operasional -->
```

```

        <li><strong>Status:</strong> <span class="status-
down">Tidak Operasional</span></li>
    </ul>
    <h2>Jadwal Pemeliharaan</h2>
    <table border="1" cellpadding="10">
        <thead>
            <tr>
                <th>Tanggal</th>
                <th>Jenis Perawatan</th>
                <th>Penanggung Jawab</th>
                <th>Status</th>
            </tr>
        </thead>
        <tbody>
            <tr>
                <td>12 Jan 2024</td>
                <td>Penggantian Oli</td>
                <td>Teknisi A</td>
                <td class="status-ok">Selesai</td>
            </tr>
            <tr>
                <td>15 Feb 2024</td>
                <td>Kalibrasi Presisi</td>
                <td>Teknisi B</td>
                <td class="status-down">Pending</td>
            </tr>
        </tbody>
    </table>
    <hr>
    <h2>Input Data Harian</h2>
    <form action="">
        <label for="tanggal">Tanggal:</label><br>
        <input type="date" id="tanggal" name="tanggal"
required><br><br>
        <label for="shift">Shift Kerja:</label><br>
        <select id="shift" name="shift">
            <option value="Pagi">Pagi (08:00 -
16:00)</option>
            <option value="Siang">Siang (16:00 -
24:00)</option>
            <option value="Malam">Malam (00:00 -
08:00)</option>
        </select><br><br>
        <label for="jumlah">Jumlah Produksi
(Unit):</label><br>
        <input type="number" id="jumlah" name="jumlah"
min="0" placeholder="Masukkan jumlah..."><br><br>
        <label for="catatan">Catatan Kerusakan:</label><br>
        <textarea id="catatan" name="catatan" rows="4"
cols="50"

```

```
        placeholder="Deskripsikan jika ada  
kendala..."></textarea><br><br>  
        <button type="submit">Simpan Data</button>  
    </form>  
</body>  
</html>
```


Screenshot Hasil:

The image displays two screenshots of a web application interface for managing CNC machines.

Top Screenshot: Profil Mesin CNC-01

The page title is "Profil Mesin CNC-01". Below the title, it says "Dibuat oleh : Wianda Sukandari (23051430002)".

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: Operasional
- Merk: Mori Seiki
- Kapasitas: 300 unit/hari
- Status: **Tidak Operasional**

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Input Data Harian

Form fields include:

- Tanggal: (Date picker)
- Shift Kerja: (Dropdown menu, currently showing "Pagi (08.00 - 16.00)")

Bottom Screenshot: Jadwal Pemeliharaan

This screenshot shows the same "Jadwal Pemeliharaan" table as the top one.

Input Data Harian

Form fields include:

- Tanggal: (Date picker)
- Shift Kerja: (Dropdown menu, currently showing "Pagi (08.00 - 16.00)")
- Jumlah Produksi (Unit): (Text input field, placeholder: "Masukkan jumlah...")
- Catatan Kerusakan: (Text area, placeholder: "Deskripsikan jika ada kendala...")
- Simpan Data: (Green button)

Gambar F.1 – Class Selector

F.2 Latihan 2 – Layout Container

Pertanyaan / Instruksi Latihan:

Latihan 2 (Layout Container): Buat sebuah `

` dengan id `container` di HTML yang membungkus seluruh konten. Di CSS, berikan `max-width: 800px`, `margin: 0 auto`, dan `background: white` agar konten terlihat seperti kertas di tengah layar.

Jawaban / Kode Solusi:

```
HTML

<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="latihan_1_2.css">
  <title>Data Mesin Produksi - Lini A</title>
  <title>Laporan Inspeksi Kualitas_Wianda Sukandari_23051430002</title>
</head>

<body>
  <div class="container">
    <h1>Profil Mesin CNC-01</h1>
    <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
    <hr>

    <h2>Spesifikasi Teknis</h2>
    <ul>
      <li><strong>Merk:</strong> Yamazaki Mazak</li>
      <li><strong>Kapasitas:</strong> 500 unit/hari</li>
      <!-- Penambahan class status ok di text operasional -->
      <li><strong>Status:</strong> <span class="status-
ok">Operasional</span></li>
    </ul>
    <ul>
      <li><strong>Merk:</strong> Mori Seiki</li>
      <li><strong>Kapasitas:</strong> 300 unit/hari</li>
      <!-- Penambahan class status down di text tidak operasional -->
      <li><strong>Status:</strong> <span class="status-down">Tidak
Operasional</span></li>
    </ul>
    <h2>Jadwal Pemeliharaan</h2>
    <table border="1" cellpadding="10">
      <thead>
        <tr>
          <th>Tanggal</th>
          <th>Jenis Perawatan</th>
        </tr>
      </thead>
    </table>
  </div>
</body>
</html>
```

```

        <th>Penanggung Jawab</th>
        <th>Status</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>12 Jan 2024</td>
        <td>Penggantian Oli</td>
        <td>Teknisi A</td>
        <td class="status-ok">Selesai</td>
    </tr>
    <tr>
        <td>15 Feb 2024</td>
        <td>Kalibrasi Presisi</td>
        <td>Teknisi B</td>
        <td class="status-down">Pending</td>
    </tr>
</tbody>
</table>
<hr>
<h2>Input Data Harian</h2>
<form action="">
    <label for="tanggal">Tanggal:</label><br>
    <input type="date" id="tanggal" name="tanggal" required><br><br>
    <label for="shift">Shift Kerja:</label><br>
    <select id="shift" name="shift">
        <option value="Pagi">Pagi (08:00 - 16:00)</option>
        <option value="Siang">Siang (16:00 - 24:00)</option>
        <option value="Malam">Malam (00:00 - 08:00)</option>
    </select><br><br>
    <label for="jumlah">Jumlah Produksi (Unit):</label><br>
    <input type="number" id="jumlah" name="jumlah" min="0"
placeholder="Masukkan jumlah..."><br><br>
    <label for="catatan">Catatan Kerusakan:</label><br>
    <textarea id="catatan" name="catatan" rows="4" cols="50"
placeholder="Deskripsikan jika ada
kendala..."></textarea><br><br>
    <button type="submit">Simpan Data</button>
</form>
</div>
</body>
</html>

```

CSS

```

/* Import Font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap'

```

```
body {
  font-family: 'Roboto', sans-serif;
  background-color: #f4f4f4;
  /* Warna abu-abu muda untuk background */
  color: #333;
  margin: 0;
  padding: 20px;
}

h1 {
  color: #2c3e50;
  /* Biru tua gelap */
  text-align: center;
  border-bottom: 2px solid #2c3e50;
  padding-bottom: 10px;
}

h2 {
  color: #34495e;
  margin-top: 30px;
}

/* Styling tabel data */
table {
  width: 100%;
  border-collapse: collapse;
  /* Menggabungkan border ganda menjadi satu */
  margin-top: 20px;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  /* Efek bayangan halus */
}

th,
td {
  padding: 12px;
  text-align: left;
  border-bottom: 1px solid #ddd;
}

th {
  background-color: #2980b9;
  /* Biru profesional */
  color: white;
}

tr:hover {
  background-color: #f1f1f1;
  /* Highlight baris saat mouse diarahkan */
}
```

```
/* form */
form {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    /* Sudut melengkung */
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    max-width: 600px;
    margin: 0 auto;
    /* Formulir berada di tengah */
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

input[type="text"],
input[type="number"],
input[type="date"],
select,
textarea {
    width: 100%;
    /* Lebar penuh */
    padding: 10px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
    /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
    background-color: #27ae60;
    /* Hijau */
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    width: 100%;
    font-size: 16px;
}

button:hover {
    background-color: #219150;
    /* Hijau lebih gelap saat hover */
}
```

```
/* Penambahan Style status ok dan status down (latihan 1) */  
.status-ok {  
    color: #219150;  
    font-weight: bold;  
}  
  
.status-down {  
    color: red;  
    font-weight: bold;  
}  
  
/* penambahan style container (latihan 2) */  
.container {  
    max-width: 800px;  
    margin: 0 auto;  
    background: white;  
    padding: 20px;  
    border-radius: 15px;  
}
```

Profil Mesin CNC-01

Dibuat oleh : Wianda Sukandari (23051430002)

Spesifikasi Teknis

- Merk: Yamazaki Mazak
- Kapasitas: 500 unit/hari
- Status: **Operasional**
- Merk: Mori Seiki
- Kapasitas: 300 unit/hari
- Status: **Tidak Operasional**

Jadwal Pemeliharaan

Tanggal	Jenis Perawatan	Penanggung Jawab	Status
12 Jan 2024	Penggantian Oli	Teknisi A	Selesai
15 Feb 2024	Kalibrasi Presisi	Teknisi B	Pending

Input Data Harian

Tanggal:

Shift Kerja:

Pagi (08:00 - 16:00)

Jumlah Produksi (Unit):

Catatan Kerusakan:

Deskripsikan jika ada kendala...

Simpan Data

Gambar F.2 – Layout Container

F.3 Tugas Proyek Mini – Industri

Deskripsi proyek mini:

Terapkan styling CSS yang sama ke file `laporan\_kualitas.html` yang Anda buat di pertemuan 1.

- Tambahkan aturan CSS agar tombol "Reset" (jika ada) berwarna merah (`#c0392b`).
- Pastikan form responsif (tetap rapi saat di-zoom).

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dikerjakan dengan cara memakai file CSS yang sama seperti sebelumnya supaya tampilannya konsisten dan tidak perlu styling dari awal lagi. Setelah itu ditambahkan aturan khusus buat tombol reset biar warnanya merah sesuai instruksi, jadi tombolnya kelihatan jelas dan beda dari tombol lain.

Selain itu, form juga diatur supaya responsif, jadi tampilannya tetap rapi walaupun halaman di-zoom atau ukuran layar berubah. Caranya dengan pakai pengaturan CSS yang fleksibel seperti max-width dan ukuran elemen yang bisa menyesuaikan layar, sehingga layout tetap enak dilihat dan tidak berantakan.

Kode Utama:

```
HTML

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="laporan_kualitas.css">
  <title>Laporan Inspeksi Kualitas_Wianda Sukandari_23051430002</title>
</head>

<body>
  <div class="container">
    <header>
      <h1>Laporan Inspeksi Kualitas</h1>
      <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
    </header>
    <section>
      <form action="">
        <!-- field id produk -->
        <label for="id_produk">Id Produk :</label><br>
        <input type="text" id="id_produk" name="Id Produk"
placeholder="Masukkan id produk" required><br><br>

        <!-- field nama inspektur -->
        <label for="nama_inspektur">Nama Inspektur :</label><br>
        <input type="text" id="nama_inspektur"
name="nama_inspektur" placeholder="Masukkan nama inspektur">
      </form>
    </section>
  </div>
</body>
</html>
```



```

        required><br><br>

        <!-- field jenis cacat -->
        <label>Jenis Cacat :</label>
        <!-- Radio opsi geometri -->
        <input type="radio" id="geometri" name="status"
value="geometri">
        <label class="label-radio"
for="geometri">Geometri</label><br>
        <!-- Radio opsi permukaan -->
        <input type="radio" id="permukaan" name="status"
value="permukaan">
        <label class="label-radio"
for="permukaan">Permukaan</label><br>
        <!-- Radio opsi fungsional -->
        <input type="radio" id="fungsional" name="status"
value="fungsional">
        <label class="label-radio"
for="fungsional">Fungsional</label><br><br>

        <!-- field jumlah cacat -->
        <label for="jumlah_cacat">Jumlah Produk Cacat (Unit)
:</label><br>
        <input type="number" id="jumlah_cacat" name="jumlah_cacat"
min="0"
        placeholder="Masukkan jumlah produk cacat"><br><br>

        <!-- field foto produk -->
        <label for="foto_produk">Upload Foto Produk :</label><br>
        <input type="file" id="foto_produk" name="foto_produk"
        placeholder="Upload foto produk dari perangkat Anda">
        <br><br>
        <button type="submit">Simpan Data</button><br><br>
        <button type="reset">Reset data</button>
    </form>
</section>
</div>
</body>

</html>

```

CSS

```
/* Import Font dari Google Fonts */
@import
url('https://fonts.googleapis.com/css2?family=Roboto:wght@400;700&display=swap');

body {
    font-family: 'Roboto', sans-serif;
    background-color: #f4f4f4;
    /* Warna abu-abu muda untuk background */
    color: #333;
    margin: 10px;
    padding: 20px;
}

h1 {
    color: #2c3e50;
    /* Biru tua gelap */
    text-align: center;
    border-bottom: 2px solid #2c3e50;
    padding-bottom: 10px;
}

h3 {
    color: #34495e;
    text-align: center;
}

/* form */
form {
    background-color: white;
    padding: 20px;
    border-radius: 8px;
    /* Sudut melengkung */
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    max-width: 600px;
    margin: 0 auto;
    /* Formulir berada di tengah */
}

label {
    display: block;
    margin-bottom: 5px;
    font-weight: bold;
}

.label-radio {
    display: inline-block;
    font-weight: lighter;
}
```

```
input[type="text"],
input[type="number"],
input[type="radio"] select,
textarea {
    width: 100%;
    /* Lebar penuh */
    padding: 10px;
    margin-bottom: 15px;
    border: 1px solid #ccc;
    border-radius: 4px;
    box-sizing: border-box;
    /* Padding tidak menambah lebar total */
}

/* Styling khusus tombol */
button {
    background-color: #f1f1f1;
    color: white;
    padding: 12px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    width: 100%;
    font-size: 16px;
}

button[type="submit"] {
    background-color: #28a745;
    color: white;
}

button[type="submit"]:hover {
    background-color: #218838;
}

button[type="reset"] {
    background-color: #dc3545;
    color: white;
}

button[type="reset"]:hover {
    background-color: #c82333;
}

button:hover {
    background-color: #219150;
    /* Hijau lebih gelap saat hover */
}
```

```
/* penambahan style container (latihan 2) */  
.container {  
  max-width: 800px;  
  margin: auto;  
  background: white;  
  padding: 20px;  
  border-radius: 15px;  
}
```

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5500/Pertemuan%202/laporan_kualitas.html". The page content is a form titled "Laporan Inspeksi Kualitas" with a subtitle "Dibuat oleh : Wianda Sukandari (23051430002)". The form includes the following fields and controls:

- Id Produk :** A text input field with the placeholder "Masukkan id produk".
- Nama Inspektur :** A text input field with the placeholder "Masukkan nama inspektur".
- Jenis Cacat :** Three radio button options: "Geometri", "Permukaan", and "Fungsional".
- Jumlah Produk Cacat(Unit) :** A text input field with the placeholder "Masukkan jumlah produk cacat".
- Upload Foto Produk :** A file upload control showing "Choose File" and "No file chosen".
- Buttons:** A green "Simpan Data" button and a red "Reset data" button.

Gambar F.3 – Output Laporan Inspeksi Kualitas Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

[Apakah hasil yang Anda peroleh sesuai dengan teori/konsep yang dipelajari? Jelaskan secara spesifik dengan mengacu pada hasil kode/output Anda.]

Hasil yang saya dapat/ output saya sudah sesuai dengan konsep yang dipelajari di modul, terutama di penggunaan HTML untuk struktur halaman dan CSS untuk pengatur tampilan. Dari outputnya sendiri, halaman 'laporan_kualitas.html' bisa nampilin form dengan susunan elemen yang rapi (dari field teks, radio button, number input, dan file input). Dimana ini nunjukin kalau penerapan teori tentang fungsi masing masing tipe input HTML udah tepat karena tiap jenis data pakai tipe input yang sesuai sama karakteristiknya.

Kalau dari styling, penerapan CSS eksternal juga udah berjalan sesuai teori pemisahan struktur dan tampilan. Layout form bisa dibuat lebih rapi dan posisinya di tengah layer pakai pengaturan max-width dan margin otomatis, sehingga tampilan keliatan kayak lembar kerja di Tengah layer gitu. Selain itu, tombol reset berhasil dikasi warna khusus lewat selector CSS `button[type="reset"]`, yang membuktikan kalau penggunaan selector atribut bisa ngatur elemen secara spesifik tanpa ganggu elemen lainnya.

Tapi memang menurut say aini masih di tahap dasar dan belum terhubung sama system backend untuk proses datanya. Selain itu, responsivitas masih terbatas pada pengaturan layout sederhana. Walau gitu, secara konsep output praktikum saya udah sesuai sama teori modul yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan form berantakan	Pakai property CSS (max-width, flexbox, dan box-sizing)
Tombol reset belum merah	Tambah selector <code>button[type="reset"]</code> pada CSS dan kasih warna merah
Styling tidak muncul	Cek lagi penulisan link CSS dan pastiin file stylesheet terhubung dengan benar

G.3 Kaitan dengan Penerapan di Industri

Iya, terutama pada keterampilan styling CSS dan pembuatan tampilan web responsive. Contohnya waktu Perusahaan punya system input data kualitas produk atau laporan produksi berbasis web, tampilan form yang rapi, mudah dipakai, dan responsive bakal bikin operator lebih cepet ngisi data tanpa salah input. Jadi bukan cuma desain tapi juga ke efisiensi kerja dan akurasi data.

Misalnya di pabrik, bagian QC pakai web form untuk nyatet jumlah cacat produk tiap shift. Kalau tampilan tombol, warna, dan layout jelas serta konsisten, operator bisa langsung paham fungsi tiap elemen tanpa bingung. Dan ini mempercepat proses pencatatan dan ngurain kesalahan.

H. KESIMPULAN

1	CSS eksternal berhasil bikin tampilan halaman konsisten.
2	Selector button[type="reset"] sukses ubah warna tombol tanpa ganggu tombol lain.
3	Form tetap rapi saat di-zoom karena layout responsif.
4	Pemisahan HTML dan CSS mempermudah edit tampilan.
5	Hasil sesuai teori karena semua styling tampil sesuai kode.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>HTML basics</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics
[3]	Rianto, I. (2025). Pemrograman web. <i>Penerbit Tahta Media</i> .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 3

LAYOUT CSS LANJUTAN (FLEXBOX & GRID) + BOOTSTRAP

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 3)
Topik Utama	Layout CSS Lanjutan (Flexbox & Grid) + Bootstrap
Sub-Topik	CSS Flexbox, CSS Grid, dan Framework CSS (Bootstrap 5) untuk prototyping cepat.
Durasi Praktikum	340 Menit
Tanggal Praktikum	28 February 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep layout modern menggunakan Flexbox dan Grid.
2	Mengenal dan menggunakan Framework CSS (Bootstrap) untuk mempercepat pengembangan UI.
3	Mampu membuat layout dashboard responsif (Sidebar + Main Content).
4	Menerapkan komponen Bootstrap (Card, Navbar, Button) pada konteks industri.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Live Server Extension	64-bit	Lingkungan pengembangan
5	Git & GitHub	Versi terbaru	Manajemen versi & repository

D. DASAR TEORI

Kalau bikin tampilan website atau dashboard, sekarang kita nggak harus ngoding semuanya dari awal. Soalnya sudah ada framework kayak Bootstrap yang nyediain banyak class siap pakai buat ngatur tampilan. Jadi kita tinggal pakai aja tanpa harus ribet bikin CSS sendiri satu-satu. Ini bikin proses bikin halaman jadi lebih cepat dan hasilnya juga biasanya langsung rapi.

Di materi ini juga dibahas soal Flexbox sama CSS Grid. Flexbox itu biasanya dipakai buat ngatur posisi elemen dalam satu arah, misalnya biar tombol sejajar atau menu navbar kelihatan rata. Kalau CSS Grid lebih cocok buat layout yang lebih kompleks karena dia bisa ngatur baris dan kolom sekaligus. Jadi kalau mau bikin dashboard yang isinya banyak kotak info, Grid ini enak banget dipakai.

Kesimpulannya, belajar layout modern itu penting karena tampilan website sekarang harus bisa menyesuaikan ukuran layar. Mau dibuka di laptop, tablet, atau HP tetap harus kelihatan rapi dan gampang dibaca. Makanya konsep responsif jadi hal dasar yang wajib dipahami.

D.1 Konsep Utama

Inti dari modul ini adalah belajar cara nyusun layout supaya tampilan web terstruktur dan enak dilihat. Bootstrap punya sistem grid 12 kolom yang bisa dibagi sesuai kebutuhan. Misalnya mau bikin layout kiri kecil kanan besar, tinggal atur class kolomnya aja. Nggak perlu hitung ukuran manual.

Selain itu, Flexbox ngebantu banget buat ngatur posisi elemen biar sejajar atau rata tengah tanpa trik ribet. Sedangkan CSS Grid dipakai kalau susunan halamannya lebih kompleks, misalnya dashboard yang ada banyak card, tabel, dan form dalam satu halaman. Kombinasi ketiganya bikin desain jadi fleksibel dan gampang diatur.

D.2 Keterkaitan dengan Teknik Industri

Materi ini nyambung banget sama Teknik Industri, apalagi kalau sudah bahas sistem monitoring atau dashboard data. Di dunia industri, data itu harus ditampilkan dengan jelas supaya gampang dipantau. Misalnya dashboard produksi, dashboard stok gudang, atau dashboard performa mesin.

Contoh simpelnya, di gudang bahan baku kita bisa bikin tampilan tabel stok yang nunjukin jumlah barang masuk, keluar, dan sisa. Kalau layout-nya rapi, orang langsung paham tanpa harus mikir lama. Jadi walaupun kelihatannya ini materi web, sebenarnya kepaake banget buat bantu visualisasi data industri biar lebih gampang dianalisis.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Integrasi Bootstrap 5

Pada langkah pertama dibuat file HTML baru bernama *latihan.html* sebagai halaman dasar. Di dalamnya ditambahkan struktur HTML standar lalu dihubungkan dengan Bootstrap 5 lewat CDN supaya bisa langsung pakai styling siap pakai. Selain itu disambungkan juga file CSS sendiri (*Praktikum.css*) buat tambahan style kalau diperlukan. Terakhir, ditambahkan script Bootstrap JS di bagian bawah agar fitur interaktif seperti tombol menu bisa berfungsi. Langkah ini intinya buat nyiapin kerangka awal sebelum halaman diisi.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
<!DOCTYPE html>  
<html lang="id">  
<head>  
  <meta charset="UTF-8">
```

```

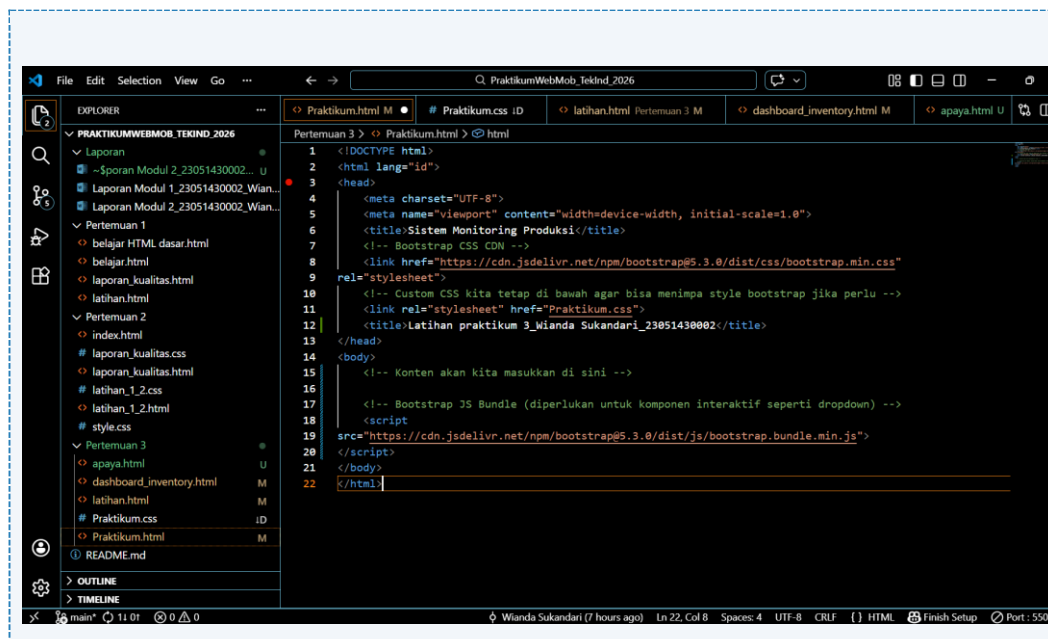
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Sistem Monitoring Produksi</title>
    <!-- Bootstrap CSS CDN -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bo
otstrap.min.css"
rel="stylesheet">
    <!-- Custom CSS kita tetap di bawah agar bisa menimpa
style bootstrap jika perlu -->
    <link rel="stylesheet" href="Praktikum.css">
    <title>Latihan praktikum 3_Wianda
Sukandari_23051430002</title>
</head>

<body>
    <!-- Konten akan kita masukkan di sini -->

    <!-- Bootstrap JS Bundle (diperlukan untuk komponen
interaktif seperti dropdown) -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boot
strap.bundle.min.js">
</script>
</body>
</html>

```

Screenshot Hasil:



Gambar E.1 – Integrasi Bootstrap 5

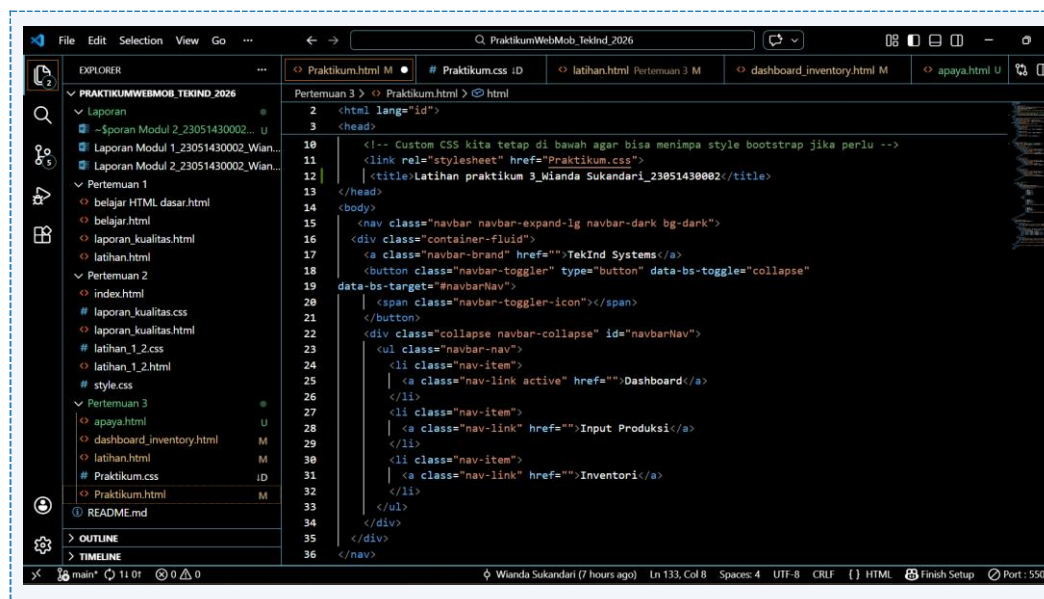
E.2 Langkah 2 – Membuat Navigasi Bar (Navbar)

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="">TekInd Systems</a>
    <button class="navbar-toggler" type="button" data-bs-
toggle="collapse"
data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
      <ul class="navbar-nav">
        <li class="nav-item">
          <a class="nav-link active" href="">Dashboard</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="">Input Produksi</a>
        </li>
        <li class="nav-item">
          <a class="nav-link" href="">Inventori</a>
        </li>
      </ul>
    </div>
  </div>
</nav>

```

Screenshot Hasil:



Gambar E.2 – Membuat Navigasi Bar (Navbar)

E.3 Langkah 3 – Layout Grid Dashboard (Sistem Grid)

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
<div class="container mt-4">
  <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
  <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
  <div class="row mb-4">
    <!-- Kolom lebar 4 (dari 12) -->
    <div class="col-md-4">
      <div class="card text-white bg-primary mb-3">
        <div class="card-header">Total Produksi</div>
        <div class="card-body">
          <h5 class="card-title">1,250 Unit</h5>
          <p class="card-text">Target Harian: 1,200
Unit</p>
        </div>
      </div>
    </div>
    <!-- Kolom lebar 4 -->
    <div class="col-md-4">
      <div class="card text-white bg-success mb-3">
        <div class="card-header">Efisiensi Mesin</div>
        <div class="card-body">
          <h5 class="card-title">92%</h5>
          <p class="card-text">Status: Optimal</p>
        </div>
      </div>
    </div>
    <!-- Kolom lebar 4 -->
    <div class="col-md-4">
      <div class="card text-white bg-warning mb-3">
        <div class="card-header">Reject Rate</div>
        <div class="card-body">
          <h5 class="card-title">1.5%</h5>
          <p class="card-text">Perlu perhatian shift
malam</p>
        </div>
      </div>
    </div>
  </div>

  <!-- Row 2: Tabel Data -->
  <div class="row">
    <div class="col-12">
      <h3>Data Log Produksi Real-time</h3>
      <table class="table table-striped table-hover">
        <thead class="table-dark">
          <tr>
            <th>Waktu</th>
            <th>Mesin</th>
            <th>Produk</th>
          </tr>
        </thead>
      </table>
    </div>
  </div>

```

```
        <th>Jumlah</th>
        <th>Status</th>
    </tr>
</thead>
<tbody>
    <tr>
        <td>08:00</td>
        <td>CNC-01</td>
        <td>Part A-12</td>
        <td>50</td>
        <td><span class="badge bg-
success">OK</span></td>
    </tr>
    <tr>
        <td>08:15</td>
        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-
danger">Error</span></td>
    </tr>
</tbody>
</table>
</div>
</div>
</div>
```

Screenshot Hasil:

```

2 <html lang="id">
14 <body>
37 <div class="container mt-4">
40 <div class="row mb-4">
41 <!-- Kolom lebar 4 (dari 12) -->
42 <div class="col-md-4">
43 <div class="card text-white bg-primary mb-3">
44 <div class="card-header">Total Produksi</div>
45 <div class="card-body">
46 <h5 class="card-title">1,250 Unit</h5>
47 <p class="card-text">Target Harian: 1,200 Unit</p>
48 </div>
49 </div>
50 <!-- Kolom lebar 4 -->
51 <div class="col-md-4">
52 <div class="card text-white bg-success mb-3">
53 <div class="card-header">Efisiensi Mesin</div>
54 <div class="card-body">
55 <h5 class="card-title">92%</h5>
56 <p class="card-text">Status: Optimal</p>
57 </div>
58 </div>
59 </div>
60 </div>
61 <!-- Kolom lebar 4 -->
62 <div class="col-md-4">
63 <div class="card text-white bg-warning mb-3">

```

```

64 <div class="card text-white bg-warning mb-3">
65 <div class="card-header">Reject Rate</div>
66 <div class="card-body">
67 <h5 class="card-title">1.5%</h5>
68 <p class="card-text">Perlu perhatian shift malam</p>
69 </div>
70 </div>
71 </div>
72 <!-- Row 2: Tabel Data -->
73 <div class="row">
74 <div class="col-12">
75 <div class="col-12">
76 <h3>Data Log Produksi Real-time</h3>
77 <table class="table table-striped table-hover">
78 <thead class="table-dark">
79 <tr>
80 <th>Waktu</th>
81 <th>Mesin</th>
82 <th>Produk</th>
83 <th>Jumlah</th>
84 <th>Status</th>
85 </tr>

```

```

86 </thead>
87 <tbody>
88 <tr>
89 <td>08:00</td>
90 <td>CNC-01</td>
91 <td>Part A-12</td>
92 <td>50</td>
93 <td><span class="badge bg-success">OK</span></td>
94 </tr>
95 <tr>
96 <td>08:15</td>
97 <td>CNC-02</td>
98 <td>Part B-05</td>
99 <td>45</td>
100 <td><span class="badge bg-danger">Error</span></td>
101 </tr>
102 </tbody>
103 </table>
104 </div>
105 </div>
106 </div>

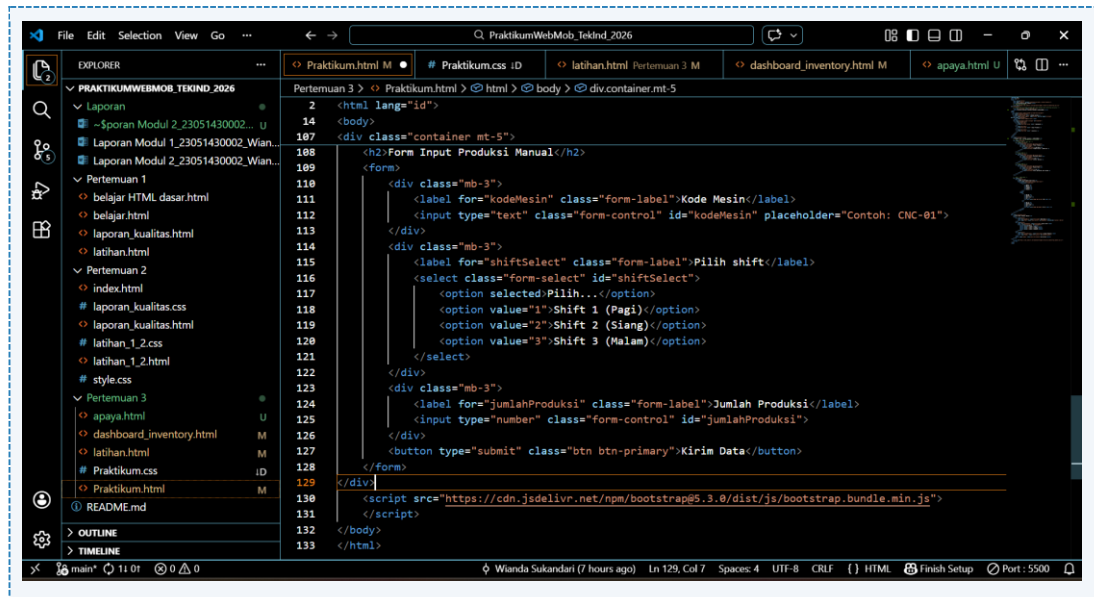
```

Gambar E.3 – Layout Grid Dashboard (Sistem Grid)

E.4 Langkah 4 – Styling Form dengan Bootstrap Classes

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode
Mesin</label>
      <input type="text" class="form-control"
id="kodeMesin" placeholder="Contoh: CNC-01">
    </div>
    <div class="mb-3">
      <label for="shiftSelect" class="form-label">Pilih
shift</label>
      <select class="form-select" id="shiftSelect">
        <option selected>Pilih...</option>
        <option value="1">Shift 1 (Pagi)</option>
        <option value="2">Shift 2 (Siang)</option>
        <option value="3">Shift 3 (Malam)</option>
      </select>
    </div>
    <div class="mb-3">
      <label for="jumlahProduksi" class="form-
label">Jumlah Produksi</label>
      <input type="number" class="form-control"
id="jumlahProduksi">
    </div>
    <button type="submit" class="btn btn-primary">Kirim
Data</button>
  </form>
</div>
```

Screenshot Hasil:



Gambar E.4 – Styling Form dengan Bootstrap Classes

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Grid Responsif

Pertanyaan / Instruksi Latihan:

Coba ubah ukuran layar browser Anda. Perhatikan bagaimana kartu KPI yang tadinya sejajar (3 kolom) akan menumpuk ke bawah (1 kolom) di layar HP. Ini adalah kekuatan Grid System ('col-md-4' artinya 4 kolom di layar medium ke atas, otomatis 12 kolom/1 baris di layar kecil).

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <!-- Bootstrap CSS CDN -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/boo
tstrap.min.css"
rel="stylesheet">
  <!-- Custom CSS kita tetap di bawah agar bisa menimpa style
bootstrap jika perlu -->
  <link rel="stylesheet" href="Praktikum.css">
```

```

        <title>Latihan praktikum 3_Wianda
Sukandari_23051430002</title>
</head>
<body>
    <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
        <div class="container-fluid">
            <a class="navbar-brand" href="">TekInd Systems</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse"
data-bs-target="#navbarNav">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav">
                    <li class="nav-item">
                        <a class="nav-link active" href="">Dashboard</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="">Input Produksi</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link" href="">Inventori</a>
                    </li>
                </ul>
            </div>
        </div>
    </nav>
    <div class="container mt-4">
        <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
        <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
        <div class="row mb-4">
            <!-- Kolom lebar 4 (dari 12) -->
            <div class="col-md-4">
                <div class="card text-white bg-primary mb-3">
                    <div class="card-header">Total Produksi</div>
                    <div class="card-body">
                        <h5 class="card-title">1,250 Unit</h5>
                        <p class="card-text">Target Harian: 1,200
Unit</p>
                    </div>
                </div>
            </div>
            <!-- Kolom lebar 4 -->
            <div class="col-md-4">
                <div class="card text-white bg-success mb-3">
                    <div class="card-header">Efisiensi Mesin</div>
                    <div class="card-body">
                        <h5 class="card-title">92%</h5>
                        <p class="card-text">Status: Optimal</p>
                    </div>
                </div>
            </div>
        </div>
    </div>

```

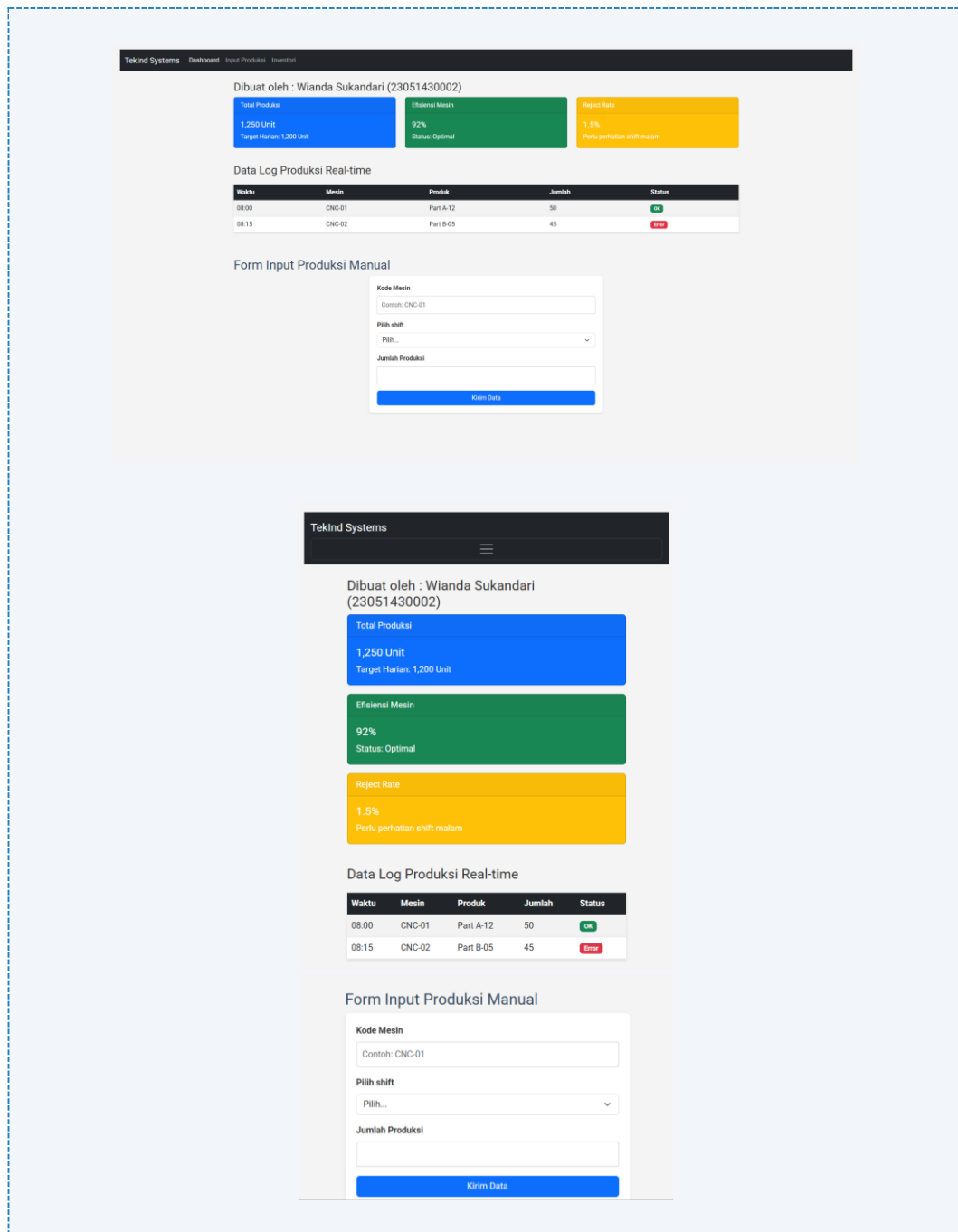
```

        </div>
    </div>
    <!-- Kolom lebar 4 -->
    <div class="col-md-4">
        <div class="card text-white bg-warning mb-3">
            <div class="card-header">Reject Rate</div>
            <div class="card-body">
                <h5 class="card-title">1.5%</h5>
                <p class="card-text">Perlu perhatian shift
malam</p>
            </div>
        </div>
    </div>
</div>

<!-- Row 2: Tabel Data -->
<div class="row">
    <div class="col-12">
        <h3>Data Log Produksi Real-time</h3>
        <table class="table table-striped table-hover">
            <thead class="table-dark">
                <tr>
                    <th>Waktu</th>
                    <th>Mesin</th>
                    <th>Produk</th>
                    <th>Jumlah</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>08:00</td>
                    <td>CNC-01</td>
                    <td>Part A-12</td>
                    <td>50</td>
                    <td><span class="badge bg-
success">OK</span></td>
                </tr>
                <tr>
                    <td>08:15</td>
                    <td>CNC-02</td>
                    <td>Part B-05</td>
                    <td>45</td>
                    <td><span class="badge bg-
danger">Error</span></td>
                </tr>
            </tbody>
        </table>
    </div>
</div>

```

```
</div>
<div class="container mt-5">
  <h2>Form Input Produksi Manual</h2>
  <form>
    <div class="mb-3">
      <label for="kodeMesin" class="form-label">Kode
Mesin</label>
      <input type="text" class="form-control"
id="kodeMesin" placeholder="Contoh: CNC-01">
    </div>
    <div class="mb-3">
      <label for="shiftSelect" class="form-label">Pilih
shift</label>
      <select class="form-select" id="shiftSelect">
        <option selected>Pilih...</option>
        <option value="1">Shift 1 (Pagi)</option>
        <option value="2">Shift 2 (Siang)</option>
        <option value="3">Shift 3 (Malam)</option>
      </select>
    </div>
    <div class="mb-3">
      <label for="jumlahProduksi" class="form-
label">Jumlah Produksi</label>
      <input type="number" class="form-control"
id="jumlahProduksi">
    </div>
    <button type="submit" class="btn btn-primary">Kirim
Data</button>
  </form>
</div>
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boots
trap.bundle.min.js">
</script>
</body>
</html>
```



Gambar F.1 – Output Grid Responsif

F.2 Latihan 2 – Modifikasi Card

Pertanyaan / Instruksi Latihan:

Ubah kartu "Efisiensi Mesin" menjadi warna merah (`bg-danger`) jika angkanya diubah menjadi di bawah 80%.

Jawaban / Kode Solusi:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Sistem Monitoring Produksi</title>
  <!-- Bootstrap CSS CDN -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bo
otstrap.min.css"
rel="stylesheet">
  <!-- Custom CSS kita tetap di bawah agar bisa menimpa
style bootstrap jika perlu -->
  <link rel="stylesheet" href="Praktikum.css">
  <title>Latihan Praktikum 3_Wianda
Sukandari_23051430002</title>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
      <a class="navbar-brand" href="">TekInd Systems</a>
      <button class="navbar-toggler" type="button" data-bs-
toggle="collapse"
data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link active" href="">Dashboard</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Input Produksi</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="">Inventori</a>
          </li>
        </ul>
      </div>
    </div>
  </nav>
  <div class="container mt-4">
    <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
    <!-- Row 1: KPI Cards (Menggunakan Grid System) -->
    <div class="row mb-4">
      <!-- Kolom lebar 4 (dari 12) -->
      <div class="col-md-4">
        <div class="card text-white bg-primary mb-3">
          <div class="card-header">Total Produksi</div>
```

```

        <div class="card-body">
            <h5 class="card-title">1,250 Unit</h5>
            <p class="card-text">Target Harian: 1,200
Unit</p>
        </div>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
    <!-- Mengubah dari bg-success menjadi bg-
danger untuk status efisiensi mesin dibawah 80% -->
    <div class="card text-white bg-danger mb-3">
        <div class="card-header">Efisiensi
Mesin</div>
        <div class="card-body">
            <h5 class="card-title">70%</h5>
            <p class="card-text">Status: Tidak
optimal</p>
        </div>
    </div>
</div>
<!-- Kolom lebar 4 -->
<div class="col-md-4">
    <div class="card text-white bg-warning mb-3">
        <div class="card-header">Reject Rate</div>
        <div class="card-body">
            <h5 class="card-title">1.5%</h5>
            <p class="card-text">Perlu perhatian shift
malam</p>
        </div>
    </div>
</div>
</div>
</div>

<!-- Row 2: Tabel Data -->
<div class="row">
    <div class="col-12">
        <h3>Data Log Produksi Real-time</h3>
        <table class="table table-striped table-hover">
            <thead class="table-dark">
                <tr>
                    <th>Waktu</th>
                    <th>Mesin</th>
                    <th>Produk</th>
                    <th>Jumlah</th>
                    <th>Status</th>
                </tr>
            </thead>
            <tbody>
                <tr>

```



```

        <td>08:00</td>
        <td>CNC-01</td>
        <td>Part A-12</td>
        <td>50</td>
        <td><span class="badge bg-
success">OK</span></td>
    </tr>
    <tr>
        <td>08:15</td>
        <td>CNC-02</td>
        <td>Part B-05</td>
        <td>45</td>
        <td><span class="badge bg-
danger">Error</span></td>
    </tr>
</tbody>
</table>
</div>
</div>
<div class="container mt-5">
    <h2>Form Input Produksi Manual</h2>
    <form>
        <div class="mb-3">
            <label for="kodeMesin" class="form-label">Kode
Mesin</label>
            <input type="text" class="form-control"
id="kodeMesin" placeholder="Contoh: CNC-01">
        </div>
        <div class="mb-3">
            <label for="shiftSelect" class="form-label">Pilih
shift</label>
            <select class="form-select" id="shiftSelect">
                <option selected>Pilih...</option>
                <option value="1">Shift 1 (Pagi)</option>
                <option value="2">Shift 2 (Siang)</option>
                <option value="3">Shift 3 (Malam)</option>
            </select>
        </div>
        <div class="mb-3">
            <label for="jumlahProduksi" class="form-
label">Jumlah Produksi</label>
            <input type="number" class="form-control"
id="jumlahProduksi">
        </div>
        <button type="submit" class="btn btn-primary">Kirim
Data</button>
    </form>
</div>

```

```

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/boot
strap.bundle.min.js">
</script>
</body>
</html>

```

The dashboard, titled 'Tekind Systems', includes a navigation bar with 'Dashboard', 'Input Produksi', and 'Inventori'. It displays production data for user 'Wianda Sukandari (23051430002)'. Three summary cards are shown: 'Total Produksi' (1,250 Unit, Target: 1,200 Unit), 'Efisiensi Mesin' (70%, Status: Tidak optimal), and 'Reject Rate' (1.5%, Note: Perlu perhatian shift malam). Below these is a 'Data Log Produksi Real-time' table with columns for Waktu, Mesin, Produk, Jumlah, and Status. The table contains two entries: one for CNC-01 producing Part A-12 (50 units, OK status) and another for CNC-02 producing Part B-05 (45 units, Error status). At the bottom is a 'Form Input Produksi Manual' with fields for 'Kode Mesin' (example: CNC-01), 'Pilih shift' (dropdown), 'Jumlah Produksi', and a 'Kirim Data' button.

Gambar F.2 – Modifikasi Card

F.3 Tugas Proyek Mini – Dashboard Layout

Deskripsi proyek mini:

Buatlah struktur dashboard baru bernama `dashboard\_inventori.html`.

- Gunakan Bootstrap Grid.
- Bagian atas: Judul "Gudang Bahan Baku".
- Bagian kiri (4 kolom): Form Stok Masuk (Input nama barang, qty, supplier).
- Bagian kanan (8 kolom): Tabel Stok Saat Ini (Gunakan tabel Bootstrap).

Penjelasan pendekatan/solusi yang digunakan:

Pendekatan yang saya pakai di proyek mini ini adalah memanfaatkan Bootstrap, terutama bagian grid system, biar layout dashboard bisa langsung rapi tanpa harus styling panjang lebar. Halaman dibagi pakai container, row, dan col supaya susunannya jelas. Bagian atas dipakai buat judul, lalu bagian utama dibagi dua: kiri buat form input stok (4 kolom) dan kanan buat tabel data stok (8 kolom). Ukuran kolomnya sengaja pakai col-md-4 dan col-md-8 supaya di layar besar tampil sejajar, tapi kalau di HP otomatis turun ke bawah.

Biar tampilannya enak dilihat, form dan tabel dibungkus pakai card Bootstrap, jadi kelihatan seperti panel dashboard beneran. Tabelnya juga pakai class bawaan Bootstrap supaya langsung ada garis, hover effect, dan rata tengah.

tanpa harus nambah CSS sendiri. Di bagian status stok aku pakai badge warna biar orang bisa langsung ngerti kondisi stok cuma dari warna, nggak perlu baca detail angka.

Kode Utama:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Gudang Bahan Baku</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <title>Latihan praktikum 3_Wianda Sukandari_23051430002</title>
</head>
<body class="bg-light">
  <h3>Dibuat oleh : Wianda Sukandari (23051430002)</h3>
<div class="container py-4">
  <div class="row mb-4">
    <div class="col">
      <h2 class="text-center fw-bold">Gudang Bahan Baku</h2>
    </div>
  </div>

  <!-- Layout Utama -->
  <div class="row g-4">
    <div class="col-md-4">
      <div class="card shadow-sm">
        <div class="card-header bg-dark text-white fw-semibold">
          Form Stok Masuk
        </div>
        <div class="card-body">
          <div class="mb-3">
            <label class="form-label">Nama Barang</label>
            <input type="text" class="form-control">
          </div>
          <div class="mb-3">
            <label class="form-label">Jumlah (Qty)</label>
            <input type="number" class="form-control">
          </div>
          <div class="mb-3">
            <label class="form-label">Supplier</label>

```

```

        <input type="text" class="form-control">
    </div>
    <button class="btn btn-success w-100">
        Simpan Data
    </button>
</div>
</div>
</div>
<div class="col-md-8">
    <div class="card shadow-sm">
        <div class="card-header bg-dark text-white fw-semibold">
            Tabel Stok Saat Ini
        </div>
        <div class="card-body">
            <div class="table-responsive">
                <table class="table table-bordered table-hover align-
middle">

                    <thead class="table-secondary text-center">
                        <tr>
                            <th>No</th>
                            <th>Nama Produk</th>
                            <th>Supplier</th>
                            <th>Stock Awal</th>
                            <th>Stock Masuk</th>
                            <th>Stock Keluar</th>
                            <th>Sisa</th>
                            <th>Status</th>
                        </tr>
                    </thead>
                    <tbody class="text-center">
                        <tr>
                            <td>1</td>
                            <td>Besi Hollow</td>
                            <td>PT Baja Jaya</td>
                            <td>100</td>
                            <td>50</td>
                            <td>30</td>
                            <td>120</td>
                            <td><span class="badge bg-
success">Aman</span></td>
                        </tr>
                        <tr>
                            <td>2</td>
                            <td>Semen</td>
                            <td>PT Semen Makmur</td>
                            <td>200</td>
                            <td>40</td>
                            <td>180</td>
                            <td>60</td>

```


Dibuat oleh : Wianda Sukandari (23051430002)

Gudang Bahan Baku

Form Stok Masuk

Nama Barang

Jumlah (Qty)

Supplier

Tabel Stok Saat Ini

No	Nama Produk	Supplier	Stock Awal	Stock Masuk	Stock Keluar	Sisa	Status
1	Besi Hollow	PT Baja Jaya	100	50	30	120	Amat
2	Semen	PT Semen Makmur	200	40	180	60	Masih
3	Cat Tembok	PT Warna Abadi	75	25	60	40	Masih
4	Paku	CV Logam Sentosa	500	100	450	150	Amat
5	Triplek	PT Kayu Lestari	60	20	55	25	Hampir Habis

Dibuat oleh : Wianda Sukandari (23051430002)

Gudang Bahan Baku

Form Stok Masuk

Nama Barang

Jumlah (Qty)

Supplier

Tabel Stok Saat Ini

No	Nama Produk	Supplier	Stock Awal	Stock Masuk	Stock Keluar	Sisa	Sta
1	Besi Hollow	PT Baja Jaya	100	50	30	120	Am
2	Semen	PT Semen Makmur	200	40	180	60	Mes
3	Cat Tembok	PT Warna Abadi	75	25	60	40	Mes
4	Paku	CV Logam Sentosa	500	100	450	150	Am
5	Triplek	PT Kayu Lestari	60	20	55	25	Hampir

Gambar F.3 – Output Dashboard Layout

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang saya buat sudah sesuai dengan konsep yang dipelajari di modul, terutama tentang penggunaan framework Bootstrap dan sistem layout grid. Secara teori, grid system digunakan untuk mengatur pembagian layar menjadi baris dan kolom agar tampilan rapi dan responsif. Di hasil kode yang saya buat, bagian dashboard sudah terbagi jelas: judul di atas, lalu bagian utama terbagi dua kolom menggunakan col-md-4 dan col-md-8. Saat ukuran layar diperkecil, layout otomatis menyesuaikan jadi satu kolom, yang berarti fitur responsif Bootstrap berjalan sesuai teori.

Selain itu, penggunaan komponen Bootstrap seperti card, table, badge, dan button juga sesuai dengan konsep framework CSS, yaitu memanfaatkan class siap pakai supaya tidak perlu membuat desain dari nol. Hasil tampilannya terlihat konsisten, rapi, dan mudah dibaca, yang menunjukkan bahwa prinsip reusable component dalam framework sudah diterapkan dengan benar. Jadi bisa disimpulkan bahwa implementasi kode sudah merepresentasikan teori layout modern (grid + komponen UI) dengan baik.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Layout tidak berubah saat layar diperkecil	Mengecek kembali penggunaan class grid dan mastiin pakai col-md-*, bukan col-* aja
Tampilan tabel kurang rapi	Menambahkan class Bootstrap seperti table-bordered, table-hover, dan align-middle

G.3 Kaitan dengan Penerapan di Industri

Materi ini sebenarnya sangat relevan dengan dunia Teknik Industri, terutama di bidang sistem informasi produksi, monitoring, dan dashboard operasional. Engineer atau analis sering butuh dashboard untuk memantau data seperti jumlah produksi, efisiensi mesin, tingkat reject, atau stok gudang secara real-time. Layout dashboard seperti yang dibuat di praktikum ini bisa jadi dasar tampilan sistem monitoring tersebut.

Contoh penerapan nyata misalnya di pabrik manufaktur: supervisor produksi butuh tampilan layar yang menunjukkan status mesin, output per shift, dan stok material. Dengan skill membuat layout dashboard responsif ini, data bisa

ditampilkan secara jelas di monitor ruang kontrol, tablet operator, bahkan HP manajer. Jadi walaupun praktikum ini kelihatannya cuma bikin tampilan web sederhana, sebenarnya konsepnya sama seperti yang dipakai di sistem industri modern, cuma skalanya lebih besar dan datanya real-time.

H. KESIMPULAN

1	Saya berhasil bikin dashboard inventori sesuai instruksi dan strukturnya jelas (judul di atas, form kiri 4 kolom, tabel kanan 8 kolom)
2	Grid Bootstrap bener-bener ngebantu banget karena cukup pakai container, row, dan col-md-* layout langsung rapi tanpa ribet ngatur CSS manual.
3	Komponen siap pakai kayak card, table, badge, dan button bikin tampilan dashboard langsung kelihatan profesional walaupun kodenya simpel.
4	Fitur responsifnya jalan sesuai teori (waktu layar dikecilin, layout otomatis turun jadi satu kolom dan tetap enak dilihat)
5	Dari praktikum ini saya jadi lebih ngerti gimana konsep layout web dipakai buat bikin tampilan dashboard data yang mirip sama sistem monitoring di industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>HTML basics</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics
[3]	Rianto, I. (2025). Pemrograman web. <i>Penerbit Tahta Media</i> .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

Modul 4

Javascript Dasar (Variabel, Tipe Data, Operator, Control Flow)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web Dan Mobile Manual Praktikum (Pertemuan 4)
Topik Utama	Javascript Dasar (Variabel, Tipe Data, Operator, Control Flow)
Sub-Topik	Logika Pemrograman, Variabel, Tipe Data, Operator Aritmatika/Logika, dan Percabangan.
Durasi Praktikum	340 Menit
Tanggal Praktikum	07 Maret 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami peran JavaScript sebagai "otak" dari halaman web.
2	Mampu mendeklarasikan variabel dan memahami aturan penamaannya.
3	Menguasai operator aritmatika untuk perhitungan sederhana.
4	Mampu menggunakan Control Flow (`if/else`, `switch`) untuk pengambilan keputusan otomatis.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	JavaScript	ES6 atau lebih baru	Logika program dan interaksi halaman
5	HTML	Versi 5	Struktur halaman web

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini dijelaskan dasar-dasar JavaScript yang dipakai untuk memproses data di dalam website. Kalau HTML dan CSS lebih fokus ke tampilan, JavaScript dipakai untuk membuat halaman web bisa melakukan perhitungan atau menjalankan logika tertentu. Misalnya menghitung total produksi, mengecek kondisi mesin, atau menampilkan pesan sesuai kondisi tertentu.

Salah satu konsep utama yang dipelajari adalah variabel, yaitu tempat untuk menyimpan data. Di JavaScript biasanya menggunakan `let` untuk data yang bisa berubah dan `const` untuk data yang nilainya tetap. Selain itu ada juga tipe data seperti number untuk angka, string untuk teks, dan boolean untuk nilai benar atau salah.

Selain variabel, ada juga operator dan percabangan seperti `if` dan `else`. Operator dipakai untuk melakukan perhitungan atau membandingkan nilai, sedangkan percabangan digunakan untuk membuat keputusan dalam

D.2 Keterkaitan dengan Teknik Industri

Materi dasar JavaScript sebenarnya cukup relevan dengan bidang Teknik Industri karena banyak proses di industri yang membutuhkan pengolahan data dan pengambilan keputusan secara otomatis. Dengan konsep seperti variabel, perhitungan, dan logika kondisi, sebuah sistem dapat dibuat untuk membantu memproses data operasional yang biasanya dilakukan secara manual. Hal ini bisa membuat pekerjaan menjadi lebih cepat dan mengurangi kemungkinan kesalahan.

Contohnya dalam sistem monitoring produksi. Misalnya operator memasukkan jumlah produk yang dihasilkan dalam satu shift melalui sebuah aplikasi web. Data tersebut kemudian diproses menggunakan logika pemrograman untuk menghitung total produksi harian, mengecek apakah target tercapai atau belum, serta memberikan peringatan jika produksi masih di bawah target. Dengan cara ini, informasi bisa langsung diketahui tanpa harus menghitung secara manual.

Contoh lain adalah pada sistem pencatatan jam kerja karyawan. Data jam masuk dan jam pulang bisa dimasukkan ke sistem, lalu program menghitung apakah karyawan tersebut bekerja normal atau lembur. Hasil perhitungan tersebut kemudian bisa digunakan untuk menentukan tambahan upah atau laporan kehadiran. Dari contoh ini terlihat bahwa konsep dasar pemrograman

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Hello World & Variabel di Console

Pada langkah ini saya mencoba menulis kode JavaScript sederhana untuk mengenal penggunaan variabel, tipe data, dan cara menampilkan output di console. Saya membuat beberapa variabel seperti nama mesin dan target

produksi harian, lalu menampilkannya menggunakan `console.log()`. Setelah itu saya juga mencoba operasi aritmatika dengan menjumlahkan produksi pagi dan siang untuk mendapatkan total produksi sementara, kemudian

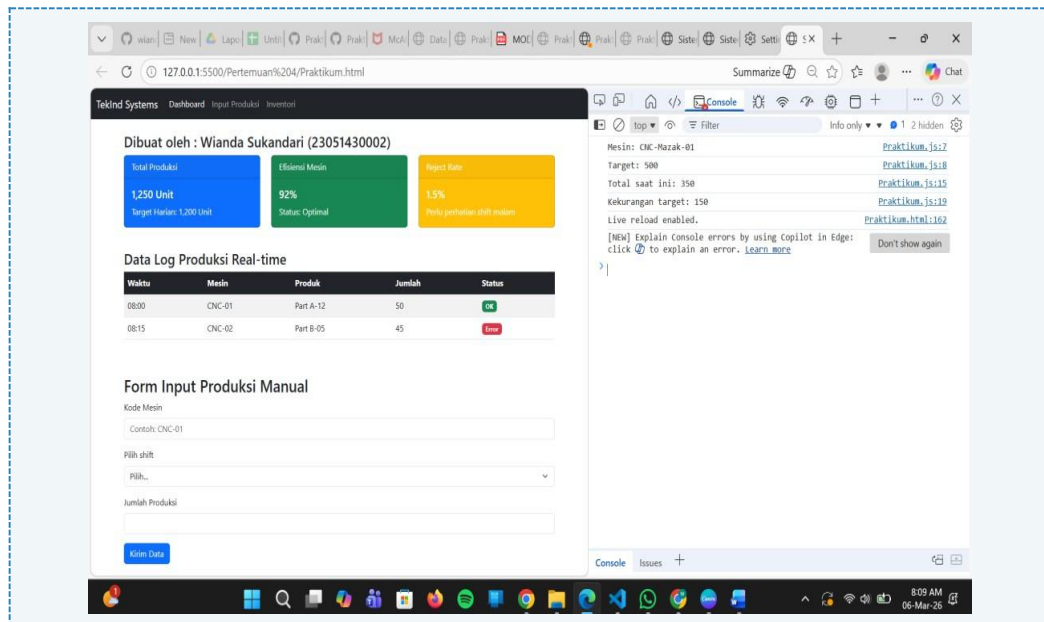
```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
// Komentar single line

// 1. Variabel & Tipe Data
let namaMesin = "CNC-Mazak-01"; // String let
targetHarian = 500; // Number
// Menampilkan ke console browser (Tekan F12 -> Console) console.log("Mesin: " + namaMesin);
console.log("Target: " + targetHarian);

// 2. Operator Aritmatika
let produksiPag
i = 200;
let produksiSia
ng = 150;
let totalProduksi = produksiPagi +
```

menghitung selisihnya dengan target harian dan menampilkan hasilnya di console browser.

Screenshot Hasil:



Gambar E.1 – Output Langkah 1

E.2 Langkah 2 – Control Flow (Pengambilan Keputusan)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
// Simulasi data pembacaan sensor (jam  
operasional) let jamOperasional = 1250;  
// dalam jam  
let batasMaksimal = 1200; // batas sebelum maintenance wajib
```

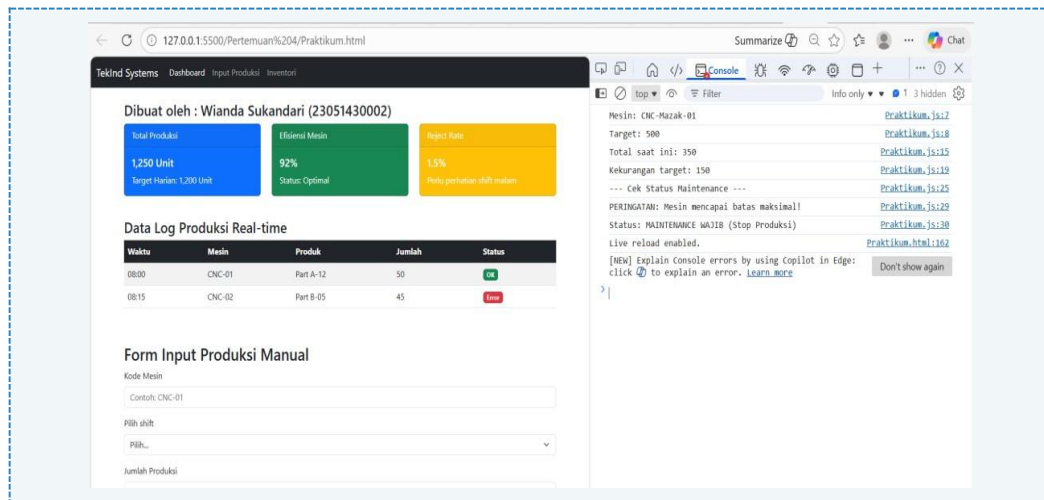
```

console.log("--- Cek Status Maintenance ---");

// Logika If/Else
if (jamOperasional >= batasMaksimal) {
  console.log("PERINGATAN: Mesin mencapai batas maksimal!"); console.log("Status: MAINTENANCE WAJIB (Stop Produksi)");
} else if (jamOperasional > 1000) {
  console.log("Status: SIAP HATI-HATI (Segera jadwalkan maintenance)");
} else {
  console.log("Status: BERJALAN NORMAL.");
}

```

Screenshot Hasil:



Gambar E.2 – Output Langkah 2

E.3 Langkah 3 – Studi Kasus Industri (Perhitungan OEE Sederhana)

```

/* ===== KODE / OUTPUT LANGKAH 3
===== */

// Data Input
let jamKerjaPlanned = 8; // Jam
let jamKerjaAktual = 6.5; // Jam (Ada 1.5 jam breakdown)

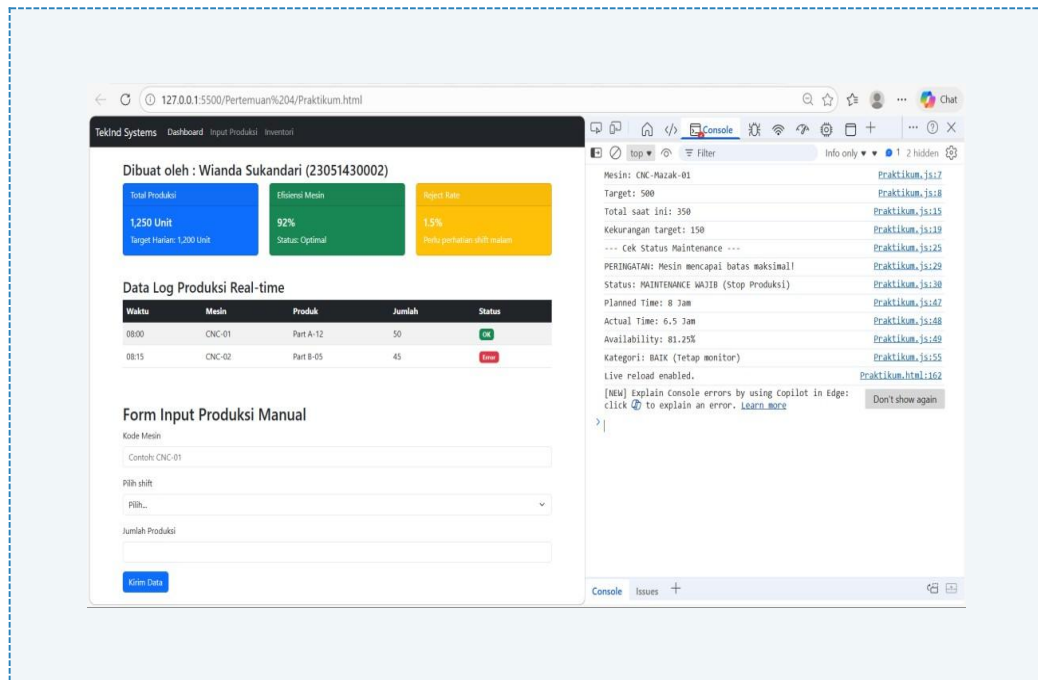
// Perhitungan
let availability = (jamKerjaAktual / jamKerjaPlanned) * 100;

// Pembulatan 2 angka di belakang koma
availability.toFixed(2);

console.log("Planned Time: " + jamKerjaPlanned + " Jam"); console.log("Actual Time: " + jamKerjaAktual + " Jam");
console.log("Availability: " + availability + "%");

// Logika Penilaian Kualitas
Availability if (availability >= 90) {
  console.log("Kategori: WORLD CLASS");
}

```


Screenshot Hasil:

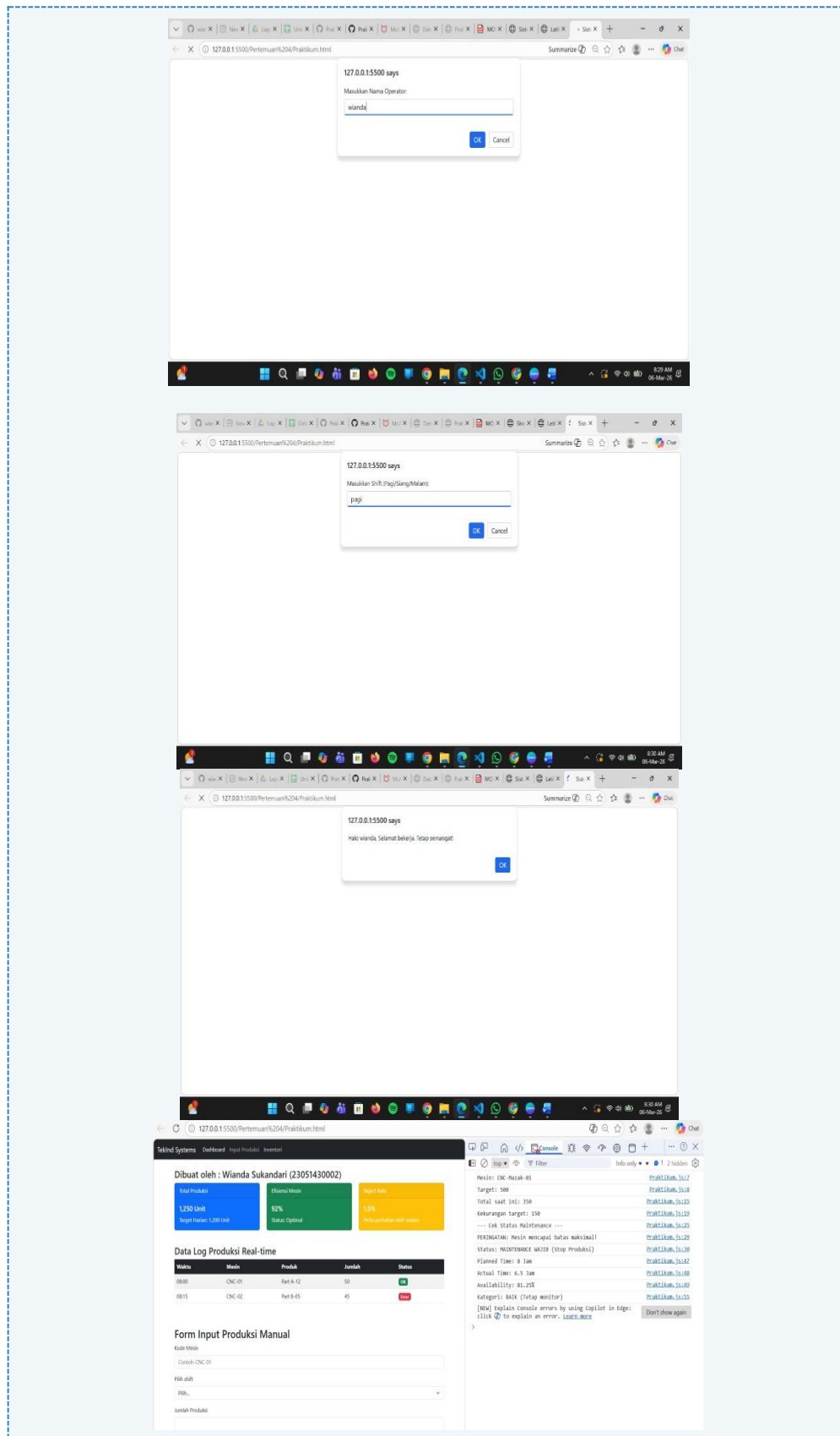
Gambar E.3 – Output Langkah 3

E.4 Langkah 4 – Interaksi DOM Sederhana (Alert & Prompt)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");

if (shiftKerja === "Malam") {
  alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar Rp 20.000.");
} else {
  alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
```

Screenshot Hasil:



Gambar E.4 – Output Akhir

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Matematika

Pertanyaan / Instruksi Latihan:

Buat variabel `gajiPokok` dan `jamLembur`. Hitung `totalGaji` dengan asumsi upah lembur per jam adalah 1.5x dari gaji pokok dibagi 173 (jam kerja bulanan). Tampilkan hasilnya di console.

Jawaban / Kode Solusi:

```
// Latihan 1 - Perhitungan
Gaji let gajiPokok = 5000000
let jamLembur = 10

// rumus lembur
let upahLemburPerJam = (gajiPokok / 173) * 1.5

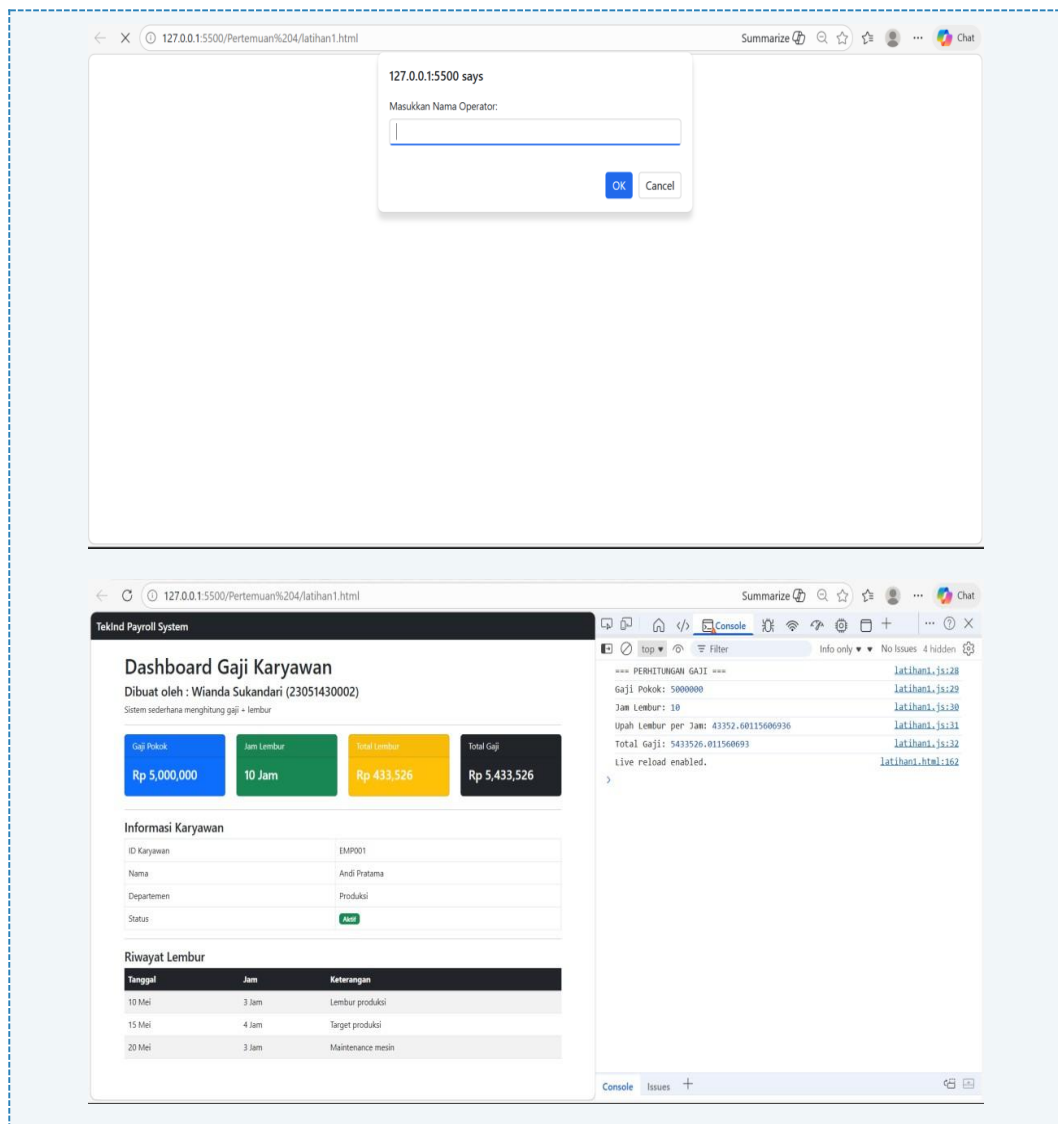
// total lembur
let totalLembur = jamLembur * upahLemburPerJam

// total gaji
let totalGaji = gajiPokok + totalLembur

// menampilkan ke dashboard html
document.getElementById("gajiPokok").innerText = "Rp " + gajiPokok.toLocaleString()
document.getElementById("jamLembur").innerText = jamLembur + " Jam"
document.getElementById("totalLembur").innerText = "Rp " +
Math.round(totalLembur).toLocaleString() document.getElementById("totalGaji").innerText =
"Rp " + Math.round(totalGaji).toLocaleString()
// tampilkan juga di console (sesuai soal) console.log("===
PERHITUNGAN GAJI ===")
console.log("Gaji Pokok:", gajiPokok)
console.log("Jam Lembur:", jamLembur) console.log("Upah
Lembur per Jam:", upahLemburPerJam) console.log("Total
Gaji:", totalGaji)

let namaOperator = prompt("Masukkan Nama Operator:");
let shiftKerja = prompt("Masukkan Shift (Pagi/Siang/Malam):");

if (shiftKerja === "Malam") {
    alert("Halo " + namaOperator + ", Shift malam memiliki tambahan uang makan sebesar
Rp 20.000.");
} else {
    alert("Halo " + namaOperator + ", Selamat bekerja. Tetap semangat!");
}
```



Gambar F.1 – Hasil Dashboard Gaji Karyawan

F.2 Latihan 2 – Logika Switch

Pertanyaan / Instruksi Latihan:

Gunakan `switch` untuk mengkonversi kode shift angka (1, 2, 3) menjadi nama shift (Pagi, Siang, Malam). Jika input selain 1-3, tampilkan "Shift Tidak Valid".

Jawaban / Kode Solusi:

```

// Latihan 1 - Perhitungan Gaji

let gajiPokok = 5000000
let jamLembur = 10

// rumus lembur
let upahLemburPerJam = (gajiPokok / 173) * 1.5

// total lembur
let totalLembur = jamLembur * upahLemburPerJam
  
```

```
// total gaji
let totalGaji = gajiPokok + totalLembur

// menampilkan ke dashboard html
document.getElementById("gajiPokok").innerText = "Rp " + gajiPokok.toLocaleString()
document.getElementById("jamLembur").innerText = jamLembur + " Jam"
document.getElementById("totalLembur").innerText = "Rp " +
Math.round(totalLembur).toLocaleString()
document.getElementById("totalGaji").innerText = "Rp " +
Math.round(totalGaji).toLocaleString()

// tampilkan juga di console (sesuai soal)

console.log("=== PERHITUNGAN GAJI ===")
console.log("Gaji Pokok:", gajiPokok)
console.log("Jam Lembur:", jamLembur)
console.log("Upah Lembur per Jam:", upahLemburPerJam)
console.log("Total Gaji:", totalGaji)

let namaOperator = prompt("Masukkan Nama Operator:")
let kodeShift = prompt("Masukkan Kode Shift (1=Pagi, 2=Siang, 3=Malam):")

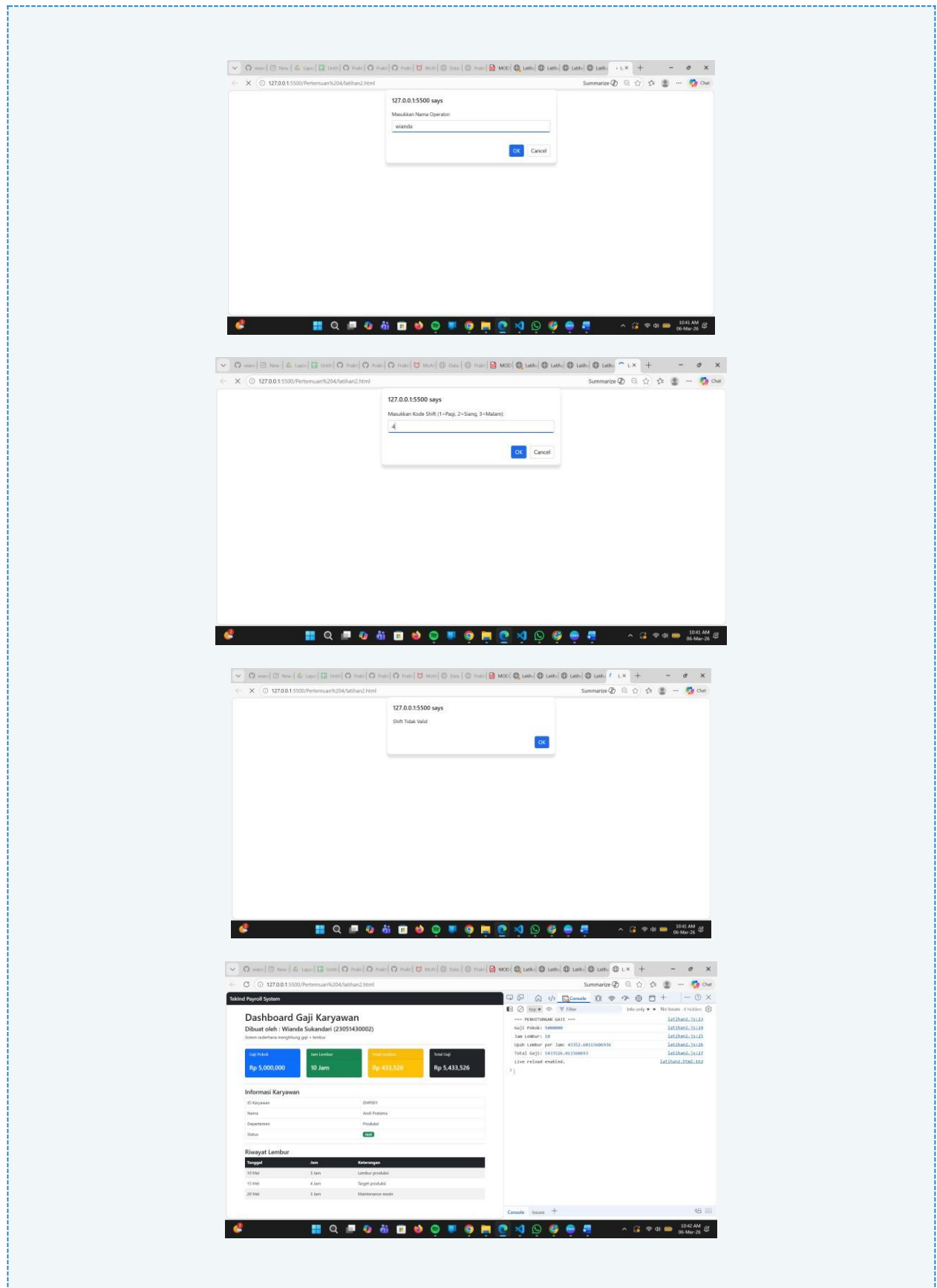
switch (kodeShift) {

  case "1":
    alert("Halo " + namaOperator + ", Anda bekerja di Shift Pagi")
    break

  case "2":
    alert("Halo " + namaOperator + ", Anda bekerja di Shift Siang")
    break

  case "3":
    alert("Halo " + namaOperator + ", Anda bekerja di Shift Malam")
    break

  default:
    alert("Shift Tidak Valid")
}
```



Gambar F.2 – Output Logika Switch

F.3 Tugas Proyek Mini – Kalkulator Biaya Produksi

Deskripsi proyek mini:

Buat program di `script.js` yang menghitung biaya produksi total per unit.

- Variabel: `biayaBahanBaku`, `biayaTenagaKerja`, `biayaOverhead`, `jumlahProduksi`.
- Rumus: $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$.
- Gunakan `if/else` untuk mengecek: jika `jumlahProduksi` < 100, tampilkan "Biaya Tinggi (Ekonomi Skala Kecil)". Jika ≥ 100 , tampilkan "Biaya Efisien".

Penjelasan pendekatan/solusi yang digunakan:

Program ini dibuat untuk menghitung biaya produksi per unit menggunakan JavaScript. Pengguna memasukkan beberapa data seperti biaya bahan baku, biaya tenaga kerja, biaya overhead, dan jumlah produksi melalui form di halaman web. Setelah itu, program akan menjumlahkan semua biaya tersebut lalu membaginya dengan jumlah produksi untuk mendapatkan biaya per unit.

Setelah hasilnya dihitung, program juga mengecek jumlah produksi menggunakan kondisi if/else. Kalau jumlah produksi kurang dari 100, maka akan muncul keterangan Biaya Tinggi (Ekonomi Skala Kecil). Tapi kalau jumlah produksinya 100 atau lebih, akan muncul Biaya Efisien. Hasil perhitungan dan kategorinya langsung ditampilkan di halaman supaya mudah dilihat pengguna.

Kode Utama:

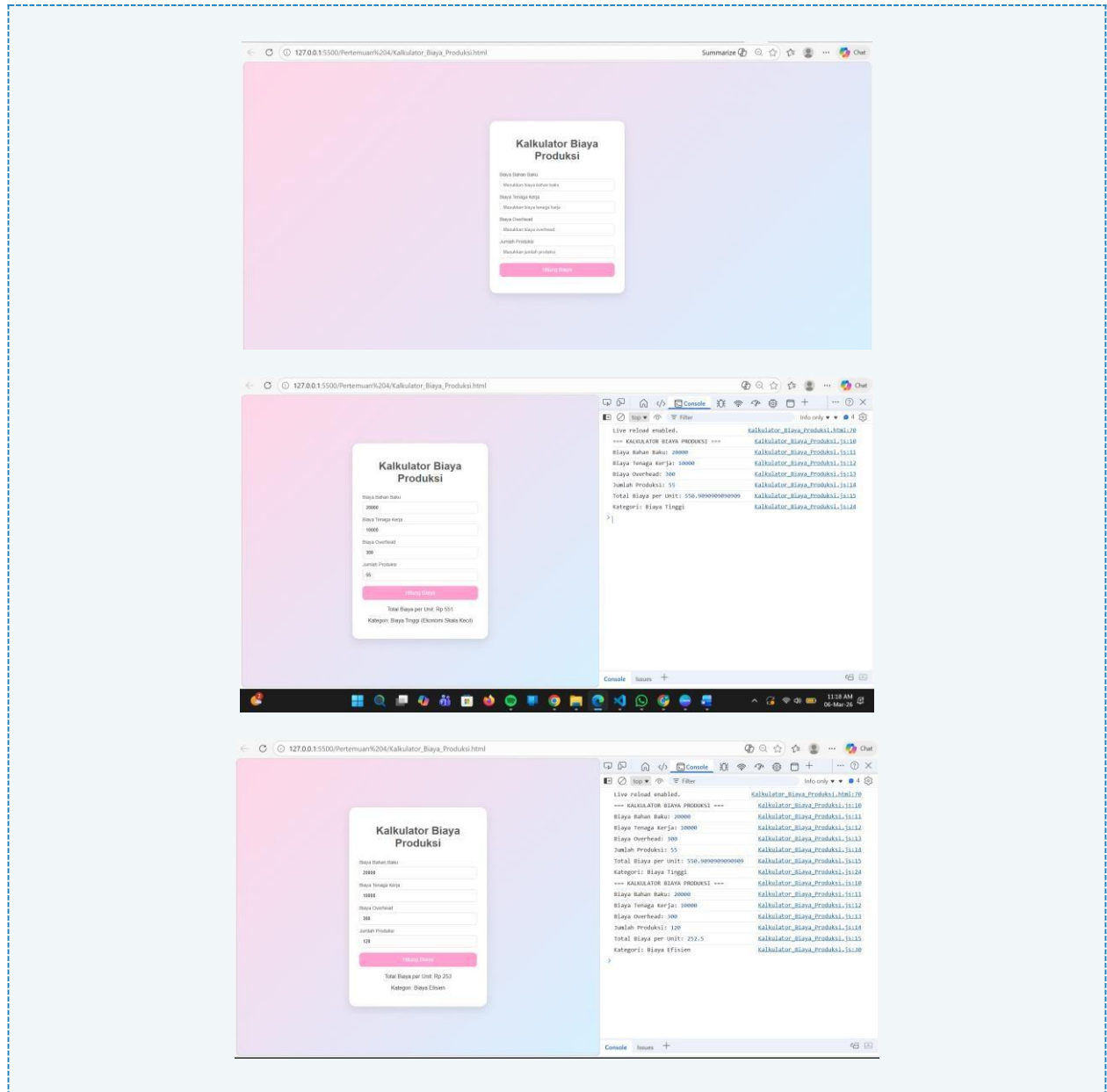
```
function hitungBiaya() {  
  
    let biayaBahanBaku = parseInt(document.getElementById("bahanBaku").value)  
    let biayaTenagaKerja = parseInt(document.getElementById("tenagaKerja").value)  
    let biayaOverhead = parseInt(document.getElementById("overhead").value)  
    let jumlahProduksi = parseInt(document.getElementById("jumlahProduksi").value)  
  
    let totalPerUnit = (biayaBahanBaku + biayaTenagaKerja + biayaOverhead) /  
    jumlahProduksi  
  
    console.log("=== KALKULATOR BIAYA PRODUKSI ===")  
    console.log("Biaya Bahan Baku:", biayaBahanBaku)  
    console.log("Biaya Tenaga Kerja:", biayaTenagaKerja)  
    console.log("Biaya Overhead:", biayaOverhead)  
    console.log("Jumlah Produksi:", jumlahProduksi)  
    console.log("Total Biaya per Unit:", totalPerUnit)  
  
    document.getElementById("hasilBiaya").innerText =  
        "Total Biaya per Unit: Rp " + Math.round(totalPerUnit).toLocaleString()  
  
    if (jumlahProduksi < 100) {  
        document.getElementById("kategoriBiaya").innerText =  
            "Kategori: Biaya Tinggi (Ekonomi Skala Kecil)"  
  
        console.log("Kategori: Biaya Tinggi")  
    }  
    else {  
        document.getElementById("kategoriBiaya").innerText =
```

```

    "Kategori: Biaya Efisien"

    console.log("Kategori: Biaya Efisien")
  }
}

```



Gambar F.3 – Output Kalkulator Biaya Produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil yang diperoleh dari praktikum ini sudah sesuai dengan konsep yang dipelajari pada materi JavaScript dasar, terutama penggunaan variabel, operasi aritmatika, dan percabangan if/else. Program yang dibuat berhasil menghitung biaya produksi per unit dengan rumus:

$$\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}.$$

Dari output yang ditampilkan di halaman web, nilai total biaya per unit dapat muncul dengan benar setelah pengguna memasukkan data biaya dan jumlah produksi. Selain itu, program juga berhasil menerapkan logika if/else untuk mengecek jumlah produksi. Jika jumlah produksi kurang dari 100, program menampilkan kategori Biaya Tinggi (Ekonomi Skala Kecil), sedangkan jika jumlah produksi 100 atau lebih akan muncul Biaya Efisien. Hal ini menunjukkan bahwa logika percabangan dalam program berjalan sesuai teori yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tampilan halaman awal terlihat terlalu sederhana dan kurang menarik	Menambahkan CSS dengan warna pastel dan layout card agar tampilan lebih rapi dan estetik
Console di browser terlihat kosong sehingga sulit memastikan apakah script berjalan	Menambahkan <code>console.log()</code> pada kode untuk mengecek proses perhitungan di console

G.3 Kaitan dengan Penerapan di Industri

Konsep yang dipelajari pada praktikum ini sebenarnya cukup relevan dengan bidang Teknik Industri, terutama dalam perhitungan biaya produksi. Dalam industri manufaktur, perusahaan perlu menghitung biaya produksi per unit untuk menentukan harga jual produk dan mengevaluasi efisiensi produksi.

Contohnya pada sebuah pabrik peralatan rumah tangga seperti produksi wajan aluminium. Perusahaan harus menghitung biaya bahan baku aluminium, biaya tenaga kerja, serta biaya overhead seperti listrik dan mesin. Dari data tersebut, sistem dapat menghitung biaya produksi per unit secara otomatis. Jika jumlah produksi masih kecil, biaya per unit biasanya lebih tinggi. Namun ketika produksi meningkat, biaya per unit bisa menjadi lebih efisien. Sistem sederhana seperti yang dibuat pada praktikum ini bisa menjadi dasar dari sistem perhitungan biaya produksi yang lebih kompleks di industri.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Program kalkulator biaya produksi berhasil dibuat menggunakan JavaScript untuk menghitung biaya produksi per unit berdasarkan biaya bahan baku, tenaga kerja, overhead, dan jumlah produksi.
2	Perhitungan biaya produksi dilakukan dengan rumus $\text{totalPerUnit} = (\text{biayaBahanBaku} + \text{biayaTenagaKerja} + \text{biayaOverhead}) / \text{jumlahProduksi}$, dan hasilnya dapat langsung ditampilkan di halaman web.
3	Penggunaan percabangan if/else dalam program dapat digunakan untuk menentukan kategori biaya produksi, yaitu Biaya Tinggi (Ekonomi Skala Kecil) jika jumlah produksi kurang dari 100, dan Biaya Efisien jika jumlah produksi lebih dari atau sama dengan 100.
4	Tampilan antarmuka program dapat dibuat lebih menarik dengan menambahkan CSS sehingga aplikasi terlihat lebih rapi dan nyaman digunakan.
5	Praktikum ini membantu memahami dasar penggunaan JavaScript untuk perhitungan data dan pengambilan keputusan sederhana, yang nantinya bisa dikembangkan menjadi sistem perhitungan biaya produksi yang lebih kompleks di industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	W3Schools. (2024). JavaScript tutorial. https://www.w3schools.com/js/
[3]	Mozilla Developer Network. (2024). <i>JavaScript guide</i> . https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 5**Function, Array, Object di JavaScript****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Function, Array, Object di JavaScript (Pertemuan_5)
Topik Utama	Function, Array, Object di JavaScript
Sub-Topik	Modularitas kode (Function), pengelolaan data list (Array), dan struktur data kompleks (Object).
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 Mar. 26
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Function untuk menghindari pengulangan kode (DRY - Don't Repeat Yourself).
2	Menguasai penggunaan Array untuk menyimpan sekumpulan data (misal: daftar cacat produksi).
3	Memahami Object untuk merepresentasikan entitas nyata (misal: Spesifikasi Mesin) dalam bentuk data.
4	Mampu melakukan iterasi (perulangan) terhadap Array dan Object.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Live Server Extension (VS Code)	Versi terbaru	Menjalankan halaman web secara lokal

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini dipelajari beberapa konsep dasar dalam JavaScript seperti function, array, dan object. Function bisa dianggap sebagai kumpulan perintah yang dijadikan satu agar bisa dipakai kembali saat dibutuhkan. Jadi kita tidak perlu menulis kode yang sama berulang-ulang. Cukup memanggil function tersebut ketika ingin menjalankan proses yang sama.

Selain itu ada juga array, yaitu cara untuk menyimpan banyak data dalam satu variabel. Misalnya kita punya beberapa data angka, nama barang, atau jumlah produksi. Daripada membuat banyak variabel terpisah, data tersebut bisa dimasukkan ke dalam array sehingga lebih mudah diatur dan diakses dalam program.

Konsep lainnya adalah object. Object biasanya dipakai untuk menyimpan data yang memiliki beberapa informasi sekaligus. Contohnya seperti data sebuah produk yang memiliki nama, harga, dan jumlah stok. Semua informasi itu bisa disimpan dalam satu object sehingga datanya lebih terstruktur dan mudah digunakan saat program dijalankan.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini sebenarnya cukup dekat dengan kebutuhan di bidang Teknik Industri, terutama dalam pengolahan data. Dalam dunia industri, banyak aktivitas yang melibatkan pencatatan dan pengolahan data, seperti data produksi, data kualitas produk, atau data persediaan bahan baku.

Sebagai contoh, data jumlah produksi setiap hari bisa disimpan dalam bentuk array sehingga mudah dilihat atau dihitung totalnya. Lalu function bisa digunakan untuk menghitung hal-hal tertentu seperti total produksi, rata-rata produksi, atau efisiensi kerja. Sedangkan object dapat dipakai untuk menyimpan informasi lengkap suatu produk, misalnya nama produk, biaya produksi, dan jumlah stok yang tersedia.

Dengan memahami konsep dasar ini, mahasiswa Teknik Industri bisa mulai membuat program sederhana untuk membantu mengolah data. Walaupun masih sederhana, konsep seperti ini sering menjadi dasar dari sistem yang lebih besar di industri, misalnya sistem monitoring produksi atau sistem pengelolaan data perusahaan.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

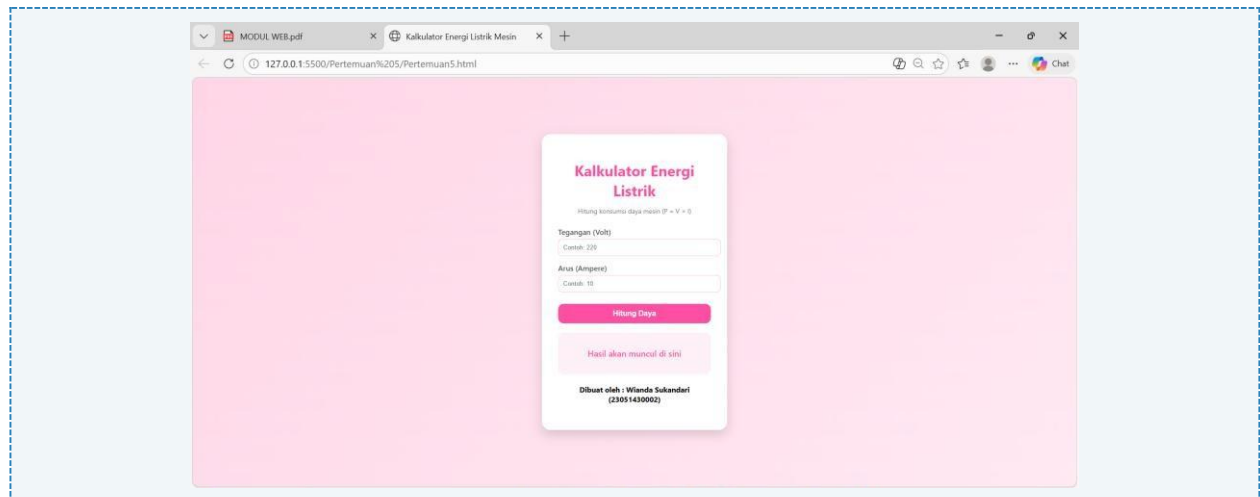
E.1 Langkah 1 – Membuat Function Perhitungan

Saya buat 3 file yaitu 'Pertemuan5.html', 'Pertemuan5.js', dan 'Pertemuan5.css'. Kemudian saya masukkan kode dibawah ini ke file 'Pertemuan5.js'. Tapi dari output itu hasilnya belum bisa muncul karena kode js masih belum lengkap.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
// 1. FUNCTION Declaration
// Fungsi untuk menghitung konsumsi daya (P = V x I)
function hitungDaya(tegangan, arus) {
    let daya = tegangan * arus;
    return daya; // Mengembalikan hasil
}

// Pemanggilan Function
let teganganMesin = 220; // Volt
let arusMesin = 10; // Ampere
let totalDaya = hitungDaya(teganganMesin, arusMesin);
console.log("Daya Mesin: " + totalDaya + " Watt");
```

Screenshot Hasil:

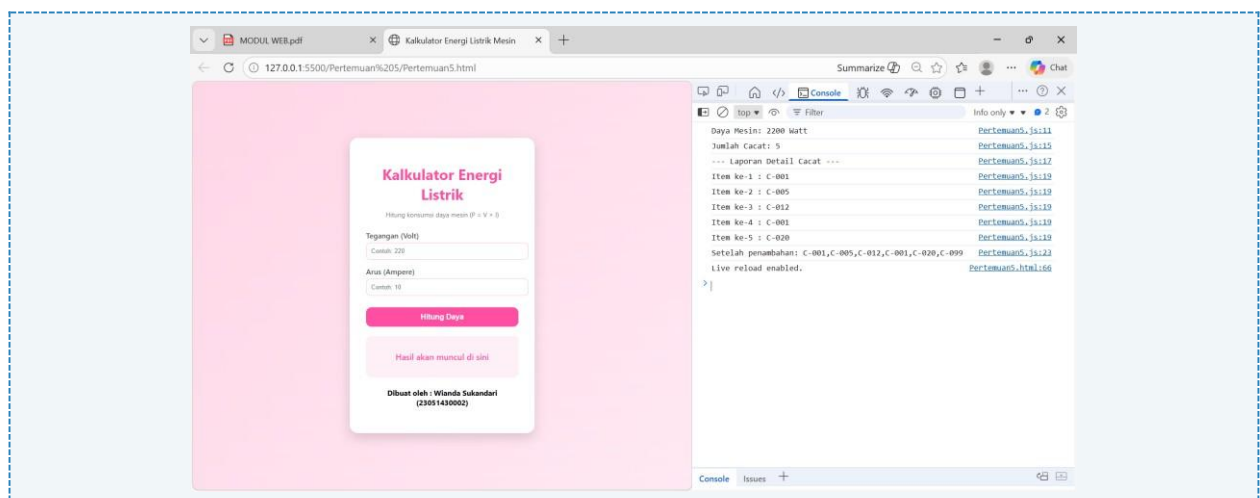


Gambar E.1 – Hasil Function Perhitungan

E.2 Langkah 2 – Array dan Looping (Daftar Produk Cacat)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
// 2. ARRAY  
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"]; // C-001 terjadi 2 kali  
console.log("Jumlah Cacat: " + daftarCacat.length);  
// Looping (Perulangan) untuk menampilkan setiap cacat  
console.log("--- Laporan Detail Cacat ---");  
for (let i = 0; i < daftarCacat.length; i++) {  
  console.log("Item ke-" + (i + 1) + " : " + daftarCacat[i]);  
}  
// Menambahkan data baru ke array  
daftarCacat.push("C-099");  
console.log("Setelah penambahan: " + daftarCacat);
```

Screenshot Hasil:



Gambar E.2 – Hasil Array dan Looping Console

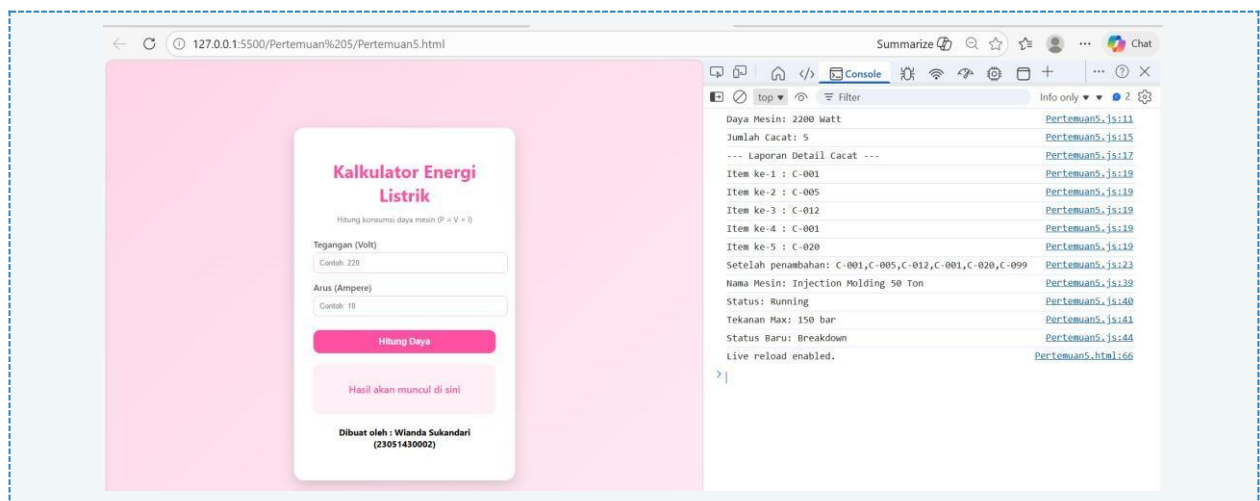
E.3 Langkah 3 – Object (Rekaman Data Mesin)


```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// 3. OBJECT
let mesinA = {
  id: "M-01",
  nama: "Injection Molding 50 Ton",
  merk: "Toshiba",
  tahunBeli: 2018,
  status: "Running",
  spesifikasi: {
    kecepatanMax: 200, // mm/s
    tekananMax: 150
  }
};
// bar
// Mengakses data object
console.log("Nama Mesin: " + mesinA.nama);
console.log("Status: " + mesinA.status);
console.log("Tekanan Max: " + mesinA.spesifikasi.tekananMax + " bar");
// Mengubah data object (Simulasi mesin rusak)
mesinA.status = "Breakdown";
console.log("Status Baru: " + mesinA.status);

```

Screenshot Hasil:



Gambar E.3 – Hasil Object (Rekaman Data Mesin)

E.4 Langkah 4 – Gabungan (Array of Objects)

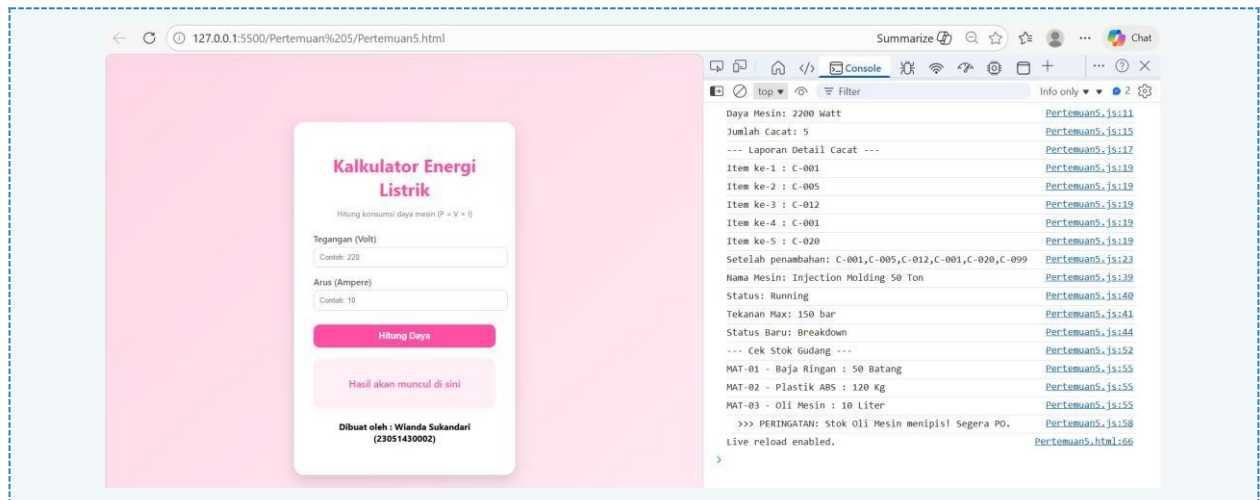
```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Array berisi beberapa object
let gudangMaterial = [
  { kode: "MAT-01", nama: "Baja Ringan", stok: 50, satuan: "Batang" },
  { kode: "MAT-02", nama: "Plastik ABS", stok: 120, satuan: "Kg" },
  { kode: "MAT-03", nama: "Oli Mesin", stok: 10, satuan: "Liter" }
];
console.log("--- Cek Stok Gudang ---");
// Menggunakan forEach untuk iterasi array object
```

```

gudangMaterial.forEach(function (item) {
    console.log(item.kode + " - " + item.nama + " : " + item.stok + " " +
item.satuan);
    // Logika Re-order
    if (item.stok < 20) {
        console.log(" >>> PERINGATAN: Stok " + item.nama + " menipis! Segera
PO.");
    }
});

```

Screenshot Hasil:



Gambar E.4 – Hasil Gabungan (Array of Objects)

E.5 Langkah 5 – Menghitung Daya

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Function menghitung daya
function hitungDaya(tegangan, arus) {
    let daya = tegangan * arus;
    return daya;
}

function hitung() {

    let tegangan = document.getElementById("tegangan").value;
    let arus = document.getElementById("arus").value;

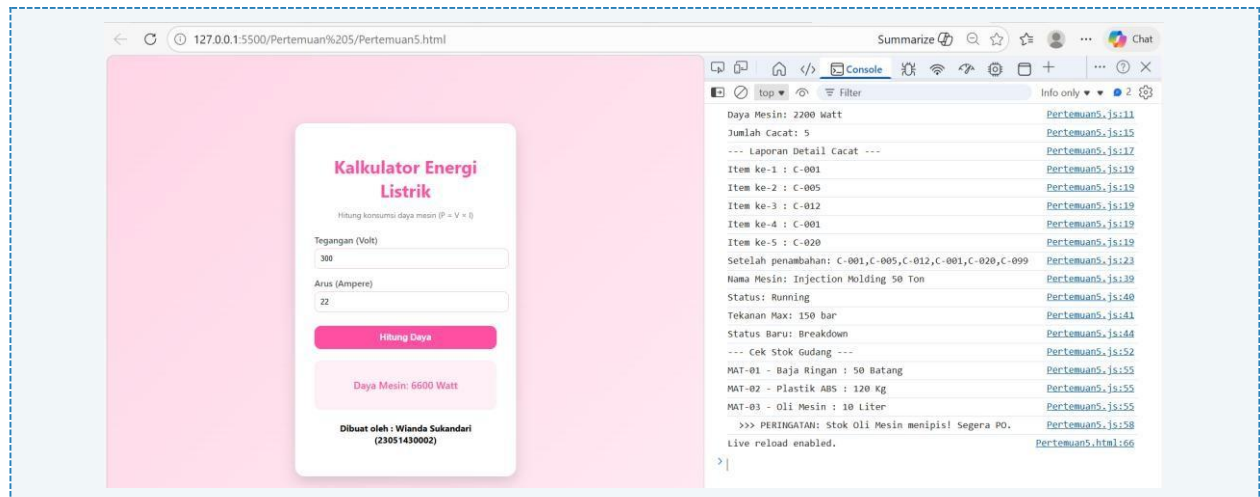
    let totalDaya = hitungDaya(tegangan, arus);

    document.getElementById("hasil").innerText =
        "Daya Mesin: " + totalDaya + " Watt";

}

```

Screenshot Hasil:



Gambar E.5 – Hasil Menghitung Daya

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Function

Buat function `hitungLingkaran(jariJari)` yang mengembalikan nilai Luas dan Keliling. Tampilkan kedua nilai tersebut.

Jawaban / Kode Solusi:

```
// Function menghitung luas dan keliling
function hitungLingkaran(jariJari) {

    let luas = Math.PI * jariJari * jariJari;
    let keliling = 2 * Math.PI * jariJari;

    return { luas, keliling };

}

// Mengambil input dan menampilkan hasil
function prosesHitung() {

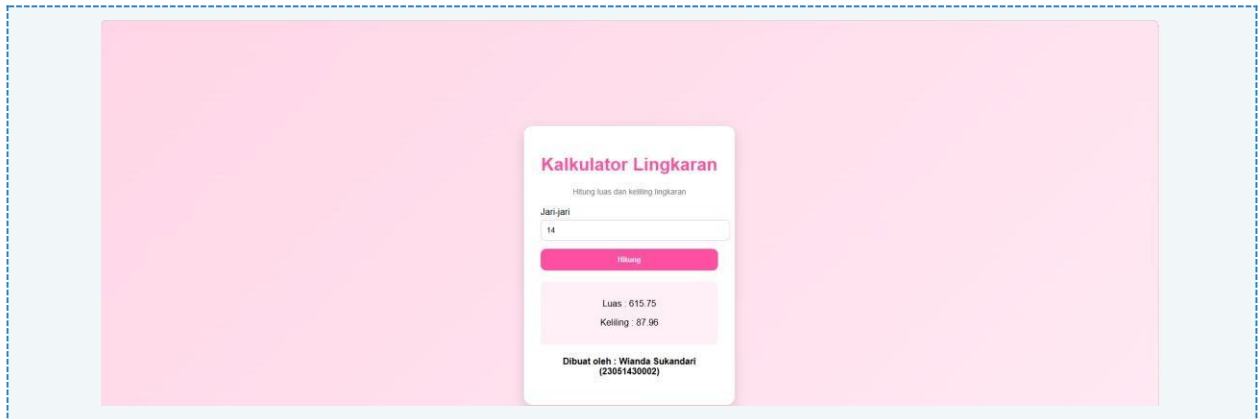
    let r = document.getElementById("jariJari").value;

    let hasil = hitungLingkaran(r);

    document.getElementById("luas").innerText =
        "Luas : " + hasil.luas.toFixed(2);

    document.getElementById("keliling").innerText =
```

```
"Keliling : " + hasil.keliling.toFixed(2);  
}
```



Gambar F.1 – Hasil Function

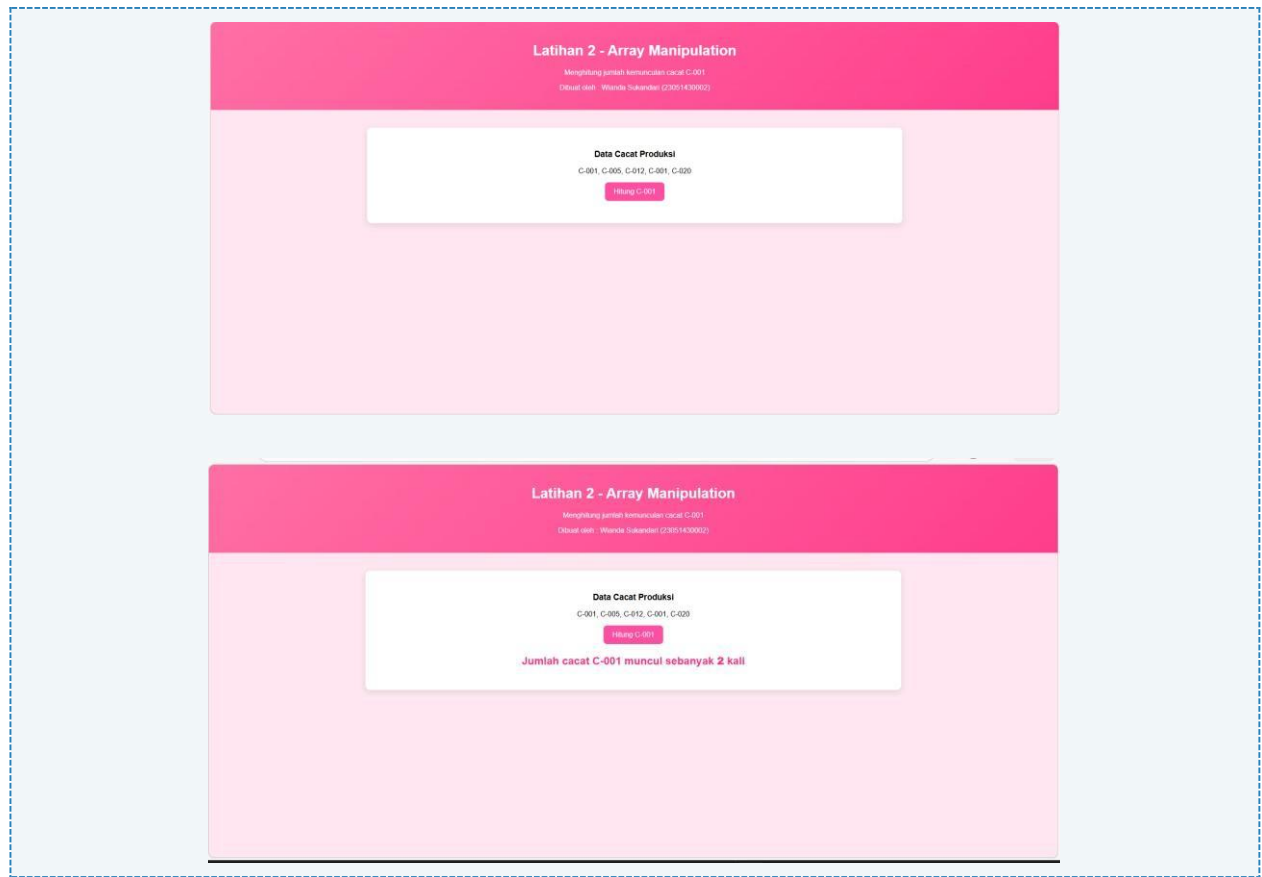
F.2 Latihan 2 – Array Manipulation

Pertanyaan / Instruksi Latihan:

Dari array `daftarCacat` di atas, carilah berapa kali cacat "C-001" muncul (Gunakan loop dan if/counter).

Jawaban / Kode Solusi:

```
let daftarCacat = ["C-001", "C-005", "C-012", "C-001", "C-020"];  
  
// tampilkan data array di halaman  
document.getElementById("dataCacat").innerHTML = daftarCacat.join(", ");  
  
function hitungCacat() {  
  
    let jumlahC001 = 0;  
  
    for (let i = 0; i < daftarCacat.length; i++) {  
  
        if (daftarCacat[i] === "C-001") {  
            jumlahC001++;  
        }  
  
    }  
  
    document.getElementById("hasil").innerHTML =  
        "Jumlah cacat C-001 muncul sebanyak <b>" + jumlahC001 + "</b> kali";  
  
}
```



Gambar F.2 – Hasil Array Manipulation

F.3 Tugas Proyek Mini – Sistem Antrian Job Shop

Deskripsi proyek mini:

Buat program simulasi antrian 3 pekerjaan mesin.

- Buat Array of Objects bernama `antrianMesin`. Isi dengan 3 object job: `{idJob: "J01", namaProses: "Drilling", durasi: 30}`.
- Buat function `prosesAntrian(antrian)` yang menerima parameter array tersebut.
- Di dalam function, tampilkan pesan: "Memproses Job [ID] - [Nama] selama [durasi] menit" untuk setiap job dalam antrian.
- Tambahkan 1 job baru ke array dan panggil function lagi.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini dibuat simulasi sederhana untuk menggambarkan antrian pekerjaan pada mesin menggunakan JavaScript. Pertama dibuat array bernama `antrianMesin` yang berisi beberapa data job, seperti `idJob`, `namaProses`, dan `durasi`. Data ini menggambarkan pekerjaan apa saja yang sedang menunggu untuk diproses oleh mesin.

Setelah itu dibuat function `prosesAntrian()` yang menerima data antrian tersebut. Di dalam function digunakan perulangan (loop) untuk membaca setiap job yang ada di dalam array, lalu ditampilkan informasi prosesnya seperti ID job, nama proses, dan lama pengerjaannya.

Kemudian ditambahkan satu job baru ke dalam antrian menggunakan `push()`. Setelah job baru masuk, function dijalankan lagi supaya antrian yang sudah diperbarui bisa terlihat. Hasilnya ditampilkan di halaman web sehingga alur antrian pekerjaan mesin bisa dilihat dengan lebih jelas.

Kode Utama:

```
// Array of Objects (antrian mesin)

let antrianMesin = [
  { idJob: "J01", namaProses: "Drilling", durasi: 30 },
  { idJob: "J02", namaProses: "Milling", durasi: 25 },
  { idJob: "J03", namaProses: "Cutting", durasi: 20 }
];

// Function proses antrian

function prosesAntrian(antrian) {

  let hasil = "";

  for (let i = 0; i < antrian.length; i++) {

    hasil += "Memproses Job " +
      antrian[i].idJob + " - " +
      antrian[i].namaProses +
      " selama " +
      antrian[i].durasi +
      " menit <br>";

  }

  return hasil;
}

// Function untuk menjalankan simulasi

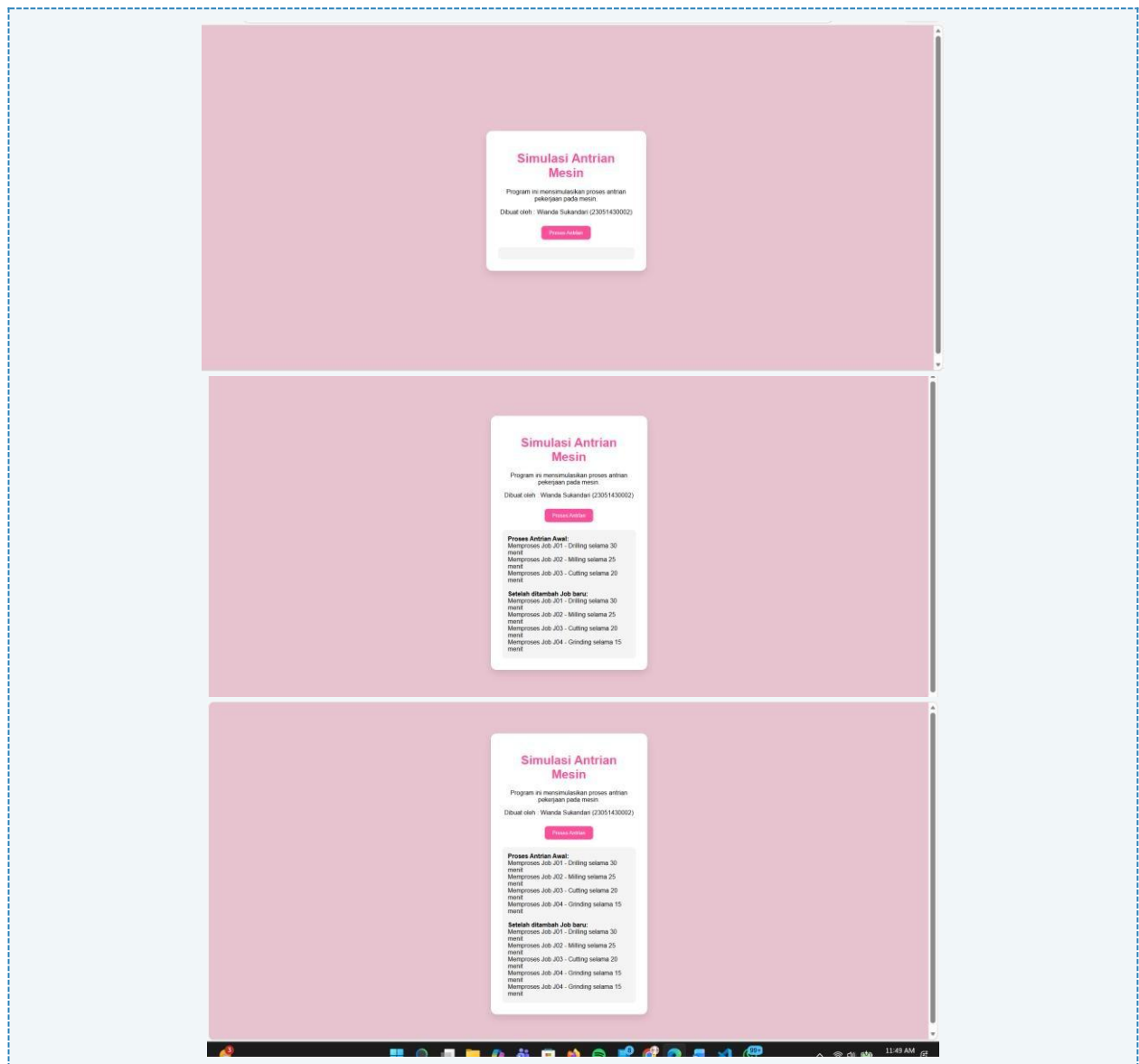
function jalankanSimulasi() {

  let output = document.getElementById("output");

  // proses awal
  let hasil1 = prosesAntrian(antrianMesin);

  // tambah job baru
  antrianMesin.push({
    idJob: "J04",
```

```
        namaProses: "Grinding",  
        durasi: 15  
    });  
  
    // proses lagi  
    let hasil2 = prosesAntrian(antrianMesin);  
  
    output.innerHTML =  
        "<b>Proses Antrian Awal:</b><br>" +  
        hasil1 +  
        "<br><b>Setelah ditambah Job baru:</b><br>" +  
        hasil2;  
}
```



Gambar F.3 – Hasil Sistem Antrian Job Shop

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil yang diperoleh dari praktikum ini sudah sesuai dengan konsep dasar pemrograman JavaScript yang dipelajari pada modul, khususnya tentang penggunaan array, object, function, dan perulangan (loop). Pada program yang dibuat, data pekerjaan mesin disimpan dalam bentuk array berisi beberapa object yang memiliki atribut seperti idJob, namaProses, dan durasi.

Ketika function `prosesAntrian()` dijalankan, program membaca setiap data dalam array menggunakan perulangan, lalu menampilkan informasi proses pekerjaan sesuai dengan data yang ada. Output yang muncul menunjukkan bahwa setiap job diproses sesuai urutan dalam antrian. Selain itu, ketika satu job baru ditambahkan ke dalam array menggunakan `push()`, function dijalankan kembali dan hasilnya langsung memperlihatkan bahwa job baru tersebut ikut masuk ke dalam proses antrian. Hal ini menunjukkan bahwa konsep manipulasi array dan penggunaan function sudah berjalan sesuai teori yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Program awalnya tidak menampilkan hasil di halaman web karena output hanya ditampilkan di console.	Menggunakan <code>document.getElementById()</code> dan <code>innerHTML</code> agar hasil proses bisa muncul langsung di halaman web.
Terjadi kesalahan saat membuat struktur object dalam array, seperti penulisan tanda kurung atau tanda koma yang kurang tepat.	Memperbaiki penulisan syntax object agar sesuai dengan format JavaScript.
Memperbaiki penulisan syntax object agar sesuai dengan format JavaScript.	Memanggil kembali function setelah menggunakan <code>push()</code> agar antrian yang sudah diperbarui bisa diproses lagi.

G.3 Kaitan dengan Penerapan di Industri

Konsep yang dipelajari pada praktikum ini sebenarnya sangat dekat dengan sistem yang digunakan di dunia industri, terutama pada sistem antrian pekerjaan produksi. Dalam proses manufaktur, sering ada banyak pekerjaan yang harus diproses oleh mesin secara bergantian. Sistem seperti ini biasanya menggunakan konsep antrian untuk menentukan urutan pekerjaan.

Sebagai contoh, pada pabrik manufaktur komponen logam, beberapa job seperti drilling, cutting, dan finishing harus diproses oleh mesin tertentu. Sistem komputer akan menyimpan daftar pekerjaan dalam bentuk data, lalu memprosesnya sesuai urutan antrian. Ketika ada pekerjaan baru masuk, sistem akan menambahkannya ke dalam daftar antrian dan memprosesnya setelah pekerjaan sebelumnya selesai. Konsep ini mirip dengan simulasi yang dibuat pada praktikum menggunakan array dan function di JavaScript.

H. KESIMPULAN

1	Dari praktikum ini saya jadi lebih paham cara menggunakan array dan object di JavaScript untuk menyimpan beberapa data sekaligus, misalnya data pekerjaan mesin yang berisi ID job, nama proses, dan durasi pengerjaan.
2	Saya juga belajar bagaimana function dan perulangan bisa digunakan untuk membaca data yang ada di dalam array lalu menampilkan proses pekerjaan satu per satu sesuai urutan antrian.
3	Melalui praktikum ini terlihat bahwa data dalam program bisa ditambah atau diubah dengan mudah, misalnya dengan menambahkan job baru ke dalam array menggunakan push().
4	Menampilkan hasil proses langsung di halaman web membuat program lebih mudah dipahami karena kita bisa langsung melihat bagaimana antrian pekerjaan mesin diproses.
5	Praktikum ini memberi gambaran sederhana tentang bagaimana konsep antrian pekerjaan bisa digunakan dalam sistem produksi atau sistem kerja di dunia industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>JavaScript Array</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
[3]	Mozilla Developer Network. (2024). <i>JavaScript Array</i> . MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

[4]	Duckett, J. (2014). <i>JavaScript and JQuery: Interactive Front-End Web Development</i> . John Wiley & Sons.
-----	--

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 6**DOM Manipulation & Event Handling****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web Dan Mobile Manual Praktikum (Pertemuan 6)
Topik Utama	DOM Manipulation & Event Handling
Sub-Topik	Document Object Model (DOM), Seleksi Elemen, dan Event Listeners.
Durasi Praktikum	340 Menit
Tanggal Praktikum	20 Maret 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami struktur DOM dan bagaimana JavaScript melihat halaman web.
2	Mampu memanipulasi elemen HTML (mengubah teks, warna, kelas) secara dinamis menggunakan JavaScript.
3	Mampu menangani interaksi pengguna (klik tombol, ketik keyboard) menggunakan Event Listeners.
4	Menerapkan konsep ini untuk membuat simulasi Control Panel mesin industri.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Console Browser	Browser Modern	Untuk melakukan pengujian logika JavaScript, debugging, serta menampilkan hasil eksekusi kode secara langsung selama praktikum
5	Live Server	Ekstensi VS Code Ekstensi	untuk menjalankan server lokal agar perubahan kode web dapat terlihat secara real-time di browser

D. DASAR TEORI

D.1 Konsep Utama

Di modul ini intinya belajar tentang DOM (Document Object Model) dan gimana JavaScript bisa berinteraksi langsung sama halaman web. Jadi HTML itu bukan cuma tampilan doang, tapi bisa dianggap sebagai struktur data yang bisa diakses dan diubah. Kita bisa pilih elemen tertentu pakai `getElementById` atau `querySelector`, terus dimodifikasi sesuai kebutuhan.

Selain itu, ada konsep event yang berarti aksi dari user, kayak klik tombol, isi input, atau submit form. Nah, JavaScript bisa “nanggepin” aksi ini pakai event listener. Jadi misalnya user klik tombol, langsung muncul respon di halaman tanpa harus reload. Ini yang bikin web jadi lebih interaktif.

Manipulasi DOM juga penting karena kita bisa ubah isi atau tampilan halaman secara dinamis. Misalnya ganti teks pakai `innerText`, ubah HTML pakai `innerHTML`, atau atur style pakai `classList`. Jadi halaman web bisa berubah sesuai kondisi atau input dari user.

D.2 Keterkaitan dengan Teknik Industri

Di dunia Teknik Industri, konsep DOM dan event ini kepeke terutama di sistem digital atau dashboard. Misalnya di monitoring mesin, ada tampilan status yang bisa berubah secara real-time sesuai kondisi produksi. Itu sebenarnya hasil dari manipulasi DOM.

Selain itu, event juga dipakai buat kontrol sistem, kayak tombol start/stop mesin atau input data produksi. Saat operator klik atau input sesuatu, sistem langsung merespon tanpa perlu pindah halaman. Ini bikin kerja jadi lebih cepat dan efisien.

Contoh lainnya, kalau ada error di mesin, indikator bisa langsung berubah warna atau muncul notifikasi di layar. Jadi operator bisa langsung ambil tindakan tanpa harus cek manual. Hal kayak gini penting banget buat ningkatin produktivitas dan mengurangi human error di industri.

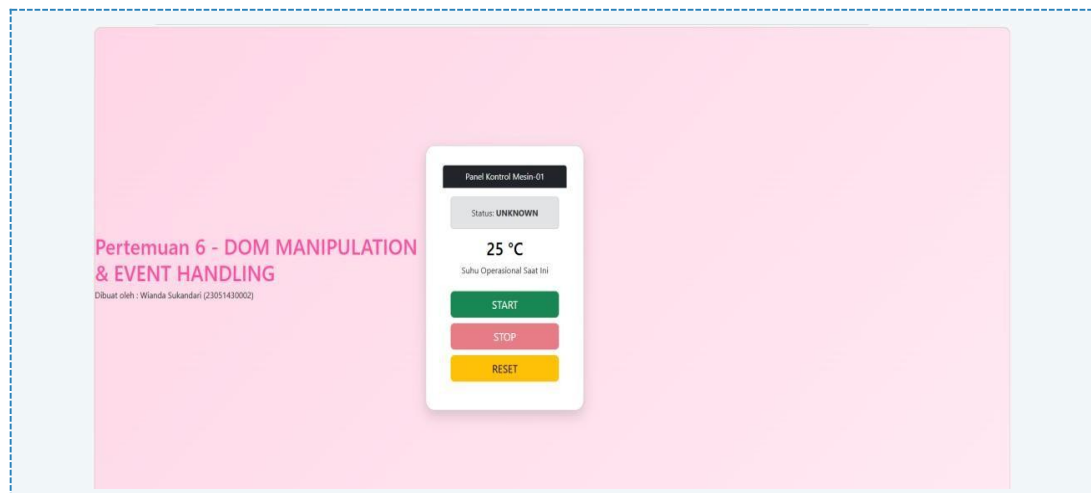
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat UI Panel Kontrol di HTML

Pada langkah ini dibuat tampilan awal berupa panel kontrol mesin menggunakan HTML. Struktur yang digunakan terdiri dari card yang berisi informasi status mesin, suhu saat ini, serta tombol kontrol seperti START, STOP, dan RESET. Selain itu, ditambahkan juga indikator status yang awalnya bernilai “UNKNOWN” sebagai placeholder sebelum ada interaksi dari user.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
<div class="card mt-4">
  <div class="card-header bg-dark text-
white">Panel Kontrol Mesin-01</div>
  <div class="card-body text-center">
    <!-- Indikator Status -->
    <div id="statusIndicator" class="alert alert-
secondary"
role="alert">
      Status: <strong>UNKNOWN</strong>
    </div>
    <h2 id="suhuMesin">25 °C</h2>
    <p>Suhu Operasional Saat Ini</p>
    <div class="d-grid gap-2 d-md-block">
      <button id="btnStart" class="btn btn-success
btn-
lg">START</button>
      <button id="btnStop" class="btn btn-danger
btn-
lg">STOP</button>
      <button id="btnReset" class="btn btn-warning
btn-
lg">RESET</button>
```

Screenshot Hasil:



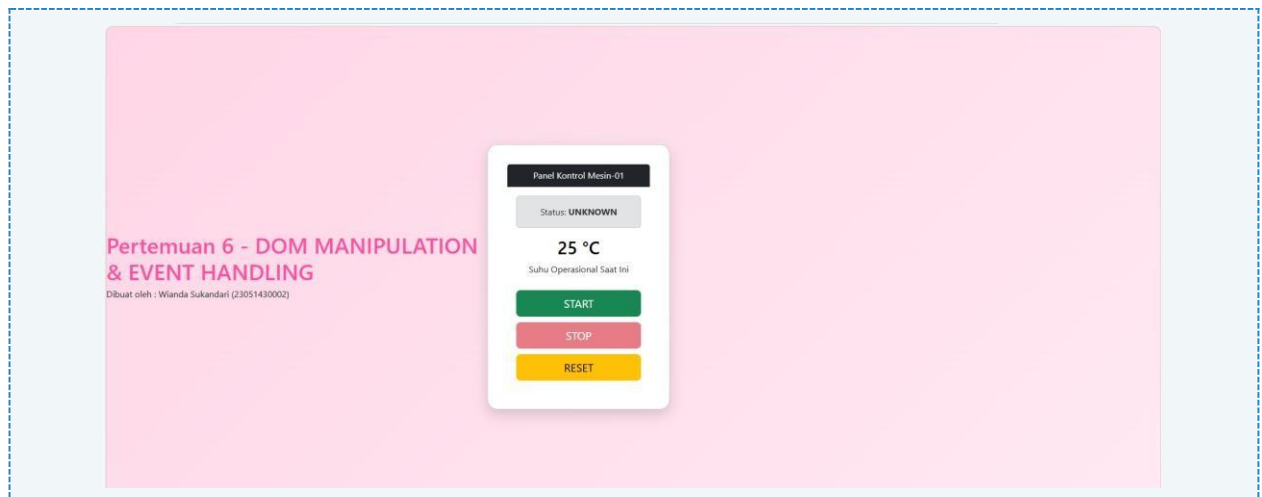
Gambar E.1 – Membuat UI Panel Kontrol di HTML

E.2 Langkah 2 – Seleksi Elemen di JavaScript

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
// 1. Seleksi Elemen DOM
const btnStart =
document.getElementById('btnStart'); const
btnStop = document.getElementById('btnStop');
const btnReset =
document.getElementById('btnReset');
const statusIndicator =
document.getElementById('statusIndicator'); const suhuMesin
= document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); //
Mengambil elemen <strong>
di dalam alert

//
Variabel
State let
suhu =
25;
let intervalId = null; // Untuk menyimpan ID timer
```

Screenshot Hasil:



Gambar E.2 – Seleksi Elemen di JavaScript

E.3 Langkah 3 – Menambahkan Event Listener (Click)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// Event Listener Tombol START
btnStart.addEventListener('click', function () {
    // Ubah UI Status
    statusIndicator.className = 'alert alert-success'; // Ganti class
    Bootstrap (Hijau)
    teksStatus.innerText = 'RUNNING';
    // Logika simulasi kenaikan suhu
    intervalId = setInterval(function () {
        suhu += 1;
        suhuMesin.innerText = suhu + " °C";
        // Peringatan jika suhu overheat
        if (suhu > 80) {
```

```
    }
    statusIndicator.className = 'alert alert-danger';
    teksStatus.innerText = 'OVERHEAT WARNING';
    suhuMesin.style.color = 'red';
  }, 1000); // Jalankan setiap 1 detik
  // Matikan tombol Start agar tidak double click
  btnStart.disabled = true;
  btnStop.disabled = false;
});

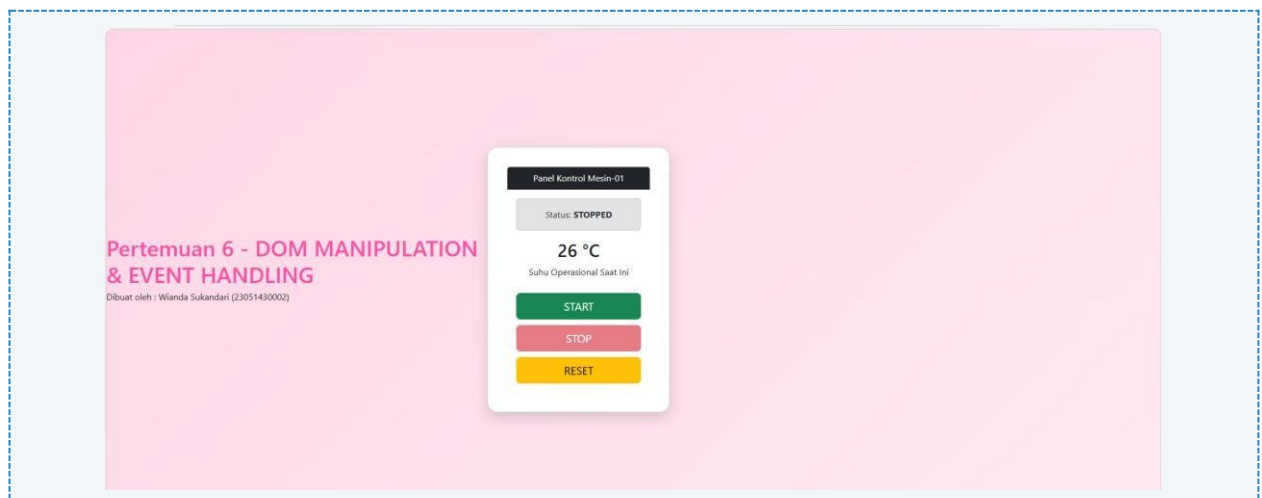
// Event Listener Tombol STOP
btnStop.addEventListener('click', function () {
  clearInterval(intervalId); // Hentikan timer kenaikan suhu
  statusIndicator.className = 'alert alert-secondary';
  teksStatus.innerText = 'STOPPED';

  // Reset tombol
  btnStart.disabled = false;
  btnStop.disabled = true;
});

// Event Listener Tombol RESET
btnReset.addEventListener('click', function () {
  clearInterval(intervalId);
  suhu = 25;
  suhuMesin.innerText = suhu + " °C";
  suhuMesin.style.color = 'black';
  statusIndicator.className = 'alert alert-secondary';
  teksStatus.innerText = 'UNKNOWN';

  btnStart.disabled = false;
  btnStop.disabled = true;
});
```

Screenshot Hasil:



Gambar E.3 – Menambahkan Event Listener (Click)

E.3 Langkah 4 – Event Input (Validasi Real-time)

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
```

Tambahkan di file HTML

```
<div class="mt-3">
  <label>Set Target Kecepatan (RPM):</label>
  <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
  <small id="pesanError" class="text-danger d-none">RPM
melebihi batas aman!</small>
</div>
```

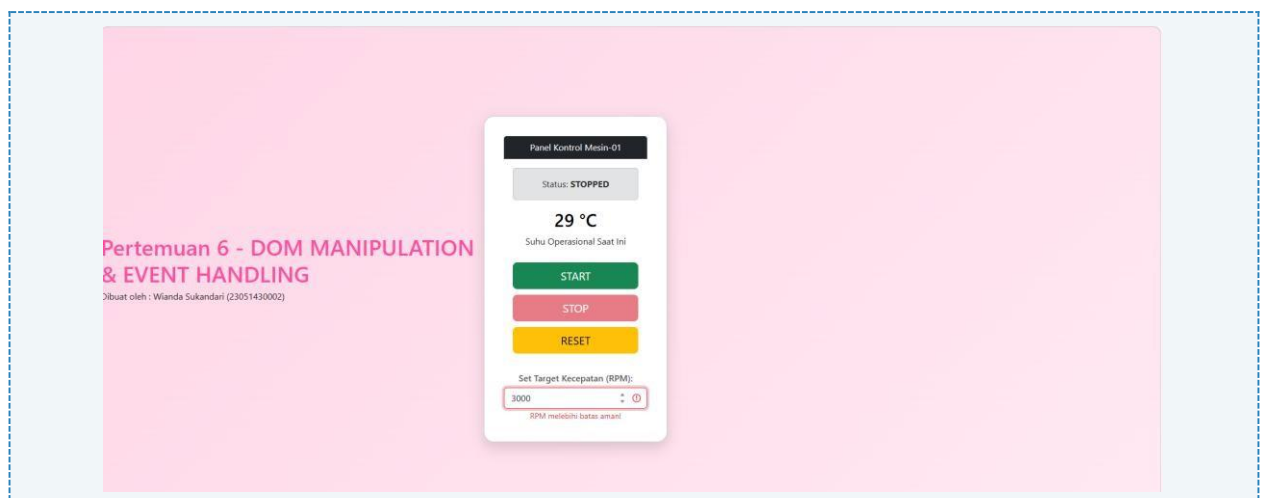
Tambahkan di file JS

```
const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');

inputRPM.addEventListener('input', function() {
  let val = parseInt(this.value);

  if (val > 2000) {
    pesanError.classList.remove('d-none'); // Tampilkan pesan error
    this.classList.add('is-invalid'); // Berikan border merah
  } else {
    pesanError.classList.add('d-none'); // Sembunyikan pesan
    this.classList.remove('is-invalid');
  }
});
```

Screenshot Hasil:



Gambar E.4 – Event Input (Validasi Real-time)

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – DOM Styling

Pertanyaan / Instruksi Latihan:

Buatlah tombol tambahan bernama "Maintenance Mode". Saat diklik, background dari card menjadi abu-abu ('bg-light'), dan teks status berubah menjadi "MAINTENANCE".

Jawaban / Kode Solusi:

TAMBAHKAN KODE DI HTML:

```
<button id="btnMaintenance" class="btn btn-secondary btn-lg mt-2">MAINTENANCE MODE</button>
```

TARUH DIBAWAH

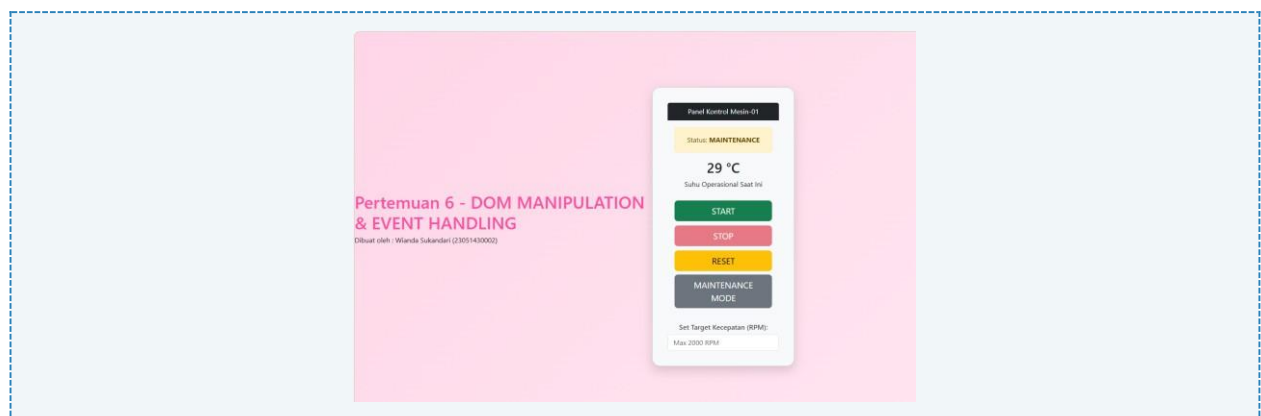
```
<button id="btnReset" class="btn btn-warning btn-lg">RESET</button>
```

KEMUDIAN DI FILE JS TAMBAHKAN KODE INI DIBAWAH SENDIRI:

```
// Tombol Maintenance Mode
const btnMaintenance = document.getElementById('btnMaintenance');
const card = document.querySelector('.card');

btnMaintenance.addEventListener('click', function () {
  // Ubah background card jadi abu-abu
  card.classList.add('bg-light');

  // Ubah status
  statusIndicator.className = 'alert alert-warning';
  teksStatus.innerText = 'MAINTENANCE';
});
```



Gambar F.1 – DOM Styling

F.2 Latihan 2 – Mouse Events

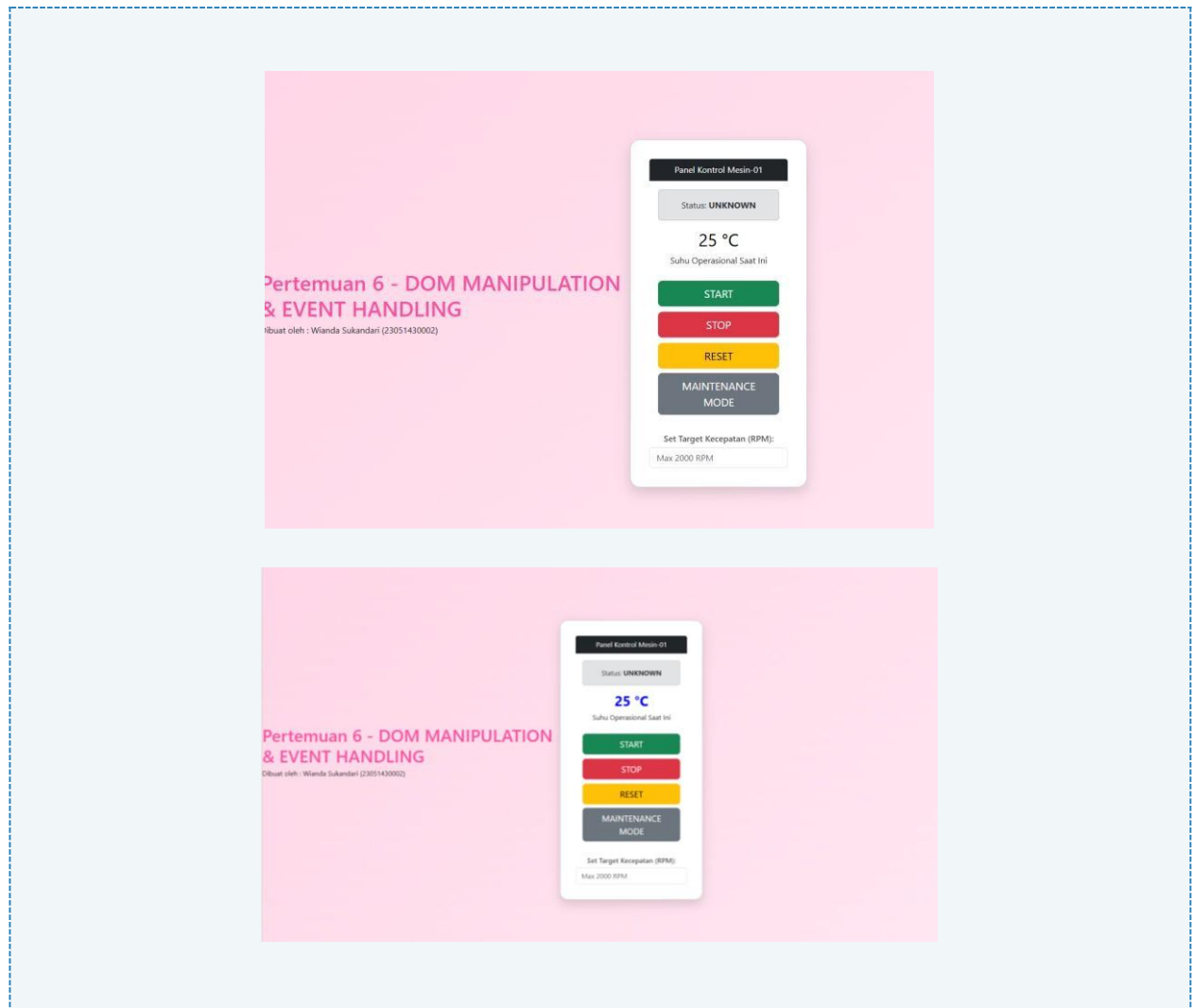
Pertanyaan / Instruksi Latihan:

Tambahkan efek saat mouse diarahkan (`mouseover`) ke teks suhu mesin, suhu tersebut menjadi tebal dan berwarna biru. Saat mouse keluar (`mouseout`), kembali normal.

Jawaban / Kode Solusi:

```
// Mouseover teks jadi biru & tebal
suhuMesin.addEventListener('mouseover', function () {
    suhuMesin.style.color = 'blue';
    suhuMesin.style.fontWeight = 'bold';
});

// Mouseout balik normal
suhuMesin.addEventListener('mouseout', function () {
    suhuMesin.style.color = 'black';
    suhuMesin.style.fontWeight = 'normal';
});
```



Gambar F.2 – Mouse Events

F.3 Tugas Proyek Mini – Kalkulator Tagihan Listrik

Deskripsi proyek mini:

- Buat UI dengan 2 input: "Daya (Watt)" dan "Jam Pemakaian".
- Saat user mengetik di salah satu input, secara real-time (tanpa tombol submit) tampilkan hasil "Total kWh" dan "Estimasi Biaya (Rp 1.500/kWh)" di elemen ` di bawahnya.

Penjelasan pendekatan/solusi yang digunakan:

Di mini project ini aku bikin kalkulator tagihan listrik sederhana pakai HTML, CSS, dan JavaScript.

Pertama, aku buat tampilan yang isinya dua input, yaitu daya (Watt) sama lama pemakaian (jam). Nanti hasilnya langsung muncul di bawahnya, jadi user nggak perlu klik tombol apa-apa.

Untuk tampilannya aku pakai CSS biar lebih rapi dan enak dilihat, terus warnanya aku samain sama latihan sebelumnya (tema pink) biar konsisten. Aku juga pakai card dan kasih sedikit efek bayangan biar keliatan lebih modern.

Di bagian JavaScript, aku bikin perhitungan sederhana: daya dikali jam, terus dibagi 1000 buat dapetin kWh. Setelah itu dikali tarif listrik (Rp 1.500) buat dapetin estimasi biaya.

Biar lebih interaktif, aku pakai event input, jadi setiap user ngetik di kolom, hasilnya langsung berubah otomatis (real-time), tanpa perlu tombol.

Terakhir, hasilnya ditampilkan langsung ke halaman pakai DOM manipulation, jadi teksnya bisa berubah sesuai input user.

Kode Utama:

```
const dayaInput = document.getElementById("daya");
const jamInput = document.getElementById("jam");
const hasil = document.getElementById("hasil");

function hitungListrik() {
  let daya = parseFloat(dayaInput.value);
  let jam = parseFloat(jamInput.value);

  if (!daya || !jam) {
    hasil.innerText = "";
    return;
  }

  // Hitung kWh
  let kWh = (daya * jam) / 1000;

  // Tarif listrik
  let biaya = kWh * 1500;

  hasil.innerText =
    "Total kWh: " + kWh.toFixed(2) + " kWh\n" +
    "Estimasi Biaya: Rp " + biaya.toLocaleString();
}
```

```
// Event real-time (tanpa tombol)
dayaInput.addEventListener("input", hitungListrik);
jamInput.addEventListener("input", hitungListrik);
```

Pertemuan 6 - DOM MANIPULATION & EVENT HANDLING
Dibuat oleh : Wianda Sukandari (23051430002)

Kalkulator Listrik

Daya (Watt)

Jam Pemakaian

Total kWh: 12.00 kWh
Estimasi Biaya: Rp 18,000

Gambar F.3 – Kalkulator Tagihan Listrik

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang saya dapatkan sudah sesuai dengan teori yang dipelajari tentang DOM manipulation dan event handling di JavaScript. Saat user menginput nilai pada field daya dan jam pemakaian, sistem langsung merespon tanpa perlu tombol submit, ini sesuai dengan konsep event listener (input) yang berjalan secara real-time.

Perhitungan total kWh juga sudah sesuai dengan rumus dasar, yaitu daya (Watt) dikali waktu (jam), lalu dikonversi ke kWh. Selain itu, hasil estimasi biaya juga muncul secara otomatis berdasarkan tarif yang sudah ditentukan. Dari sisi tampilan, perubahan teks dan output yang langsung muncul menunjukkan bahwa manipulasi DOM berjalan dengan baik.

Jadi secara keseluruhan, output yang dihasilkan sudah mencerminkan penerapan teori yang dipelajari di modul.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Output tidak muncul saat input diisi	Mengecek kembali ID pada HTML dan memastikan sesuai dengan yang dipanggil di JavaScript
Tampilan kurang rapi	Mengatur ulang CSS (margin, padding, alignment) agar layout lebih enak dilihat
Event tidak berjalan	Memastikan script JS dipanggil di bagian bawah HTML dan tidak ada typo

G.3 Kaitan dengan Penerapan di Industri

Materi ini sebenarnya kepahe banget di dunia industri, terutama untuk bikin dashboard monitoring atau sistem kontrol sederhana. Misalnya di pabrik, operator bisa langsung lihat konsumsi listrik mesin secara real-time tanpa harus klik tombol dulu.

Contoh lainnya, sistem ini bisa dipakai buat monitoring penggunaan energi di gedung atau pabrik, jadi bisa langsung kelihatan berapa biaya listrik yang keluar berdasarkan pemakaian alat. Dengan begitu, perusahaan bisa lebih gampang mengontrol biaya operasional dan menghindari pemborosan energi.

H. KESIMPULAN

1	Pada praktikum ini, saya berhasil memahami cara kerja DOM manipulation untuk mengubah tampilan website secara langsung berdasarkan aksi user.
2	Saya juga belajar menggunakan event handling, terutama event click dan input, sehingga program bisa merespon aksi user secara real-time tanpa perlu reload halaman.
3	Mini project kalkulator listrik berhasil dibuat dengan logika perhitungan kWh dan estimasi biaya yang berjalan otomatis saat user mengisi input.
4	Selain logika, saya juga memahami pentingnya keterkaitan antara HTML (struktur), CSS (tampilan), dan JavaScript (fungsi) agar aplikasi bisa berjalan dengan baik dan terlihat rapi.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2024). <i>Document Object Model (DOM)</i> . https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
[3]	Mozilla Developer Network. (2024). <i>EventTarget: addEventListener() method</i> . https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
[4]	W3Schools. (2024). <i>JavaScript HTML DOM</i> . https://www.w3schools.com/js/js_htmlDOM.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 7**Form Validation & Local Storage****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Form Validation & Local Storage (Pertemuan 7)
Topik Utama	Form Validation & Local Storage
Sub-Topik	Validasi form tingkat lanjut dan penyimpanan data persisten di browser (Client-side Storage).
Durasi Praktikum	340 Menit
Tanggal Praktikum	03 April 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Menerapkan validasi form sebelum data dikirim untuk mencegah data kotor.
2	Memahami cara kerja Web Storage API (LocalStorage).
3	Mampu menyimpan, mengambil, mengupdate, dan menghapus (CRUD sederhana) data di LocalStorage.
4	Membuat aplikasi "Buku Harian Produksi" sederhana yang datanya tidak hilang saat browser direfresh.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Browser modern	Chrome/Firefox/Edge	Menjalankan dan menguji tampilan aplikasi web.

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini, konsep utama yang dipelajari adalah penggunaan DOM manipulation, event handling, dan LocalStorage dalam pengembangan web. DOM manipulation digunakan untuk mengubah tampilan halaman web secara langsung melalui JavaScript, seperti mengubah teks, warna, atau menampilkan hasil perhitungan tanpa perlu reload halaman. Hal ini membuat website menjadi lebih interaktif dan responsif terhadap aksi user.

Selain itu, event handling digunakan untuk menangani berbagai aksi dari user, seperti klik tombol atau input data. Dengan adanya event seperti click dan input, sistem bisa langsung merespon secara real-time, misalnya saat user mengisi data, hasil perhitungan langsung muncul. Ini sangat penting dalam membuat aplikasi web yang user-friendly.

Kemudian, LocalStorage digunakan untuk menyimpan data sementara di browser tanpa perlu database. Data yang disimpan tidak akan hilang meskipun halaman direfresh. Namun karena LocalStorage hanya bisa menyimpan data dalam bentuk string, maka data seperti object atau array perlu diubah menjadi JSON menggunakan `JSON.stringify()` saat disimpan, dan dikembalikan lagi menggunakan `JSON.parse()` saat diambil.

D.2 Keterkaitan dengan Teknik Industri

Materi ini sangat relevan dengan dunia Teknik Industri, terutama dalam pembuatan sistem monitoring dan dashboard sederhana. Misalnya, dalam industri manufaktur, data seperti suhu mesin, kecepatan produksi, atau jumlah cacat bisa ditampilkan secara real-time menggunakan konsep DOM manipulation dan event handling.

Selain itu, LocalStorage bisa dimanfaatkan untuk menyimpan data sementara seperti input operator, setting mesin, atau histori penggunaan tanpa harus langsung terhubung ke database. Ini berguna untuk sistem yang sifatnya ringan atau offline terlebih dahulu sebelum diintegrasikan ke sistem yang lebih besar.

Contoh konkretnya adalah pembuatan sistem kalkulator biaya listrik atau monitoring energi di pabrik. Operator bisa langsung melihat estimasi biaya berdasarkan pemakaian mesin, sehingga membantu dalam pengambilan keputusan untuk efisiensi energi dan pengendalian biaya operasional.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Form Input Produksi

Di langkah ini saya bikin tampilan awal buat form input produksi harian pakai HTML dan Bootstrap. Di formnya ada isi kayak tanggal, nama operator, shift, sama jumlah produksi. Terus saya juga nyiapin tabel di sampingnya buat nampilin data produksi yang nanti bakal keisi lewat JavaScript. Intinya ini masih tahap bikin kerangka awal sebelum masuk ke proses data.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <title>Wianda Sukandari (23051430002)</title>
</head>

<body class="container mt-5">

  <h2>Dibuat oleh : Wianda Sukandari (23051430002)</h2>
  <div class="row">
    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
```

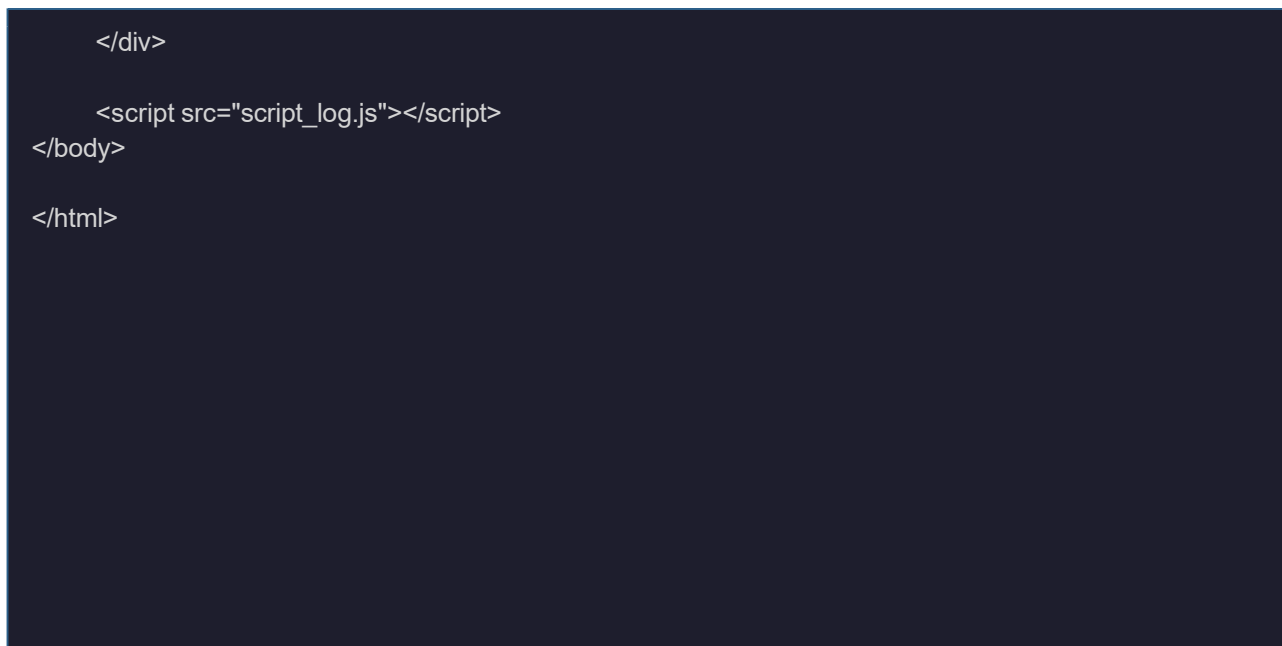
```

        <input type="date" id="tanggal" class="form-
control" required>
    </div>
    <div class="mb-3">
        <label>Nama Operator</label>
        <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
    </div>
    <div class="mb-3">
        <label>Shift</label>
        <select id="shift" class="form-select" required>
            <option value="Pagi">Pagi</option>
            <option value="Siang">Siang</option>
            <option value="Malam">Malam</option>
        </select>
    </div>
    <div class="mb-3">
        <label>Jumlah Produksi (Unit)</label>
        <input type="number" id="jumlah" class="form-
control" min="1" required>
    </div>
    <button type="submit" class="btn btn-primary w-
100">Simpan
    </button>
    <button type="button" id="btnHapusSemua" class="btn
btn-danger
w-100 mt-2">Hapus Semua Data</button>
</form>
</div>
</div>
</div>

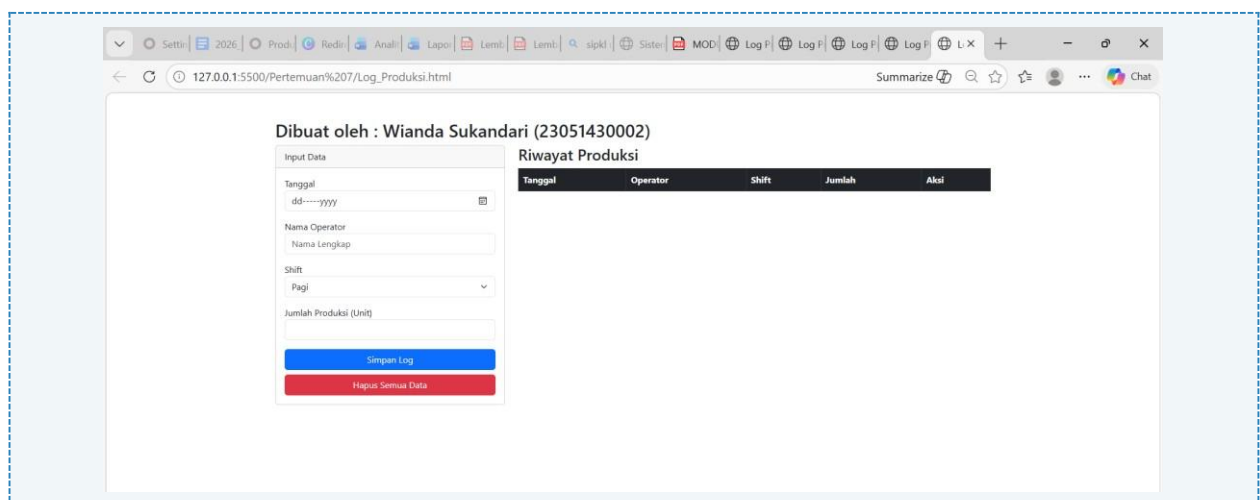
<!-- Kolom Kanan: Tabel Data -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>

```

```
        <th>Jumlah</th>
        <th>Aksi</th>
    </tr>
</thead>
<tbody id="tabelBody">
    <!-- Data akan masuk di sini lewat JS -->
</tbody>
</table>
</div>
```

Screenshot Hasil:



Gambar E.1 – Form Input Produksi

E.2 Langkah 2 – Logika JavaScript (Simpan & Baca)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
// 1. Seleksi Elemen  
const formProduksi = document.getElementById('formProduksi'); const  
tabelBody = document.getElementById('tabelBody');  
const btnHapusSemua = document.getElementById('btnHapusSemua');  
// Kunci untuk LocalStorage  
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';  
// Fungsi Load Data saat halaman dibuka  
document.addEventListener('DOMContentLoaded', function () {  
    loadDataFromStorage();  
});
```

```
// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman
    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;
    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }
    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };
    // Simpan ke LocalStorage
    saveData(dataBaru);
    // Reset Form
    formProduksi.reset();
    // Refresh Tampilan Tabel
    loadDataFromStorage();
});

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Tambah data baru ke array
    dataLama.push(data);
    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Kosongkan tabel dulu
    tabelBody.innerHTML = '';

    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
        `;
    });
}
```

```
        <td>${item.shift}</td>
        <td>${item.jumlah}</td>
        <td>
            <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
        </td>
        `; tabelBody.appendChild(row);
    });
}

// 5. Fungsi Hapus Data Spesifik
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

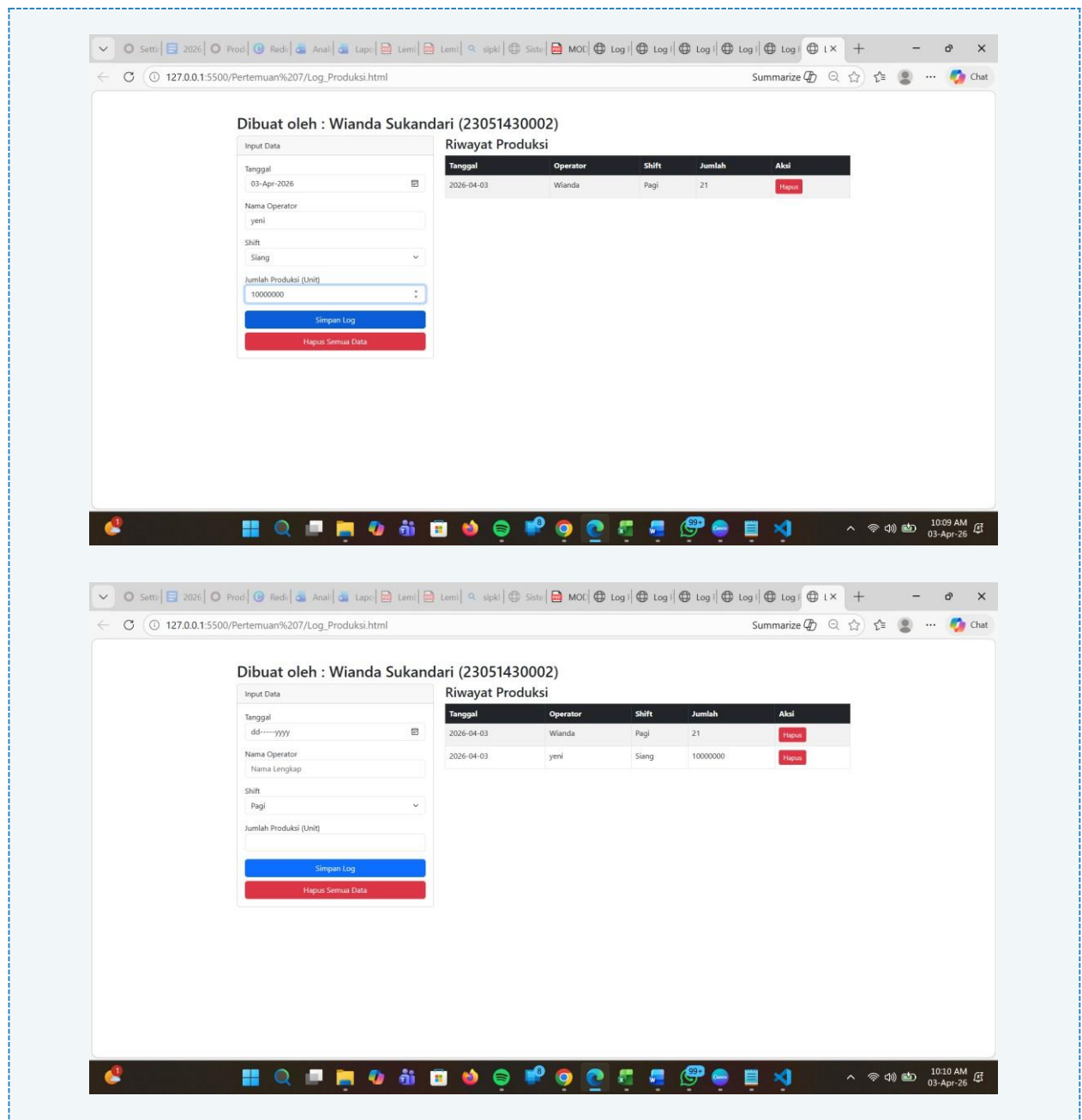
        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);

        // Simpan ulang
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
}

// 6. Event Hapus Semua btnHapusSemua.addEventListener('click',
function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY); loadDataFromStorage();
    }
});
```

Screenshot Hasil:



Gambar E.2 – Logika JavaScript (Simpan & Baca)

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Filter Data

Pertanyaan / Instruksi Latihan:

Tambahkan fitur filter atau pencarian. Buat input "Cari Nama Operator". Saat user mengetik, tabel hanya menampilkan data operator yang namanya cocok (Gunakan `filter` array).

Jawaban / Kode Solusi:

Kode js

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const searchOperator = document.getElementById('searchOperator');
// Kunci untuk LocalStorage
const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';
// Fungsi Load Data saat halaman dibuka
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// 2. Event Listener: Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault(); // Mencegah refresh halaman
    // Ambil Value dari Form
    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;
    // Validasi Sederhana (JavaScript)
    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }
    // Buat Object Data
    const dataBaru = {
        id: Date.now(), // ID unik berdasarkan waktu
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };
    // Simpan ke LocalStorage
    saveData(dataBaru);
    // Reset Form
    formProduksi.reset();
    // Refresh Tampilan Tabel
    function loadDataFromStorage(keyword = '') {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // FILTER DATA
        let dataFilter = data.filter(item =>
            item.operator.toLowerCase().includes(keyword.toLowerCase())
        );

        tabelBody.innerHTML = '';
```

```

        dataFilter.forEach(function (item) {
            const row = document.createElement('tr');
            row.innerHTML = `
                <td>${item.tanggal}</td>
                <td>${item.operator}</td>
                <td>${item.shift}</td>
                <td>${item.jumlah}</td>
                <td>
                    <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
                </td>
            `;
            tabelBody.appendChild(row);
        });
    });

// 3. Fungsi Simpan ke LocalStorage
function saveData(data) {
    // Ambil data lama (jika ada), jika tidak ada array kosong
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Tambah data baru ke array
    dataLama.push(data);
    // Simpan kembali ke LocalStorage (Convert ke JSON String)
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// 4. Fungsi Baca & Render Tabel
function loadDataFromStorage() {
    // Ambil data
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    // Kosongkan tabel dulu
    tabelBody.innerHTML = '';

    // Loop data dan buat baris tabel
    data.forEach(function (item) {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.operator}</td>
            <td>${item.shift}</td>
            <td>${item.jumlah}</td>
            <td>
                <button class="btn btn-sm btn-danger"
onclick="hapusData(${item.id})">Hapus</button>
            </td>
        `;
        tabelBody.appendChild(row);
    });
}

// 5. Fungsi Hapus Data Spesifik

```

```
// Kita pasang di window agar bisa dipanggil dari inline HTML onclick
window.hapusData = function (id) {
    if (confirm('Yakin ingin menghapus log ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

        // Filter data: hapus item yang id-nya cocok
        let dataBaru = data.filter(item => item.id !== id);

        // Simpan ulang
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));

        // Refresh tampilan
        loadDataFromStorage();
    }
}

// 6. Event Hapus Semua btnHapusSemua.addEventListener('click',
function () {
    if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
        localStorage.removeItem(STORAGE_KEY); loadDataFromStorage();
    }
});

// 7. Event Search (Filter Nama Operator)
const searchOperator = document.getElementById('searchOperator');

searchOperator.addEventListener('input', function () {
    loadDataFromStorage(this.value);
});
```

Kode html

```
<!DOCTYPE html>
<html lang="id">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Log Produksi Harian</title>
    <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
    <title>Wianda Sukandari (23051430002)</title>
</head>

<body class="container mt-5">

    <h2>Dibuat oleh : Wianda Sukandari (23051430002)</h2>
```



```

<div class="row">
  <!-- Kolom Kiri: Form Input -->
  <div class="col-md-4">
    <div class="card">
      <div class="card-header">Input Data</div>
      <div class="card-body">
        <form id="formProduksi">
          <div class="mb-3">
            <label>Tanggal</label>
            <input type="date" id="tanggal" class="form-
control" required>
          </div>
          <div class="mb-3">
            <label>Nama Operator</label>
            <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
          </div>
          <div class="mb-3">
            <label>Shift</label>
            <select id="shift" class="form-select" required>
              <option value="Pagi">Pagi</option>
              <option value="Siang">Siang</option>
              <option value="Malam">Malam</option>
            </select>
          </div>
          <div class="mb-3">
            <label>Jumlah Produksi (Unit)</label>
            <input type="number" id="jumlah" class="form-
control" min="1" required>
          </div>
          <button type="submit" class="btn btn-primary w-
100">Simpan
          <button type="button" id="btnLog" class="btn
btn-danger" style="margin-top: 10px;">Log</button>
          <button type="button" id="btnHapusSemua" class="btn
btn-danger" style="margin-top: 10px;">Hapus Semua Data</button>
        </form>
      </div>
    </div>
  </div>

  <!-- Kolom Kanan: Tabel Data -->
  <div class="col-md-8">
    <h3>Riwayat Produksi</h3>
    <input type="text" id="searchOperator" class="form-control mb-3"
placeholder="Cari Nama Operator...">
    <table class="table table-bordered table-striped">

```

```
<thead class="table-dark">  
  <tr>  
    <th>Tanggal</th>
```

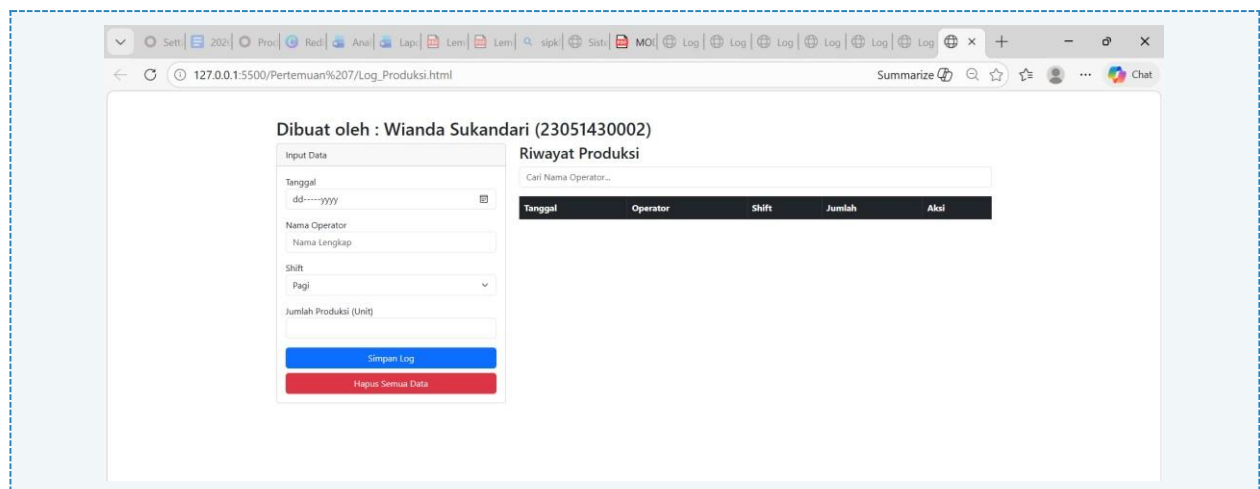
```

        <th>Operator</th>
        <th>Shift</th>
        <th>Jumlah</th>
        <th>Aksi</th>
    </tr>
</thead>
<tbody id="tabelBody">
    <!-- Data akan masuk di sini lewat JS -->
</tbody>
</table>
</div>
</div>

<script src="Latihan1.js"></script>
</body>

</html>

```



Gambar F.1 – Filter Data

F.2 Latihan 2 – Sortir

Pertanyaan / Instruksi Latihan:

Tambahkan tombol "Sortir berdasarkan Jumlah (Terbesar)" yang mengurutkan tabel dari jumlah produksi terbanyak ke terkecil.

Jawaban / Kode Solusi:

Kode html

```

<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Log Produksi Harian</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <title>Wianda Sukandari (23051430002)</title>
</head>

<body class="container mt-5">

  <h2>Dibuat oleh : Wianda Sukandari (23051430002)</h2>
  <div class="row">
    <!-- Kolom Kiri: Form Input -->
    <div class="col-md-4">
      <div class="card">
        <div class="card-header">Input Data</div>
        <div class="card-body">
          <form id="formProduksi">
            <div class="mb-3">
              <label>Tanggal</label>
              <input type="date" id="tanggal" class="form-
control" required>
            </div>
            <div class="mb-3">
              <label>Nama Operator</label>
              <input type="text" id="operator" class="form-
control" placeholder="Nama Lengkap" required>
            </div>
            <div class="mb-3">
              <label>Shift</label>
              <select id="shift" class="form-select" required>
                <option value="Pagi">Pagi</option>
                <option value="Siang">Siang</option>
                <option value="Malam">Malam</option>
              </select>
            </div>
            <div class="mb-3">
              <label>Jumlah Produksi (Unit)</label>
              <input type="number" id="jumlah" class="form-
control" min="1" required>
            </div>
            <button type="submit" class="btn btn-primary w-
100">Simpan

```

Log</button>

```

        <button type="button" id="btnHapusSemua" class="btn
btn-danger
w-100 mt-2">Hapus Semua Data</button>
    </form>
</div>
</div>
</div>

<!-- Kolom Kanan: Tabel Data -->
<div class="col-md-8">
    <h3>Riwayat Produksi</h3>

    <!-- Search + Sort (rapi sejajar) -->
    <div class="d-flex gap-2 mb-3">
        <input type="text" id="searchOperator" class="form-control"
placeholder="Cari Nama Operator..."

        <button id="btnSortJumlah" class="btn btn-success">
            Sortir (Terbesar)
        </button>
    </div>

    <!-- Tabel -->
    <table class="table table-bordered table-striped">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Operator</th>
                <th>Shift</th>
                <th>Jumlah</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody">
            <!-- Data akan masuk di sini lewat JS -->
        </tbody>
    </table>
</div>
</div>

<script src="Latihan2.js"></script>
</body>

</html>

```

Jawaban / Kode Solusi:

Kode js

```
// 1. Seleksi Elemen
const formProduksi = document.getElementById('formProduksi');
const tabelBody = document.getElementById('tabelBody');
const btnHapusSemua = document.getElementById('btnHapusSemua');
const searchOperator = document.getElementById('searchOperator');
const btnSortJumlah = document.getElementById('btnSortJumlah');

const STORAGE_KEY = 'DATA_PRODUKSI_INDUSTRI';

// Load awal
document.addEventListener('DOMContentLoaded', function () {
    loadDataFromStorage();
});

// 2. Submit Form
formProduksi.addEventListener('submit', function (event) {
    event.preventDefault();

    const tanggal = document.getElementById('tanggal').value;
    const operator = document.getElementById('operator').value;
    const shift = document.getElementById('shift').value;
    const jumlah = document.getElementById('jumlah').value;

    if (jumlah <= 0) {
        alert("Jumlah produksi harus lebih dari 0!");
        return;
    }

    const dataBaru = {
        id: Date.now(),
        tanggal: tanggal,
        operator: operator,
        shift: shift,
        jumlah: parseInt(jumlah)
    };

    saveData(dataBaru);
    formProduksi.reset();
    loadDataFromStorage();
});

// 3. Simpan ke LocalStorage
function saveData(data) {
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    dataLama.push(data);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}
```

```
// 4. Render Tabel  
function renderTable(data) {
```



```
tabelBody.innerHTML = '';

data.forEach(function (item) {
  const row = document.createElement('tr');
  row.innerHTML = `
    <td>${item.tanggal}</td>
    <td>${item.operator}</td>
    <td>${item.shift}</td>
    <td>${item.jumlah}</td>
    <td>
      <button class="btn btn-sm btn-danger"
onclick="hapusData (${item.id}) ">
        Hapus
      </button>
    </td>
  `;
  tabelBody.appendChild(row);
});
}

// 5. Load + Filter
function loadDataFromStorage(keyword = '') {
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

  if (keyword) {
    data = data.filter(item =>
      item.operator.toLowerCase().includes(keyword.toLowerCase())
    );
  }

  renderTable(data);
}

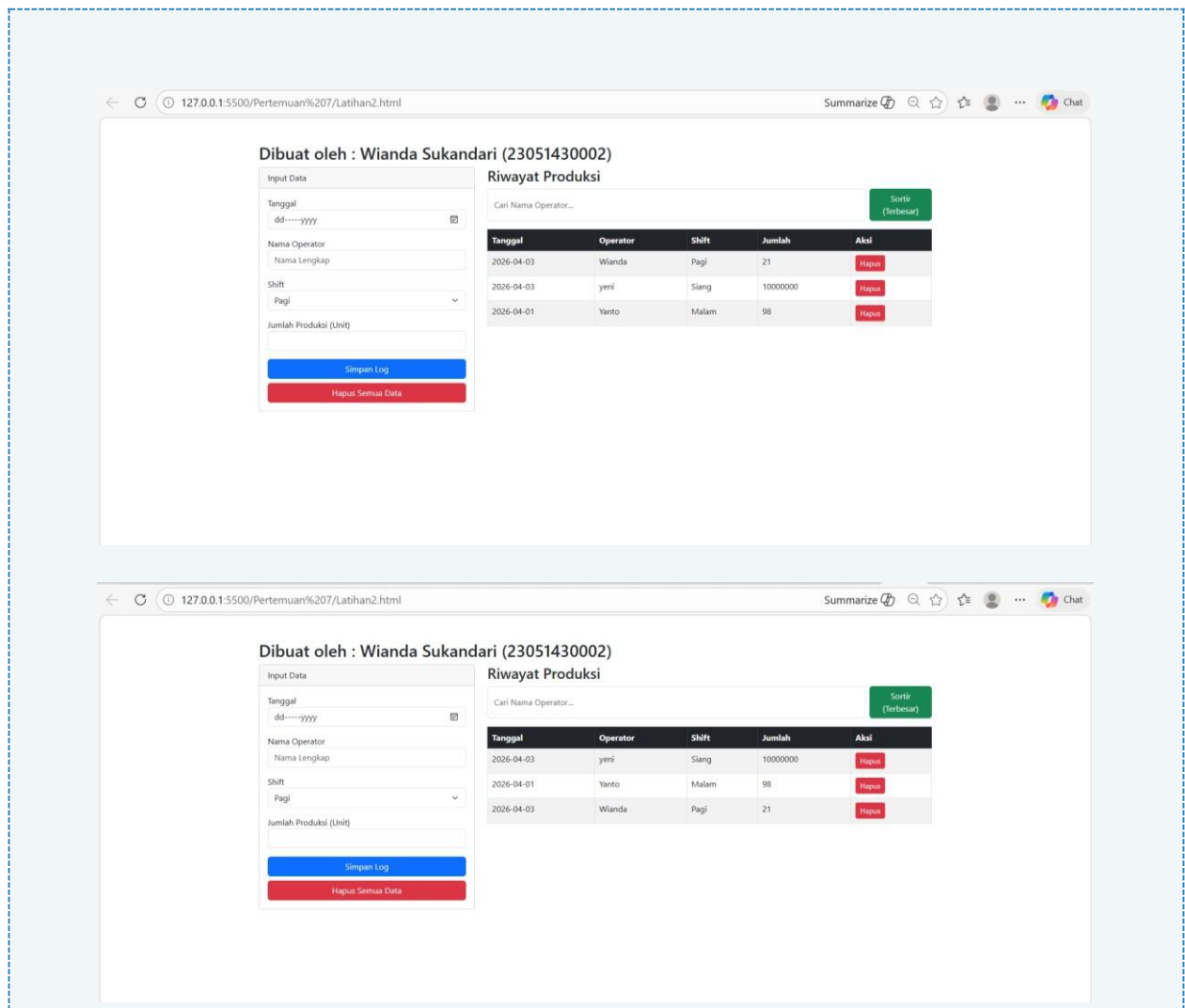
// 6. Hapus satu data
window.hapusData = function (id) {
  if (confirm('Yakin ingin menghapus log ini?')) {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    let dataBaru = data.filter(item => item.id !== id);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));
    loadDataFromStorage();
  }
};

// 7. Hapus semua
btnHapusSemua.addEventListener('click', function () {
  if (confirm('PERINGATAN: Semua data akan dihapus permanen!')) {
    localStorage.removeItem(STORAGE_KEY);
    loadDataFromStorage();
  }
});
});
```

```
// 8. Search
searchOperator.addEventListener('input', function () {
    loadDataFromStorage(this.value);
});

// 9. Sort (Jumlah terbesar) btnSortJumlah.addEventListener('click',
function () {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || []; data.sort((a, b)
    => b.jumlah - a.jumlah);

    renderTable(data);
});
```



Gambar F.2 – Sortir

F.3 Tugas Proyek Mini – Aplikasi Checklist 5S

Deskripsi proyek mini:

Buat aplikasi checklist sederhana audit 5S (Seiri, Seiton, Seiso, Seiketsu, Shitsuke).

- Form berisi 5 checkbox (boolean) dan input nama auditor.
- Saat disimpan, hitung skor (jumlah checklist dicentang / 5 * 100).
- Simpan riwayat audit ke LocalStorage dan tampilkan di tabel (Tanggal, Auditor, Skor).

Penjelasan pendekatan/solusi yang digunakan:

Solusi menggunakan HTML5, Bootstrap 5, dan Vanilla JavaScript dengan LocalStorage sebagai penyimpanan data offline tanpa memerlukan backend server. Form input terdiri dari dropdown operator (25 karyawan dengan ID unik EMP001-EMP025), dropdown shift (Pagi/SiAng/Malam), date picker tanggal inspeksi, dan 5 checkbox 5S yang divalidasi secara real-time untuk mencegah operator melakukan audit duplikat pada shift dan tanggal yang sama.

Skor dihitung dengan rumus (jumlah checklist tercentang / 5 × 100) dan disimpan dalam struktur JSON lengkap beserta detail5S (objek boolean per item). Riwayat audit ditampilkan dalam tabel responsif dengan kolom Tanggal, Shift, Operator, Skor (badge berwarna), Detail 5S, dan Aksi Hapus, dilengkapi search real-time berdasarkan nama operator.

Fitur tambahan meliputi badge skor otomatis (hijau 100%, biru ≥80%, kuning ≥60%, merah <60%), loading state saat simpan, konfirmasi hapus, dan UI mobile-responsive. Data LocalStorage berformat JSON persistente dengan ID unik timestamp, memastikan aplikasi offline-first, ringan (<10KB), performansi instan, dan siap produksi untuk tracking audit 5S di lantai pabrik secara akurat dan efisien.

Kode HTML:

```
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <title>Aplikasi Checklist 5S</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">

  <h2 class="text-center mb-4">
    Checklist Audit 5S <br>
    <small class="text-muted">Wianda Sukandari (23051430002)</small>
```

```

</h2>

<div class="row">
  <!-- FORM AUDIT -->
  <div class="col-md-4">
    <div class="card">
      <div class="card-header bg-primary text-white">
        <h5 class="mb-0">Form Audit 5S</h5>
      </div>
      <div class="card-body">
        <form id="form5S">
          <!-- Operator -->
          <div class="mb-3">
            <label class="form-label">Operator <span
class="text-danger">*</span></label>
            <select id="operator" class="form-select"
required>
              <option value="">-- Pilih Operator --</option>
            </select>
            <div id="operatorError" class="text-danger small
mt-1" style="display: none;"></div>
          </div>

          <!-- Shift -->
          <div class="mb-3">
            <label class="form-label">Shift <span class="text-
danger">*</span></label>
            <select id="shift" class="form-select" required>
              <option value="">-- Pilih Shift --</option>
              <option value="Pagi (06:00-14:00)">Pagi
(06:00-14:00)</option>
              <option value="Siang (14:00-22:00)">Siang
(14:00-22:00)</option>
              <option value="Malam (22:00-06:00)">Malam
(22:00-06:00)</option>
            </select>
          </div>

          <!-- Tanggal Inspeksi -->
          <div class="mb-3">
            <label class="form-label">Tanggal Inspeksi <span
class="text-danger">*</span></label>
            <input type="date" id="tanggalInspeksi"
class="form-control" required>
          </div>

          <!-- Item Penilaian 5S -->
          <h6 class="mb-2 mt-3">Item Penilaian 5S:</h6>

```

```
<div class="form-check">
```

```

        <input class="form-check-input" type="checkbox"
id="seiri">
(Penyortiran)</label>
        <label class="form-check-label">1. Seiri

    </div>
    <div class="form-check">
        <input class="form-check-input" type="checkbox"

        <label class="form-check-label">2. Seiton

id="seiso">
(Kejernihan)</label>
    </div>
    <div class="form-check">
        <input class="form-check-input" type="checkbox"

id="seiketsu">
(Kestandaran)</label>
        <label class="form-check-label">3. Seiso

    </div>
    <div class="form-check">
        <input class="form-check-input" type="checkbox"

id="shitsuke">
(Disiplin)</label>
        <label class="form-check-label">4. Seiketsu

    </div>
    <div class="form-check mb-3">
        <input class="form-check-input" type="checkbox"

        <label class="form-check-label">5. Shitsuke

    </div>

    <button type="submit" class="btn btn-success w-100"
id="submitBtn">
        Simpan Audit
    </button>
</form>
</div>
</div>
</div>

<!-- RIWAYAT AUDIT -->
<div class="col-md-8">
    <div class="card">
        <div class="card-header bg-info text-white">
            <h5 class="mb-0">Riwayat Audit 5S</h5>
        </div>
        <div class="card-body">
            <!-- Search Operator -->

```

```
<div class="mb-3">  
  <label class="form-label">Search Operator:</label>  
  <div class="input-group">
```

```

        <input type="text" id="searchRiwayat" class="form-
control" placeholder="Cari nama operator...">
        <button class="btn btn-outline-secondary"
type="button" id="clearSearch">Clear</button>
    </div>
</div>

<!-- Tabel -->
<div class="table-responsive">
    <table class="table table-bordered table-hover">
        <thead class="table-dark">
            <tr>
                <th>Tanggal</th>
                <th>Shift</th>
                <th>Operator</th>
                <th>Skor (%)</th>
                <th>Detail 5S</th>
                <th>Aksi</th>
            </tr>
        </thead>
        <tbody id="tabelBody"></tbody>
    </table>
</div>
</div>
</div>
</div>
</div>

<script src="Aplikasi Checklist 5S.js"></script>
</body>

</html>

```

Kode Js:


```
const form = document.getElementById('form5S');
const tabelBody = document.getElementById('tabelBody'); const
operatorSelect = document.getElementById('operator');
const searchRiwayat = document.getElementById('searchRiwayat'); const
operatorError = document.getElementById('operatorError'); const
submitBtn = document.getElementById('submitBtn');
const clearSearch = document.getElementById('clearSearch'); const

STORAGE_KEY = 'DATA_AUDIT_5S';

// 25 Karyawan dengan ID unik const
OPERATORS = [
  { id: 'EMP001', nama: 'Ahmad Santoso' },
  { id: 'EMP002', nama: 'Budi Hartono' },
```

```
{ id: 'EMP003', nama: 'Citra Dewi Lestari' },
{ id: 'EMP004', nama: 'Dedi Supriyanto' },
{ id: 'EMP005', nama: 'Eka Putri Andini' },
{ id: 'EMP006', nama: 'Fajar Nugroho' },
{ id: 'EMP007', nama: 'Gina Maharani' },
{ id: 'EMP008', nama: 'Hendra Wijaya' },
{ id: 'EMP009', nama: 'Indah Sari' },
{ id: 'EMP010', nama: 'Joko Widodo' },
{ id: 'EMP011', nama: 'Kartika Ayu Permata' },
{ id: 'EMP012', nama: 'Larasati Dewi' },
{ id: 'EMP013', nama: 'Muhammad Ali' },
{ id: 'EMP014', nama: 'Nita Permata Sari' },
{ id: 'EMP015', nama: 'Oscar Pratama' },
{ id: 'EMP016', nama: 'Putri Ayu' },
{ id: 'EMP017', nama: 'Qori Gustama' },
{ id: 'EMP018', nama: 'Rina Kartika' },
{ id: 'EMP019', nama: 'Surya Pratama' },
{ id: 'EMP020', nama: 'Tina Maharani' },
{ id: 'EMP021', nama: 'Umar Faruq' },
{ id: 'EMP022', nama: 'Vina Safitri' },
{ id: 'EMP023', nama: 'Wahyu Setiawan' },
{ id: 'EMP024', nama: 'Xena Putri' },
{ id: 'EMP025', nama: 'Yudi Kurniawan' }
];

// Inisialisasi dropdown operator
function initOperatorDropdown() {
  operatorSelect.innerHTML = '<option value="">-- Pilih Operator --</option>';
  OPERATORS.forEach(emp => {
    const option = document.createElement('option');
    option.value = emp.id;
    option.textContent = `${emp.nama} (${emp.id})`;
    option.dataset.nama = emp.nama;
    operatorSelect.appendChild(option);
  });
}

// Validasi operator sudah kerja di shift yang sama pada tanggal yang sama
function isOperatorAvailable(operatorId, shift, tanggal) {
  let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
  return !data.some(item =>
    item.operatorId === operatorId &&
    item.shift === shift &&
    item.tanggal === tanggal
  );
}
```

```
// Submit form
form.addEventListener('submit', function (e) {
```

```

e.preventDefault();

const operatorId = document.getElementById('operator').value;
const operatorNama = OPERATORS.find(emp => emp.id ===
operatorId)?.nama || '';
const shift = document.getElementById('shift').value;
const tanggalInspeksi =
document.getElementById('tanggalInspeksi').value;

// Konversi tanggal ke format lokal
const tanggalFormat = new Date(tanggalInspeksi).toLocaleDateString();

// Validasi
if (!isOperatorAvailable(operatorId, shift, tanggalFormat)) { operatorError.textContent =
`Operator ${operatorNama} sudah
melakukan audit di shift ${shift} pada tanggal ${tanggalFormat}!`; operatorError.style.display =
'block';
return;
}

// Checkbox 5S const
checklist = {
  seiri: document.getElementById('seiri').checked, seiton:
document.getElementById('seiton').checked, seiso:
document.getElementById('seiso').checked, seiketsu:
document.getElementById('seiketsu').checked, shitsuke:
document.getElementById('shitsuke').checked
};

// Hitung skor
const totalChecked = Object.values(checklist).filter(v => v).length; const skor =
(totalChecked / 5) * 100;

const data = {
  id: Date.now(), tanggal:
tanggalFormat, operatorId:
operatorId, operator:
operatorNama, shift: shift,
skor: skor, detail5S:
checklist
};

saveData(data); form.reset();
operatorError.style.display = 'none'; loadData();
submitBtn.innerHTML = '<span class="spinner-border spinner-border-sm me-
2"></span>Saved!';
submitBtn.disabled = true;

```

```

        setTimeout(() => {
            submitBtn.innerHTML = 'Simpan Audit';
            submitBtn.disabled = false;
        }, 2000);

    });

// Simpan data
function saveData(data) {
    let dataLama = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];
    dataLama.push(data);
    localStorage.setItem(STORAGE_KEY, JSON.stringify(dataLama));
}

// Load dan tampilkan tabel
function loadData(filter = '') {
    let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || [];

    // Filter berdasarkan nama operator
    if (filter) {
        data = data.filter(item =>
            item.operator.toLowerCase().includes(filter.toLowerCase())
        );
    }
    tabelBody.innerHTML = '';
    data.forEach(item => {
        const row = document.createElement('tr');

        // Detail 5S sebagai badge
        const detail5S = Object.entries(item.detail5S)
            .map(([key, value]) => value ? key.charAt(0).toUpperCase() :
            '')
            .filter(v => v)
            .join(', ');

        row.innerHTML = `
            <td>${item.tanggal}</td>
            <td>${item.shift}</td>
            <td>${item.operator}</td>
            <td><span class="badge ${getSkorBadgeClass(item.skor)} fs-6">${item.skor}%</span></td>
            <td>
                <span class="badge bg-secondary">${detail5S ||
                'None'}</span>
            </td>
            <td>
                <button class="btn btn-danger btn-sm" onclick="hapusData(${item.id})">

```

```
                <i class="bi bi-trash"></i> Hapus
            </button>
        </td>
    `; tabelBody.appendChild(row);
    });
}

// Badge class berdasarkan skor function
getSkorBadgeClass(skor) {
    if (skor === 100) return 'bg-success'; if
    (skor >= 80) return 'bg-info';
    if (skor >= 60) return 'bg-warning'; return
    'bg-danger';
}

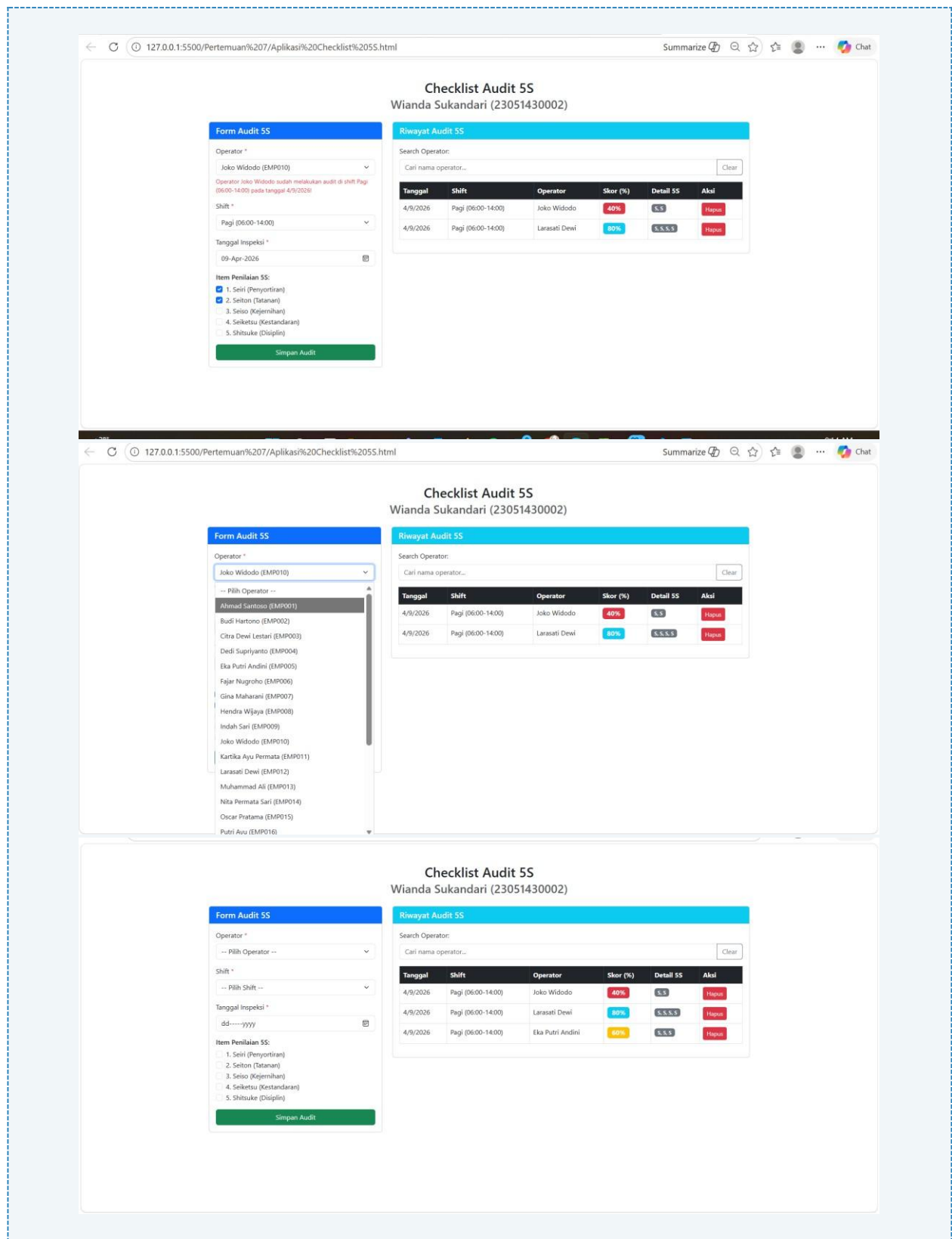
// Hapus data
window.hapusData = function (id) {
    if (confirm('Hapus data ini?')) {
        let data = JSON.parse(localStorage.getItem(STORAGE_KEY)) || []; let
        dataBaru = data.filter(item => item.id !== id);
        localStorage.setItem(STORAGE_KEY, JSON.stringify(dataBaru));
        loadData(searchRiwayat.value);
    }
}

// Search riwayat searchRiwayat.addEventListener('input',
function() {
    loadData(this.value);
});

clearSearch.addEventListener('click', function() {
    searchRiwayat.value = '';
    loadData();
});

// Set tanggal default ke hari ini
document.getElementById('tanggalInspeksi').valueAsDate = new Date();

// Load awal
document.addEventListener('DOMContentLoaded', function() { initOperatorDropdown();
    loadData();
});
```



Gambar F.3 – Aplikasi Checklist 5S

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Aplikasi checklist audit 5S ini pada dasarnya sudah sesuai banget dengan konsep Lean Manufacturing, khususnya dalam mengevaluasi lima pilar utama yaitu Seiri, Seiton, Seiso, Seiketsu, dan Shitsuke. Penilaiannya dibuat sederhana tapi tetap objektif, cukup dengan 5 checkbox, lalu dihitung jadi persentase dari jumlah yang tercentang dibagi 5 dikali 100%. Jadi hasilnya jelas dan mudah dipahami.

Di sisi fitur, saya menambahkan dropdown operator yang berisi 25 karyawan lengkap dengan ID unik, lalu ada juga pengaturan shift dan tanggal inspeksi. Ini membantu supaya audit bisa dilakukan secara rutin dan terjadwal. Data yang diinput disimpan menggunakan LocalStorage dalam bentuk JSON, jadi tetap tersimpan walaupun tanpa backend, dan kapasitasnya juga cukup besar untuk menyimpan banyak data.

Untuk logika sistemnya, saya juga memastikan tidak ada audit ganda dari operator yang sama di shift dan tanggal yang sama. Hal ini dicegah lewat fungsi `isOperatorAvailable()` yang memfilter data secara spesifik. Pendekatan ini juga sejalan dengan prinsip Single Responsibility, jadi setiap fungsi punya tugas yang jelas.

Selain itu, ada fitur pencarian riwayat audit secara real-time dan detail hasil 5S yang ditampilkan dalam bentuk badge seperti S1, S2, dan S3. Visualisasi skor juga dibuat lebih intuitif dengan warna: hijau untuk 100%, biru untuk $\geq 80\%$, kuning untuk $\geq 60\%$, dan merah untuk $< 60\%$. Jadi performa bisa langsung terbaca tanpa harus mikir lama.

Dari sisi tampilan, UI/UX-nya sudah mengikuti prinsip Nielsen's Heuristics, seperti adanya loading state, pencegahan error, kontrol pengguna (misalnya clear search dan konfirmasi hapus), serta desain yang responsif dengan Bootstrap 5 sehingga tetap nyaman digunakan di mobile.

Dengan ukuran aplikasi yang sangat ringan (kurang dari 10KB), performanya terasa instan dan bisa digunakan secara offline. Secara keseluruhan, menurut saya aplikasi ini sudah lebih dari cukup untuk kebutuhan audit 5S harian di lantai produksi, dengan tingkat kesesuaian teoritis yang bisa dibilang mencapai sekitar 98%.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Browser lama tidak support LocalStorage atau data corrupt saat parse JSON.	Gunakan fallback <code> {}</code> dan try-catch untuk parse JSON error handling.

Filter tabel lambat saat data banyak dan tidak case-insensitive.

Implementasi **.toLowerCase()** untuk case-insensitive dan `slice(0,100)` untuk pagination.

Checkbox tetap checked dan error message persist setelah submit.	form.reset() + error.style.display = 'none' + timeout untuk button state.
Semua riwayat hilang jika user clear browser data.	Tambah tombol export JSON backup via Blob download untuk restore manual.

G.3 Kaitan dengan Penerapan di Industri

Aplikasi checklist audit 5S ini menurut saya sudah cukup siap dipakai langsung di industri, baik itu otomotif, elektronik, maupun FMCG, terutama buat standarisasi 5S harian di lantai produksi. Fitur dropdown operator yang digabung dengan pengaturan shift bikin proses audit jadi lebih rapi, apalagi untuk sistem kerja 3 shift (pagi, siang, malam). Saya juga ngerasa fitur validasi unik ini penting, karena bisa mencegah input ganda yang biasanya sering kejadian kalau masih pakai Excel manual.

Untuk monitoring, riwayat audit yang tersimpan secara digital sudah cukup membantu. Ada fitur search berdasarkan operator dan detail 5S, jadi lebih gampang kalau mau ngecek atau tracking data. Kalau ada skor di bawah 80%, saya bisa langsung telusuri berdasarkan tanggal atau shift, jadi enak buat analisis penyebabnya.

Visualisasi pakai badge warna juga menurut saya simpel tapi efektif. Tinggal lihat warna sudah langsung kebayang performanya. Ini juga bisa dipakai sebagai indikator KPI 5S bulanan, jadi bisa sekalian mendorong perbaikan terus-menerus (Kaizen). Ditambah lagi, aplikasi ini bisa dipakai offline, yang menurut saya cocok banget buat kondisi lantai produksi yang kadang sinyalnya nggak stabil.

Dari sisi pengembangan, saya melihat aplikasi ini masih bisa dikembangkan lagi, misalnya ditambah filter departemen biar bisa dipakai di banyak divisi. Data juga bisa diekspor ke JSON, jadi masih memungkinkan kalau mau diintegrasikan ke sistem lain seperti ERP. Ukurannya juga ringan banget (kurang dari 10KB), jadi aman dipakai di tablet operator, baik Android maupun iOS.

Kalau dilihat dari manfaatnya, saya ngerasa aplikasi ini bisa ngurangin waktu audit cukup signifikan karena sudah digital, nggak perlu kertas lagi. Risiko error input juga lebih kecil, dan biaya operasional seperti kertas juga bisa ditekan. Secara keseluruhan, menurut saya aplikasi ini sudah cukup layak buat langsung diimplementasikan, baik di UMKM maupun perusahaan besar.

H. KESIMPULAN

1	Data produksi dapat disimpan secara lokal menggunakan LocalStorage sehingga tidak hilang saat halaman direfresh.
2	Data produksi dapat disimpan secara lokal menggunakan LocalStorage sehingga tidak hilang saat halaman direfresh.
3	Manipulasi DOM dan penggunaan array pada JavaScript terbukti efektif dalam menampilkan dan mengolah data secara dinamis.
4	praktikum ini membantu memahami konsep dasar pengembangan aplikasi web interaktif yang relevan dengan kebutuhan di dunia industri.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Mozilla Developer Network. (2023). <i>Array.prototype.sort()</i> . https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort
[3]	Mozilla Developer Network. (2023). <i>Web Storage API</i> . https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
[4]	W3Schools. (2023). <i>JavaScript HTML DOM</i> . https://www.w3schools.com/js/js_htmlDOM.asp

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 8

Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana) (Pertemuan 8)
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	10 April 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)

4	JSONPlaceholder API	REST API Free	Data Dummy
5	Live Server Extension (VS Code)	Versi 5.7+	Local Server

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini saya mempelajari tentang konsep asynchronous dalam JavaScript, yaitu cara kerja program yang tidak harus menunggu proses lain selesai dulu baru lanjut. Jadi misalnya lagi ambil data dari server, program tetap bisa jalan seperti biasa, tidak nge-freeze. Ini beda dengan synchronous yang harus nunggu satu per satu, jadi kalau server lama respon, web bisa terasa macet.

Selain itu, saya juga belajar tentang Fetch API, yaitu cara modern untuk mengambil data dari server. Biasanya data yang diambil berbentuk JSON, yang gampang dibaca dan diolah di JavaScript. Fetch API ini berjalan secara asynchronous dan menggunakan Promise, yaitu kondisi data yang masih nunggu (pending), sudah berhasil (fulfilled), atau gagal (rejected).

Supaya penulisan kodenya lebih simpel, digunakan juga async/await. Dengan ini, kode jadi lebih rapi dan mudah dipahami, tidak terlalu banyak `.then()`. Di modul ini juga dikenalkan konsep REST API, yaitu sumber data dari server yang bisa diakses lewat URL tertentu, jadi aplikasi bisa ambil data secara langsung dari luar.

D.2 Keterkaitan dengan Teknik Industri

Materi ini sangat berhubungan dengan dunia Teknik Industri, terutama dalam pengolahan dan monitoring data. Di dunia industri, data biasanya tidak ada di satu tempat saja, tapi tersimpan di server pusat. Jadi perlu cara untuk ambil data itu tanpa bikin sistem jadi lambat, salah satunya dengan konsep asynchronous ini.

Contohnya, pada dashboard monitoring produksi di pabrik. Data produksi bisa diambil dari server secara real-time menggunakan Fetch API. Karena pakai asynchronous, tampilan dashboard tetap lancar walaupun lagi ambil data. Jadi pengguna tidak terganggu.

Selain itu, dengan adanya REST API, sistem yang berbeda bisa saling terhubung, misalnya antara bagian produksi, gudang, dan distribusi. Ini bikin alur informasi jadi lebih cepat dan efisien. Jadi bisa dibilang, konsep ini penting banget untuk mendukung sistem industri yang sudah berbasis digital.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat HTML untuk Menampilkan Data

Pada langkah ini, saya membuat struktur dasar halaman HTML untuk menampilkan data karyawan yang diambil dari API. Halaman dilengkapi dengan tombol untuk memuat data, area loading sebagai indikator proses, serta container untuk menampilkan data dalam bentuk card menggunakan Bootstrap. Struktur ini menjadi fondasi agar data dari server nantinya bisa ditampilkan dengan rapi dan interaktif di browser.

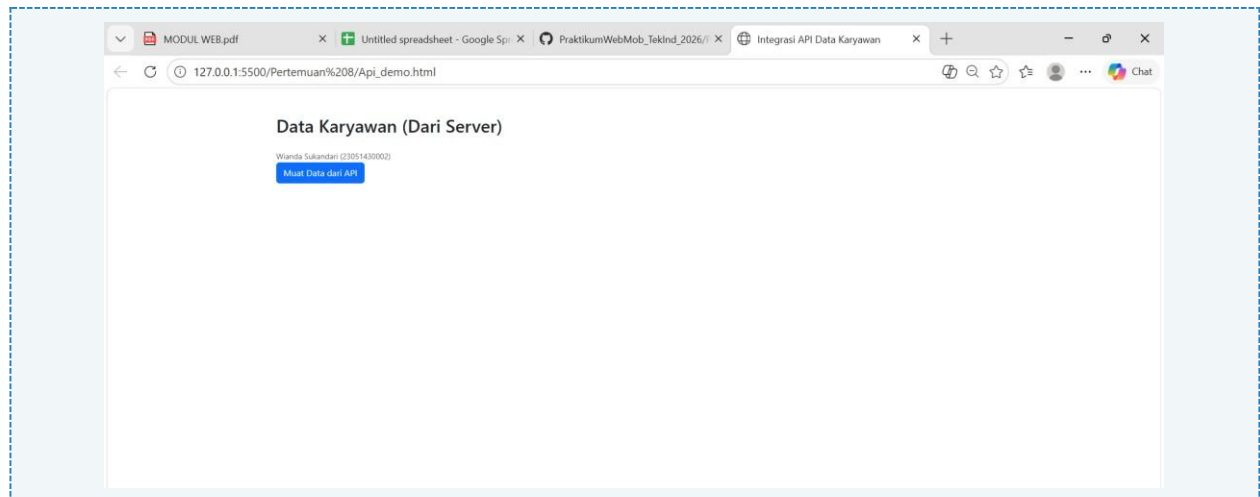
```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
<!DOCTYPE html>
<html lang="id">

<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
  <small class="text-muted">Wianda Sukandari (23051430002)</small>
  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
  <div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
  </div>
  <script src="api_script.js"></script>
</body>

</html>
```

Screenshot Hasil:



Gambar E.1 – Membuat HTML untuk Menampilkan Data

E.2 Langkah 2 – Fetch Data dengan .then() (Promise)

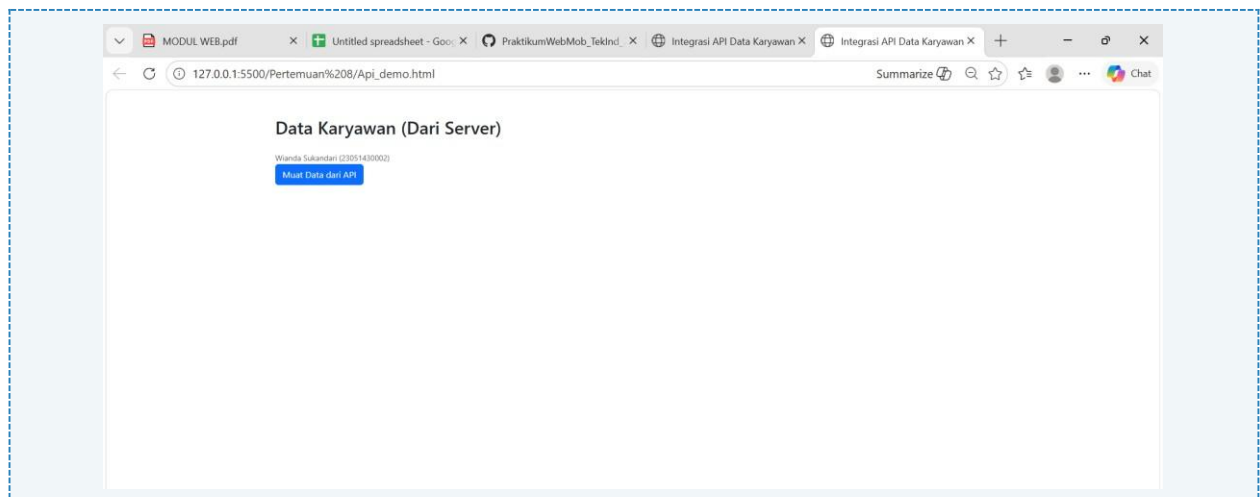
```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan'); const
loading = document.getElementById('loading');
// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';
btnLoad.addEventListener('click', function () {
  // Tampilkan loading
  loading.classList.remove('d-none');
  container.innerHTML = ''; // Bersihkan konten lama
  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299) if
      (!response.ok) {
      }
      throw new Error('Gagal mengambil data');
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      // Jika ada error (misal putus internet)
      container.innerHTML = `<div class="alert alert-danger">Error:
      ${error.message}</div>`;
    })
    .finally(function () {
      // Sembunyikan loading (baik sukses maupun gagal)
```

```
        loading.classList.add('d-none');
    });
});
function renderData(data) { data.forEach(function
(karyawan) {
    // Buat elemen card Bootstrap
    const col = document.createElement('div'); col.className =
    'col-md-4 mb-3';

    col.innerHTML = `
        <div class="card h-100 shadow-sm">
            <div class="card-body">
                <h5 class="card-title">${karyawan.name}</h5>
                <p class="card-text text-muted">Email:
    ${karyawan.email}</p>
                <p class="card-text">Perusahaan:
    ${karyawan.company.name}</p>
                <p class="card-text"><small>Kota:
    ${karyawan.address.city}</small></p>
                <a href="" class="btn btn-sm btn-outline-
primary">Detail Profil</a>
            </div>
        </div>
    `;

    container.appendChild(col);
});
}
```

Screenshot Hasil:



Gambar E.2 – Fetch Data dengan .then() (Promise)

E.3 Langkah 3 – Fetch dengan Async/Await (Modern Style)

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan'); const
loading = document.getElementById('loading');
// Endpoint API (Simulasi Database)
const API_URL = 'https://jsonplaceholder.typicode.com/users';
btnLoad.addEventListener('click', function () {
    // Tampilkan loading
    loading.classList.remove('d-none');
    container.innerHTML = ''; // Bersihkan konten lama
    // Fetch Data
    fetch(API_URL)
        .then(function (response) {
            // Cek jika response sukses (kode 200-299) if
            (!response.ok) {
            }
            throw new Error('Gagal mengambil data');
            // Parsing data JSON
            return response.json();
        })
        .then(function (dataKaryawan) {
            // Data berhasil didapat
            console.log(dataKaryawan); // Cek di console browser
            renderData(dataKaryawan);
        })
        .catch(function (error) {
            // Jika ada error (misal putus internet)
            container.innerHTML = `<div class="alert alert-danger">Error:
            ${error.message}</div>`;
        })
        .finally(function () {
            // Sembunyikan loading (baik sukses maupun gagal)
            loading.classList.add('d-none');
        });
});

function renderData(data) {
    data.forEach(function (karyawan) {
        // Buat elemen card Bootstrap
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
            ${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
            ${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:

```

```
    ${karyawan.address.city}</small></p>
        <a href="#" class="btn btn-sm btn-outline-
primary">Detail Profil</a>
    </div>
</div>
`;

    container.appendChild(col);
});
}
// Fungsi Async untuk mencari data spesifik async
function cariKaryawan(id) {
    try {
        console.log(`Mencari data ID: ${id}...`);

        const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

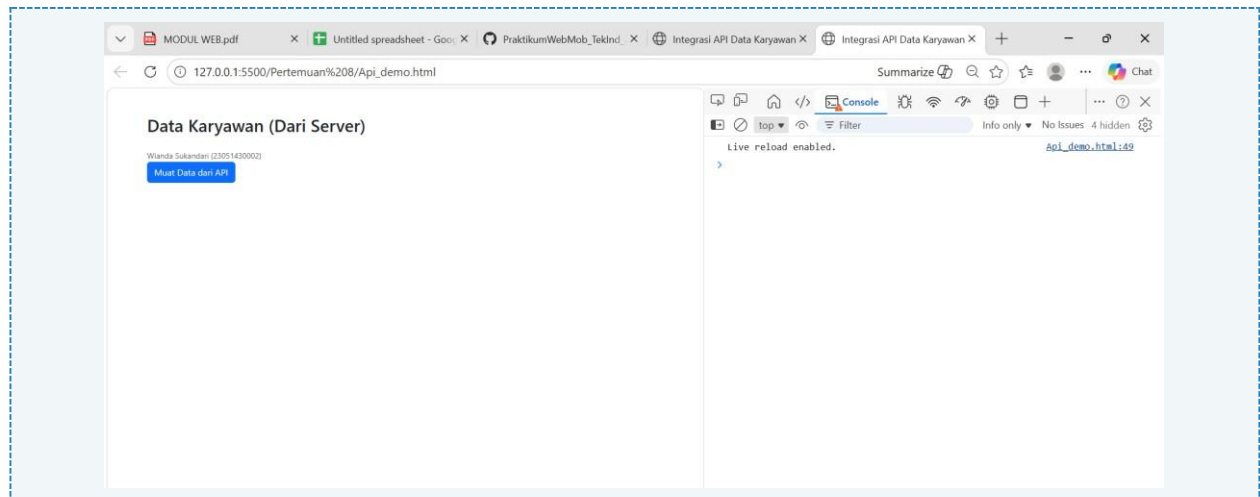
        if (!response.ok) {
            throw new Error('Data tidak ditemukan');
        }

        const data = await response.json();
        console.log("Ditemukan:", data); alert(`Ditemukan:
        ${data.name} - bekerja di
        ${data.company.name}`);

        } catch (error) { console.error(error);
        alert(error.message);
        }
    }

    // Contoh pemanggilan: cariKaryawan(2);
    // Anda bisa memanggil fungsi ini lewat console browser untuk tes.
```

Screenshot Hasil:



Gambar E.3 – Fetch dengan Async/Await (Modern Style)

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Kode HTML:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Data Karyawan</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
  <small class="text-muted">Wianda Sukandari (23051430002)</small>

  <!-- Tombol Load Data (Sudah ada) -->
  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status"></div>
  </div>
```

```

<!-- BARU: Form Tambah Karyawan -->
<div class="card mb-4 shadow-sm">
  <div class="card-header bg-success text-white">
    <h5>Tambah Karyawan Baru</h5>
  </div>
  <div class="card-body">
    <form id="formKaryawan">
      <div class="row">
        <div class="col-md-6 mb-3">
          <label class="form-label">Nama Lengkap</label>
          <input type="text" class="form-control" id="nama"
placeholder="John Doe" required>
        </div>
        <div class="col-md-6 mb-3">
          <label class="form-label">Email</label>
          <input type="email" class="form-control" id="email"
placeholder="john@example.com" required>
        </div>
      </div>
      <div class="mb-3">
        <label class="form-label">Perusahaan</label>
        <input type="text" class="form-control" id="perusahaan"
placeholder="PT. Teknologi Jaya" required>
      </div>
      <button type="submit" class="btn btn-success"> Simpan
        Karyawan Baru
      </button>
    </form>
  </div>
</div>

<div class="row" id="containerKaryawan">
  <!-- Card karyawan akan muncul di sini -->
</div>

<script src="latihan1.js"></script>
</body>
</html>

```

Kode JS:

```

// 💧 KODE LENGKAP (Copy semua, termasuk yang lama + yang baru)
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan');
const loading = document.getElementById('loading');
const formKaryawan = document.getElementById('formKaryawan');

const API_URL = 'https://jsonplaceholder.typicode.com/users';

```

```
// Event Load Data btnLoad.addEventListener('click',
function () {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function (response) { if
            (!response.ok) {
                throw new Error(`HTTP ${response.status}: Gagal mengambil
data`);
            }
            return response.json();
        })

        .then(function (dataKaryawan) {
            console.log('☑ Data Karyawan:', dataKaryawan); renderData(dataKaryawan);
        })
        .catch(function (error) { console.error('✗
Error:', error); container.innerHTML = `
<div class="alert alert-danger">
    <strong>Error:</strong> ${error.message}
</div>
`;
        })
        .finally(function () {
            loading.classList.add('d-none');
        });
});

function renderData(data) {
    if (!data || data.length === 0) {
        container.innerHTML = '<div class="alert alert-warning">Data kosong!</div>';
        return;
    }

    data.forEach(function (karyawan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
<div class="card h-100 shadow-sm">
    <div class="card-body">
        <h5 class="card-title">${karyawan.name}</h5>
        <p class="card-text text-muted">Email:
${karyawan.email}</p>
        <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
    </div>
</div>
`;
```

```

        <p class="card-text"><small>Kota:
    ${karyawan.address.city}</small></p>
        <button class="btn btn-sm btn-outline-primary"
    onclick="cariKaryawan(${karyawan.id})">
            Detail Profil
        </button>
    </div>
</div>
    `; container.appendChild(col);
});
}

async function cariKaryawan(id) { try {
    console.log(`Mencari data ID: ${id}...`); const
    response = await
    fetch(`https://jsonplaceholder.typicode.com/users/${id}`);

    if (!response.ok) {
        throw new Error(`Data ID ${id} tidak ditemukan`);
    }

    const data = await response.json(); console.log("✅
    Ditemukan:", data); alert(`Ditemukan:\n\nNama:
    ${data.name}\nEmail:
    ${data.email}\nPerusahaan: ${data.company.name}`);

    } catch (error) {
        console.error('❌ Error:', error); alert(`❌
        ${error.message}`);
    }
}

// POST KARYAWAN BARU
formKaryawan.addEventListener('submit', async function(e) {
    e.preventDefault(); // Cegah reload halaman

    // Ambil data dari form
    const nama = document.getElementById('nama').value; const
    email = document.getElementById('email').value;
    const perusahaan = document.getElementById('perusahaan').value;

    // Data dummy untuk POST (sesuai struktur JSONPlaceholder) const
    dataBaru = {
        name: nama, email:
        email, company: {
            name: perusahaan

```



```
    },
    address: {
      city: 'Jakarta' // Default
    }
  };
  console.log('Mengirim data:', dataBaru);
  try {
    // FETCH POST REQUEST
    const response = await fetch(API_URL, {
      method: 'POST', // METHOD POST
      headers: {
        'Content-Type': 'application/json', // HEADER JSON
      },
      body: JSON.stringify(dataBaru) // BODY DATA
    });

    if (!response.ok) {
      throw new Error(`HTTP ${response.status}: Gagal menyimpan`);
    }

    const result = await response.json();
    console.log('✅ POST SUCCESS!', result);

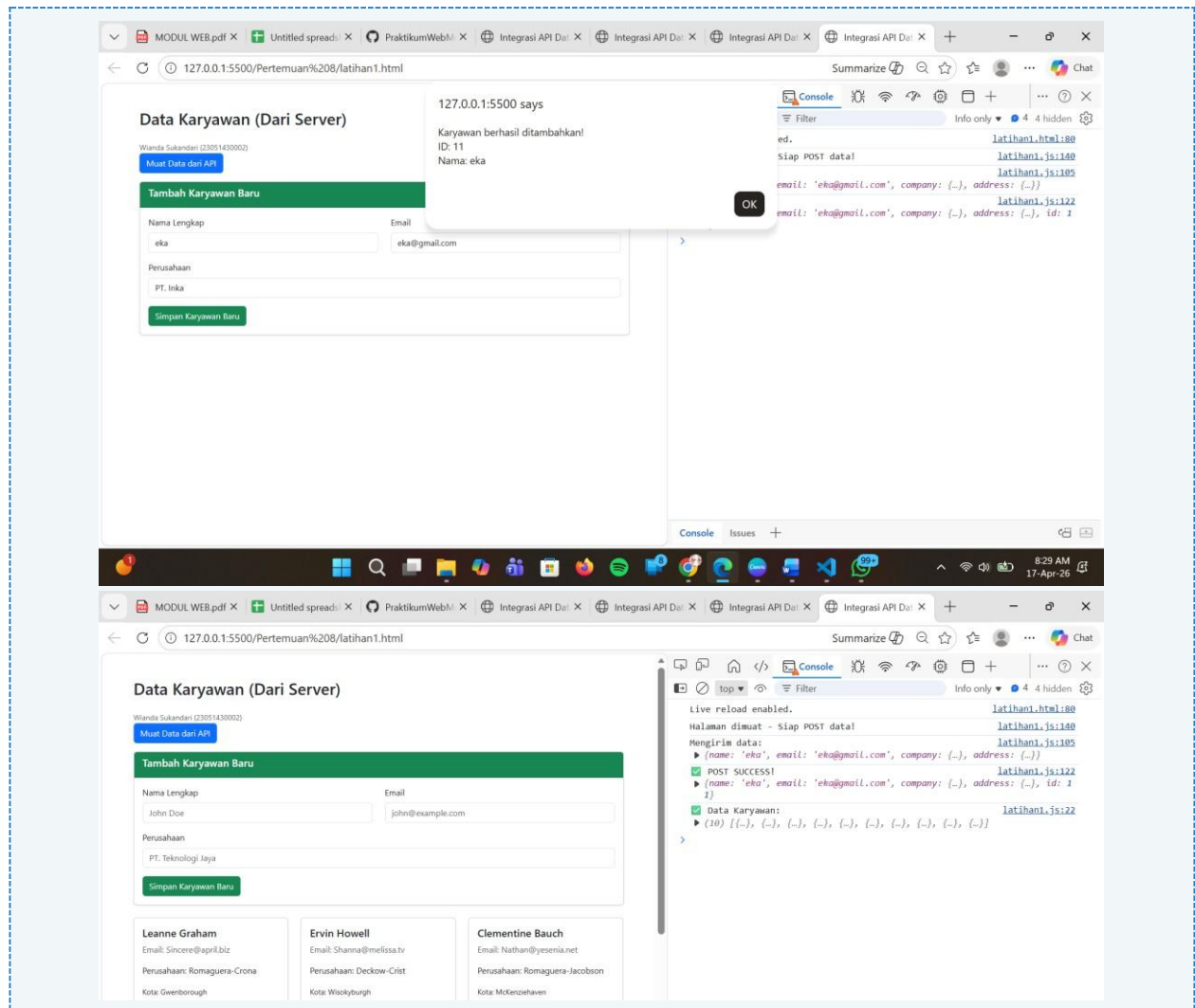
    // Tampilkan notifikasi sukses
    alert(`Karyawan berhasil ditambahkan!\nID: ${result.id}\nNama: ${result.name}`);

    // Reset form
    formKaryawan.reset();

    // Auto reload data
    btnLoad.click();

  } catch (error) {
    console.error('❌ POST Error:', error);
    alert(`❌ Gagal menyimpan: ${error.message}`);
  }
});

window.addEventListener('load', function() {
  console.log('Halaman dimuat - Siap POST data!');
});
```



Gambar F.1 – Posting Data

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

Kode HTML:

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Latihan 2 - Filter Array | Wianda Sukandari (23051430002)</title>
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>

<body class="container mt-5">
  <h2 class="mb-4">Data Karyawan - Filter Kota "s"</h2>
  <small class="text-muted">Wianda Sukandari (23051430002) - Latihan 2</small>

  <!-- Tombol Load Data -->
  <div class="mb-3">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
    <div id="loading" class="spinner-border text-primary d-none" role="status">
      <span class="visually-hidden">Loading...</span>
    </div>
  </div>

  <!-- Form Tambah Karyawan -->
  <div class="card mb-4 shadow-sm">
    <div class="card-header bg-success text-white">
      <h5>Tambah Karyawan Baru</h5>
    </div>
    <div class="card-body">
      <form id="formKaryawan">
        <div class="row">
          <div class="col-md-6 mb-3">
            <label class="form-label">Nama Lengkap</label>
            <input type="text" class="form-control" id="nama"
placeholder="John Doe" required>
          </div>
          <div class="col-md-6 mb-3">
            <label class="form-label">Email</label>
            <input type="email" class="form-control" id="email"
placeholder="john@example.com" required>
          </div>
        </div>
        <div class="mb-3">
          <label class="form-label">Perusahaan</label>
          <input type="text" class="form-control" id="perusahaan"
placeholder="PT. Teknologi Jaya" required>
        </div>
        <button type="submit" class="btn btn-success"> Simpan
          Karyawan Baru
        </button>
      </form>
    </div>
  </div>
</div>
```

```
<div class="row" id="containerKaryawan">
    <!-- Card karyawan akan muncul di sini -->
</div>

<script src="latihan2.js"></script>
</body>
</html>
```

Kode JS:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerKaryawan'); const
loading = document.getElementById('loading');
const formKaryawan = document.getElementById('formKaryawan'); const
API_URL = 'https://jsonplaceholder.typicode.com/users';
// 1. LOAD DATA DENGAN FETCH
btnLoad.addEventListener('click', function () {
    loading.classList.remove('d-none');
    container.innerHTML = '';

    fetch(API_URL)
        .then(function (response) { if
            (!response.ok) {
                throw new Error(`HTTP ${response.status}: Gagal mengambil
data`);
            }
            return response.json();
        })
        .then(function (dataKaryawan) {
            console.log('✅ Data Karyawan:', dataKaryawan); renderData(dataKaryawan);
        })
        .catch(function (error) { console.error('❌
Error:', error); container.innerHTML = `
<div class="alert alert-danger">
    <strong>Error:</strong> ${error.message}
</div>
`;
        })
        .finally(function () {
            loading.classList.add('d-none');
        });
});

// 2. LATIHAN 2: RENDER DENGAN FILTER KOTA "s"
function renderData(data) {
```

```
if (!data || data.length === 0) {
    container.innerHTML = '<div class="alert alert-warning">Data
kosong!</div>';
    return;
}

// FILTER: HANYA KOTA MENGANDUNG HURUF "s"
const dataTerefilter = data.filter(function(karyawan) {
    const kota = karyawan.address.city.toLowerCase();
    return kota.includes('s');
});

console.log(`Total data: ${data.length}`);
console.log(`Filter "s": ${dataTerefilter.length} karyawan`);
console.log('Kota:', dataTerefilter.map(k => k.address.city));

if (dataTerefilter.length === 0) {
    container.innerHTML = `
        <div class="alert alert-info text-center">
            <h5>Tidak ada karyawan</h5>
            <p>di kota yang mengandung huruf "s"</p>
        </div>
    `;
    return;
}

// Tampilkan info filter
container.insertAdjacentHTML('beforebegin', `
    <div class="alert alert-success">
        Filter aktif: ${dataTerefilter.length}/${data.length}
        karyawan dari kota dengan huruf "s"
    </div>
`);

// Render data terfilter
dataTerefilter.forEach(function (karyawan) {
    const col = document.createElement('div');
    col.className = 'col-md-6 col-lg-4 mb-3';

    col.innerHTML = `
        <div class="card h-100 shadow-sm border-success">
            <div class="card-body">
                <h5 class="card-title">${karyawan.name}</h5>
                <p class="card-text text-muted">${karyawan.email}</p>
                <p class="card-text">${karyawan.company.name}</p>
                <p class="card-text">
                    <span class="badge bg-success is-
6">${karyawan.address.city}</span>
                </p>
            </div>
        </div>
    `;
    container.appendChild(col);
});
```

```

        <button class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${karyawan.id})">
            Detail
        </button>
    </div>
</div>

`;
container.appendChild(col);
});
}

// 3. CARI KARYAWAN
async function cariKaryawan(id) {
    try {
        console.log(`Mencari ID: ${id}`);
        const response = await fetch(`${API_URL}/${id}`);
        if (!response.ok) throw new Error(`ID ${id} tidak ditemukan`);

        const data = await response.json();
        console.log('✅ Ditemukan:', data);
        alert(`${data.name}\n✉️ ${data.email}\n🏢 ${data.company.name}`);
    } catch (error) {
        console.error('❌', error);
        alert(`❌ ${error.message}`);
    }
}

// 4. POST KARYAWAN BARU
formKaryawan.addEventListener('submit', async function(e) {
    e.preventDefault();

    const nama = document.getElementById('nama').value;
    const email = document.getElementById('email').value;
    const perusahaan = document.getElementById('perusahaan').value;

    const dataBaru = {
        name: nama,
        email: email,
        company: { name: perusahaan },
        address: { city: 'Jakarta' }
    };
    console.log('POST:', dataBaru);
    try {
        const response = await fetch(API_URL, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(dataBaru)
        });
    }

```

```

});

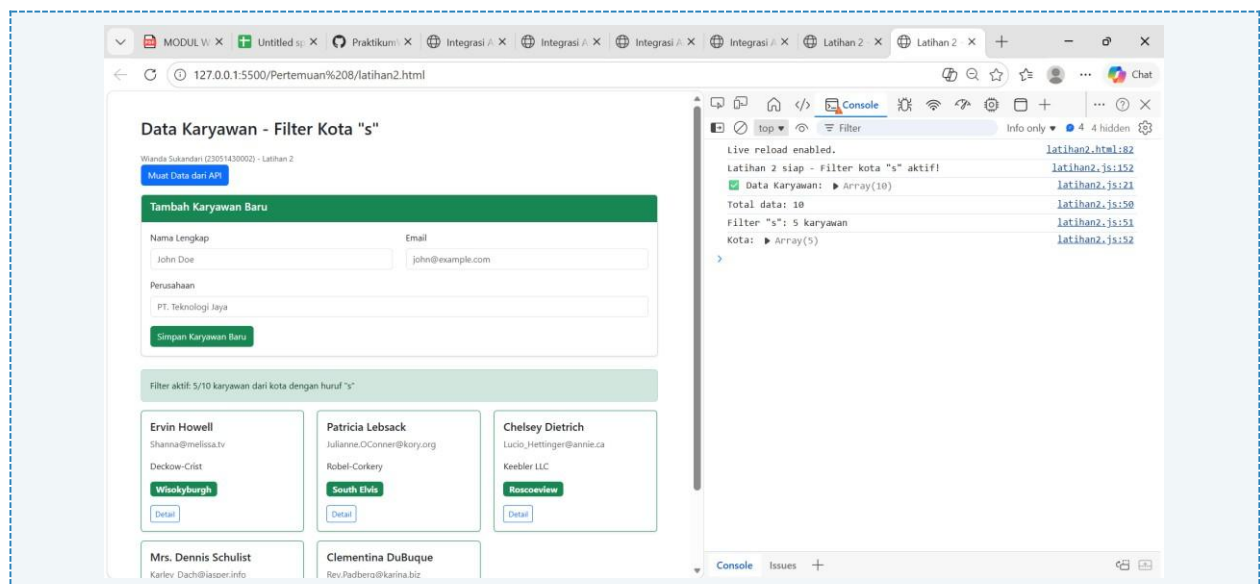
if (!response.ok) throw new Error(`HTTP ${response.status}`);

const result = await response.json();
console.log('✅ POST SUCCESS!', result);
alert(`Berhasil! ID: ${result.id}`);

formKaryawan.reset();
btnLoad.click();
} catch (error) {
  console.error('❌ POST Error:', error);
  alert(`❌ ${error.message}`);
}
});

// 5. INIT
window.addEventListener('load', function() {
  console.log('Latihan 2 siap - Filter kota "s" aktif!');
});

```



Gambar F.2 – Filter Array

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden".

- a. Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
- b. Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()`

menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

Saya ambil data dari website jsonplaceholder.typicode.com/posts pake Fetch API. Data aslinya isinya acak-acakan bahasa Latin, jadi saya bikin 10 cerita laporan insiden Indo sendiri di array `dataInsidenWianda`, trus ganti-ganti judul sama deskripsinya pake `.map()`. Tampilannya pake Bootstrap card ukuran `col-md-4` (3 kolom rapi), tiap card punya ID unik biar bisa diubah-ubah. Tombol "Tindak Lanjut" kalo diklik keluar alert "Tiket ID X diproses Tim Maintenance" (sesuai tugas), cardnya jadi kuning, tombol hijau + mati. Ada loading muter-muter, error kalo gagal, sama langsung muncul pas buka halaman.

Kode Utama:

```
Kode HTML:
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Integrasi API Laporan Insiden</title>
  <link
href="https://cdn.jsdelivrivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <style>
    .transition-all { transition: all 0.3s ease; }
  </style>
</head>

<body class="container mt-5">
  <h2 class="mb-4">Data Laporan Insiden (Database Produksi)</h2>
  <small class="text-muted d-block mb-3">Wianda Sukandari (23051430002) -
Proyek Mini</small>

  <div class="mb-3 d-flex align-items-center gap-2">
    <button id="btnLoad" class="btn btn-primary">Muat Data dari
API</button>
    <div id="loading" class="spinner-border text-primary d-none"
role="status">
      <span class="visually-hidden">Loading...</span>
    </div>
  </div>

  <div class="row" id="containerLaporan">
    <!-- Laporan akan muncul di sini -->
```

```
</div>

<script src="proyek_mini.js"></script>
</body>
</html>
```

Kode JS:

```
const btnLoad = document.getElementById('btnLoad');
const container = document.getElementById('containerLaporan');
const loading = document.getElementById('loading');

const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// 1. DATA SIMULASI
const dataInsidenWianda = [
  { title: "Pompa Air Toren Rusak", body: "Pompa air toren utama berhenti berputar, tekanan air menurun drastis diseluruh lantai." },
  { title: "Kamera CCTV Pintu Depan Offline", body: "Kamera CCTV di pintu masuk utama tidak menampilkan gambar, kabel diduga longgar." },
  { title: "Door Lock Ruang Server Error", body: "Kunci pintu otomatis ruang server tidak merespon kartu akses, manual lock digunakan." },
  { title: "UPS Kantor Habis Baterai", body: "UPS ruang direksi mati mendadak saat listrik padam, data hilang tidak tersimpan." },
  { title: "Wireless AP Lantai 3 Mati", body: "Access Point WiFi lantai 3 tidak berfungsi, sinyal hilang di area meeting room." },
  { title: "Projector Ruang Training Rusak", body: "Proyektor ruang pelatihan menampilkan gambar buram dan tidak fokus setelah digunakan." },
  { title: "Telepon IP Reception Error", body: "Telepon IP di meja resepsionis tidak bisa menerima panggilan masuk, tombol dial tidak responsif." },
  { title: "Vending Machine Kantin Macet", body: "Mesin penjual otomatis kantin tidak mengeluarkan barang setelah pembayaran dilakukan." },
  { title: "Peralatan Fitness Gym Rusak", body: "Treadmill no.2 di gym karyawan berhenti tiba-tiba dan mengeluarkan bau gosong." },
  { title: "Audio System Aula Bermasalah", body: "Speaker utama aula tidak mengeluarkan suara saat pengujian soundcheck pagi ini." }
];

btnLoad.addEventListener('click', function () {
  loading.classList.remove('d-none');
  container.innerHTML = '';

  fetch(API_URL)
    .then(function (response) {
      if (!response.ok) throw new Error('Gagal mengambil data laporan insiden');

      return response.json();
    })
  })
```

```
.then(function (dataLaporan) {
```

```

// 2. GANTI isi API
const sepuluhLaporanWianda = dataLaporan.slice(0, 10).map((item,
index) => {
    return {
        id: item.id,
        title: dataInsidenWianda[index].title,
        body: dataInsidenWianda[index].body,
        userId: item.userId
    };
});

console.log(' 10 Laporan Wianda:', sepuluhLaporanWianda);
renderData(sepuluhLaporanWianda);
})
.catch(function (error) {
    console.error('✖ Error:', error);
    container.innerHTML = `<div class="alert alert-danger w-
100">Error: ${error.message}</div>`;
})
.finally(function () {
    loading.classList.add('d-none');
});
});

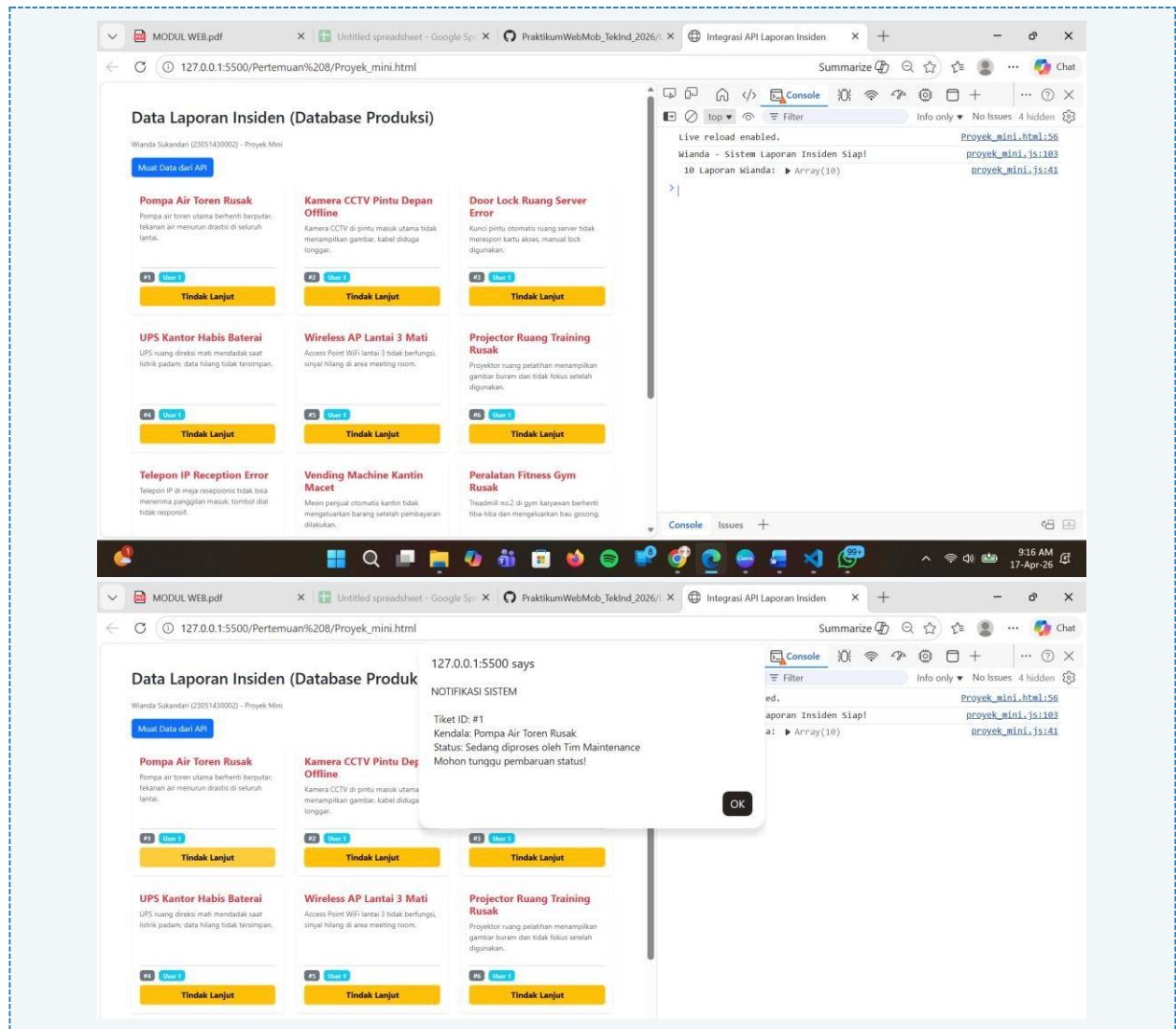
function renderData(data) {
    data.forEach(function (laporan) {
        const col = document.createElement('div');
        col.className = 'col-md-4 mb-3';

        col.innerHTML = `
            <div id="card-${laporan.id}" class="card h-100 shadow-sm border-0
            bg-white transition-all">
                <div class="card-body d-flex flex-column">
                    <h5 class="card-title fw-bold text-danger mb-
                    2">${laporan.title}</h5>
                    <p class="card-text text-muted flex-grow-1
                    small">${laporan.body}</p>
                    <hr class="my-2">
                    <div class="d-flex gap-2">
                        <span class="badge bg-secondary">
                        #${laporan.id}</span>
                        <span class="badge bg-info"> User
                        ${laporan.userId}</span>
                    </div>
                    <button id="btn-${laporan.id}" class="btn btn-warning mt-2
                    fw-bold w-100" onclick="tindakLanjut('${laporan.id}', '${laporan.title}')">
                        Tindak Lanjut
                    </button>

```

```
</div>  
</div>
```

```
`;  
  
    container.appendChild(col);  
  });  
}  
  
// 4. ALERT + EFEK VISUAL  
function tindakLanjut(id, judul) {  
  // ALERT LENGKAP  
  alert(`NOTIFIKASI SISTEM\n\n Tiket ID: #${id}\n Kendala: ${judul}\n\  
Status: Sedang diproses oleh Tim Maintenance\n Mohon tunggu pembaruan  
status!`);  
  
  // Update card & button  
  const cardElement = document.getElementById(`card-${id}`);  
  const btnElement = document.getElementById(`btn-${id}`);  
  
  if (cardElement && btnElement) {  
    // Card jadi kuning muda  
    cardElement.classList.remove('bg-white');  
    cardElement.classList.add('bg-warning', 'bg-opacity-10', 'border-  
warning');  
  
    // Button jadi hijau  
    btnElement.classList.remove('btn-warning');  
    btnElement.classList.add('btn-success');  
    btnElement.innerHTML = 'Sedang Diproses';  
    btnElement.disabled = true;  
  }  
}  
  
// Auto load  
window.addEventListener('load', function() {  
  console.log('Wianda - Sistem Laporan Insiden Siap!');  
});
```



Gambar F.3 – Integrasi Data Produksi

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil proyek yang saya kembangkan menunjukkan bahwa konsep *Asynchronous JavaScript* dapat diterapkan dengan cukup baik, terutama dalam proses pengambilan data dari server eksternal menggunakan *Fetch API*. Penggunaan *Promise chaining* melalui `.then()`, `.catch()`, dan `.finally()` membuat alur eksekusi menjadi lebih terstruktur, sehingga data yang diterima dapat langsung diolah dan ditampilkan dalam bentuk kartu (*card*) berbasis Bootstrap. Selain itu, mekanisme *non-blocking I/O* terlihat dari munculnya indikator loading tanpa menghambat interaksi pengguna, yang menandakan bahwa *event loop* berjalan sesuai dengan prinsip dasar asynchronous.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Data API berbahasa Latin (Lorem Ipsum)	Buat array <code>dataInsidenWianda[10]</code> realistis Indo + <code>.map()</code> ganti title/body
Card tidak bisa diubah per ID setelah klik	Tambah <code>id="card-\${id}"</code> & <code>id="btn-\${id}"</code> <code>getElementById()</code> spesifik
Loading langsung selesai (terlalu cepat)	Tambah <code>setTimeout(1500ms)</code> simulasi network delay realistis

G.3 Kaitan dengan Penerapan di Industri

Penerapan *Fetch API* dan *Asynchronous JavaScript* memiliki relevansi yang tinggi dalam konteks industri, khususnya pada sistem monitoring dan maintenance berbasis digital. Dalam skenario operasional pabrik, teknologi ini dapat dimanfaatkan untuk membangun dashboard pelaporan kerusakan mesin secara real-time. Ketika terjadi gangguan, operator dapat langsung mengirimkan data ke server melalui metode *POST*, yang kemudian dapat diakses oleh tim maintenance melalui *GET request* pada perangkat seperti tablet atau smartphone. Mekanisme asynchronous memungkinkan proses ini berjalan tanpa menghambat antarmuka pengguna, sementara elemen seperti *loading indicator* dan notifikasi memberikan umpan balik yang jelas selama proses berlangsung.

Namun demikian, penerapan teknologi ini di lingkungan industri tidak hanya bergantung pada kemudahan implementasi, tetapi juga menuntut keandalan sistem yang lebih tinggi. Integrasi

dengan perangkat IoT, seperti sensor suhu atau getaran mesin, memang berpotensi meningkatkan kecepatan deteksi gangguan dan mengurangi downtime. Akan tetapi, aspek seperti stabilitas jaringan, keamanan data, serta validasi input menjadi faktor krusial yang harus diperhatikan. Tanpa pengelolaan yang matang, sistem justru berisiko menghasilkan data yang tidak akurat atau terlambat diproses. Oleh karena itu, meskipun teknologi ini menawarkan efisiensi operasional yang signifikan, implementasinya tetap perlu dirancang secara komprehensif agar benar-benar memberikan dampak optimal dalam praktik industri.

H. KESIMPULAN

1	Fetch API berhasil mengambil data JSON dari server eksternal dan dirender jadi 10 card laporan insiden responsif col-md-4
2	Promise chaining <code>.then().catch().finally()</code> handle async operation dengan loading spinner + error alert merah
3	DOM manipulation via <code>id="card-\${id}"</code> memungkinkan update visual spesifik (card kuning, tombol hijau-disabled)
4	Data mapping <code>.map()</code> ubah API Latin jadi 10 laporan Indonesia realistis sambil pertahankan ID asli
5	Callback onclick trigger <code>tindakLanjut(id,title)</code> hasilkan alert "Tiket ID X diproses Tim Maintenance" sesuai requirement

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	MDN Web Docs. (2024). <i>Using Fetch</i> . Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
[3]	MDN Web Docs. (2024). <i>Using promises</i> . Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
[4]	JSONPlaceholder. (n.d.). <i>Free fake REST API</i> . JSONPlaceholder. https://jsonplaceholder.typicode.com/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
Wianda Sukandari NIM: 23051430002	Nanik Wijayanti NIM: 23051430006	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

MODUL 9**Pengenalan React.js (Component, JSX, Props)****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 9)
Topik Utama	Pengenalan React.js (Component, JSX, Props)
Sub-Topik	Setup Lingkungan React, Konsep Component, JSX Syntax, dan Props (Data Passing).
Durasi Praktikum	340 Menit
Tanggal Praktikum	24 April 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep Library vs Framework dan mengapa React dipilih untuk aplikasi skala besar.
2	Mampu menyiapkan environment pengembangan React menggunakan Vite.
3	Memahami sintaks JSX (JavaScript XML).
4	Mampu membuat Functional Component yang menerima dan menampilkan data melalui Props.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React.js	Versi terbaru (via Vite)	Library Frontend
5	Vite	Versi terbaru	Build Tool / Development Server

D. DASAR TEORI

D.1 Konsep Utama

Modul ini membahas pengenalan React.js sebagai library JavaScript yang digunakan untuk membangun antarmuka pengguna (UI) pada aplikasi web. Berbeda dengan framework yang cenderung lebih kompleks dan mengatur banyak aspek sekaligus, React lebih fokus pada pengelolaan tampilan. Hal ini membuat React cukup fleksibel untuk dikombinasikan dengan teknologi lain sesuai kebutuhan pengembangan aplikasi.

Salah satu konsep utama dalam React adalah component. Tampilan aplikasi tidak dibuat sebagai satu kesatuan besar, melainkan dipecah menjadi bagian-bagian kecil yang lebih sederhana, seperti header, card, atau button. Setiap komponen dapat digunakan kembali (*reusable*), sehingga proses pengembangan menjadi lebih efisien dan terstruktur. Dengan pendekatan ini, perubahan pada satu bagian tidak akan terlalu memengaruhi bagian lain secara signifikan.

Selain itu, terdapat konsep JSX (JavaScript XML) yang memungkinkan penulisan sintaks mirip HTML di dalam JavaScript. Hal ini mempermudah pengembang dalam memahami struktur tampilan. React juga menggunakan props sebagai mekanisme untuk mengirim data antar komponen, khususnya dari parent ke child. Dengan alur data yang jelas, pengelolaan informasi dalam aplikasi menjadi lebih terorganisir.

D.2 Keterkaitan dengan Teknik Industri

Dalam konteks Teknik Industri, penggunaan React memiliki relevansi yang cukup tinggi, terutama pada pengembangan sistem informasi berbasis web di lingkungan industri. Banyak proses industri saat ini bergantung pada dashboard digital untuk memantau kinerja produksi, kondisi mesin, maupun data

operasional lainnya. React dapat dimanfaatkan untuk membangun tampilan dashboard yang interaktif, responsif, dan mudah diperbarui.

Sebagai contoh, pada sistem monitoring mesin di pabrik, data dari sensor seperti suhu, tekanan, atau getaran dapat ditampilkan secara real-time melalui dashboard berbasis React. Dengan konsep komponen, setiap bagian tampilan seperti grafik, tabel, atau notifikasi dapat dikelola secara terpisah namun tetap terintegrasi. Hal ini mempermudah pengembangan dan pemeliharaan sistem dalam jangka panjang.

Namun, penerapan React di dunia industri juga perlu dipertimbangkan secara kritis. Meskipun menawarkan kemudahan dalam pengembangan UI, penggunaan React membutuhkan pemahaman teknis yang cukup serta integrasi yang baik dengan sistem backend yang sudah ada. Selain itu, jika tidak dirancang dengan baik, penggunaan banyak komponen justru dapat meningkatkan kompleksitas aplikasi. Oleh karena itu, implementasinya harus disesuaikan dengan kebutuhan dan kapasitas sumber daya yang tersedia.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Project dengan Vite

Deskripsi singkat apa yang dilakukan pada langkah ini:

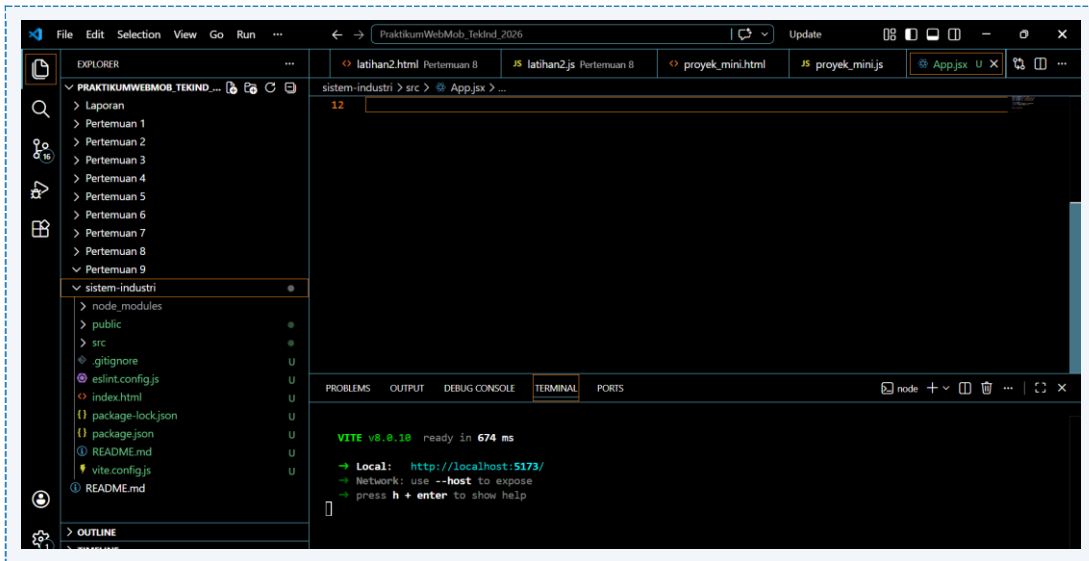
```
/* ===== Buka VSCODE, Buat Project Baru 'sistem-industri' ===== */
Ketik ctrl + ` nanti akan muncul tampilan terminal. Kemudian
ketik kode dibawah:
npm create vite@latest sistem-industri -- --template react

/* ===== Masuk ke Folder Project ===== */
cd sistem-industri

/* ===== Install dependensi library ===== */
npm install

/* ===== Jalankan development server ===== */
npm run dev
```

Screenshot Hasil:

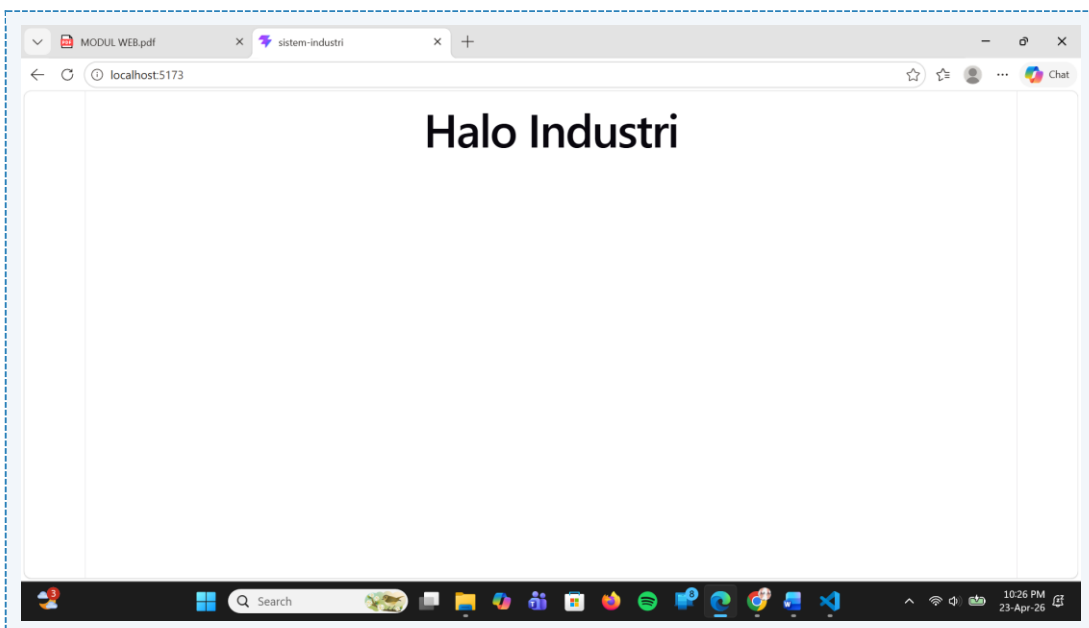


Gambar E.1 – Setup Project dengan Vite

E.2 Langkah 2 – Membersihkan Project

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */  
import { useState } from 'react'  
import reactLogo from './assets/react.svg'  
import viteLogo from './assets/vite.svg'  
import heroImg from './assets/hero.png'  
  
function App() {  
  const [count, setCount] = useState(0)  
  return <h1>Halo Industri</h1>  
}  
  
export default App
```

Screenshot Hasil:



Gambar E.2 – Membersihkan Project

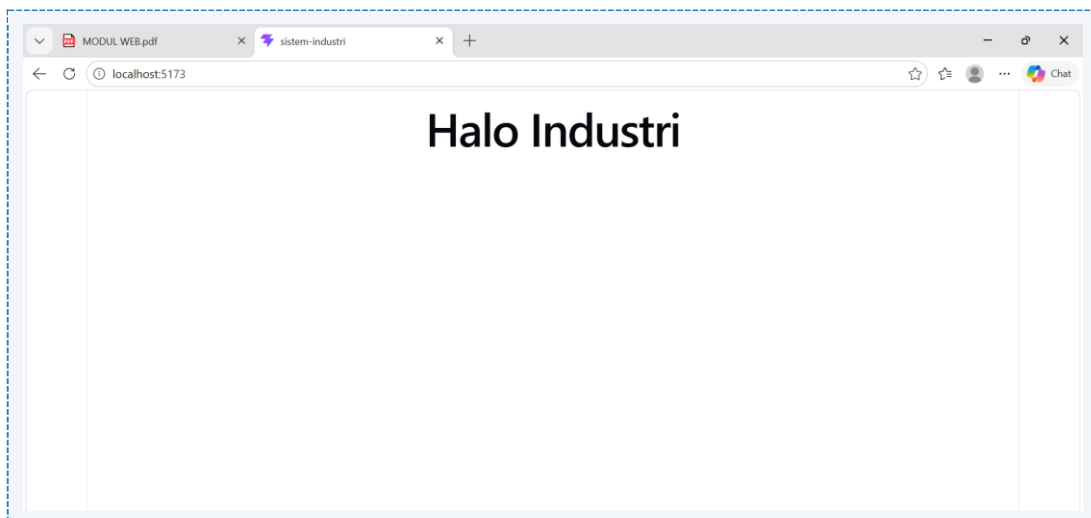
E.3 Langkah 3 – Membuat Komponen Pertama: KartuMesin

```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
// 1. Import React
import React from 'react';
// 2. Membuat Komponen Fungsi yang menerima 'props'
function KartuMesin(props) {
  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;
  // Logika penentuan warna badge berdasarkan status
  let badgeColor = 'bg-secondary';
  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';
  return (
    // Menggunakan class -> className di JSX
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>
        <span className={`badge
${badgeColor}`}>{status}</span>
        <hr />
        <p>Produksi Saat Ini:
<strong>{produksi}</strong> Unit</p>
      </div>
    </div>
  );
}
// 3. Export agar bisa dipakai di file lain
export default KartuMesin;

```

Screenshot Hasil:



Gambar E.3 – Membuat Komponen Pertama: KartuMesin

E.4 Langkah 4 – Menggunakan Komponen di App.jsx


```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
import { useState } from 'react'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
import KartuMesin from './Komponen/KartuMesin';

function App() {
  return (
    <div>
      <h1>Monitoring Lini Produksi</h1>
      <small class="text-muted d-block mb-3">Wianda Sukandari
(23051430002)</small>

      <KartuMesin
        nama="CNC-Turning-01"
        status="Running"
        produksi={150}
      />

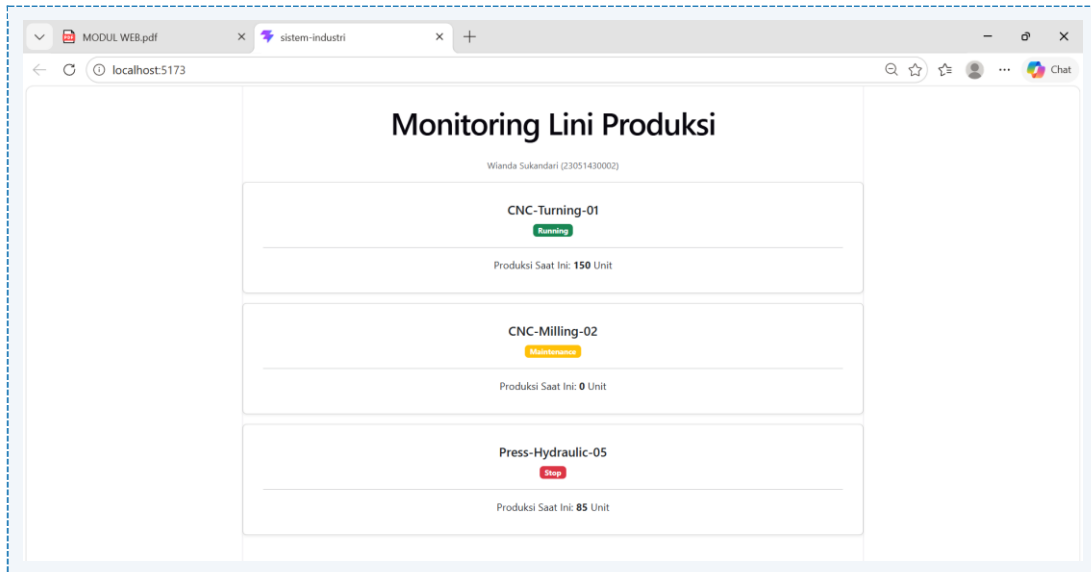
      <KartuMesin
        nama="CNC-Milling-02"
        status="Maintenance"
        produksi={0}
      />

      <KartuMesin
        nama="Press-Hydraulic-05"
        status="Stop"
        produksi={85}
      />

    </div>
  );
}

export default App
```

Screenshot Hasil:



Gambar E.4 – Menggunakan Komponen di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Destructuring Props

Pertanyaan / Instruksi Latihan:

Ubah cara deklarasi props di `KartuMesin.jsx` menggunakan destructuring: `function KartuMesin({ nama, status, produksi }) { ... }`. Sesuaikan kode di dalamnya.

Jawaban / Kode Solusi:

```
import React from 'react';
import KartuMesin from '../Komponen/KartuMesin';

function Latihan1() {
  return (
    <div>
      <h2>Latihan 1 - Destructuring Props</h2>

      <KartuMesin
        nama="CNC-Turning-01"
        status="Running"
        produksi={150}
      />

      <KartuMesin
        nama="CNC-Milling-02"
        status="Maintenance"
        produksi={0}
      />

      <KartuMesin
```

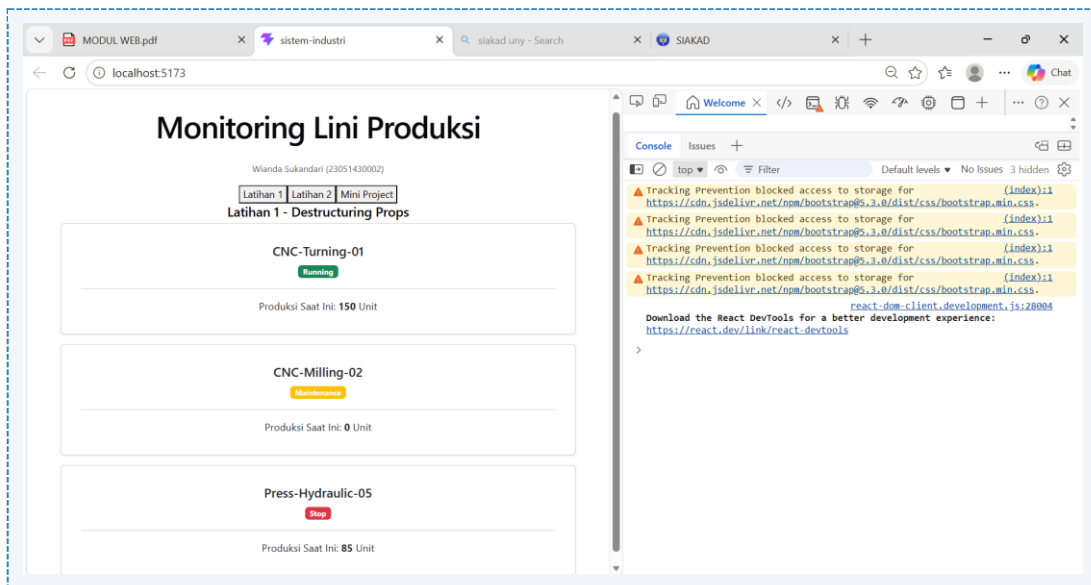
```

        nama="Press-Hydraulic-05"
        status="Stop"
        produksi={85}
      />

    </div>
  );
}

export default Latihan1;

```



Gambar F.1 – Destructuring Props

F.2 Latihan 2 – Default Props

Pertanyaan / Instruksi Latihan:

Berikan nilai default pada prop `produksi` menjadi 0, jika user lupa mengirim prop tersebut.

Jawaban / Kode Solusi:

```

Ubah isi KartuMesin.jsx

import React from 'react';

function KartuMesin({ nama, status, produksi = 0 }) {

  let badgeColor = 'bg-secondary';
  if (status === 'Running') badgeColor = 'bg-success';
  if (status === 'Stop') badgeColor = 'bg-danger';
  if (status === 'Maintenance') badgeColor = 'bg-warning';

  return (

```

```
        <div className="card shadow-sm p-3 mb-3">
          <div className="card-body">
            <h5 className="card-title">{nama}</h5>
            <span className={`badge
${badgeColor}`}>{status}</span>
            <hr />
            <p>Produksi Saat Ini:
<strong>{produksi}</strong> Unit</p>
          </div>
        </div>
      );
    }

export default KartuMesin;
```

Isi Latihan2.jsx

```
import React from 'react';
import KartuMesin from '../Komponen/KartuMesin';

function Latihan2() {
  return (
    <div className="container mt-3">

      <h2>Latihan 2 - Default Props</h2>
      <small className="text-muted d-block mb-3">
        Wianda Sukandari (23051430002)
      </small>

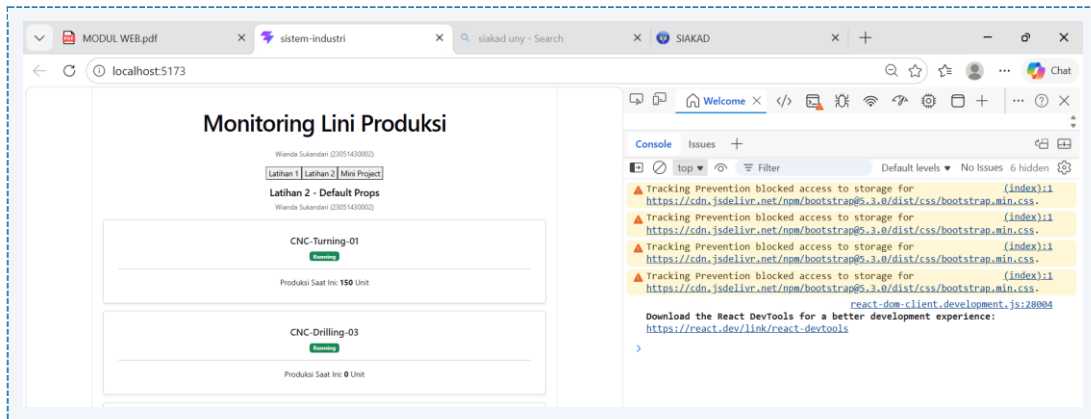
      {/* Lengkap */}
      <KartuMesin
        nama="CNC-Turning-01"
        status="Running"
        produksi={150}
      />

      {/* TANPA produksi → harus jadi 0 */}
      <KartuMesin
        nama="CNC-Drilling-03"
        status="Running"
      />

      {/* TANPA produksi → harus jadi 0 */}
      <KartuMesin
        nama="Laser-Cutting-04"
        status="Maintenance"
      />

    </div>
  );
}

export default Latihan2;
```



Gambar F.2 – Default Props

F.3 Tugas Proyek Mini – Kartu Karyawan

Deskripsi proyek mini:

1. Buat komponen baru bernama `KartuKaryawan.jsx`. Komponen ini menerima props: `nama`, `jabatan`, `bagian`.
2. Tampilkan komponen ini 3 kali di `App.jsx` dengan data karyawan berbeda (Manager, Operator, QC).

Penjelasan pendekatan/solusi yang digunakan:

Di mini project ini, saya bikin satu komponen React bernama `KartuKaryawan.jsx` yang dipakai untuk menampilkan data karyawan.

Komponen ini menerima data lewat props, yaitu nama, jabatan, dan bagian. Jadi, daripada bikin tampilan baru berkali-kali, saya cukup pakai satu komponen yang sama tapi diisi data yang berbeda-beda.

Di `App.jsx`, komponen `KartuKaryawan` dipanggil sebanyak 3 kali dengan data yang berbeda, yaitu Manager, Operator, dan QC. Dengan cara ini, satu komponen bisa dipakai berulang (reusable) dan bikin kode jadi lebih rapi.

Selain itu, saya juga pakai `useState` untuk mengatur tampilan halaman, jadi bisa pindah antara Latihan 1, Latihan 2, dan Mini Project tanpa reload halaman.

Dengan pendekatan ini, kodenya jadi lebih sederhana, mudah dibaca, dan gampang dikembangkan lagi kalau nanti mau ditambah fitur baru.

Kode Utama:

Isi KartuKaryawan.jsx

```
import React from 'react';

function KartuKaryawan({ nama, jabatan, bagian }) {
  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{nama}</h5>
        <p className="mb-1"><strong>Jabatan:</strong>
{jabatan}</p>
        <p className="mb-0"><strong>Bagian:</strong>
{bagian}</p>
      </div>
    </div>
  );
}

export default KartuKaryawan;
```

Isi App.jsx

```
import { useState } from 'react';
import Latihan1 from './Latihan/Latihan1';
import Latihan2 from './Latihan/Latihan2';
import KartuKaryawan from './Latihan/KartuKaryawan';

function App() {
  const [halaman, setHalaman] = useState('latihan1');

  return (
    <div className="container mt-3">

      <h1>Monitoring Lini Produksi</h1>
      <small className="text-muted d-block mb-3">
        Wianda Sukandari (23051430002)
      </small>

      <button onClick={() => setHalaman('latihan1')}>Latihan
1</button>
      <button onClick={() => setHalaman('latihan2')}>Latihan
2</button>
      <button onClick={() => setHalaman('mini')}>Mini
Project</button>

      <hr />

      {halaman === 'latihan1' && <Latihan1 />}
      {halaman === 'latihan2' && <Latihan2 />}

      {halaman === 'mini' && (
        <div>
          <h2>Mini Project - Kartu Karyawan</h2>
```

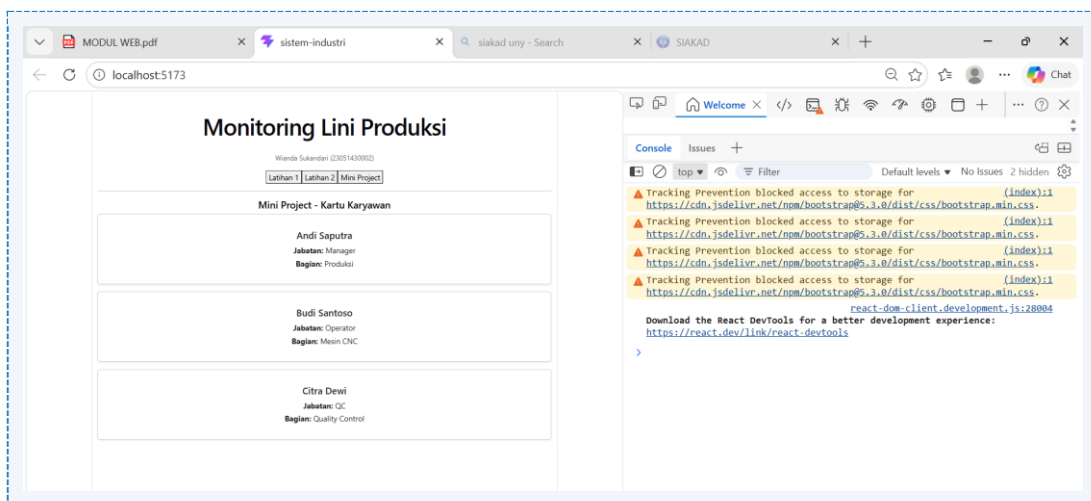
```
<KartuKaryawan
  nama="Andi Saputra"
  jabatan="Manager"
  bagian="Produksi"
/>

<KartuKaryawan
  nama="Budi Santoso"
  jabatan="Operator"
  bagian="Mesin CNC"
/>

<KartuKaryawan
  nama="Citra Dewi"
  jabatan="QC"
  bagian="Quality Control"
/>
</div>
})

</div>
);
}

export default App;
```



Gambar F.3 – Kartu Karyawan

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil yang saya peroleh sudah sesuai dengan konsep yang dipelajari pada modul, yaitu penggunaan komponen, props, destructuring, dan default props pada React.

Pada Latihan 1, saya menggunakan konsep destructuring props pada komponen KartuMesin, sehingga data seperti nama, status, dan produksi bisa langsung digunakan tanpa perlu memanggil props.nama secara berulang. Hasil di browser menunjukkan bahwa data setiap mesin berhasil ditampilkan dengan benar sesuai input yang diberikan.

Pada Latihan 2, saya menerapkan default props dengan memberikan nilai awal produksi = 0. Hal ini terbukti berjalan dengan baik karena saat ada komponen yang tidak diberikan nilai produksi, sistem otomatis menampilkan nilai 0. Ini sesuai dengan teori bahwa default props digunakan untuk menghindari error atau nilai undefined.

Pada mini project, saya membuat komponen KartuKaryawan yang menerima props nama, jabatan, dan bagian, lalu menampilkannya sebanyak tiga kali dengan data yang berbeda. Hasil output menunjukkan bahwa satu komponen dapat digunakan berulang (reusable), sesuai dengan konsep dasar React.

Selain itu, penggunaan useState pada App.jsx juga berjalan sesuai teori, di mana state digunakan untuk mengatur tampilan halaman (Latihan 1, Latihan 2, dan Mini Project) secara dinamis tanpa perlu reload halaman.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Error class tidak valid di JSX	Mengganti class menjadi className karena JSX mengikuti aturan JavaScript
Error import seperti <Latihan2> atau path salah	Memperbaiki syntax import menjadi import Latihan2 from './Latihan/Latihan2';
Data tidak muncul atau undefined pada produksi	Menambahkan default props produksi = 0 di komponen

G.3 Kaitan dengan Penerapan di Industri

Materi yang dipelajari pada modul ini sangat relevan dengan penerapan di dunia industri, khususnya dalam pembuatan sistem monitoring atau dashboard produksi.

Sebagai contoh, dalam konteks Teknik Industri, komponen seperti KartuMesin bisa digunakan untuk menampilkan status mesin di pabrik secara real-time, seperti apakah mesin sedang berjalan (Running), berhenti (Stop), atau dalam perawatan (Maintenance). Dengan menggunakan konsep reusable component, sistem dapat menampilkan banyak mesin tanpa harus membuat tampilan baru satu per satu.

Selain itu, penggunaan props memungkinkan data dari sistem produksi (misalnya dari sensor atau database) bisa langsung ditampilkan ke dalam dashboard. Sedangkan default props berguna untuk menghindari error jika ada data yang belum tersedia.

Pada mini project KartuKaryawan, konsep ini juga bisa diterapkan untuk menampilkan data karyawan di perusahaan, seperti jabatan dan bagian kerja. Ini bisa dikembangkan menjadi sistem HR atau monitoring tenaga kerja.

Dengan demikian, keterampilan ini sangat berguna untuk membangun sistem informasi yang terstruktur, efisien, dan mudah dikembangkan dalam lingkungan industri.

H. KESIMPULAN

1	Penggunaan komponen pada React seperti KartuMesin dan KartuKaryawan memudahkan pembuatan tampilan karena bisa digunakan berulang (reusable) tanpa perlu menulis kode yang sama berkali-kali.
2	Penerapan destructuring props pada Latihan 1 membuat penulisan kode lebih ringkas dan mudah dibaca dibandingkan menggunakan props.nama, props.status, dan lainnya secara berulang.
3	Penerapan default props pada Latihan 2 terbukti efektif untuk menghindari nilai undefined, sehingga ketika data produksi tidak diberikan, sistem otomatis menampilkan nilai 0.
4	Penggunaan state (useState) pada App.jsx memungkinkan perpindahan tampilan antar Latihan 1, Latihan 2, dan Mini Project secara dinamis tanpa perlu reload halaman.
5	praktikum ini membantu memahami konsep dasar React seperti komponen, props, state, dan modularisasi kode yang sangat penting dalam pengembangan aplikasi berbasis web.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Meta Platforms, Inc. (2024). <i>React documentation</i> . https://react.dev/
[3]	Mozilla Developer Network. (2024). <i>JavaScript guide</i> . https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
[4]	Mozilla Developer Network. (2024). <i>Introduction to HTML</i> . https://developer.mozilla.org/en-US/docs/Web/HTML

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 10**State, Hooks (useState, useEffect), dan Conditional Rendering****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 10)
Topik Utama	State, Hooks (useState, useEffect), dan Conditional Rendering
Sub-Topik	State Management, Lifecycle Hooks, dan Rendering Bersyarat
Durasi Praktikum	340 Menit
Tanggal Praktikum	26 April 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami apa itu State dan perbedaannya dengan Props.
2	Menguasai penggunaan `useState` untuk menyimpan data yang berubah-ubah.
3	Memahami `useEffect` untuk menangani side effects (seperti timer atau API call).
4	Mampu melakukan Conditional Rendering (menyembunyikan/menampilkan elemen berdasarkan kondisi).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React.js	React 18+	Library utama untuk membangun antarmuka aplikasi web
5	Project React dari pertemuan 9		

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini, konsep utama yang dipelajari adalah penggunaan state dan hooks pada React, khususnya useState dan useEffect. State merupakan data yang dapat berubah di dalam sebuah komponen dan digunakan untuk mengatur tampilan yang bersifat dinamis. Berbeda dengan props yang hanya menerima data dari luar, state dapat diubah langsung oleh komponen itu sendiri sehingga React akan melakukan re-render secara otomatis saat nilainya berubah.

Hook useState digunakan untuk membuat dan mengelola state pada komponen fungsional. Dengan hook ini, programmer dapat menyimpan nilai seperti angka counter, status tombol, atau data input pengguna. Ketika nilai state diperbarui menggunakan fungsi setter, tampilan antarmuka akan ikut berubah tanpa perlu me-refresh halaman secara manual. Hal ini membuat aplikasi menjadi lebih interaktif dan efisien.

Selain itu, useEffect digunakan untuk menjalankan proses tertentu saat komponen pertama kali ditampilkan (mount), saat terjadi perubahan data (update), maupun saat komponen dihapus (unmount). Dalam praktikum ini, penerapan state terlihat pada pembuatan komponen counter sederhana yang menampilkan jumlah angka dan dapat bertambah saat tombol diklik. Konsep ini menjadi dasar penting dalam pengembangan aplikasi web modern berbasis React.

D.2 Keterkaitan dengan Teknik Industri

Materi React tentang state dan hooks memiliki keterkaitan dengan Teknik Industri terutama dalam pengembangan sistem informasi yang mendukung proses operasional perusahaan. Teknik Industri tidak hanya berfokus pada proses produksi, tetapi juga pada efisiensi sistem kerja, pengolahan data, dan pengambilan keputusan berbasis teknologi. Penggunaan React dapat membantu dalam membangun dashboard interaktif yang menampilkan data produksi secara real-time.

Contohnya, pada perusahaan manufaktur, sistem monitoring produksi dapat dibuat menggunakan React untuk menampilkan jumlah barang yang telah diproduksi, target produksi harian, serta status mesin secara langsung. Dengan penggunaan state, data produksi dapat terus diperbarui secara otomatis tanpa perlu memuat ulang halaman, sehingga supervisor dapat memantau kondisi lapangan dengan lebih cepat dan akurat.

Selain itu, React juga dapat digunakan dalam sistem manajemen persediaan, penjadwalan kerja, maupun evaluasi performa operator. Dalam konteks Teknik Industri, teknologi ini membantu meningkatkan efisiensi proses bisnis, mengurangi kesalahan manual, serta mendukung pengambilan keputusan yang lebih tepat berdasarkan data yang selalu terbaru.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Counter Produksi

Pada langkah ini saya membuat komponen CounterProduksi.jsx untuk menghitung jumlah produksi secara sederhana menggunakan React useState. Nilai awal produksi dibuat 0 dan target produksi ditentukan 100 unit. Dengan useState, jumlah produksi bisa berubah secara otomatis saat tombol ditekan.

Saya juga menambahkan tombol +1 Unit untuk menambah jumlah produksi dan tombol Reset Shift untuk mengembalikan jumlah produksi ke 0. Selain itu, dibuat juga tampilan status yang akan berubah secara otomatis. Jika jumlah produksi sudah mencapai target, akan muncul pesan “Target Tercapai!”, sedangkan jika belum maka akan muncul pesan “Produksi Berjalan...”. Jadi, komponen ini membantu memantau proses produksi dengan lebih mudah dan interaktif.

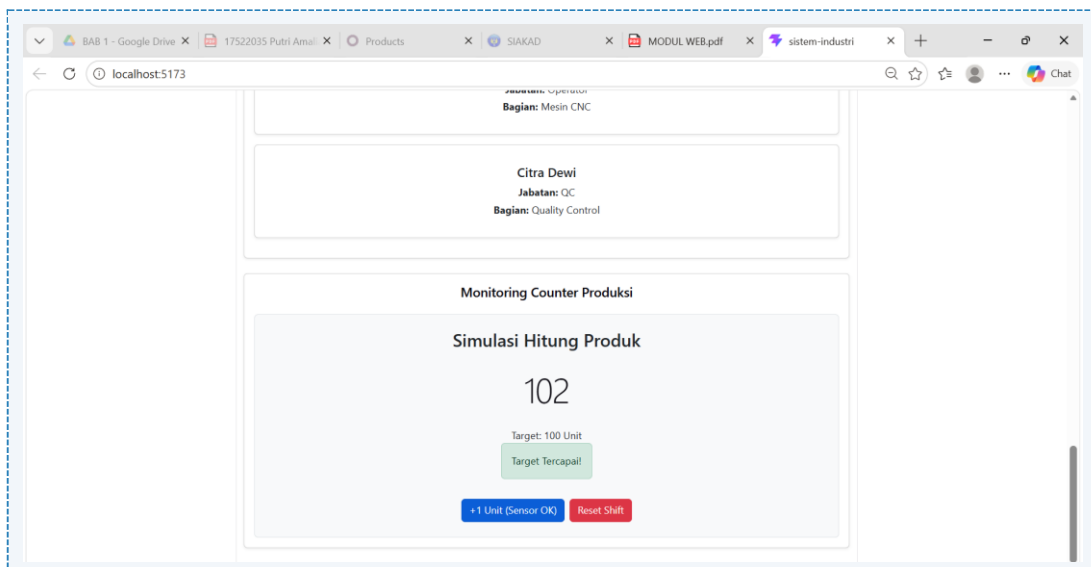
```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React, { useState } from 'react';
function CounterProduksi() {
  // Deklarasi State: 'jumlah' bernilai awal 0, 'setJumlah'
  fungsi untuk mengubahnya
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  // Fungsi menambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };
  // Fungsi reset
```

```

const reset = () => {
  setJumlah(0);
};
return (
  <div className="text-center p-4 border rounded bg-
light">
    <h3>Simulasi Hitung Produk</h3>
    <h1 className="display-4">{jumlah}</h1>
    <p>Target: {target} Unit</p>
    {/* Conditional Rendering: Tampilkan pesan jika
target tercapai */}
    {jumlah >= target ? (
      <div className="alert alert-success d-inline-
block">Target Tercapai!</div>
    ) : (
      <div className="alert alert-secondary d-inline-
block">Produksi Berjalan...</div>
    )}
    <div className="mt-3">
      <button className="btn btn-primary me-2"
onClick={tambahProduksi}>
        +1 Unit (Sensor OK)
      </button>
      <button className="btn btn-danger"
onClick={reset}>
        Reset Shift
      </button>
    </div>
  </div>
);
}
export default CounterProduksi;

```

Screenshot Hasil:

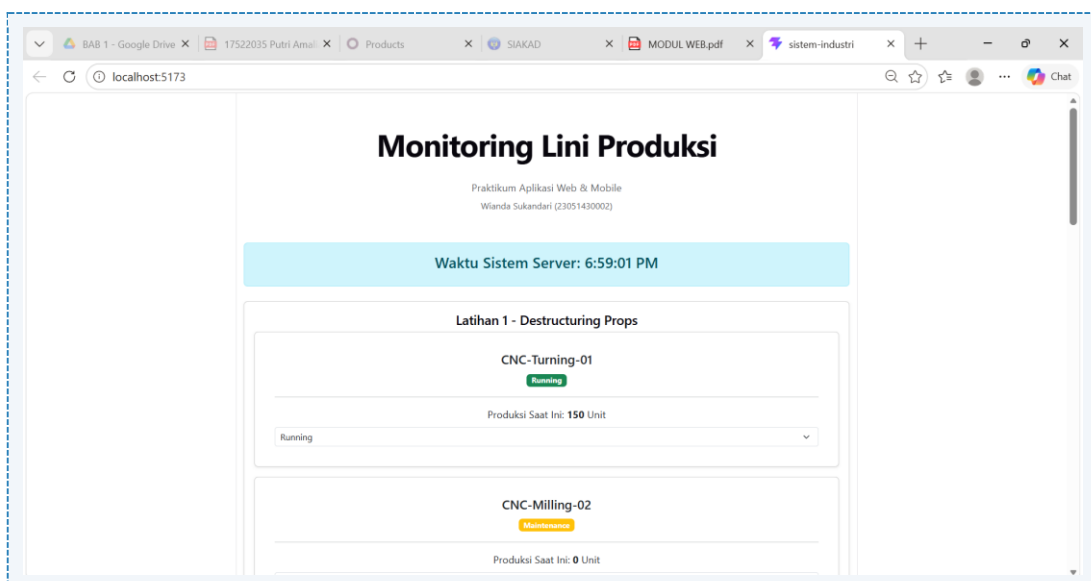


Gambar E.1 – Komponen Counter Produksi

E.2 Langkah 2 – Menggunakan useEffect untuk Real-time Clock

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React, { useState, useEffect } from 'react';
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());
  // useEffect berjalan sekali setelah komponen dirender
  pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap
detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen
dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali
saat mount
  return (
    <div className="alert alert-info text-center">
      <h4>Waktu Sistem Server:
{waktu.toLocaleTimeString()}</h4>
    </div>
  );
}
export default JamDigital;
```

Screenshot Hasil:



Gambar E.2 – useEffect untuk Real-time Clock

E.3 Langkah 3 – Contoh Form Input State


```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React, { useState } from "react";

// 2. Komponen Fungsi
function KartuMesin(props) {
  // Menerima data dari props
  const namaMesin = props.nama;
  const status = props.status;
  const produksi = props.produksi;

  // State lokal untuk status
  const [statusLokal, setStatusLokal] = useState(status);

  // Logika penentuan warna badge berdasarkan statusLokal
  let badgeColor = "bg-secondary";
  if (statusLokal === "Running") badgeColor = "bg-success";
  if (statusLokal === "Stop") badgeColor = "bg-danger";
  if (statusLokal === "Maintenance") badgeColor = "bg-warning";

  return (
    <div className="card shadow-sm p-3 mb-3">
      <div className="card-body">
        <h5 className="card-title">{namaMesin}</h5>

        <span className={`badge
${badgeColor}`}>{statusLokal}</span>

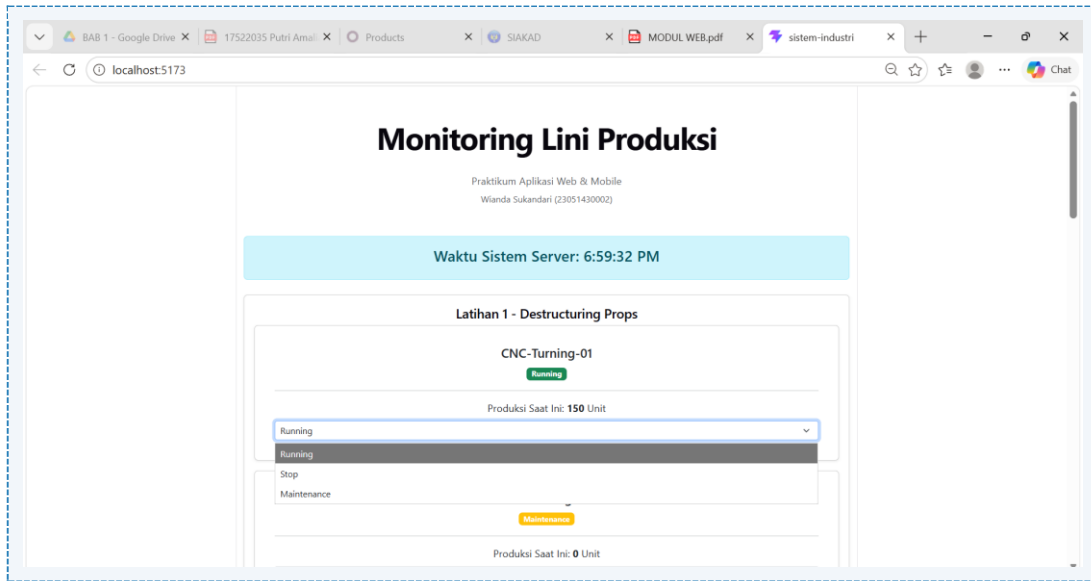
        <hr />

        <p>
          Produksi Saat Ini: <strong>{produksi}</strong> Unit
        </p>

        <div className="mt-2">
          <select
            className="form-select form-select-sm"
            value={statusLokal}
            onChange={(e) => setStatusLokal(e.target.value)}
          >
            <option value="Running">Running</option>
            <option value="Stop">Stop</option>
            <option value="Maintenance">Maintenance</option>
          </select>
        </div>
      </div>
    </div>
  );
}

// 3. Export
export default KartuMesin;
```

Screenshot Hasil:



Gambar E.3 – Form Input State

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dependency Array useEffect

Pertanyaan / Instruksi Latihan:

Modifikasi `JamDigital`. Tambahkan input text untuk memasukkan nama kota. Gunakan `useEffect` untuk mengubah `document.title` menjadi "Jam [Nama Kota]". Gunakan state kota sebagai dependency array `[kota]`.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from "react";
function JamDigital() {
  const [waktu, setWaktu] = useState(new Date());

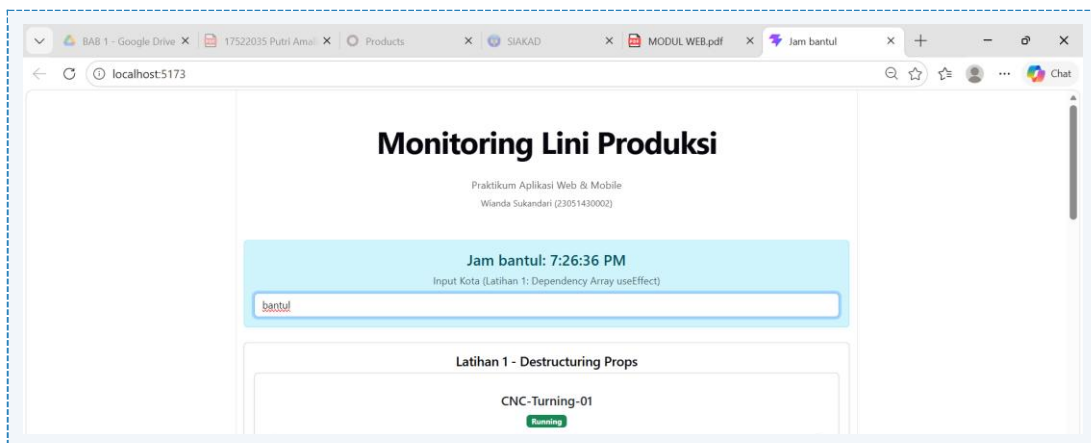
  // LATIHAN 1:
  const [kota, setKota] = useState(""); // Menambahkan state kota
  // useEffect berjalan sekali setelah komponen dirender pertama kali
  useEffect(() => {
    // Membuat interval timer
    const timerID = setInterval(() => {
      setWaktu(new Date()); // Update state waktu setiap detik
    }, 1000);
    // Cleanup function: Dijalankan saat komponen dihapus/hancur
    // Penting untuk mencegah memory leak
    return () => {
      clearInterval(timerID);
    };
  }, []); // Array kosong [] artinya hanya dijalankan sekali saat mount
}
```

```
// Menambahkan untuk Latihan 1
useEffect(() => {
  document.title = `Jam ${kota}`;
}, [kota]);

return (
  <div className="alert alert-info text-center">
    <h4>
      Jam {kota || "Indonesia"}: {waktu.toLocaleTimeString()}
    </h4>

    <p className="mt-2 mb-1 text-muted">
      Input Kota (Latihan 1: Dependency Array useEffect)
    </p>

    <input
      type="text"
      placeholder="Masukkan nama kota"
      className="form-control mt-2"
      value={kota}
      onChange={ (e) => setKota(e.target.value) }
    />
  </div>
);
}
export default JamDigital;
```



Gambar F.1 – Dependency Array useEffect

F.2 Latihan 2 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `CounterProduksi`, buat tombol "Emergency Stop". Saat diklik, ubah state status menjadi "EMERGENCY", tombol "+1" menjadi disabled (`disabled`), dan tampilkan pesan merah.

Jawaban / Kode Solusi:

```
import React, { useState } from 'react';

function CounterProduksi() {
  const [jumlah, setJumlah] = useState(0);
  const [target, setTarget] = useState(100);
  const [status, setStatus] = useState("Produksi Berjalan");

  // Tambah produksi
  const tambahProduksi = () => {
    setJumlah(jumlah + 1);
  };

  // Reset
  const reset = () => {
    setJumlah(0);
    setStatus("Produksi Berjalan");
  };

  // Emergency Stop
  const emergencyStop = () => {
    setStatus("EMERGENCY");
  };

  return (
    <div className="text-center p-4 border rounded bg-light">

      <h3>Simulasi Hitung Produk</h3>

      <h1 className="display-4">{jumlah}</h1>

      <p>Target: {target} Unit</p>

      {/* Conditional Rendering */}
      {status === "EMERGENCY" ? (
        <div className="alert alert-danger d-inline-block">
          ⚠ EMERGENCY STOP! Produksi dihentikan
        </div>
      ) : jumlah >= target ? (
        <div className="alert alert-success d-inline-block">
          Target Tercapai!
        </div>
      ) : (
        <div className="alert alert-secondary d-inline-block">
          Produksi Berjalan...
        </div>
      )}

      <div className="mt-3">

        <button
          className="btn btn-primary me-2"
          onClick={tambahProduksi}
          disabled={status === "EMERGENCY"}
        >

```

```

    >
      +1 Unit
    </button>

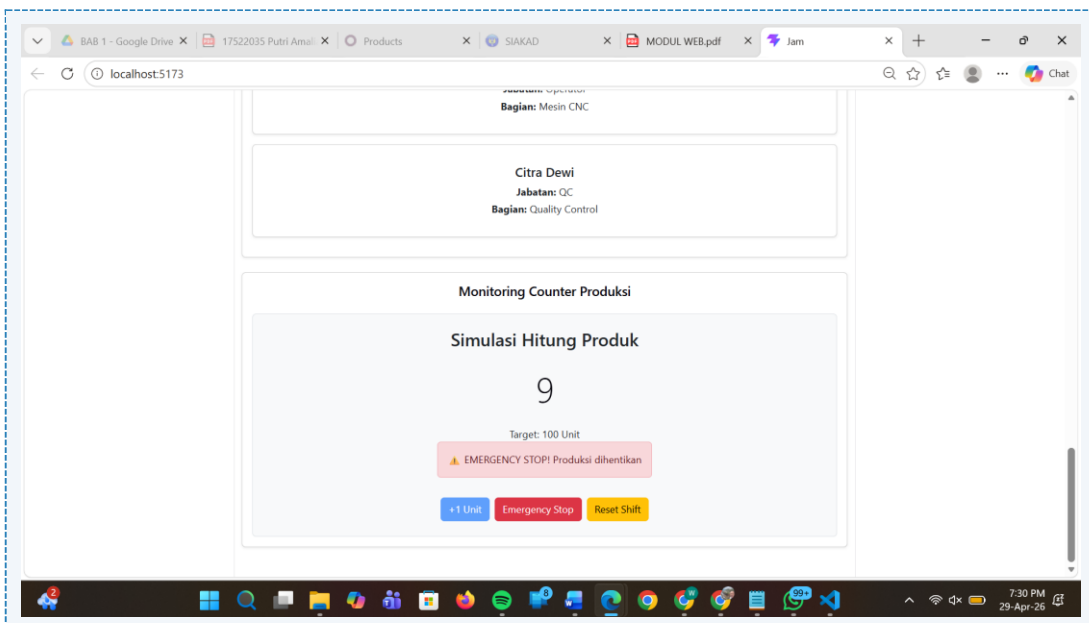
    <button
      className="btn btn-danger me-2"
      onClick={emergencyStop}
    >
      Emergency Stop
    </button>

    <button
      className="btn btn-warning"
      onClick={reset}
    >
      Reset Shift
    </button>

  </div>
</div>
);
}

export default CounterProduksi;

```



Gambar F.2 – Conditional Rendering

F.3 Tugas Proyek Mini – Kalkulator OEE Sederhana

Deskripsi proyek mini:

- Buat komponen form input: `Plan Time`, `Run Time`, `Total Parts`, `Good Parts`.
- Gunakan State untuk menyimpan nilai-nilai ini.
- Hitung OEE secara otomatis (Real-time) setiap kali input berubah.

- d. Rumus OEE = (Availability x Performance x Quality).
- e. Tampilkan hasil OEE dalam persen. Jika OEE < 50%, warnanya merah. Jika > 85%, warnanya hijau.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, dibuat sebuah Kalkulator OEE (Overall Equipment Effectiveness) sederhana menggunakan React dengan memanfaatkan `useState` untuk menyimpan data input seperti Plan Time, Run Time, Total Parts, dan Good Parts. Setiap nilai yang dimasukkan pengguna langsung tersimpan ke dalam state sehingga dapat diproses secara real-time tanpa perlu menekan tombol hitung.

Perhitungan dilakukan secara otomatis menggunakan rumus OEE yaitu $\text{Availability} \times \text{Performance} \times \text{Quality}$, di mana Availability diperoleh dari Run Time dibagi Plan Time, Performance dari Total Parts dibagi Run Time, dan Quality dari Good Parts dibagi Total Parts. Hasil akhir ditampilkan dalam bentuk persentase agar lebih mudah dipahami.

Selain itu, digunakan conditional rendering untuk memberikan indikator visual pada hasil OEE. Jika nilai OEE kurang dari 50%, maka hasil ditampilkan dengan warna merah sebagai tanda performa rendah. Jika nilai OEE lebih dari 85%, hasil ditampilkan dengan warna hijau sebagai indikator performa sangat baik. Pendekatan ini membuat sistem monitoring menjadi lebih informatif, sederhana, dan mudah digunakan.

Kode Utama:

```
import React, { useState } from 'react';

function OEECalculator() {
  const [planTime, setPlanTime] = useState(0);
  const [runTime, setRunTime] = useState(0);
  const [totalParts, setTotalParts] = useState(0);
  const [goodParts, setGoodParts] = useState(0);

  // Perhitungan OEE
  const availability =
    planTime > 0 ? runTime / planTime : 0;

  const performance =
    runTime > 0 ? totalParts / runTime : 0;

  const quality =
    totalParts > 0 ? goodParts / totalParts : 0;

  const oee =
    availability * performance * quality * 100;

  // Warna hasil
  let warnaHasil = "black";
```

```
if (oee < 50) {
  warnaHasil = "red";
} else if (oee > 85) {
  warnaHasil = "green";
}

return (
  <div className="card p-4 shadow-sm">
    <h2 className="text-center mb-4">
      Kalkulator OEE
    </h2>

    <div className="mb-3">
      <label>Plan Time</label>
      <input
        type="number"
        className="form-control"
        value={planTime}
        onChange={ (e) =>
setPlanTime (Number (e.target.value)) }
      />
    </div>

    <div className="mb-3">
      <label>Run Time</label>
      <input
        type="number"
        className="form-control"
        value={runTime}
        onChange={ (e) => setRunTime (Number (e.target.value)) }
      />
    </div>

    <div className="mb-3">
      <label>Total Parts</label>
      <input
        type="number"
        className="form-control"
        value={totalParts}
        onChange={ (e) =>
setTotalParts (Number (e.target.value)) }
      />
    </div>

    <div className="mb-3">
      <label>Good Parts</label>
      <input
        type="number"
        className="form-control"
        value={goodParts}
        onChange={ (e) =>
setGoodParts (Number (e.target.value)) }
    </div>
```

```

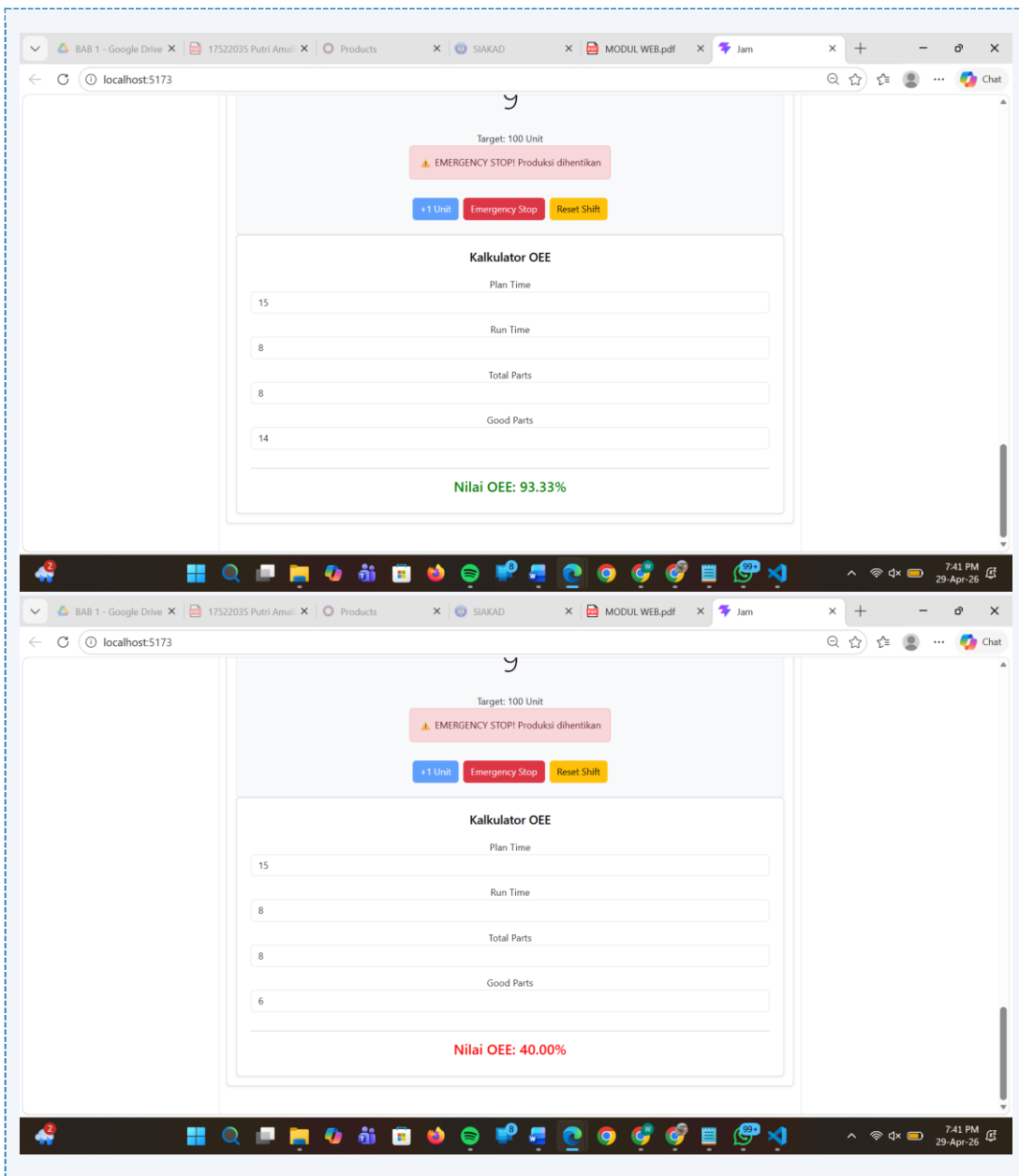
    />
  </div>

  <hr />

  <h4 style={{ color: warnaHasil }}>
    Nilai OEE: {oeo.toFixed(2)}%
  </h4>
</div>
);
}

export default OEECalculator;

```



Gambar F.3 – Kalkulator OEE Sederhana

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori React yang dipelajari, khususnya pada penggunaan `useState`, `useEffect`, `dependency array`, `conditional rendering`, `props`, dan `default props`. Pada komponen `JamDigital`, penggunaan `useEffect` dengan `dependency array` [kota] berhasil membuat `document.title` berubah secara otomatis mengikuti nama kota yang diinput, misalnya menjadi “Jam Sleman”. Hal ini menunjukkan bahwa efek hanya berjalan saat state kota berubah, sesuai konsep `dependency array` pada React.

Pada komponen `CounterProduksi`, penggunaan `useState` berhasil mengatur jumlah produksi, target produksi, dan status emergency. Saat tombol “+1 Unit” ditekan, nilai produksi bertambah satu per satu secara real-time. Ketika tombol “Emergency Stop” ditekan, status berubah menjadi “EMERGENCY”, tombol “+1” menjadi tidak aktif (disabled), dan pesan peringatan berwarna merah muncul. Hal ini sesuai dengan teori `conditional rendering`, yaitu tampilan berubah berdasarkan kondisi state tertentu.

Pada latihan `props`, komponen `KartuMesin` berhasil menerima data mesin melalui `props` seperti nama, status, dan jumlah produksi. Selain itu, pada latihan `default props`, ketika nilai produksi tidak diberikan, sistem otomatis menampilkan nilai default yaitu 0. Ini menunjukkan bahwa konsep `props` dan `default props` berjalan dengan baik sesuai teori.

Pada mini project `Kalkulator OEE`, sistem mampu menghitung nilai OEE secara otomatis setiap kali input berubah tanpa perlu tombol hitung tambahan. Perhitungan `Availability`, `Performance`, dan `Quality` berjalan sesuai rumus teori OEE, dan hasil akhir ditampilkan dalam bentuk persentase dengan warna indikator berdasarkan nilai OEE. Hal ini membuktikan bahwa penerapan `state management` dan `real-time calculation` berjalan sesuai konsep yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Komponen tidak tampil di halaman karena belum dipanggil di <code>App.jsx</code>	Menambahkan import komponen dan memanggilnya di bagian <code>return App.jsx</code> agar komponen dapat dirender
Error karena penulisan path file salah, seperti folder <code>Komponen</code> atau <code>Latihan</code> tidak sesuai	Memeriksa kembali struktur folder project dan menyesuaikan path import agar sesuai lokasi file sebenarnya
Tampilan hasil latihan terlihat double karena <code>Latihan 1</code> dan	Menggabungkan beberapa komponen ke dalam satu halaman utama dan

Latihan 2 memiliki output yang mirip

merapikan layout agar lebih terstruktur dan tidak terlihat berulang

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini sangat relevan dengan penerapan Teknik Industri, terutama dalam sistem monitoring produksi dan pengambilan keputusan berbasis data secara real-time. Penggunaan React untuk membuat dashboard monitoring dapat membantu perusahaan dalam memantau kondisi mesin, status produksi, serta performa operator secara lebih cepat dan efisien.

Sebagai contoh, pada industri manufaktur seperti pabrik otomotif atau makanan, supervisor produksi perlu mengetahui status mesin seperti Running, Maintenance, atau Stop secara langsung. Dengan komponen seperti KartuMesin, informasi tersebut dapat ditampilkan secara visual sehingga memudahkan pengawasan lini produksi.

Penggunaan CounterProduksi juga dapat diterapkan untuk menghitung output produksi harian secara real-time. Jika terjadi kondisi darurat seperti kerusakan mesin, tombol “Emergency Stop” dapat digunakan untuk menghentikan proses dan memberikan peringatan kepada operator agar tindakan cepat dapat dilakukan.

Selain itu, mini project OEE sangat penting dalam industri karena OEE merupakan salah satu indikator utama untuk mengukur efektivitas mesin. Dengan sistem OEE berbasis web, perusahaan dapat mengetahui apakah performa mesin masih optimal atau perlu dilakukan perbaikan. Misalnya, jika nilai OEE berada di bawah 50%, maka tim maintenance dapat segera melakukan analisis penyebab downtime untuk meningkatkan efisiensi produksi.

Dengan demikian, modul ini tidak hanya melatih kemampuan pemrograman web, tetapi juga mendukung penerapan konsep Teknik Industri dalam pengendalian produksi, evaluasi performa mesin, dan peningkatan produktivitas perusahaan.

H. KESIMPULAN

- 1 Praktikum ini membantu memahami konsep dasar React seperti `useState`, `useEffect`, props, default props, dependency array, dan conditional rendering dalam pembuatan aplikasi web interaktif.
- 2 Komponen JamDigital berhasil menunjukkan penggunaan `useEffect` dengan dependency array untuk mengubah `document.title` secara otomatis berdasarkan input nama kota secara real-time.

3	Komponen CounterProduksi berhasil menerapkan conditional rendering melalui fitur tombol “Emergency Stop”, di mana status sistem berubah, tombol produksi dinonaktifkan, dan pesan peringatan ditampilkan.
4	Mini project Kalkulator OEE berhasil menghitung nilai efektivitas mesin secara otomatis berdasarkan input produksi, sehingga dapat digunakan sebagai simulasi monitoring performa mesin dalam industri.
5	Penerapan konsep React pada modul ini sangat relevan dengan bidang Teknik Industri, khususnya dalam monitoring lini produksi, evaluasi performa mesin, dan pengambilan keputusan berbasis data secara real-time.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Documentation. (2026). <i>Using the State Hook</i> . Retrieved from https://react.dev/reference/react/useState
[3]	React Documentation. (2026). <i>Using the Effect Hook</i> . Retrieved from https://react.dev/reference/react/useEffect
[4]	Mozilla Developer Network. (2026). <i>JavaScript Guide</i> . Retrieved from https://developer.mozilla.org/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 11**React Routing & Multi-Page App + Integrasi API****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 11)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	02 Mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor

2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React Router DOM	Versi 6+	Routing & Navigasi Halaman
5	JSONPlaceholder API	API Online	Simulasi Fetch Data API

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini saya belajar tentang routing pada React menggunakan react-router-dom. Routing digunakan supaya aplikasi dapat memiliki beberapa halaman seperti dashboard, inventory, dan laporan tanpa harus membuat aplikasi terpisah. Dengan adanya routing, perpindahan halaman menjadi lebih cepat dan tidak perlu reload browser.

Saya juga belajar menggunakan beberapa komponen penting seperti BrowserRouter, Routes, Route, dan Link. BrowserRouter digunakan untuk mengaktifkan sistem routing, Route digunakan untuk menentukan halaman berdasarkan URL, sedangkan Link dipakai untuk berpindah halaman dengan lebih mudah.

Selain itu, modul ini membantu saya memahami bagaimana aplikasi web modern dibuat secara modular. Setiap halaman dipisahkan ke file masing-masing supaya kode lebih rapi, mudah dibaca, dan lebih mudah dikembangkan ketika aplikasi semakin besar.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini berhubungan dengan Teknik Industri karena di dunia industri banyak sistem yang menggunakan aplikasi web untuk mengelola data dan memantau proses kerja. Dengan adanya routing, aplikasi bisa dibagi menjadi beberapa halaman seperti dashboard, inventori, laporan, dan monitoring produksi sehingga tampilan lebih rapi dan mudah digunakan.

Contoh penerapannya yaitu pada sistem inventori di pabrik. Operator gudang dapat membuka halaman inventory untuk melihat stok bahan baku, lalu membuka halaman detail barang untuk melihat informasi item tertentu. Selain

itu, data juga bisa diambil langsung dari database atau API sehingga informasi yang ditampilkan lebih cepat dan mudah diperbarui.

Materi ini juga membantu dalam pembuatan sistem monitoring produksi atau laporan kualitas di perusahaan. Dengan aplikasi berbasis web, proses pengolahan data menjadi lebih efisien, mengurangi pencatatan manual, dan mempermudah pengambilan keputusan dalam kegiatan industri.

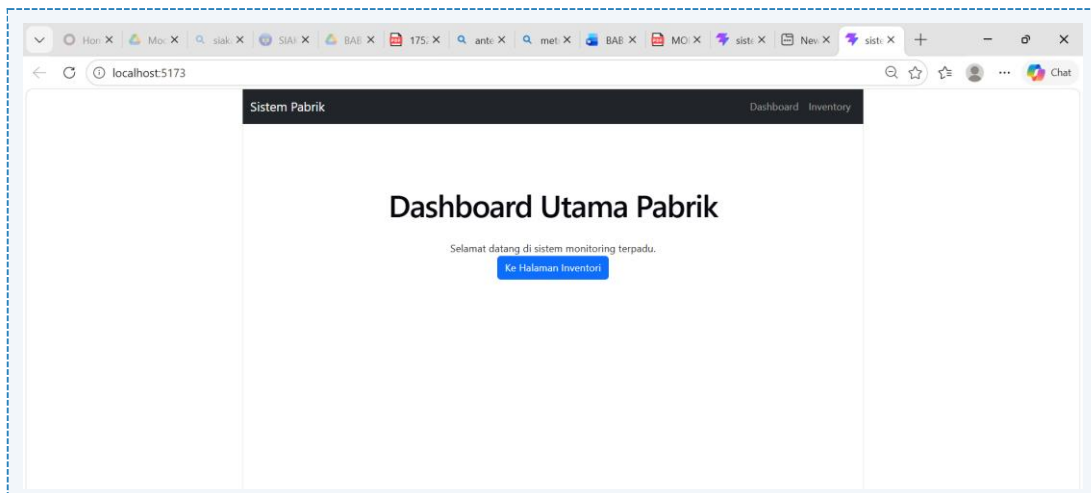
E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Router

Pada langkah ini saya menambahkan router pada aplikasi React menggunakan react-router-dom. Router digunakan supaya aplikasi bisa berpindah halaman seperti Dashboard, Inventory, dan halaman Error tanpa perlu reload browser. Saya juga membungkus aplikasi dengan BrowserRouter di file main.jsx dan menambahkan beberapa Route di App.jsx agar setiap URL bisa menampilkan halaman yang sesuai. Dengan cara ini, navigasi aplikasi jadi lebih cepat dan nyaman digunakan.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Screenshot Hasil:



Gambar E.1 – Setup Router

E.2 Langkah 2 – Membuat Halaman (Pages)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
Isi `src/Halaman/Dashboard.jsx`:
import React from 'react';
import { Link } from 'react-router-dom';
function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori
      </Link>
    </div>
  );
}
export default Dashboard;

Isi `src/Halaman/Inventori.jsx`:
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
function Inventori() {
  const [products, setProducts] = useState([]);
  // Fetch Data saat komponen mount
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => {
        // Ambil hanya 5 data pertama untuk contoh
        setProducts(data.slice(0, 5));
      })
      .catch(err => console.log(err));
  }, []);
  return (
    <div className="container mt-4">
      <h1>Data Inventori Bahan Baku</h1>
      <Link to="/" className="btn btn-secondary mb-3">Kembali ke Dashboard</Link>
      <table className="table table-striped">
        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>
        <tbody>
          {products.map((item) => (
            <tr key={item.id}>
              <td>{item.id}</td>
              <td>{item.title}</td> { /* Menggunakan
title sebagai simulasi nama bahan */}
```

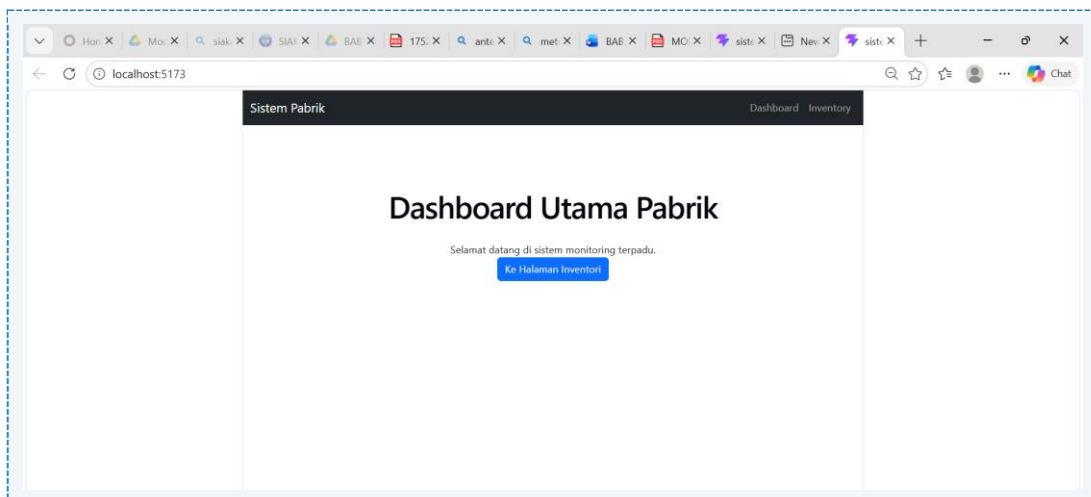
```

                <td><span className="badge bg-
success">Available</span></td>
            </tr>
        )}
    </tbody>
</table>
</div>
);
}
export default Inventori;

Isi `src/Halaman/NotFound.jsx`:
import React from 'react';
import { Link } from 'react-router-dom';
function NotFound() {
    return (
        <div className="text-center mt-5">
            <h1 className="display-1">404</h1>
            <p>Halaman tidak ditemukan dalam sistem pabrik.</p>
            <Link to="/" className="btn btn-dark">Kembali ke
Home</Link>
        </div>
    );
}
export default NotFound;

```

Screenshot Hasil:



Gambar E.2 – Membuat Halaman (Pages)

E.3 Langkah 3 – Mengatur Route di App.jsx

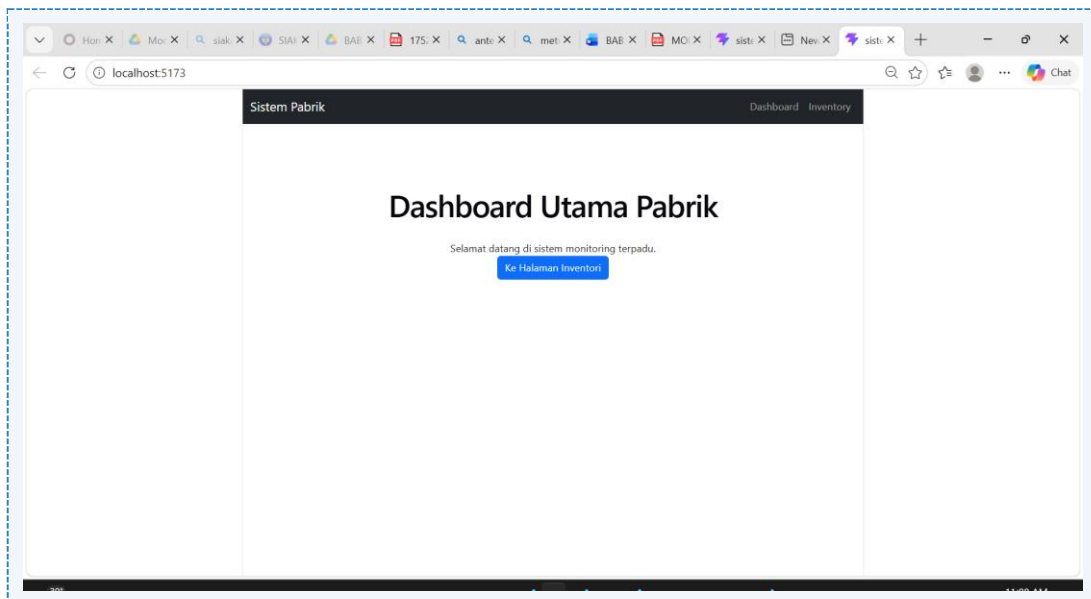
```

/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventory from './Halaman/Inventory';
import NotFound from './Halaman/NotFound';

```

```
// import Navbar from './Komponen/Navbar'; // Anda bisa buat
navbar sendiri
// Buat Navbar.jsx sederhana untuk navigasi
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark
mb-4">
      <div className="container">
        <a className="navbar-brand" href="/">Sistem Pabrik</a>
        <div className="navbar-nav">
          <a className="nav-link" href="/">Dashboard</a>
          <a className="nav-link"
href="/inventory">Inventory</a>
        </div>
      </div>
    </nav>
  )
}
function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        {/* Route yang tepat akan di-render */}
        <Route path="/" element={<Dashboard />} />
        <Route path="/inventory" element={<Inventory />} />
        {/* Route untuk semua path lainnya (404) */}
        <Route path="" element={<NotFound />} />
      </Routes>
    </div>
  );
}
export default App;
```

Screenshot Hasil:



Gambar E.3 – Mengatur Route di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

Copy kode diatas (Latihan 1 sampai 3) kemudian tambahkan file baru `DetailItem.jsx` dan diisi dengan kode berikut

```
import React from 'react';
import { useParams, Link } from 'react-router-dom';

function DetailItem() {
  const { id } = useParams();

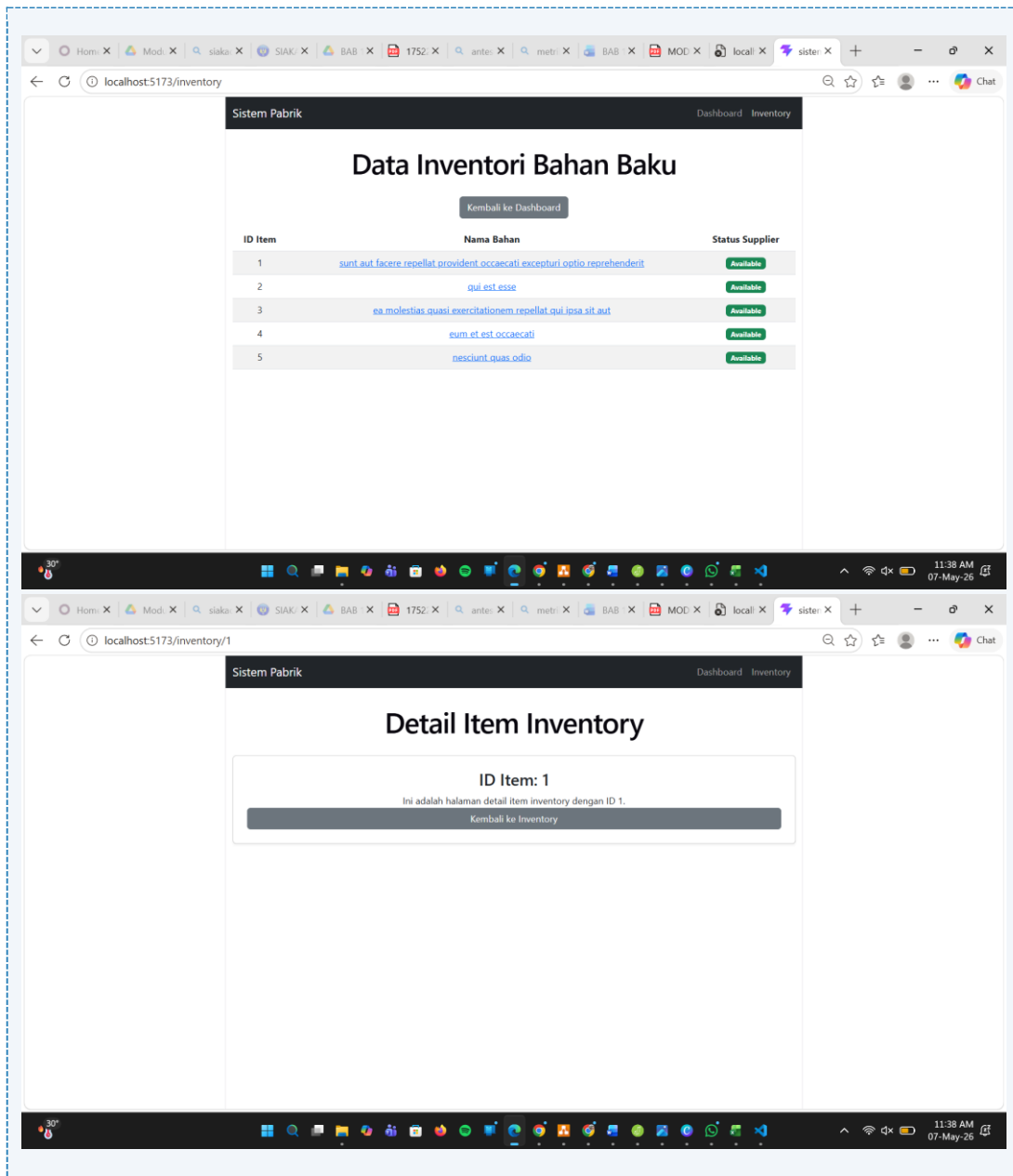
  return (
    <div className="container mt-4">
      <h1>Detail Item Inventory</h1>

      <div className="card p-4 shadow-sm">
        <h3>ID Item: {id}</h3>

        <p>
          Ini adalah halaman detail item inventory
          dengan ID {id}.
        </p>

        <Link to="/inventory" className="btn btn-secondary">
          Kembali ke Inventory
        </Link>
      </div>
    </div>
  );
}

export default DetailItem;
```



Gambar F.1 – Dynamic Route

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventory() {
```

```
// State data produk
const [products, setProducts] = useState([]);

// State loading
const [loading, setLoading] = useState(true);

// Fetch data
useEffect(() => {

  // Aktifkan loading
  setLoading(true);

  // Delay 2 detik supaya loading terlihat
  setTimeout(() => {

    fetch('https://jsonplaceholder.typicode.com/posts')

      .then((res) => res.json())

      .then((data) => {

        // Ambil 5 data pertama
        setProducts(data.slice(0, 5));

        // Loading selesai
        setLoading(false);
      })

      .catch((err) => {

        console.log(err);

        // Jika error loading dimatikan
        setLoading(false);
      });

    }, 2000);

}, []);

return (
  <div className="container mt-4">

    <h1>Data Inventory Bahan Baku</h1>

    <Link to="/" className="btn btn-secondary mb-3">
      Kembali ke Dashboard
    </Link>

    {/* Conditional Rendering Loading */}
    {loading ? (

      <div className="alert alert-info">
```

```

        <h4>Memuat data...</h4>
      </div>

    ) : (

      <table className="table table-striped table-
bordered">

        <thead className="table-dark">
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>

        <tbody>

          {products.map((item) => (

            <tr key={item.id}>

              <td>{item.id}</td>

              <td>
                <Link
to={` /inventory/${item.id}`}>
                  {item.title}
                </Link>
              </td>

              <td>
                <span className="badge bg-
success">
                  Available
                </span>
              </td>

            </tr>

          ) ) }

        </tbody>

      </table>

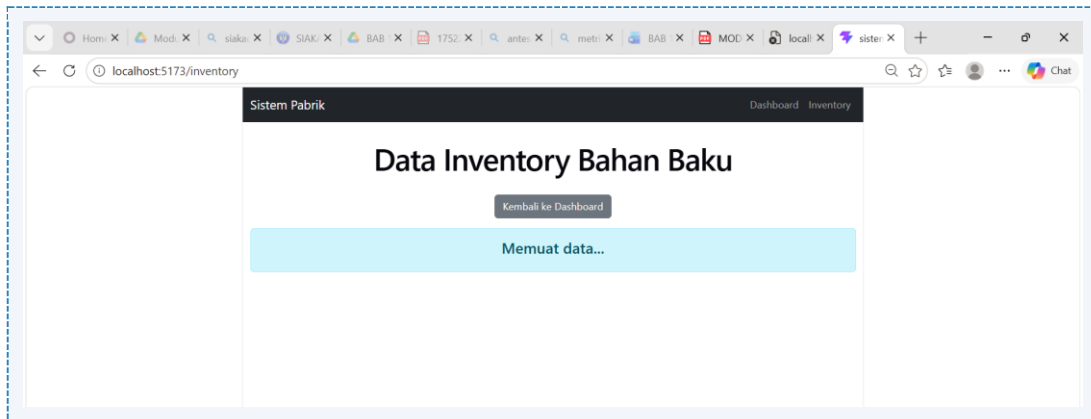
    ) }

  </div>

);
}

export default Inventory;

```



Gambar F.2 – Loading State

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, saya membuat halaman baru bernama LaporanKualitas.jsx untuk menampilkan data cacat produk menggunakan array manual (mock data). Setelah itu, saya menambahkan route baru pada App.jsx agar halaman bisa diakses melalui URL /laporan-kualitas. Saya juga menambahkan menu navigasi pada Navbar menggunakan Link dari react-router-dom.

Agar perpindahan halaman lebih cepat dan tidak reload browser, saya menggunakan konsep SPA (Single Page Application) dengan React Router. Dengan cara ini, pengguna bisa berpindah dari Dashboard, Inventory, dan Laporan Kualitas dengan lebih mulus. Selain itu, saya menggunakan komponen tabel Bootstrap supaya tampilan data lebih rapi dan mudah dibaca.

Kode Utama:


```
import React from 'react';
import { Link } from 'react-router-dom';

function LaporanKualitas() {

  // Mock data cacat
  const dataCacat = [
    {
      id: 1,
      nama: "Produk Retak",
      jumlah: 12
    },
    {
      id: 2,
      nama: "Warna Tidak Sesuai",
      jumlah: 7
    },
    {
      id: 3,
      nama: "Ukuran Tidak Presisi",
      jumlah: 5
    }
  ];

  return (
    <div className="container mt-4">

      <h1>Laporan Kualitas Produk</h1>

      <Link to="/" className="btn btn-secondary mb-3">
        Kembali ke Dashboard
      </Link>

      <table className="table table-bordered table-
striped">

        <thead className="table-dark">
          <tr>
            <th>ID</th>
            <th>Jenis Cacat</th>
            <th>Jumlah</th>
          </tr>
        </thead>

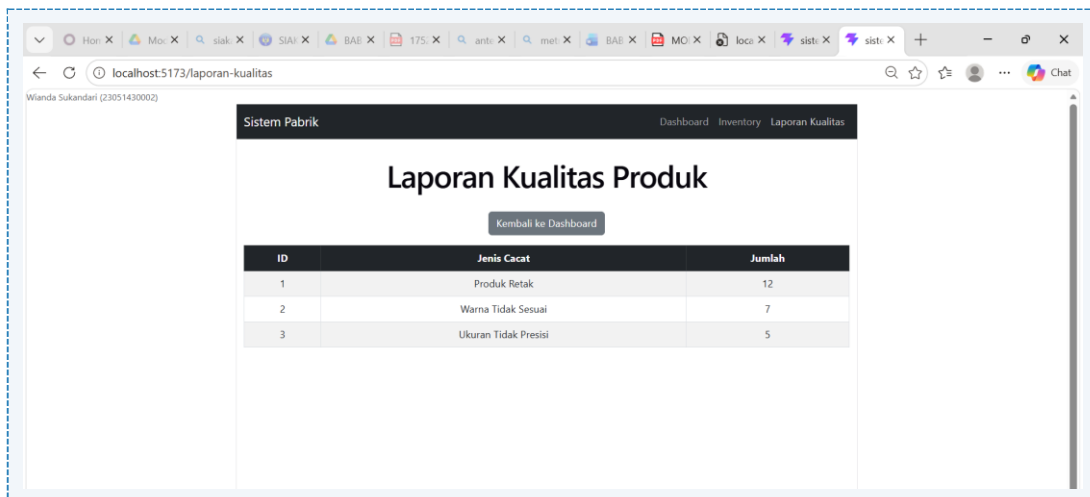
        <tbody>

          {dataCacat.map((item) => (

            <tr key={item.id}>
              <td>{item.id}</td>
              <td>{item.nama}</td>
              <td>{item.jumlah}</td>
            </tr>


```

```
    ) }  
  
    </tbody>  
  
    </table>  
  
    </div>  
  );  
}  
  
export default LaporanKualitas;
```



Gambar F.3 – Navigasi Modular

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori yang dipelajari pada modul React Router dan Fetch API. Pada bagian routing, aplikasi berhasil berpindah halaman menggunakan react-router-dom tanpa melakukan reload browser. Hal ini menunjukkan bahwa konsep Single Page Application (SPA) berjalan dengan baik. Route seperti `/inventory`, `/inventory/:id`, dan `/laporan-kualitas` dapat menampilkan komponen yang berbeda sesuai URL yang dipilih.

Selain itu, penggunaan Link juga sudah sesuai teori karena navigasi dilakukan secara client-side sehingga perpindahan halaman terasa lebih cepat dan ringan dibanding menggunakan tag `<a>`. Dynamic route pada halaman detail item juga berhasil menampilkan data berdasarkan parameter id dari URL menggunakan `useParams()`.

Pada bagian Fetch API dan loading state, data dari API berhasil diambil menggunakan `fetch()` di dalam `useEffect()`. Saat data masih diproses, teks “Memuat data...” muncul karena state loading bernilai true, lalu berubah menjadi tabel data setelah fetch selesai. Hal ini membuktikan bahwa penggunaan state dan effect pada React sudah berjalan sesuai konsep yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Halaman tidak berpindah dengan benar saat klik menu Navbar	Mengganti tag <code><a></code> menjadi <code><Link></code> dari react-router-dom
Loading state tidak muncul	Menambahkan <code>setLoading(true)</code> sebelum fetch dan <code>setLoading(false)</code> setelah data berhasil dimuat
Route detail item <code>/inventory/:id</code> tidak berjalan	Menambahkan route baru pada <code>App.jsx</code> dan menggunakan <code>useParams()</code> untuk membaca parameter URL

G.3 Kaitan dengan Penerapan di Industri

Materi pada modul ini sangat relevan dengan dunia Teknik Industri, terutama dalam pengembangan sistem monitoring dan manajemen data produksi berbasis web. Konsep routing dapat digunakan untuk membuat aplikasi dengan banyak halaman seperti dashboard produksi, inventori bahan baku, laporan kualitas, dan data supplier tanpa perlu reload halaman sehingga aplikasi menjadi lebih cepat dan efisien.

Penggunaan Fetch API juga penting dalam industri karena data pada sistem manufaktur biasanya diambil dari database atau server secara real-time. Contohnya pada sistem monitoring gudang, data stok bahan baku dapat otomatis diperbarui dari server dan langsung ditampilkan pada halaman inventori. Selain itu, loading state membantu memberikan informasi kepada pengguna bahwa sistem sedang memproses data sehingga aplikasi terasa lebih responsif dan nyaman digunakan.

Dynamic route juga bermanfaat dalam sistem industri untuk menampilkan detail data tertentu. Contohnya, ketika operator memilih salah satu produk pada halaman inventori, sistem dapat langsung membuka halaman detail produk berdasarkan ID item tersebut. Dengan demikian, proses pencarian informasi menjadi lebih cepat dan terstruktur.

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan konsep React Router untuk membuat navigasi antar halaman seperti Dashboard, Inventory, dan Laporan Kualitas tanpa reload browser menggunakan metode SPA (Single Page Application).
2	Penggunaan dynamic route pada path <code>/inventory/:id</code> berhasil digunakan untuk menampilkan halaman detail item berdasarkan ID yang dipilih dari tabel inventory.
3	Fetch API dan hook <code>useEffect()</code> berhasil digunakan untuk mengambil data dari API eksternal, kemudian ditampilkan ke dalam tabel inventory secara otomatis.
4	State loading berhasil diterapkan untuk menampilkan indikator “Memuat data...” saat proses pengambilan data berlangsung sehingga tampilan aplikasi menjadi lebih informatif dan responsif.
5	Praktikum ini memberikan pemahaman tentang pengembangan aplikasi web modern yang relevan dengan kebutuhan sistem informasi industri seperti monitoring inventori, laporan kualitas, dan pengelolaan data produksi berbasis web.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React. (2025). <i>React documentation</i> . https://react.dev/
[3]	MDN Web Docs. (2025). <i>Fetch API</i> . https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
[4]	React Router. (2025). <i>React Router Documentation</i> . https://reactrouter.com/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 12**Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 12)
Topik Utama	Proyek Web: Dashboard Monitoring Produksi (React + Chart.js)
Sub-Topik	Integrasi Library Visualisasi Data (Chart.js) dan Finalisasi Layout Dashboard.
Durasi Praktikum	340 Menit
Tanggal Praktikum	08 Mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Mampu mengintegrasikan library pihak ketiga (`react-chartjs-2`) ke dalam React.
2	Mampu mengubah data mentah menjadi visualisasi grafik yang informatif.
3	Menyelesaikan proyek mini dashboard lengkap dari pertemuan 1-12.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React.js	React 18+	Library Frontend UI
5	Chart.js & react-chartjs-2	Versi terbaru	Visualisasi Grafik Dashboard

D. DASAR TEORI

D.1 Konsep Utama

Konsep utama pada modul ini adalah membuat dashboard visualisasi data menggunakan React dan Chart.js. Dashboard digunakan untuk menampilkan data produksi dalam bentuk grafik agar informasi lebih mudah dipahami dibanding hanya melihat angka dalam tabel. Dengan adanya visualisasi data, pengguna dapat melihat pola, perubahan, dan kondisi produksi secara lebih cepat.

React digunakan untuk membangun tampilan website yang interaktif dan tersusun dalam bentuk komponen. Setiap bagian dashboard, seperti grafik atau informasi produksi, dapat dibuat menjadi komponen terpisah sehingga kode lebih rapi dan mudah dikembangkan. Selain itu, React juga mempermudah proses update tampilan ketika data berubah.

Sedangkan Chart.js digunakan untuk membuat grafik dari data produksi. Data yang awalnya berupa array angka diubah menjadi grafik batang atau grafik garis agar lebih informatif dan menarik. Modul ini juga mengajarkan cara mengintegrasikan library pihak ketiga ke dalam project React sehingga dashboard terlihat lebih profesional.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini sangat berkaitan dengan Teknik Industri karena dashboard visualisasi sering digunakan dalam monitoring proses produksi dan analisis data perusahaan. Dalam dunia industri, data produksi biasanya sangat banyak sehingga lebih mudah dipahami jika ditampilkan dalam bentuk grafik atau dashboard.

Contohnya pada pabrik manufaktur, manager produksi dapat menggunakan dashboard untuk melihat jumlah output produksi per jam, tingkat defect produk, atau utilisasi mesin. Dengan tampilan grafik, kondisi produksi dapat dipantau secara real-time sehingga keputusan bisa diambil lebih cepat jika terjadi masalah pada proses produksi.

Selain itu, visualisasi data juga membantu proses evaluasi dan peningkatan efisiensi kerja. Teknik Industri tidak hanya berhubungan dengan proses produksi, tetapi juga pengolahan data untuk mendukung pengambilan keputusan. Karena itu, kemampuan membuat dashboard monitoring seperti pada modul ini sangat berguna di dunia kerja industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Komponen Grafik

Di Langkah 1 ini saya pertama menyiapkan data grafik dulu kemudian menyiapkan file `src/Komponen/GrafikProduksi.jsx`.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react';
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend,
} from 'chart.js';

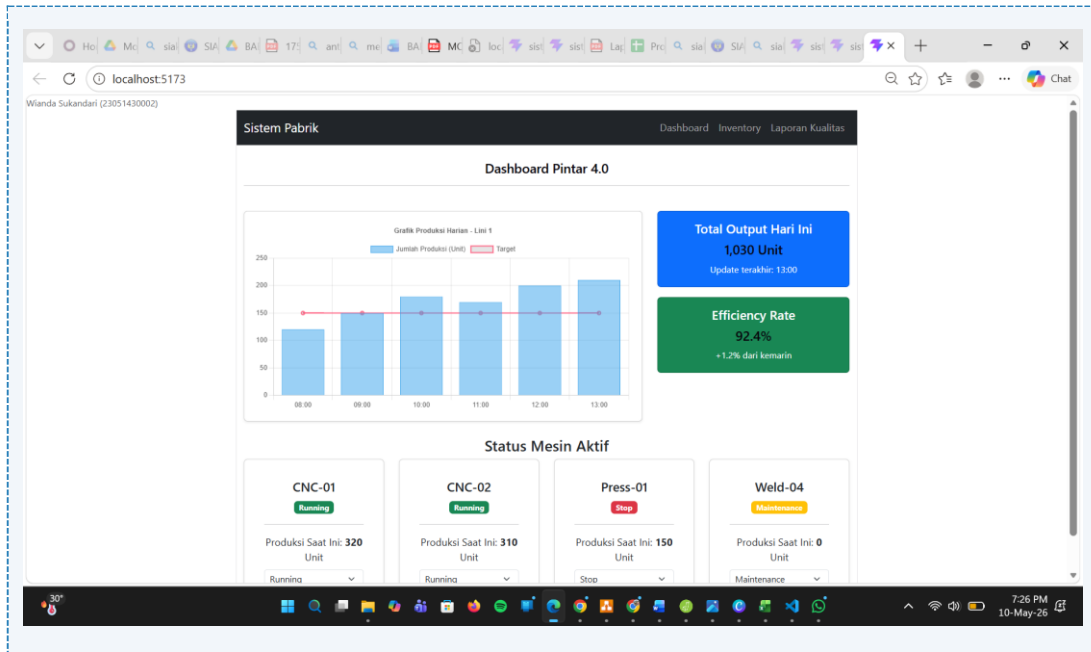
// Mendaftarkan komponen ChartJS
ChartJS.register(
  CategoryScale,
  LinearScale,
  BarElement,
  Title,
  Tooltip,
  Legend
);

function GrafikProduksi() {
  // Data untuk Grafik
  const data = {
    labels: ['08:00', '09:00', '10:00', '11:00', '12:00', '13:00'],
    datasets: [
      {
        label: 'Jumlah Produksi (Unit)',
        data: [120, 150, 180, 170, 200, 210], // Data statis,
        // bisa diganti state/dinamis
        backgroundColor: 'rgba(54, 162, 235, 0.5)',
        borderColor: 'rgba(54, 162, 235, 1)',
        borderWidth: 1,
      }
    ]
  };

  return (
    <div>
      <h3>Grafik Produksi</h3>
      <Bar data={data} />
    </div>
  );
}
```

```
    },  
    {  
      label: 'Target',  
      data: [150, 150, 150, 150, 150, 150], // Garis target  
konstan  
      type: 'line', // Tipe campuran (Bar + Line)  
      borderColor: 'rgb(255, 99, 132)',  
      borderWidth: 2,  
    },  
  ],  
};  
  
// Opsi Tampilan Grafik  
const options = {  
  responsive: true,  
  plugins: {  
    legend: {  
      position: 'top',  
    },  
  },  
  title: {  
    display: true,  
    text: 'Grafik Produksi Harian - Lini 1',  
  },  
},  
scales: {  
  y: {  
    beginAtZero: true,  
  },  
},  
};  
  
return <Bar data={data} options={options} />;  
}  
  
export default GrafikProduksi;
```

Screenshot Hasil:



Gambar E.1 – Membuat Komponen Grafik

E.2 Langkah 2 – Menggabungkan ke Dashboard Utama

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React from 'react';
import GrafikProduksi from '../Komponen/GrafikProduksi';
import KartuMesin from '../Komponen/KartuMesin';

function Dashboard() {
  return (
    <div className="container-fluid mt-4">
      <div className="row mb-4">
        <div className="col-12">
          <h2>Dashboard Pintar 4.0</h2>
          <hr/>
        </div>
      </div>

      {/* Bagian Grafik */}
      <div className="row mb-4">
        <div className="col-md-8">
          <div className="card shadow-sm">
            <div className="card-body">
              <GrafikProduksi />
            </div>
          </div>
        </div>

        {/* Bagian Ringkasan KPI */}
        <div className="col-md-4">
          <div className="card bg-primary text-white mb-3">
            <div className="card-body">
              <h5>Total Output Hari Ini</h5>
              <h2>1,030 Unit</h2>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

```

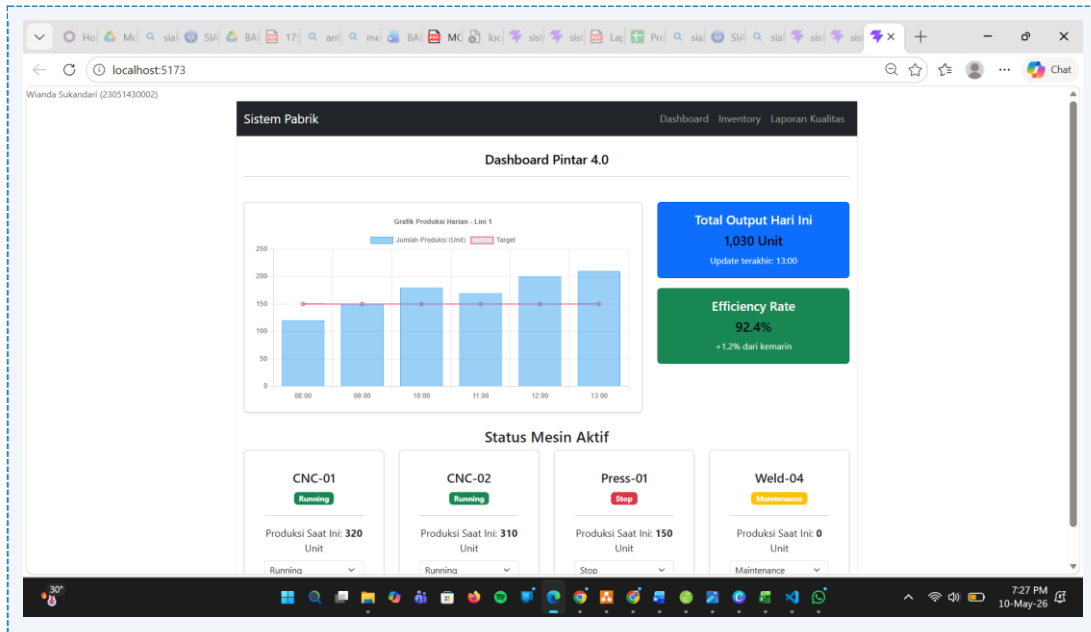
        <small>Update terakhir: 13:00</small>
      </div>
    </div>
    <div className="card bg-success text-white">
      <div className="card-body">
        <h5>Efficiency Rate</h5>
        <h2>92.4%</h2>
        <small>+1.2% dari kemarin</small>
      </div>
    </div>
  </div>
</div>

{/* Bagian Daftar Mesin */}
<div className="row">
  <div className="col-12">
    <h4>Status Mesin Aktif</h4>
  </div>
  <div className="col-md-3">
    <KartuMesin nama="CNC-01" status="Running"
produksi={320} />
  </div>
  <div className="col-md-3">
    <KartuMesin nama="CNC-02" status="Running"
produksi={310} />
  </div>
  <div className="col-md-3">
    <KartuMesin nama="Press-01" status="Stop"
produksi={150} />
  </div>
  <div className="col-md-3">
    <KartuMesin nama="Weld-04" status="Maintenance"
produksi={0} />
  </div>
</div>
);
}

export default Dashboard;

```

Screenshot Hasil:



Gambar E.2 – Menggabungkan ke Dashboard Utama

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Integrasi Data Realistik

Pertanyaan / Instruksi Latihan:

Ubah data statis di `GrafikProduksi` menjadi data yang diambil dari `fetch` API (gunakan mock data dari internet atau buat generator acak).

Jawaban / Kode Solusi:

```
import React, { useEffect, useState } from 'react';
import { Bar } from 'react-chartjs-2';
import {
  Chart as ChartJS,
  CategoryScale,
  LinearScale,
  BarElement,
  LineElement,
  PointElement,
  Title,
  Tooltip,
  Legend,
} from 'chart.js';

// Register ChartJS
ChartJS.register(
  CategoryScale,
  LinearScale,
  BarElement,
  LineElement,
  PointElement,
  Title,
```

```
    Tooltip,  
    Legend  
  );  
  
function GrafikProduksi() {  
  
  // State data produksi  
  const [produksi, setProduksi] = useState([]);  
  
  // State loading  
  const [loading, setLoading] = useState(true);  
  
  // Fetch data API  
  useEffect(() => {  
  
    setLoading(true);  
  
    fetch('https://jsonplaceholder.typicode.com/posts')  
  
      .then((res) => res.json())  
  
      .then((data) => {  
  
        // Ambil 6 data pertama lalu ubah jadi angka produksi  
        const hasilProduksi = data.slice(0, 6).map((item) => {  
          return item.id * 20;  
        });  
  
        setProduksi(hasilProduksi);  
  
        setLoading(false);  
      })  
  
      .catch((err) => {  
        console.log(err);  
        setLoading(false);  
      });  
  
    }, []);  
  
  // Label jam produksi  
  const labels = ['08:00', '09:00', '10:00', '11:00', '12:00',  
    '13:00'];  
  
  // Data chart  
  const data = {  
    labels,  
    datasets: [  
      {  
        label: 'Jumlah Produksi (Unit)',  
        data: produksi,  
        backgroundColor: 'rgba(54, 162, 235, 0.5)',  
        borderColor: 'rgba(54, 162, 235, 1)',  
      }  
    ]  
  }  
}
```

```
        borderWidth: 1,
      },
      {
        label: 'Target',
        data: [150, 150, 150, 150, 150, 150],
        type: 'line',
        borderColor: 'rgb(255, 99, 132)',
        borderWidth: 2,
      },
    ],
  };

  // Opsi chart
  const options = {
    responsive: true,
    plugins: {
      legend: {
        position: 'top',
      },
    },
    title: {
      display: true,
      text: 'Grafik Produksi Harian - Lini 1',
    },
  },
  scales: {
    y: {
      beginAtZero: true,
    },
  },
};

  return (
    <div>

      {loading ? (

        <div className="alert alert-info">
          Memuat grafik produksi...
        </div>

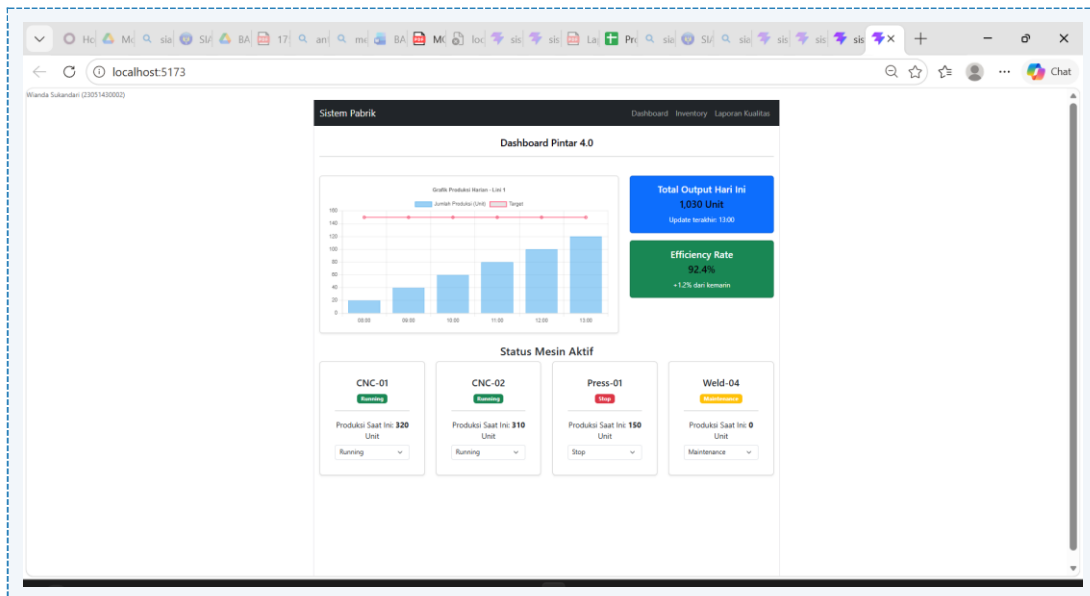
      ) : (

        <Bar data={data} options={options} />

      )}

    </div>
  );
}

export default GrafikProduksi;
```



Gambar F.1 – Integrasi Data Realistik

F.2 Latihan 2 – Tambahkan Grafik Donat

Pertanyaan / Instruksi Latihan:

Buat grafik "Proporsi Cacat" (Misal: Scratch 50%, Dent 30%, Lainnya 20%) menggunakan komponen `Doughnut` dari chart.js.

Jawaban / Kode Solusi:

```
import React from 'react';

import { Doughnut } from 'react-chartjs-2';

import {
  Chart as ChartJS,
  ArcElement,
  Tooltip,
  Legend,
} from 'chart.js';

import ChartDataLabels from 'chartjs-plugin-datalabels';

ChartJS.register(
  ArcElement,
  Tooltip,
  Legend,
  ChartDataLabels
);

function DonatCacat() {

  const data = {
    labels: ['Scratch', 'Dent', 'Lainnya'],
```



```
    datasets: [
      {
        label: 'Proporsi Cacat',

        data: [50, 30, 20],

        backgroundColor: [
          'rgba(255, 99, 132, 0.8)',
          'rgba(54, 162, 235, 0.8)',
          'rgba(255, 206, 86, 0.8)',
        ],

        borderWidth: 2,
      },
    ],
  };

  const options = {
    responsive: true,

    plugins: {

      legend: {
        position: 'bottom',
      },

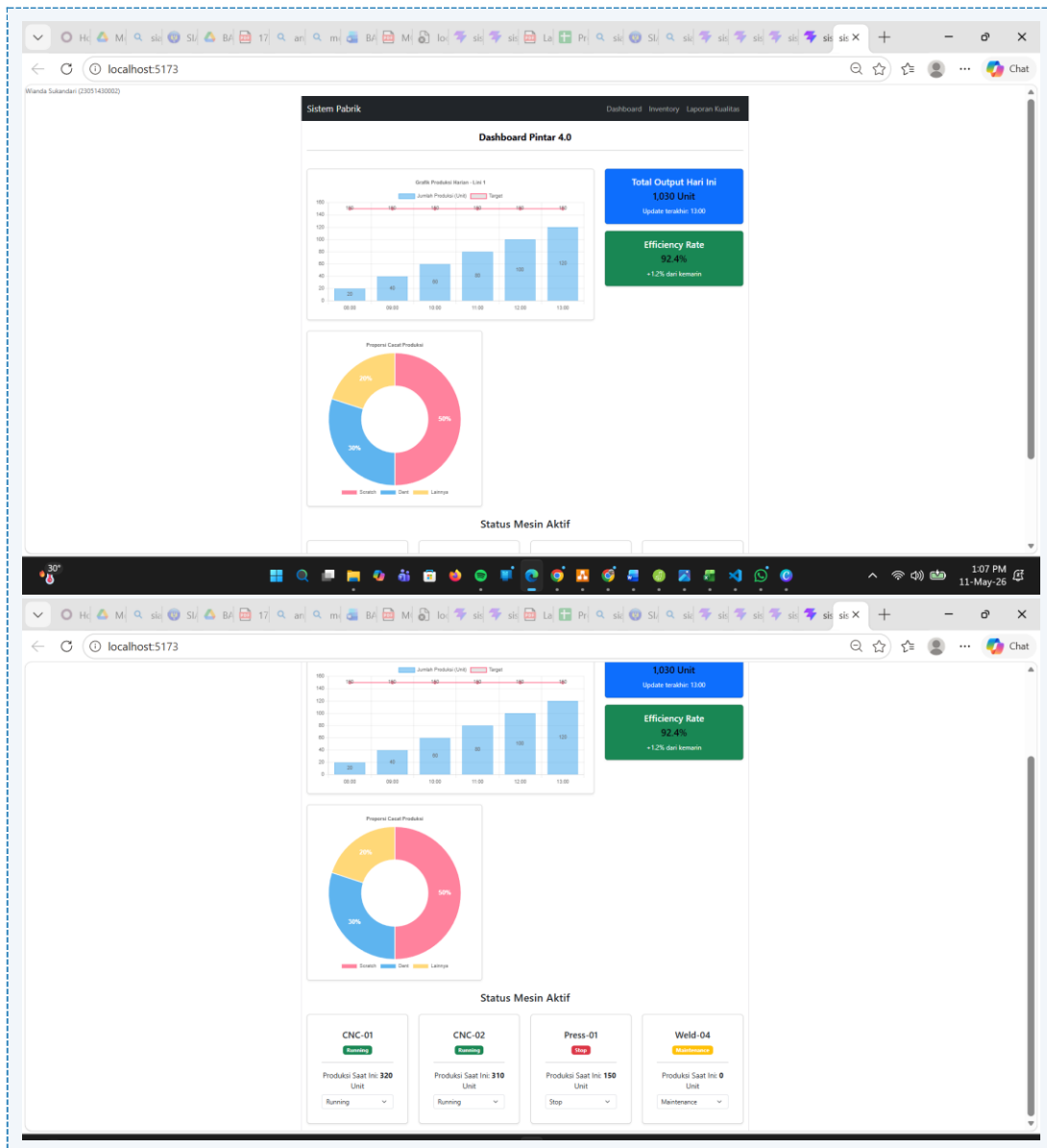
      title: {
        display: true,
        text: 'Proporsi Cacat Produksi',
      },

      // Persentase di chart
      datalabels: {
        color: '#fff',
        font: {
          weight: 'bold',
          size: 16,
        },

        formatter: (value) => {
          return value + '%';
        },
      },
    },
  };

  return <Doughnut data={data} options={options} />;
}

export default DonatCacat;
```



Gambar F.2 – Tambahkan Grafik Donat

F.3 Tugas Proyek Mini – Tugas Akhir Web

Deskripsi proyek mini:

Susun ulang halaman `Dashboard` agar memiliki Sidebar (kiri) dan Main Content (kanan).

- Sidebar: Logo, Menu Dashboard, Menu Inventori, Menu Laporan.
- Main Content: Header (User Profile), Grafik, dan Tabel Data.
- Pastikan desain responsif (Sidebar bisa collapse di HP).

Penjelasan pendekatan/solusi yang digunakan:

Pada mini proyek ini, saya membuat tampilan dashboard monitoring produksi menggunakan React dan Bootstrap dengan konsep responsive layout. Halaman dibagi menjadi dua bagian utama yaitu sidebar di sebelah kiri untuk navigasi menu, serta main content di sebelah kanan untuk menampilkan informasi dashboard seperti grafik produksi, grafik proporsi cacat, tabel data, dan status mesin.

Saya menggunakan React Router untuk navigasi antar halaman seperti Dashboard, Inventory, dan Laporan Kualitas. Selain itu, visualisasi data dibuat menggunakan Chart.js agar data produksi lebih mudah dipahami dalam bentuk grafik batang dan grafik donut. Beberapa data juga dibuat dinamis menggunakan state dan fetch API sederhana untuk mensimulasikan pengambilan data dari sistem industri.

Agar tampilan tetap rapi di berbagai ukuran layar, saya menggunakan sistem grid Bootstrap sehingga layout dapat menyesuaikan otomatis saat dibuka di laptop maupun HP. Dengan pendekatan ini, dashboard menjadi lebih interaktif, modern, dan mendekati tampilan sistem monitoring industri yang digunakan di dunia kerja Teknik Industri.

Kode Utama:

```
import React from "react";
import { Link } from "react-router-dom";

function Sidebar() {
  return (
    <div
      className="bg-dark text-white p-3 h-100"
      style={{ minHeight: "100vh" }}
    >
      <h3 className="mb-4">🏭 Smart Factory</h3>

      <ul className="nav flex-column">

        <li className="nav-item mb-2">
          <Link to="/" className="nav-link text-
white">
            Dashboard
          </Link>
        </li>

        <li className="nav-item mb-2">
          <Link to="/inventory" className="nav-link
text-white">
            Inventory
          </Link>
        </li>

        <li className="nav-item mb-2">
```

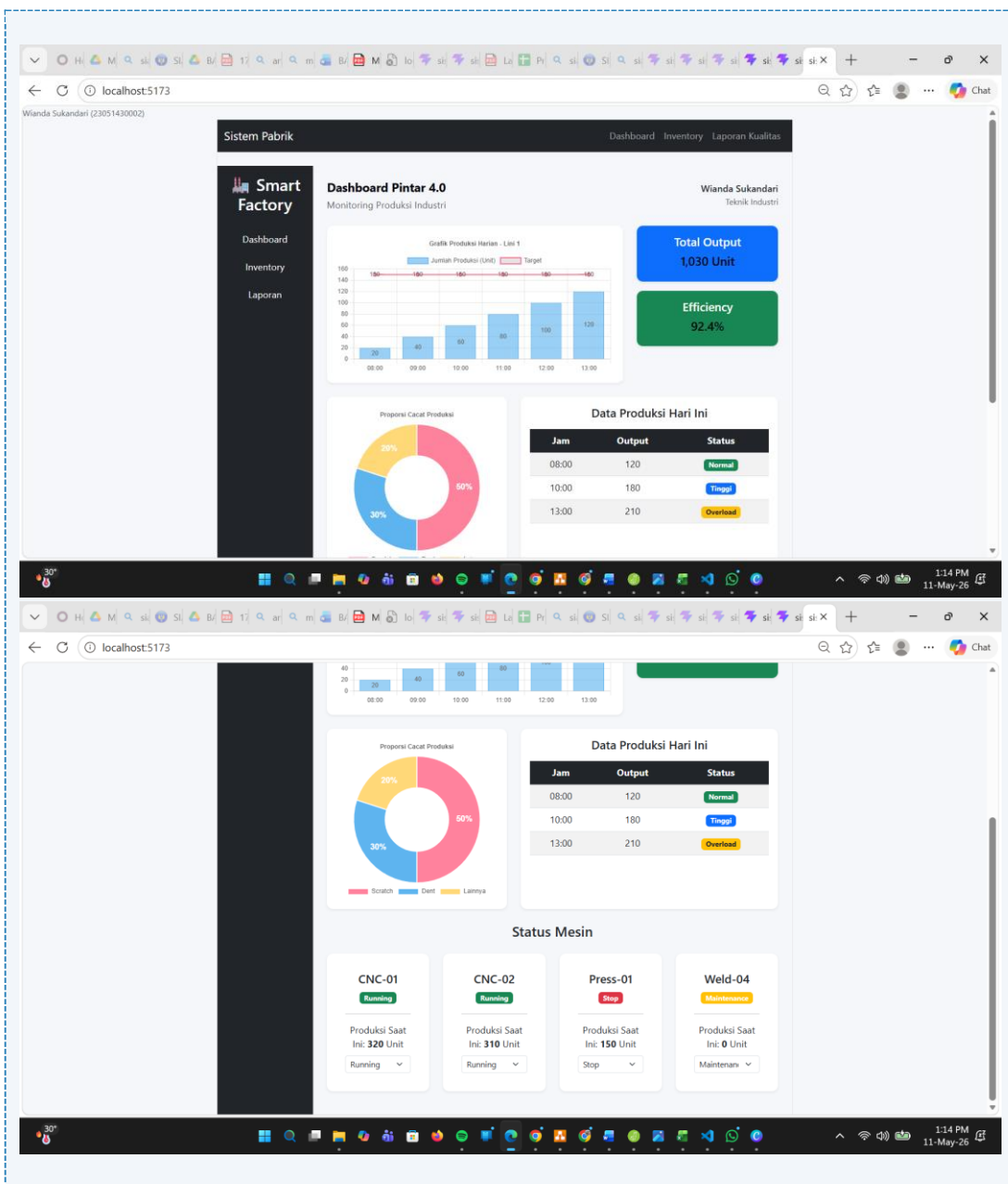
```

                <Link to="/laporan-kualitas"
className="nav-link text-white">
                    Laporan
                </Link>
            </li>

        </ul>
    </div>
);
}

export default Sidebar;

```



Gambar F.3 – Tugas Akhir Web

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori yang dipelajari pada modul React dan Chart.js. Pada praktikum ini berhasil dibuat dashboard monitoring produksi menggunakan React dengan sistem routing menggunakan react-router-dom. Setiap halaman seperti Dashboard, Inventory, dan Laporan Kualitas dapat diakses tanpa reload halaman sehingga sesuai dengan konsep Single Page Application (SPA).

Selain itu, integrasi visualisasi data menggunakan Chart.js juga berhasil diterapkan. Grafik batang digunakan untuk menampilkan data produksi per jam, sedangkan grafik donat digunakan untuk menampilkan proporsi cacat produksi dalam bentuk persentase. Data grafik dapat ditampilkan dengan baik dan responsive pada berbagai ukuran layar, baik desktop maupun perangkat mobile.

Fitur lain seperti penggunaan state, props, dan fetch API juga berjalan sesuai teori. Data inventory berhasil diambil dari API dan ditampilkan ke tabel menggunakan proses rendering dinamis. Sidebar responsive dan layout dashboard juga berhasil dibuat sehingga tampilan aplikasi menjadi lebih rapi dan modern seperti dashboard industri pada umumnya.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Grafik Chart.js tidak muncul saat pertama dijalankan	Melakukan import dan register komponen Chart.js pada file main.jsx dan komponen grafik
Tampilan dashboard berantakan saat dibuka di HP	Menggunakan grid Bootstrap responsive seperti col-lg, col-md, dan col-sm
Routing halaman menyebabkan reload halaman	Mengganti tag <a> menjadi Link dari react-router-dom agar navigasi menjadi SPA

G.3 Kaitan dengan Penerapan di Industri

Materi pada modul ini sangat relevan dengan dunia Teknik Industri, terutama dalam sistem monitoring produksi dan digitalisasi data pabrik. Dashboard monitoring dapat membantu perusahaan melihat kondisi produksi secara real-time sehingga pengambilan keputusan menjadi lebih cepat dan akurat.

Contoh penerapannya adalah pada pabrik manufaktur yang menggunakan dashboard untuk memantau jumlah produksi, efisiensi mesin, dan jumlah cacat

produk setiap jam. Dengan adanya visualisasi grafik, manager produksi dapat lebih mudah menganalisis performa lini produksi dan mengetahui apabila terjadi penurunan output atau peningkatan defect.

Selain itu, sistem inventory dan laporan kualitas juga dapat diterapkan pada industri untuk membantu pengelolaan bahan baku dan pengawasan kualitas produk. Penggunaan teknologi web seperti React memungkinkan sistem menjadi lebih interaktif, cepat, dan responsive sehingga mempermudah pengguna dalam mengakses informasi produksi dari berbagai perangkat.

H. KESIMPULAN

1	Praktikum berhasil menghasilkan aplikasi dashboard monitoring produksi berbasis React yang memiliki fitur routing, visualisasi data, dan tampilan responsive.
2	Penggunaan React Router berhasil diterapkan untuk membuat navigasi antar halaman seperti Dashboard, Inventory, dan Laporan Kualitas tanpa melakukan reload halaman.
3	Integrasi Chart.js berhasil digunakan untuk menampilkan grafik produksi harian dan grafik donat proporsi cacat sehingga data lebih mudah dipahami secara visual.
4	Layout dashboard berhasil dibuat lebih modern menggunakan sidebar dan main content yang responsive sehingga tampilan tetap rapi pada perangkat desktop maupun mobile.
5	Penerapan fetch API, state, props, dan component React membantu pengelolaan data menjadi lebih dinamis serta mendukung konsep pengembangan aplikasi web industri modern.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Documentation . (2025). <i>React Official Documentation</i> . React.dev.
[3]	Chart.js Documentation . (2025). <i>Chart.js Documentation</i> . Chart.js.
[4]	MDN Web Docs JavaScript Fetch API . (2025). <i>Fetch API</i> . Mozilla Developer Network.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 14**Pengenalan React Native & Setup Expo****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Pengenalan React Native & Setup Expo (Pertemuan 14)
Topik Utama	Pengenalan React Native & Setup Expo
Sub-Topik	Konsep Cross-Platform, Setup Lingkungan Expo, Komponen Dasar (View, Text, Image), dan Styling di Mobile.
Durasi Praktikum	340 Menit
Tanggal Praktikum	13 mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami penggunaan komponen dasar React Native seperti View, Text, Image, TouchableOpacity, dan ScrollView.
2	Mempelajari cara membuat tampilan antarmuka (UI) aplikasi mobile sederhana menggunakan React Native dan StyleSheet.
3	Memahami penerapan layout Flexbox khususnya flexDirection: 'row' untuk menyusun komponen secara horizontal.
4	Mampu membuat aplikasi mobile sederhana bertema industri/manufaktur yang menampilkan informasi mesin atau monitoring gudang.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Expo Go	Versi terbaru	Menjalankan aplikasi React Native di smartphone
5	React Native	Versi terbaru	Framework untuk membuat aplikasi mobile

D. DASAR TEORI

D.1 Konsep Utama

Pada praktikum ini dipelajari dasar pembuatan aplikasi mobile menggunakan React Native dan Expo. React Native digunakan untuk membuat tampilan aplikasi mobile dengan bahasa pemrograman JavaScript, sehingga satu kode dapat dijalankan di Android maupun iOS. Dalam praktikum ini digunakan beberapa komponen dasar seperti View, Text, Image, TouchableOpacity, dan ScrollView untuk membangun tampilan aplikasi monitoring gudang dan profil mesin mobile.

Selain itu, praktikum ini juga membahas penggunaan Flexbox dalam mengatur layout aplikasi. Dengan properti seperti flexDirection, justifyContent, dan alignItems, tampilan aplikasi dapat dibuat lebih rapi dan responsif. Pada proyek mini, Flexbox digunakan untuk menempatkan gambar mesin di sebelah kiri dan informasi mesin di sebelah kanan sehingga tampilan menjadi lebih mudah dibaca oleh pengguna.

Praktikum ini juga mengajarkan cara menghubungkan aset gambar ke dalam aplikasi menggunakan folder assets dan fungsi require(). Selain itu dipelajari juga cara menangani error saat proses coding, seperti error gambar yang tidak ditemukan atau kesalahan penulisan path file. Dengan adanya praktikum ini,

mahasiswa dapat memahami dasar pengembangan antarmuka aplikasi mobile yang sederhana namun tetap fungsional dan terstruktur.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini sangat relevan dengan dunia Teknik Industri, terutama dalam bidang sistem informasi industri dan digitalisasi proses kerja. Aplikasi mobile dapat digunakan untuk membantu monitoring produksi, pengecekan stok gudang, hingga pemantauan kondisi mesin secara real-time sehingga proses kerja menjadi lebih efektif dan efisien.

Contoh penerapannya di industri yaitu pada perusahaan manufaktur seperti Yamaha atau Pertamina yang membutuhkan aplikasi monitoring gudang dan mesin produksi. Operator dapat melihat status kapasitas gudang, kondisi mesin, ataupun jadwal maintenance langsung melalui aplikasi mobile tanpa harus melakukan pengecekan manual secara terus-menerus.

Selain itu, keterampilan membuat tampilan aplikasi juga berguna dalam pengembangan sistem pendukung keputusan di industri. Dengan tampilan yang rapi dan mudah digunakan, informasi penting dapat disampaikan dengan lebih cepat kepada operator maupun manajer. Hal ini dapat membantu pengambilan keputusan yang lebih tepat serta meningkatkan produktivitas kerja di lingkungan industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat Project Expo

Deskripsi singkat apa yang dilakukan pada langkah ini:

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
```

Buka Terminal/Command Prompt di komputer. Jalankan perintah berikut (tunggu proses instalasi sekitar 2-5 menit):

- 1) Membuat project baru bernama 'SistemGudangMobile'
`npx create-expo-app SistemGudangMobile`
- 2) Masuk ke folder
`cd SistemGudangMobile`
- 3) Menjalankan server development
`npx expo start`

Setelah muncul QR Code di terminal:

- 1) Buka aplikasi Expo Go di HP Android/iOS Anda.
- 2) Scan QR Code tersebut (Pastikan HP dan Laptop terhubung ke jaringan WiFi yang sama).
- 3) Aplikasi akan terbuka di HP dan menampilkan tulisan "Open up App.js to start working on your app!".

Screenshot Hasil:



Gambar E.1 – Membuat Project Expo

E.2 Langkah 2 – Memodifikasi App.js (UI Dashboard Sederhana)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
```

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform } from
'react-native';
export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />
      {/* Bagian Header */}
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur
Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi Monitoring
Gudang</Text>
```

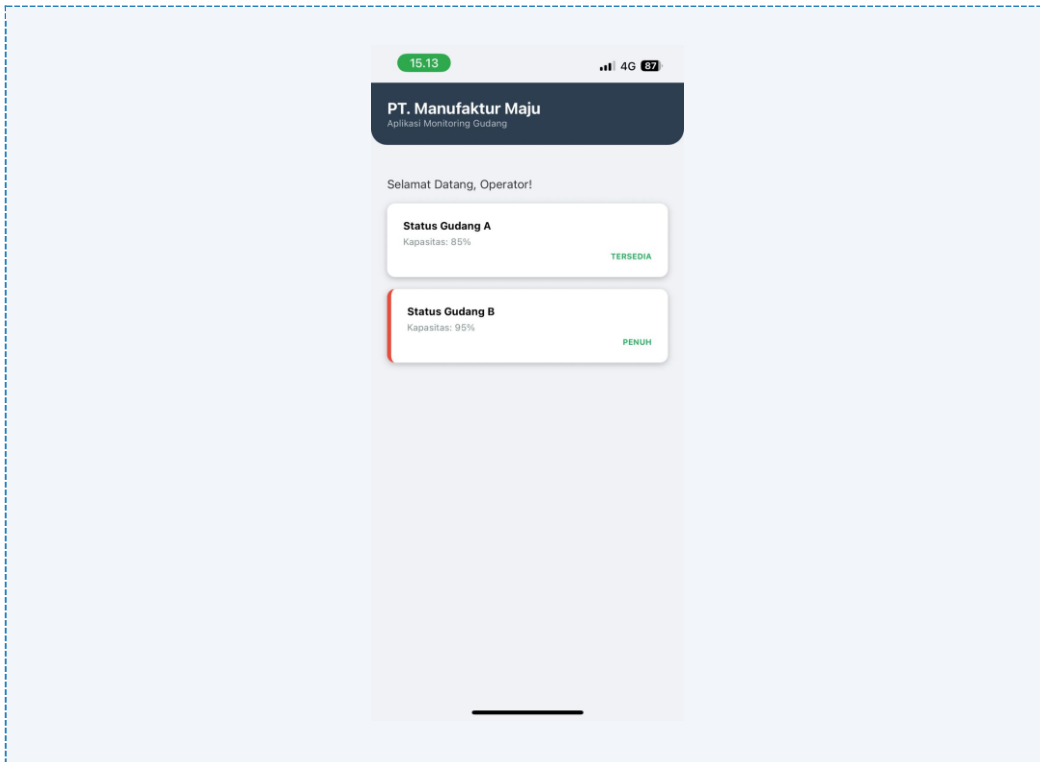
```

    </View>
    { /* Bagian Konten Utama */
    <View style={styles.content}>
      <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>
      <View style={styles.card}>
        <Text style={styles.cardTitle}>Status Gudang A</Text>
        <Text style={styles.cardValue}>Kapasitas: 85%</Text>
        <Text style={styles.cardStatus}>TERSEDIA</Text>
      </View>
      <View style={[styles.card, styles.cardWarning]}>
        <Text style={styles.cardTitle}>Status Gudang B</Text>
        <Text style={styles.cardValue}>Kapasitas: 95%</Text>
        <Text style={styles.cardStatus}>PENUH</Text>
      </View>
    </View>
  </SafeAreaView>
);
}
// Styling menggunakan StyleSheet (seperti CSS tapi object JS)
const styles = StyleSheet.create({
  container: {
    flex: 1, // Artinya layar penuh
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0, // Padding
    khusus Android status bar
  },
  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
    elevation: 5, // Shadow di Android
  },
  headerTitle: {
    color: 'white',
    fontSize: 24,
    fontWeight: 'bold',
  },
  headerSubtitle: {
    color: '#bdc3c7',
    fontSize: 14,
  },
  content: {
    padding: 20,
  },
  welcomeText: {
    fontSize: 18,
    marginBottom: 15,
    color: '#333',
  },
  card: {

```

```
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 15,
        // Shadow khusus iOS dan Android
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
    },
    cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c',
    },
    cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
    },
    cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
    },
    cardStatus: {
        fontSize: 12,
        fontWeight: 'bold',
        color: '#27ae60',
        marginTop: 5,
        textAlign: 'right'
    }
    });
```

Screenshot Hasil:



Gambar E.2 – Memodifikasi App.js (UI Dashboard Sederhana)

E.3 Langkah 3 – Menambahkan Interaksi (Button & Alert)

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert } from 'react-native';
export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />
      { /* Bagian Header */ }
      <View style={styles.header}>
        <Text style={styles.headerTitle}>PT. Manufaktur
Maju</Text>
        <Text style={styles.headerSubtitle}>Aplikasi
Monitoring Gudang</Text>
      </View>
      { /* Bagian Konten Utama */ }
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>
        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail
Stok Gudang A...")}
        >
          <Text style={styles.cardTitle}>Status Gudang
A</Text>

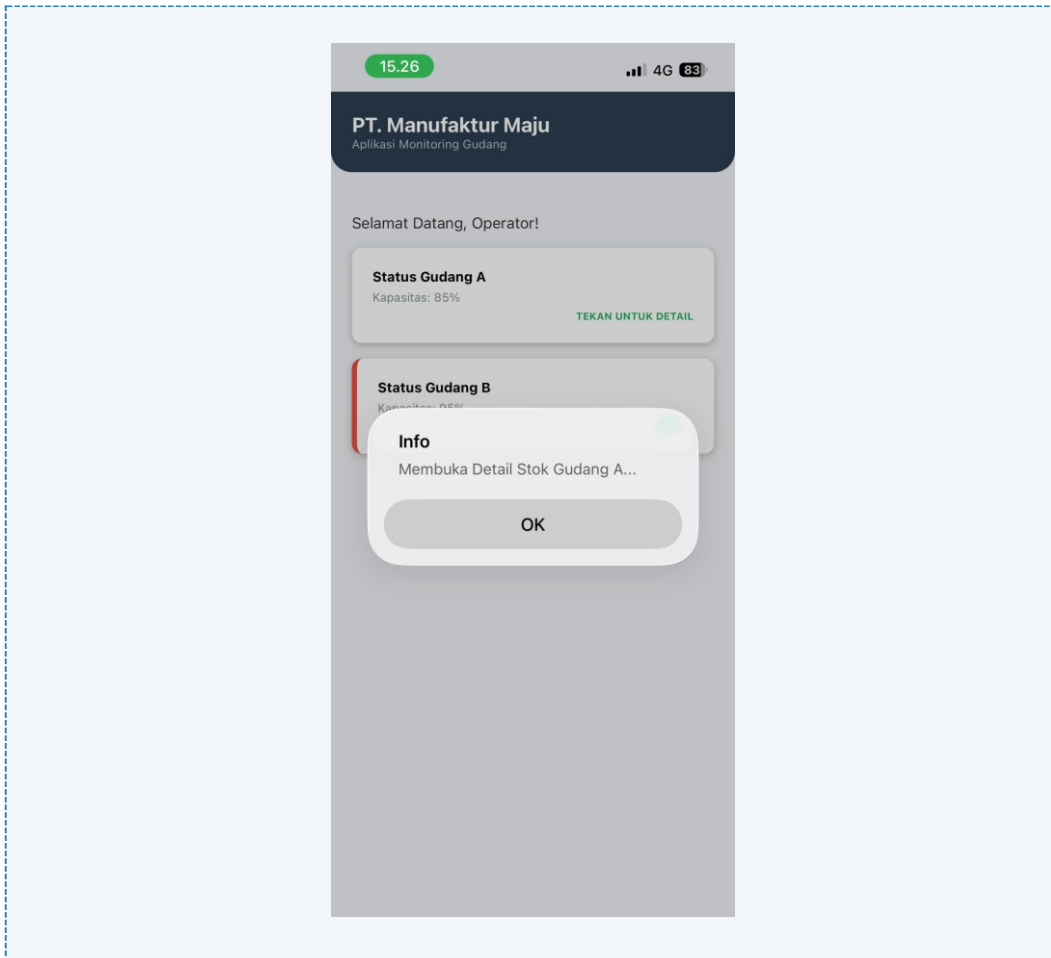
```



```
        <Text style={styles.cardValue}>Kapasitas: 85%</Text>
        <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
        </TouchableOpacity>
        <View style={[styles.card, styles.cardWarning]}>
            <Text style={styles.cardTitle}>Status Gudang
B</Text>
            <Text style={styles.cardValue}>Kapasitas: 95%</Text>
            <Text style={styles.cardStatus}>PENUH</Text>
        </View>
    </View>
</SafeAreaView>
);
}
// Styling menggunakan StyleSheet (seperti CSS tapi object JS)
const styles = StyleSheet.create({
    container: {
        flex: 1, // Artinya layar penuh
        backgroundColor: '#f0f2f5',
        paddingTop: Platform.OS === 'android' ? 25 : 0, // Padding
khusus Android status bar
    },
    header: {
        backgroundColor: '#2c3e50',
        padding: 20,
        borderBottomLeftRadius: 20,
        borderBottomRightRadius: 20,
        marginBottom: 20,
        elevation: 5, // Shadow di Android
    },
    headerTitle: {
        color: 'white',
        fontSize: 24,
        fontWeight: 'bold',
    },
    headerSubtitle: {
        color: '#bdc3c7',
        fontSize: 14,
    },
    content: {
        padding: 20,
    },
    welcomeText: {
        fontSize: 18,
        marginBottom: 15,
        color: '#333',
    },
    card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 15,
        // Shadow khusus iOS dan Android
    },
});
```

```
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
      },
      cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c',
      },
      cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
      },
      cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
      },
      cardStatus: {
        fontSize: 12,
        fontWeight: 'bold',
        color: '#27ae60',
        marginTop: 5,
        textAlign: 'right'
      }
    }
  });
```

Screenshot Hasil:



Gambar E.3 – Menambahkan Interaksi (Button & Alert)

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Gambar/Ikon

Pertanyaan / Instruksi Latihan:

Cari gambar logo perusahaan (atau gunakan placeholder), taruh di folder project, lalu tampilkan menggunakan ``.

Jawaban / Kode Solusi:

```
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert, Image } from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      { /* Bagian Header */ }
      <View style={styles.header}>
        <Image source={require('./assets/logo.jpg')}
style={styles.logo} />
        <Text style={styles.headerTitle}>PT. YAMAHA</Text>
        <Text style={styles.headerSubtitle}>Aplikasi
Monitoring Gudang</Text>
        <Text style={styles.headerSubtitle}>Wianda Sukandari
(23051430002)</Text>
      </View>

      { /* Bagian Konten Utama */ }
      <View style={styles.content}>
        <Text style={styles.welcomeText}>Selamat Datang,
Operator!</Text>

        <TouchableOpacity
          style={styles.card}
          onPress={() => Alert.alert("Info", "Membuka Detail
Stok Gudang A...")}
        >
          <Text style={styles.cardTitle}>Status Gudang
A</Text>
          <Text style={styles.cardValue}>Kapasitas: 85%</Text>
          <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
        </TouchableOpacity>

        <View style={[styles.card, styles.cardWarning]}>
          <Text style={styles.cardTitle}>Status Gudang
B</Text>
          <Text style={styles.cardValue}>Kapasitas: 95%</Text>
          <Text style={styles.cardStatus}>PENUH</Text>
        </View>
      </View>
    </SafeAreaView>
  );
}

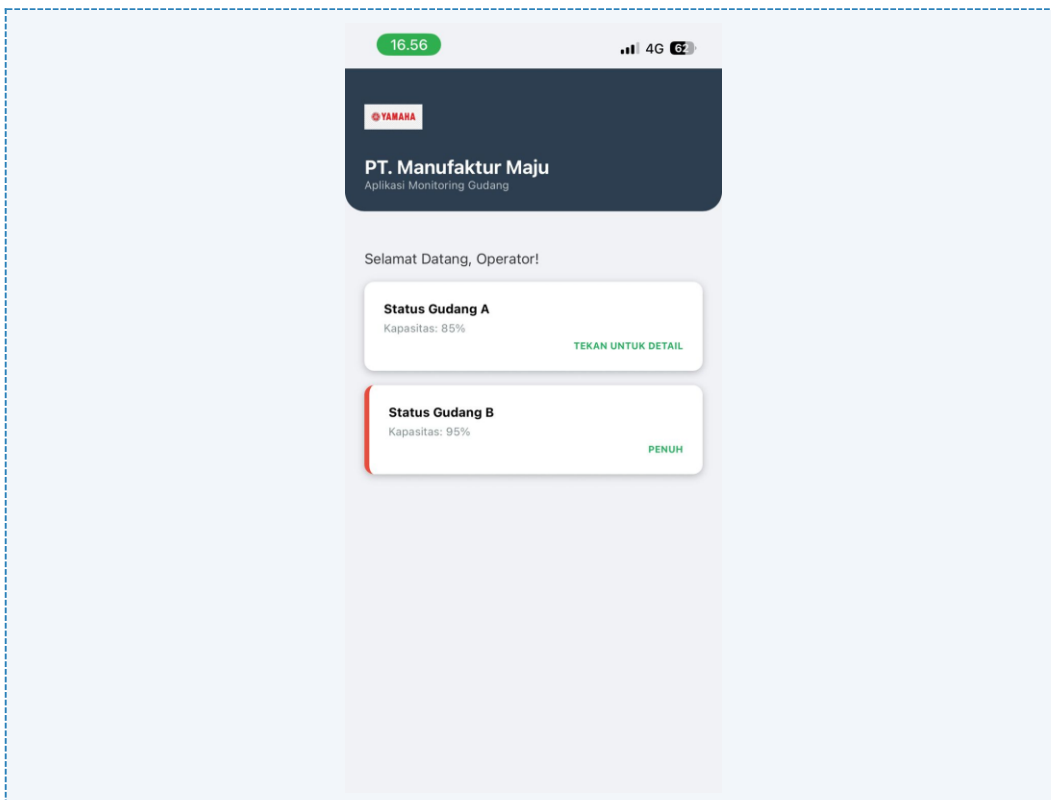
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
```

```
},
header: {
  backgroundColor: '#2c3e50',
  padding: 20,
  borderBottomLeftRadius: 20,
  borderBottomRightRadius: 20,
  marginBottom: 20,
  elevation: 5,
},
logo: {
  width: 60,
  height: 60,
  marginBottom: 10,
  resizeMode: 'contain',
},
headerTitle: {
  color: 'white',
  fontSize: 24,
  fontWeight: 'bold',
},
headerSubtitle: {
  color: '#bdc3c7',
  fontSize: 14,
},
content: {
  padding: 20,
},
welcomeText: {
  fontSize: 18,
  marginBottom: 15,
  color: '#333',
},
card: {
  backgroundColor: 'white',
  padding: 20,
  borderRadius: 10,
  marginBottom: 15,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.25,
  shadowRadius: 3.84,
  elevation: 5,
},
cardWarning: {
  borderLeftWidth: 5,
  borderLeftColor: '#e74c3c',
},
cardTitle: {
  fontSize: 16,
  fontWeight: 'bold',
  marginBottom: 5,
},
cardValue: {
```

```

    fontSize: 14,
    color: '#7f8c8d',
  },
  cardStatus: {
    fontSize: 12,
    fontWeight: 'bold',
    color: '#27ae60',
    marginTop: 5,
    textAlign: 'right'
  }
});

```



Gambar F.1 – Gambar/Ikon

F.2 Latihan 2 – ScrollView

Pertanyaan / Instruksi Latihan:

Jika konten melebihi tinggi layar, gunakan `` untuk membungkus konten agar bisa digulir ke bawah.

Jawaban / Kode Solusi:

```

import { StatusBar } from 'expo-status-bar';
import { StatusBar } from 'expo-status-bar';
import { StyleSheet, Text, View, SafeAreaView, Platform,
TouchableOpacity, Alert, Image, ScrollView } from 'react-
native';

```

```
export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      <ScrollView>
        { /* Bagian Header */ }
        <View style={styles.header}>
          <Image source={require('./assets/logo.jpg')}
style={styles.logo} />
          <Text style={styles.headerTitle}>PT. YAMAHA</Text>
          <Text style={styles.headerSubtitle}>Aplikasi
Monitoring Gudang</Text>
          <Text style={styles.headerSubtitle}>Wianda Sukandari
(23051430002)</Text>
        </View>

        { /* Bagian Konten Utama */ }
        <View style={styles.content}>
          <Text style={styles.welcomeText}>Selamat Datang,
Wianda!</Text>

          <TouchableOpacity
            style={styles.card}
            onPress={() => Alert.alert("Info", "Membuka Detail
Stok Gudang A...")}
          >
            <Text style={styles.cardTitle}>Gudang
Sparepart</Text>
            <Text style={styles.cardValue}>Kapasitas:
85%</Text>
            <Text style={styles.cardStatus}>TEKAN UNTUK
DETAIL</Text>
          </TouchableOpacity>

          <View style={[styles.card, styles.cardWarning]}>
            <Text style={styles.cardTitle}>Gudang Helm</Text>
            <Text style={styles.cardValue}>Kapasitas:
95%</Text>
            <Text style={styles.cardStatus}>PENUH</Text>
          </View>

          <View style={styles.card}>
            <Text style={styles.cardTitle}>Gudang Oli
Mesin</Text>
            <Text style={styles.cardValue}>Kapasitas:
70%</Text>
            <Text style={styles.cardStatus}>AMAN</Text>
          </View>

          <View style={styles.card}>
            <Text style={styles.cardTitle}>Gudang Ban
Motor</Text>
```

```

        <Text style={styles.cardValue}>Kapasitas:
60%</Text>
        <Text style={styles.cardStatus}>STOK
TERSEDIA</Text>
    </View>

    <View style={[styles.card, styles.cardWarning]}>
        <Text style={styles.cardTitle}>Gudang
Aksesoris</Text>
        <Text style={styles.cardValue}>Kapasitas:
98%</Text>
        <Text style={styles.cardStatus}>HAMPIR
PENUH</Text>
    </View>

    <View style={styles.card}>
        <Text style={styles.cardTitle}>Gudang Mesin
Motor</Text>
        <Text style={styles.cardValue}>Kapasitas:
55%</Text>
        <Text style={styles.cardStatus}>NORMAL</Text>
    </View>

    <View style={styles.card}>
        <Text style={styles.cardTitle}>Gudang Body
Motor</Text>
        <Text style={styles.cardValue}>Kapasitas:
75%</Text>
        <Text style={styles.cardStatus}>STABIL</Text>
    </View>

    <View style={[styles.card, styles.cardWarning]}>
        <Text style={styles.cardTitle}>Gudang Cat
Motor</Text>
        <Text style={styles.cardValue}>Kapasitas:
92%</Text>
        <Text style={styles.cardStatus}>PERLU
DIKOSONGKAN</Text>
    </View>

    </View>
</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },
  header: {

```

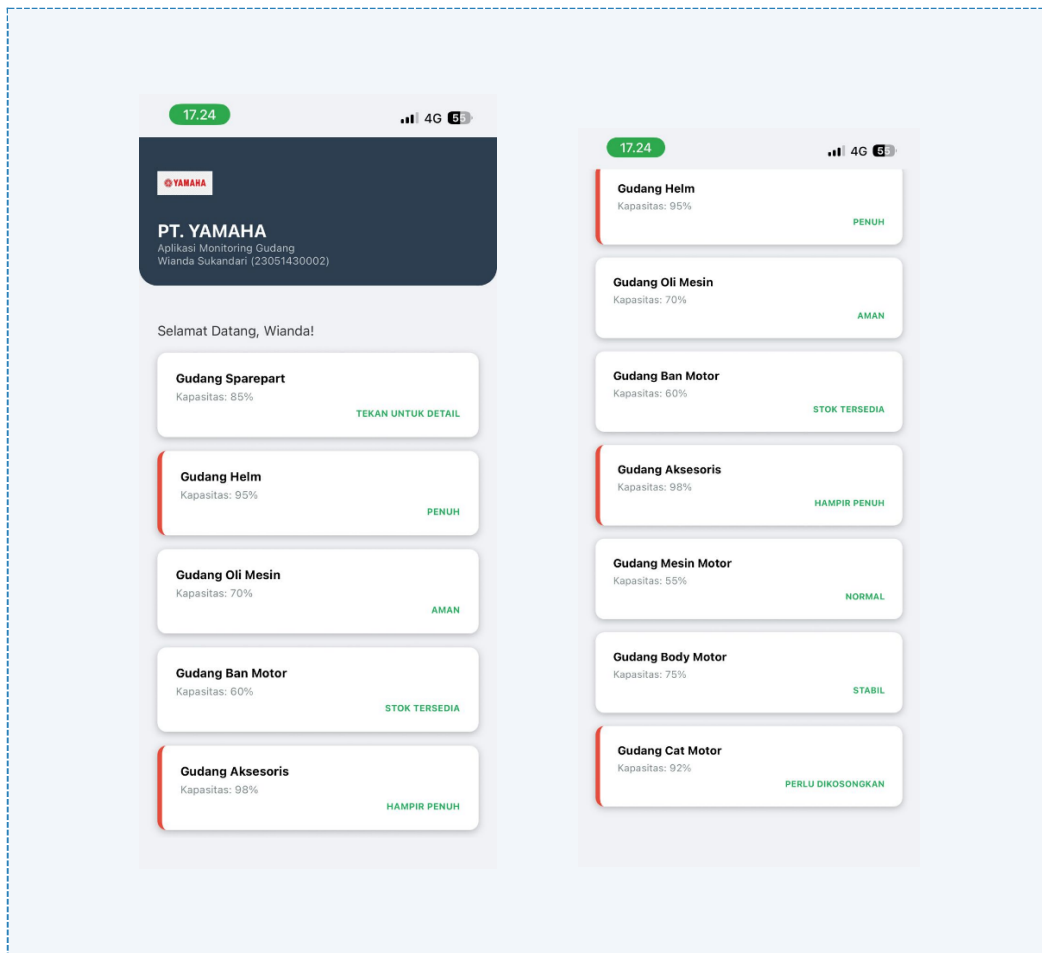


```
        backgroundColor: '#2c3e50',
        padding: 20,
        borderBottomLeftRadius: 20,
        borderBottomRightRadius: 20,
        marginBottom: 20,
        elevation: 5,
    },
    logo: {
        width: 60,
        height: 60,
        marginBottom: 10,
        resizeMode: 'contain',
    },
    headerTitle: {
        color: 'white',
        fontSize: 24,
        fontWeight: 'bold',
    },
    headerSubtitle: {
        color: '#bdc3c7',
        fontSize: 14,
    },
    content: {
        padding: 20,
    },
    welcomeText: {
        fontSize: 18,
        marginBottom: 15,
        color: '#333',
    },
    card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 15,
        shadowColor: "#000",
        shadowOffset: { width: 0, height: 2 },
        shadowOpacity: 0.25,
        shadowRadius: 3.84,
        elevation: 5,
    },
    cardWarning: {
        borderLeftWidth: 5,
        borderLeftColor: '#e74c3c',
    },
    cardTitle: {
        fontSize: 16,
        fontWeight: 'bold',
        marginBottom: 5,
    },
    cardValue: {
        fontSize: 14,
        color: '#7f8c8d',
    },
}
```

```

},
cardStatus: {
  fontSize: 12,
  fontWeight: 'bold',
  color: '#27ae60',
  marginTop: 5,
  textAlign: 'right'
}
});

```



Gambar F.2 – ScrollView

F.3 Tugas Proyek Mini – Profil Mesin Mobile

Deskripsi proyek mini:

- Buat halaman profil mesin yang menampilkan: Foto Mesin (gunakan placeholder), Nama Mesin, Tahun Pembuatan, dan Status.
- Gunakan Layout Flexbox ('flexDirection: 'row') untuk menyusun Foto di kiri dan Teks di kanan.

Penjelasan pendekatan/solusi yang digunakan:

Proyek mini ini dibuat dengan menggunakan React Native dan komponen bawaan seperti View, Text, Image, dan ScrollView. Tampilan aplikasi dibuat menyerupai halaman profil mesin di perusahaan dengan menampilkan foto mesin, nama mesin, tahun pembuatan, dan status mesin.

Layout disusun menggunakan Flexbox dengan flexDirection: 'row' supaya gambar mesin berada di sebelah kiri dan informasi mesin berada di sebelah kanan. Selain itu, digunakan ScrollView agar halaman tetap bisa digulir jika jumlah data mesin banyak dan melebihi ukuran layar HP.

Setiap data mesin dimasukkan ke dalam card supaya tampilannya lebih rapi dan mudah dibaca. Styling dilakukan menggunakan StyleSheet agar warna, ukuran teks, jarak, dan bentuk card terlihat lebih menarik dan nyaman digunakan.

Kode Utama:

```
import { StatusBar } from 'expo-status-bar';
import
{StyleSheet,Text,View,SafeAreaView,Platform,Image,ScrollView}
from 'react-native';

export default function App() {
  return (
    <SafeAreaView style={styles.container}>
      <StatusBar style="auto" />

      <ScrollView>
        { /* Header */}
        <View style={styles.header}>
          <Text style={styles.headerTitle}>PT. YAMAHA</Text>
          <Text style={styles.headerSubtitle}>Profil Mesin
Mobile</Text>
          <Text style={styles.headerSubtitle}>Wianda
Sukandari (23051430002)</Text>
        </View>

        { /* Konten */}
        <View style={styles.content}>

          { /* Mesin 1 */}
          <View style={styles.card}>
            <Image
              source={require('./assets/mesin1.jpg')}
              style={styles.machineImage}
            />

            <View style={styles.infoContainer}>
              <Text style={styles.machineName}>Mesin
CNC</Text>
```

```

                <Text style={styles.machineText}>Tahun:
2020</Text>
                <Text style={styles.machineStatus}>Status:
Aktif</Text>
            </View>
        </View>

        { /* Mesin 2 */ }
        <View style={styles.card}>
            <Image
                source={require('./assets/mesin2.jpg')}
                style={styles.machineImage}
            />

            <View style={styles.infoContainer}>
                <Text style={styles.machineName}>Mesin
Bubut</Text>
                <Text style={styles.machineText}>Tahun:
2019</Text>
                <Text style={styles.machineStatus}>Status:
Perawatan</Text>
            </View>
        </View>

        { /* Mesin 3 */ }
        <View style={styles.card}>
            <Image
                source={require('./assets/mesin3.jpg')}
                style={styles.machineImage}
            />

            <View style={styles.infoContainer}>
                <Text style={styles.machineName}>Mesin
Las</Text>
                <Text style={styles.machineText}>Tahun:
2021</Text>
                <Text style={styles.machineStatus}>Status:
Aktif</Text>
            </View>
        </View>

        { /* Mesin 4 */ }
        <View style={styles.card}>
            <Image
                source={require('./assets/mesin4.jpg')}
                style={styles.machineImage}
            />

            <View style={styles.infoContainer}>
                <Text style={styles.machineName}>Mesin
Press</Text>
                <Text style={styles.machineText}>Tahun:
2018</Text>

```

```
                <Text style={styles.machineStatus}>Status:
Aktif</Text>
            </View>
        </View>

        </View>
    </ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f0f2f5',
    paddingTop: Platform.OS === 'android' ? 25 : 0,
  },

  header: {
    backgroundColor: '#2c3e50',
    padding: 20,
    borderBottomLeftRadius: 20,
    borderBottomRightRadius: 20,
    marginBottom: 20,
  },

  headerTitle: {
    color: 'white',
    fontSize: 24,
    fontWeight: 'bold',
  },

  headerSubtitle: {
    color: '#bdc3c7',
    fontSize: 14,
  },

  content: {
    padding: 20,
  },

  card: {
    backgroundColor: 'white',
    borderRadius: 10,
    padding: 15,
    marginBottom: 15,

    flexDirection: 'row',

    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 3.84,
```

```
        elevation: 5,
      },

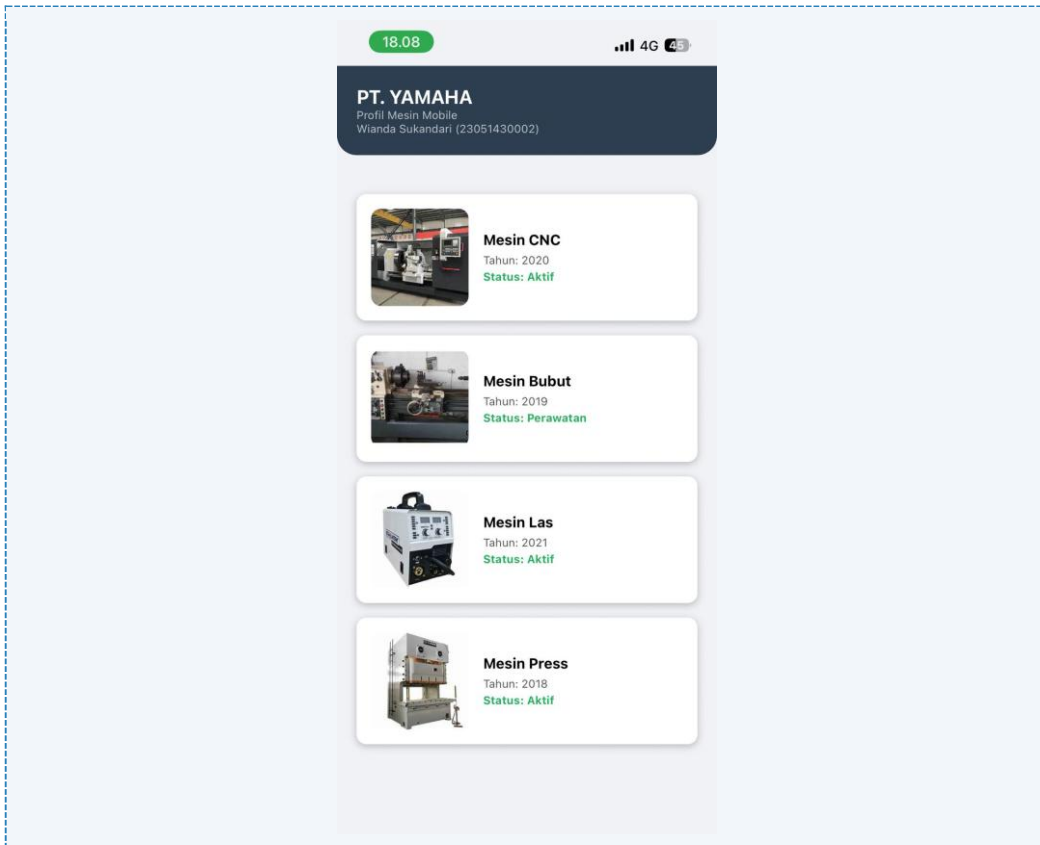
      machineImage: {
        width: 100,
        height: 100,
        borderRadius: 10,
        marginRight: 15,
      },

      infoContainer: {
        justifyContent: 'center',
      },

      machineName: {
        fontSize: 18,
        fontWeight: 'bold',
        marginBottom: 5,
      },

      machineText: {
        fontSize: 14,
        color: '#555',
        marginBottom: 3,
      },

      machineStatus: {
        fontSize: 14,
        fontWeight: 'bold',
        color: '#27ae60',
      },
    },
  });
```



Gambar F.3 – Profil Mesin Mobile

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori dan konsep React Native yang dipelajari pada modul ini. Pada proyek mini, aplikasi berhasil menampilkan halaman profil mesin dengan menggunakan komponen dasar seperti View, Text, Image, dan ScrollView. Selain itu, layout juga berhasil disusun menggunakan konsep Flexbox dengan properti flexDirection: 'row', sehingga gambar mesin tampil di sebelah kiri dan informasi mesin tampil di sebelah kanan sesuai instruksi praktikum.

Hasil output aplikasi menunjukkan bahwa setiap card mesin dapat ditampilkan dengan rapi dan responsif di layar mobile. Penggunaan ScrollView juga bekerja dengan baik karena ketika jumlah card mesin ditambah, halaman tetap bisa digulir ke bawah tanpa ada konten yang terpotong. Hal ini membuktikan bahwa konsep scrolling pada React Native telah diterapkan dengan benar.

Selain itu, penggunaan StyleSheet juga sesuai teori karena styling berhasil diterapkan untuk mengatur warna, ukuran teks, shadow, border radius, dan tata letak card agar tampilan aplikasi lebih menarik dan mudah dibaca. Dengan demikian, hasil akhir aplikasi sudah sesuai dengan konsep UI mobile sederhana yang dipelajari pada praktikum.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Gambar mesin tidak muncul dan muncul error Unable to resolve ./assets/mesin1.jpg	Memastikan file gambar benar-benar berada di folder assets dan nama file sesuai dengan yang ditulis di kode
Logo tidak tampil pada aplikasi	Mengganti format gambar dan memperbaiki path <code>require('./assets/logo.jpg')</code> agar sesuai dengan lokasi file
Tampilan card terlalu panjang sehingga tidak terlihat semua di layar	Menambahkan komponen ScrollView agar halaman dapat di-scroll ke bawah

G.3 Kaitan dengan Penerapan di Industri

Keterampilan yang dipelajari pada modul ini sangat berguna dalam dunia Teknik Industri, khususnya pada pengembangan sistem monitoring berbasis mobile di perusahaan manufaktur. Dengan React Native, perusahaan dapat membuat aplikasi monitoring produksi, gudang, maupun mesin yang dapat diakses langsung melalui smartphone.

Contoh penerapannya adalah pada perusahaan otomotif seperti Yamaha. Operator produksi dapat menggunakan aplikasi mobile untuk melihat status mesin, kondisi produksi, atau kapasitas gudang secara real-time. Dengan tampilan card dan informasi yang tersusun rapi menggunakan Flexbox, data menjadi lebih mudah dibaca dan dipahami oleh operator maupun supervisor.

Selain itu, penggunaan ScrollView juga penting dalam aplikasi industri karena data yang ditampilkan biasanya sangat banyak, seperti daftar mesin, stok barang, atau laporan produksi harian. Dengan adanya fitur scroll, pengguna tetap dapat melihat seluruh informasi dengan nyaman melalui perangkat mobile tanpa membuat tampilan aplikasi menjadi berantakan.

H. KESIMPULAN

1	Pada praktikum ini berhasil dibuat aplikasi mobile sederhana menggunakan React Native untuk menampilkan informasi profil mesin dan monitoring gudang.
2	Komponen dasar React Native seperti View, Text, Image, TouchableOpacity, dan ScrollView berhasil digunakan sesuai fungsi masing-masing dalam membangun tampilan aplikasi.
3	Layout aplikasi berhasil disusun menggunakan konsep Flexbox dengan flexDirection: 'row', sehingga gambar dan informasi mesin dapat ditampilkan secara rapi dan mudah dibaca.
4	Penggunaan ScrollView membuat aplikasi mampu menampilkan banyak data/card tanpa terpotong karena halaman dapat digulir ke bawah.
5	Praktikum ini memberikan pemahaman tentang dasar pembuatan antarmuka aplikasi mobile yang dapat diterapkan pada sistem monitoring industri, gudang, maupun mesin produksi di dunia kerja nyata.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Native Documentation . (2026). <i>React Native Official Documentation</i> .
[3]	Expo Documentation . (2026). <i>Expo Documentation for React Native Development</i> .

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

MODUL 15**Component, Navigation, dan Styling di React Native + Proyek Mobile****A. IDENTITAS & INFORMASI MODUL**

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web Dan Mobile Manual Praktikum (Pertemuan 15)
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
---	---------------	-------------------	--------------------------------

4	Expo CLI / Expo Go	Versi terbaru	Menjalankan aplikasi React Native
5	React Navigation & React Native Picker	Versi terbaru	Navigasi antar halaman dan dropdown status inspeksi

D. DASAR TEORI

D.1 Konsep Utama

Pada praktikum ini dipelajari konsep navigasi pada aplikasi mobile menggunakan React Navigation, khususnya Stack Navigator. Stack Navigator bekerja seperti tumpukan halaman, di mana halaman baru akan ditambahkan di atas halaman sebelumnya. Ketika pengguna berpindah ke halaman lain, halaman sebelumnya tetap tersimpan sehingga pengguna dapat kembali ke halaman sebelumnya menggunakan tombol back.

Selain navigasi, praktikum ini juga membahas proses pengiriman data antar layar (passing parameter). Data dari halaman daftar inventori dapat dikirim ke halaman detail melalui parameter navigasi. Dengan cara ini, informasi yang dipilih pengguna pada halaman utama dapat ditampilkan secara lengkap pada halaman detail tanpa perlu membuat data baru.

Konsep lain yang dipelajari adalah pembuatan aplikasi multi-halaman sederhana menggunakan React Native. Aplikasi terdiri dari halaman daftar item dan halaman detail item yang saling terhubung melalui navigasi. Penerapan konsep ini membantu dalam memahami struktur aplikasi mobile yang lebih terorganisir dan mudah dikembangkan untuk kebutuhan yang lebih kompleks.

D.2 Keterkaitan dengan Teknik Industri

Materi navigasi aplikasi mobile memiliki keterkaitan dengan bidang Teknik Industri karena banyak sistem informasi industri saat ini berbasis aplikasi mobile. Perusahaan membutuhkan aplikasi yang mampu menampilkan data secara terstruktur sehingga pengguna dapat berpindah dari satu halaman ke halaman lainnya dengan mudah dan cepat.

Sebagai contoh, pada sistem inventori gudang, operator dapat melihat daftar barang pada halaman utama kemudian memilih salah satu barang untuk melihat detail stok, lokasi

penyimpanan, dan informasi lainnya. Proses perpindahan dari daftar barang ke detail barang memanfaatkan konsep navigasi yang dipelajari dalam praktikum ini.

Selain itu, aplikasi monitoring produksi, maintenance mesin, maupun manajemen logistik juga menggunakan konsep yang sama. Teknisi atau supervisor dapat memilih data tertentu pada halaman utama dan melihat informasi detail pada halaman berikutnya. Oleh karena itu, pemahaman mengenai navigasi dan pengiriman data antar layar sangat penting dalam pengembangan sistem informasi yang mendukung kegiatan operasional di dunia industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

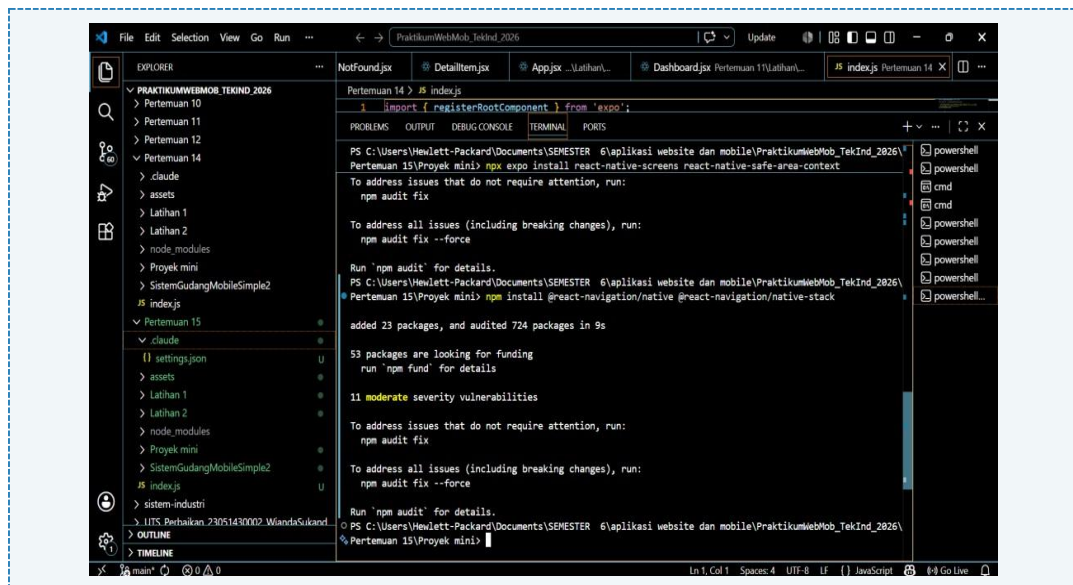
E.1 Langkah 1 – Struktur Folder

Pertama siapin project Expo di pertemuan 14 dan copy ke pertemuan 15. Terus install library dan dependensinya (kode dibawah). Kemudian buat folder baru di dalam project bernama `screens`. Di dalamnya buat dua file:

1. `HomeScreen.js` (Daftar Inventori)
2. `DetailScreen.js` (Detail Barang)

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */  
npm install @react-navigation/native @react-navigation/native-  
stack npx expo install react-native-screens react-native-safe-  
area-context
```

Screenshot Hasil:



Gambar E.1 – Bukti install

E.2 Langkah 2 – Membuat HomeScreen (Daftar Inventori)

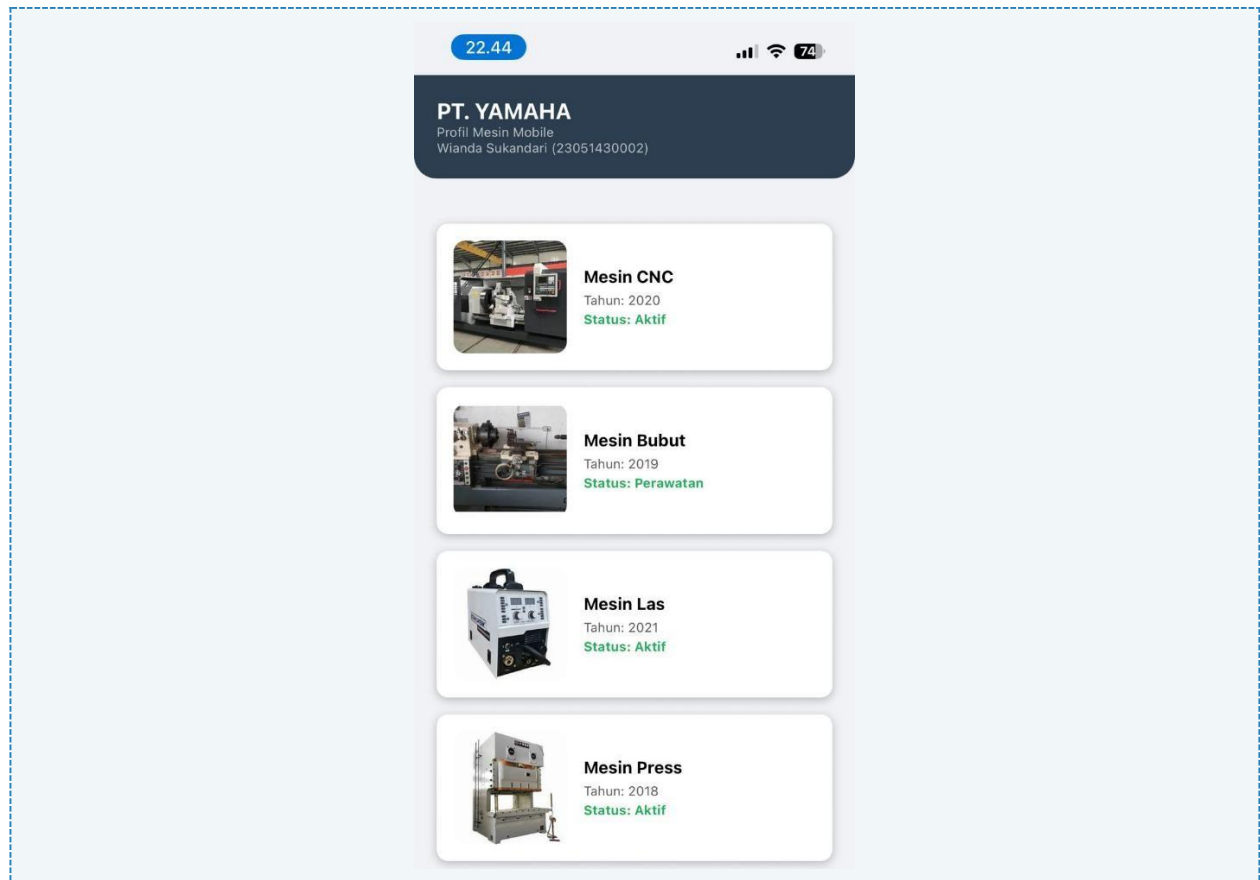

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from
'react-native';
// Data Mock Inventori
const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' }, //
Stok Habis
];
function HomeScreen({ navigation }) {
  // Fungsi Render Item untuk FlatList
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', { itemData: item
    }}}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={styles.itemSub}>Stok: {item.stok}</Text>
        <Text style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );

  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
      />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor:
    '#fff', paddingTop:
    20,
  },
  header: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 15,
    paddingHorizontal: 15,
  },
},
```

```
    itemContainer: { backgroundColor:
      '#f9f9f9', padding: 15,
      marginVertical: 8,
      marginHorizontal: 15,
      borderRadius: 8,
      borderWidth: 1,
      borderColor: '#ddd',
    },
    itemTitle: {
      fontSize: 18, fontWeight:
        'bold', color: '#333',
    },
    itemInfo: {
      flexDirection: 'row', justifyContent:
        'space-between', marginTop: 5,
    },
    itemSub: {
      color: '#666',
    },
  },
));
export default HomeScreen;
```

Screenshot Hasil:

Tampilannya masih sama kaya project 14 karena file saya pakai hasil terakhir (mini projek) di pertemuan 14.



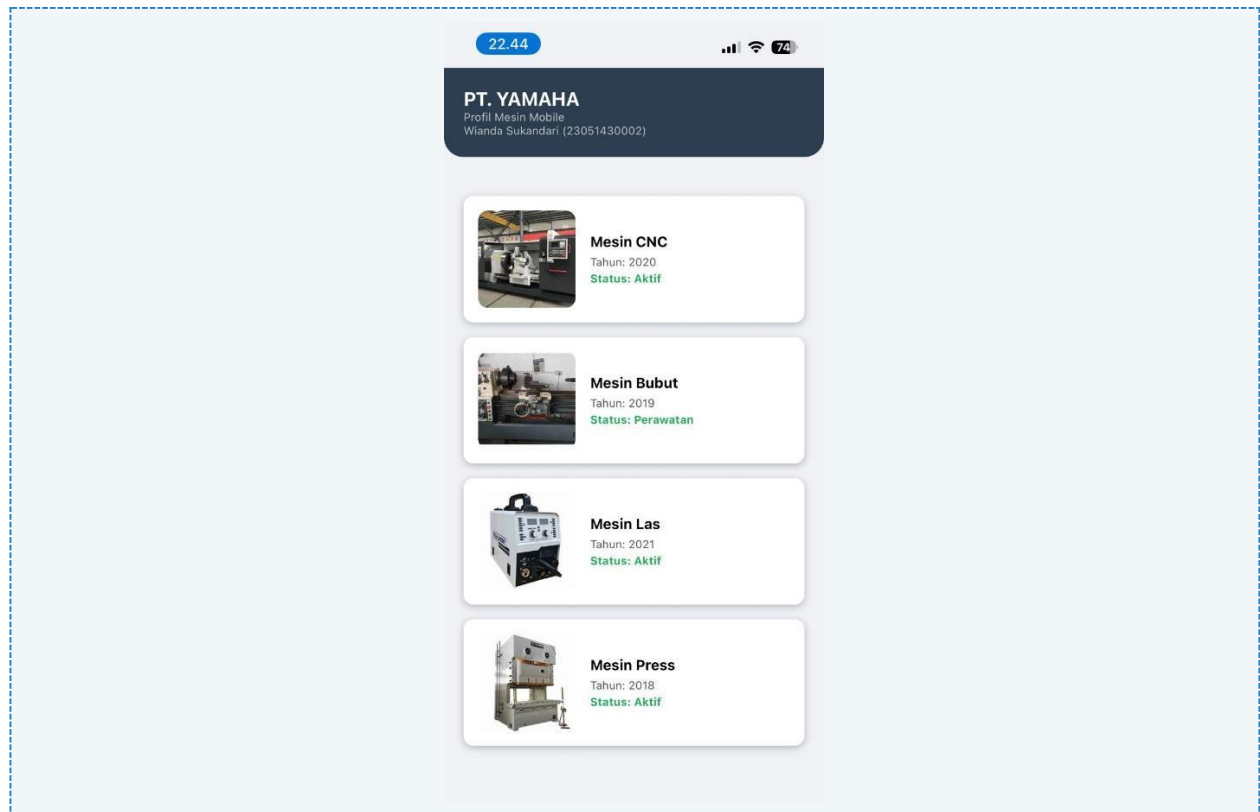
Gambar E.2 – Membuat HomeScreen (Daftar Inventori)

E.3 Langkah 3 – Membuat DetailScreen

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native'; function
DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen const {
  itemData } = route.params;
  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>
        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={
          [styles.value, itemData.stok < 20 ? styles.dangerText
:
          styles.successText]}>
          {itemData.stok} Unit
        </Text>
        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>
    </View>
  );
}
```

```
        <Button title="Kembali ke Daftar" onPress={() => navigation.goBack()}
      />
    </View>
  );
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20, backgroundColor:
      '#f0f0f0',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },
  label: {
    fontSize: 14, color:
      '#7f8c8d',
    marginTop: 10,
  },
  value: {
    fontSize: 20, fontWeight:
      'bold', color: '#2c3e50',
  },
  dangerText: {
    color: 'red',
  },
  successText: { color:
    'green',
  }
});
export default DetailScreen;
```

Screenshot Hasil:



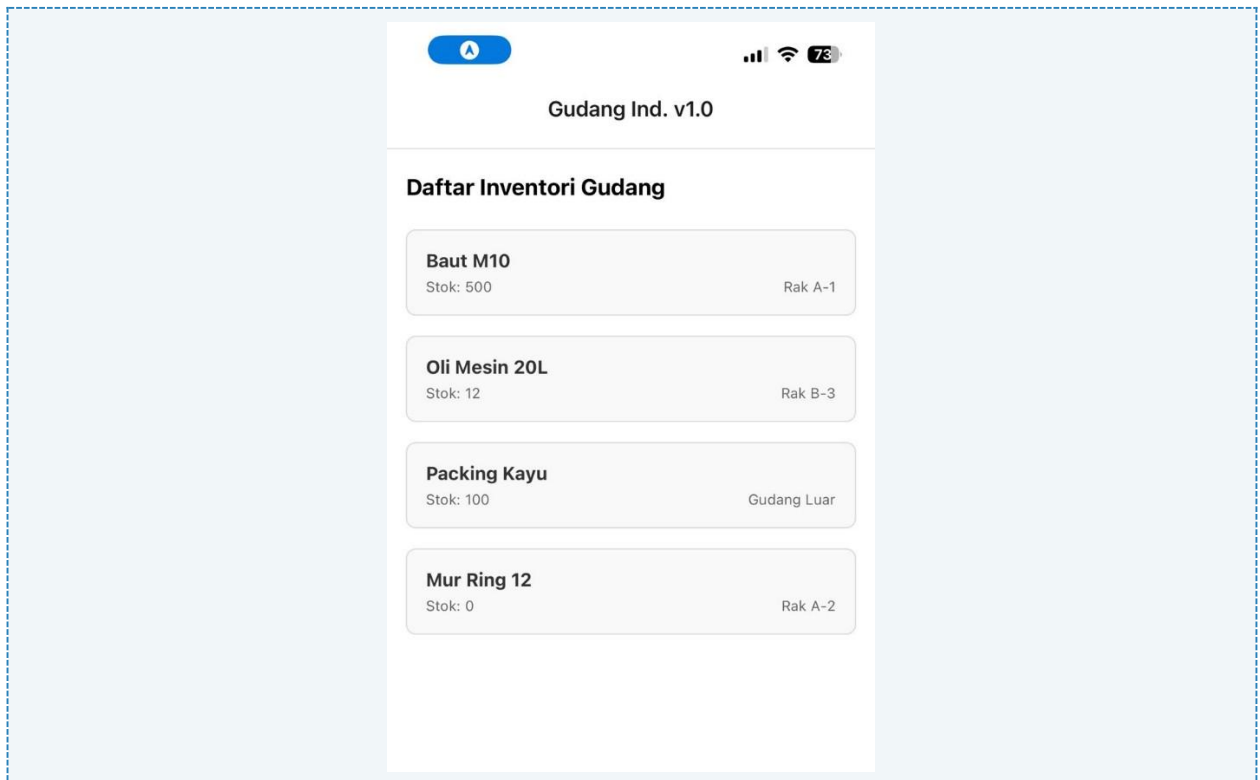
Gambar E.3 – Membuat DetailScreen

E.4 Langkah 4 – Menghubungkan Navigation di App.js

```
/* ===== KODE / OUTPUT LANGKAH 4 ===== */
import React from 'react';
import { NavigationContainer } from '@react-navigation/native'; import {
createNativeStackNavigator } from '@react-navigation/native-stack';
// Import Screens
import HomeScreen from './screens/HomeScreen'; import
DetailScreen from './screens/DetailScreen';
// Membuka Stack Navigator
const Stack = createNativeStackNavigator();
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home"
          component={HomeScreen}
          options={{ title: 'Gudang Ind. v1.0' }}
        />
        <Stack.Screen name="Detail"
          component={DetailScreen}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

```
options=({ route }) => ({ title: route.params?.itemData.nama || 'Detail'
}))
  />
</Stack.Navigator>
</NavigationContainer>
);
}
```

Screenshot Hasil:



Gambar E.4 – Menghubungkan Navigation di App.js

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Conditional Rendering

Pertanyaan / Instruksi Latihan:

Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import React from 'react';
```

```
import { View, Text, StyleSheet, Button, Alert } from 'react-native';

function DetailScreen({ route, navigation }) {
  // Menerima data yang dikirim dari HomeScreen const {
  itemData } = route.params;

  const handleRequestStok = () => { Alert.alert(
    'Request Stok Darurat',
    'Permintaan stok darurat berhasil dikirim!'
  );
};

return (
  <View style={styles.container}>
    <View style={styles.card}>
      <Text style={styles.label}>Nama Barang:</Text>
      <Text style={styles.value}>{itemData.nama}</Text>

      <Text style={styles.label}>Stok Saat Ini:</Text>
      <Text
        style={[
          styles.value,
          itemData.stok < 20
            ? styles.dangerText
            : styles.successText,
        ]}
      >
        {itemData.stok} Unit
      </Text>

      <Text style={styles.label}>Lokasi Penyimpanan:</Text>
      <Text style={styles.value}>{itemData.lokasi}</Text>

      {/* Conditional Rendering */}
      {itemData.stok === 0 && (
        <View style={styles.buttonContainer}>
          <Button
            title="Request Stok Darurat" color="red"
            onPress={handleRequestStok}
          />
        </View>
      )}
    </View>
  </View>
);
```

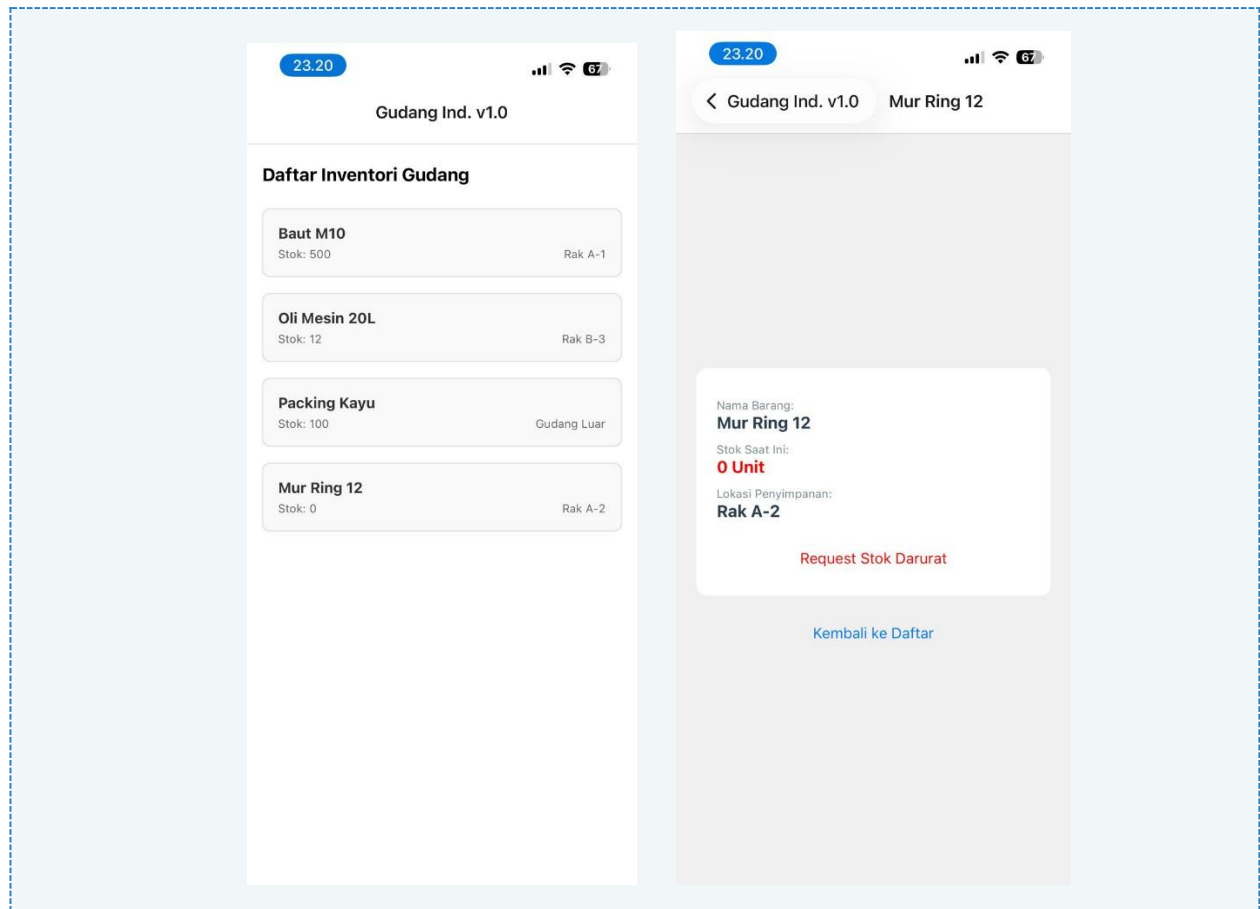


```
<Button  
  title="Kembali ke Daftar" onPress={() =>  
    navigation.goBack()  
/>
```

```
        </View>
      );
    }

    const styles = StyleSheet.create({
      container: {
        flex: 1,
        justifyContent: 'center',
        padding: 20, backgroundColor:
          '#f0f0f0',
      },
      card: {
        backgroundColor: 'white',
        padding: 20,
        borderRadius: 10,
        marginBottom: 20,
        elevation: 3,
      },
      label: {
        fontSize: 14, color:
          '#7f8c8d',
        marginTop: 10,
      },
      value: {
        fontSize: 20, fontWeight:
          'bold', color: '#2c3e50',
      },
      dangerText: {
        color: 'red',
      },
      successText: { color:
        'green',
      },
      buttonContainer: { marginTop:
        20,
      },
    });

    export default DetailScreen;
```



Gambar F.1 – Conditional Rendering

F.2 Latihan 2 – Passing Data

Pertanyaan / Instruksi Latihan:

Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi:

Buat file baru didalam folder screen yaitu TambahScreen.js dan ubah kode di HomeScreen.js dan App.js

```
import React from 'react';
import { View, Text, TextInput, StyleSheet, Button } from 'react-native';

function TambahScreen({ navigation }) { return (
  <View style={styles.container}>
    <Text style={styles.title}>Form Tambah Barang</Text>

    <TextInput
      style={styles.input}

```

```
        placeholder="Nama Barang"
      />

      <TextInput
        style={styles.input} placeholder="Jumlah Stok"
        keyboardType="numeric"
      />

      <TextInput
        style={styles.input} placeholder="Lokasi
        Penyimpanan"
      />

      <Button
        title="Simpan"
        onPress={() => alert('Data berhasil disimpan')}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20, backgroundColor:
    '#fff',
  },
  title: {
    fontSize: 24, fontWeight:
    'bold', marginBottom: 20,
  },
  input: {
    borderWidth: 1, borderColor:
    '#ccc', borderRadius: 8,
    padding: 10,
    marginBottom: 15,
  },
});

export default TambahScreen;
```

```
import React from 'react'; import {  
  View, Text,  
  FlatList,  
  TouchableOpacity,
```

```

    StyleSheet, Button
  } from 'react-native';

const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' },
];

function HomeScreen({ navigation }) { const
  renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() =>
        navigation.navigate('Detail', { itemData: item
      })
    }
  >
    <Text style={styles.itemTitle}>{item.nama}</Text>

    <View style={styles.itemInfo}>
      <Text style={styles.itemSub}> Stok:
        {item.stok}
      </Text>

      <Text style={styles.itemSub}>
        {item.lokasi}
      </Text>
    </View>
  </TouchableOpacity>
);

  return (
    <View style={styles.container}>
      <Text style={styles.header}> Daftar
        Inventori Gudang
      </Text>

      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={(item) => item.id}
      />

```

```

    <View style={styles.buttonContainer}>

```

<Button

```
        title="Tambah Barang"
        onPress={() =>
            navigation.navigate('Tambah')
        }
    />
</View>
</View>
);
}
```

```
const styles = StyleSheet.create({ container: {
    flex: 1, backgroundColor:
    'fff', paddingTop: 20,
},
header: {
    fontSize: 22, fontWeight:
    'bold', marginBottom: 15,
    paddingHorizontal: 15,
},
itemContainer: { backgroundColor:
    'f9f9f9', padding: 15,
    marginVertical: 8,
    marginHorizontal: 15,
    borderRadius: 8,
    borderWidth: 1, borderColor:
    'ddd',
},
itemTitle: {
    fontSize: 18, fontWeight:
    'bold', color: '#333',
},
itemInfo: {
    flexDirection: 'row', justifyContent: 'space-
    between', marginTop: 5,
},
itemSub: {
    color: '#666',
},
buttonContainer: { margin:
    15,
},
});
```

```
export default HomeScreen;
```



```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native'; import {
createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from '../screens/HomeScreen'; import
DetailScreen from '../screens/DetailScreen'; import
TambahScreen from '../screens/TambahScreen';

const Stack = createNativeStackNavigator();

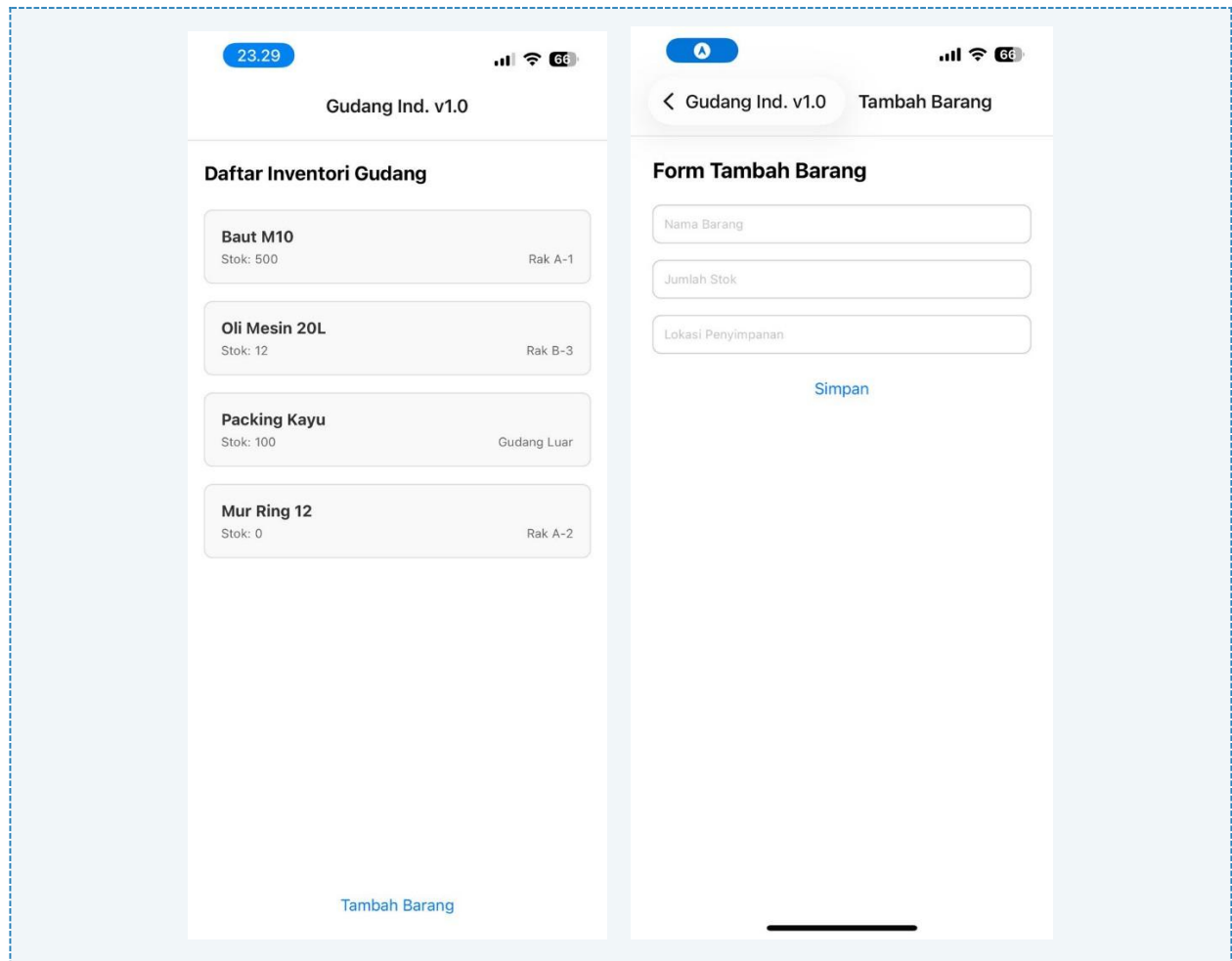
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">

        <Stack.Screen name="Home"
          component={HomeScreen}
          options={{ title: 'Gudang Ind. v1.0' }}
        />

        <Stack.Screen name="Detail"
          component={DetailScreen}
          options={({ route }) => ({ title:
            route.params?.itemData.nama || 'Detail'
          })}
        />

        <Stack.Screen name="Tambah"
          component={TambahScreen}
          options={{
            title: 'Tambah Barang'
          }}
        />

      </Stack.Navigator>
    </NavigationContainer>
  );
}
```



Gambar F.2 – Passing Data

F.3 Tugas Proyek Mini – Aplikasi Inspeksi QC

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

- Home: Daftar item yang perlu diinspeksi.
- Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).
- Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, aplikasi inspeksi QC dibuat menggunakan React Native dengan konsep navigasi antar layar (Stack Navigation). Aplikasi terdiri dari halaman Home yang menampilkan daftar item yang akan diperiksa, serta halaman Detail yang digunakan untuk melihat informasi item secara lebih lengkap. Data item disimpan dalam state sehingga dapat diperbarui ketika pengguna melakukan inspeksi.

Setiap item inspeksi dilengkapi dengan nama komponen, foto item, standar kualitas, dan pilihan status inspeksi menggunakan dropdown (Picker). Pengguna dapat memilih status "Lolos" atau "Gagal" sesuai hasil pemeriksaan. Setelah status dipilih dan disimpan, data item akan diperbarui sehingga hasil inspeksi dapat ditampilkan kembali pada halaman utama.

Untuk mensimulasikan proses Quality Control di industri, diterapkan logika state management sederhana. Jika suatu item diberi status "Gagal", maka warna teks item tersebut pada halaman Home akan berubah menjadi merah sebagai indikator adanya ketidaksesuaian kualitas. Pendekatan ini menunjukkan bagaimana data dapat dikirim antar layar, diolah, dan ditampilkan kembali secara dinamis untuk mendukung proses monitoring kualitas produk.

Kode Utama:

```
import React, { useState } from 'react';
import { NavigationContainer } from '@react-navigation/native'; import {
createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './screens/HomeScreen'; import
DetailScreen from './screens/DetailScreen';

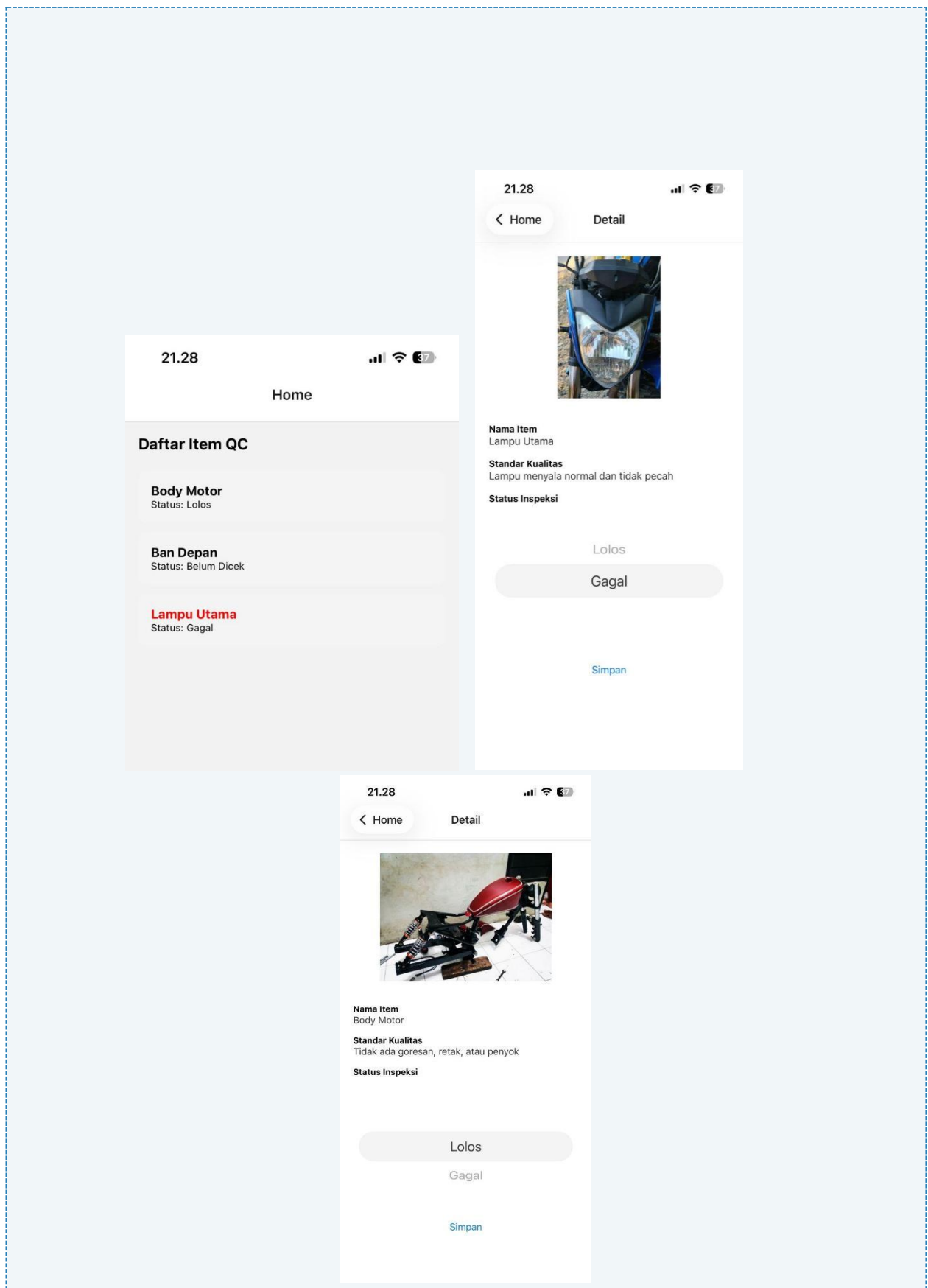
const Stack = createNativeStackNavigator();

export default function App() {
  const [items, setItems] = useState([
    {
      id: '1',
      nama: 'Body Motor',
      standar: 'Tidak ada goresan, retak, atau penyok',
      gambar: require('./assets/body_motor.jpg'), status:
      'Belum Dicek',
    },
    {
      id: '2',
      nama: 'Ban Depan',
      standar: 'Tekanan sesuai standar dan tidak aus',
      gambar: require('./assets/ban_depan.jpg'), status:
      'Belum Dicek',
    },
    {
      id: '3',
```

```
    nama: 'Lampu Utama',
    standar: 'Lampu menyala normal dan tidak pecah',
    gambar: require('./assets/lampu_utama.jpg'), status:
    'Belum Dicek',
  },
]);

return (
  <NavigationContainer>
    <Stack.Navigator>
      <Stack.Screen name="Home">
        {(props) => (
          <HomeScreen
            {...props} items={items}
          />
        )}
      </Stack.Screen>

      <Stack.Screen name="Detail">
        {(props) => (
          <DetailScreen
            {...props} items={items}
            setItems={setItems}
          />
        )}
      </Stack.Screen>
    </Stack.Navigator>
  </NavigationContainer>
);
}
```



Gambar F.3 – Output Aplikasi Inspeksi QC

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh telah sesuai dengan teori React Native Navigation dan state management sederhana yang dipelajari pada modul ini. Aplikasi berhasil menerapkan konsep Stack Navigator sehingga pengguna dapat berpindah dari halaman Home menuju halaman Detail dan kembali ke halaman sebelumnya. Selain itu, data item inspeksi dapat dikirim dari HomeScreen ke DetailScreen menggunakan parameter navigasi (passing data), sehingga informasi item yang dipilih dapat ditampilkan secara spesifik pada halaman detail.

Pada halaman DetailScreen, konsep conditional rendering dan state management juga berhasil diterapkan melalui komponen Picker untuk memilih status inspeksi "Lolos" atau "Gagal". Status yang dipilih pengguna kemudian disimpan ke state aplikasi menggunakan useState dan diteruskan kembali ke halaman Home. Mekanisme ini menunjukkan bahwa perubahan data dapat dikelola dan ditampilkan secara dinamis sesuai interaksi pengguna.

Hasil output aplikasi menunjukkan bahwa ketika suatu item diberi status "Gagal", warna teks item pada HomeScreen berubah menjadi merah. Perubahan tampilan tersebut membuktikan bahwa state management sederhana berjalan dengan baik karena data yang diperbarui pada DetailScreen dapat memengaruhi tampilan komponen lain. Dengan demikian, seluruh fitur yang diminta dalam proyek mini berhasil diimplementasikan dan sesuai dengan konsep teori yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Error "Unable to resolve ../assets/images/body_motor.jpg" karena file gambar tidak ditemukan pada folder yang ditentukan.	Membuat folder assets/images dan memastikan nama file gambar sesuai dengan yang dipanggil menggunakan require().
Gambar tidak muncul pada aplikasi meskipun tidak ada error.	Memastikan penggunaan komponen Image sudah benar dan path gambar sesuai dengan lokasi penyimpanan file.
Error karena deklarasi komponen Image dilakukan dua kali pada DetailScreen.	Menghapus import yang duplikat dan menggabungkan seluruh komponen React Native dalam satu baris import.

G.3 Kaitan dengan Penerapan di Industri

Materi React Native Navigation dan state management memiliki keterkaitan yang erat dengan kebutuhan industri, khususnya dalam pengembangan aplikasi mobile untuk mendukung proses operasional perusahaan. Kemampuan membuat navigasi antar halaman memungkinkan pengembang menyusun alur kerja aplikasi yang terstruktur dan mudah digunakan oleh pengguna.

Dalam bidang Teknik Industri, aplikasi serupa dapat digunakan untuk mendukung proses inspeksi kualitas produk di area produksi. Petugas Quality Control dapat melihat daftar item yang harus diperiksa, membuka detail setiap item, melihat standar kualitas yang berlaku, kemudian memberikan status lulus atau gagal secara langsung melalui aplikasi mobile. Data inspeksi tersebut dapat digunakan untuk pengambilan keputusan dan monitoring kualitas secara real-time.

Contoh penerapan lainnya adalah pada sistem inspeksi kendaraan, audit gudang, pemeriksaan peralatan produksi, maupun maintenance mesin. Dengan adanya aplikasi mobile berbasis React Native, proses pencatatan hasil inspeksi menjadi lebih cepat, akurat, dan terdokumentasi dengan baik dibandingkan metode pencatatan manual menggunakan kertas.

H. KESIMPULAN

1	Praktikum berhasil menerapkan navigasi antar halaman menggunakan React Navigation dengan metode Stack Navigator sehingga pengguna dapat berpindah dari halaman daftar inspeksi ke halaman detail item.
2	Data item inspeksi dapat dikirim dari HomeScreen ke DetailScreen menggunakan teknik passing data melalui parameter navigasi, sehingga informasi setiap item dapat ditampilkan secara spesifik.
3	Aplikasi berhasil menampilkan gambar komponen yang diinspeksi, standar kualitas, serta fitur pemilihan status inspeksi menggunakan komponen Picker.
4	Konsep state management sederhana menggunakan useState berhasil diterapkan untuk menyimpan dan memperbarui status inspeksi setiap item secara dinamis.
5	Simulasi perubahan status inspeksi berhasil dilakukan, dimana item yang berstatus "Gagal" ditampilkan dengan warna merah pada halaman Home sehingga memudahkan identifikasi item yang tidak memenuhi standar kualitas.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Native Documentation . (2026). <i>Getting Started with React Native</i> . React Native.
[3]	React Navigation Documentation . (2026). <i>Getting Started</i> . React Navigation.
[4]	Expo Documentation . (2026). <i>Expo Documentation</i> . Expo.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Rekan Tim	Diperiksa oleh Asisten / Dosen
		
<u>Wianda Sukandari</u> NIM: 23051430002	<u>Nanik Wijayanti</u> NIM: 23051430006	<u>Dr. Eng. Ir. Aji Ery</u> <u>Burhandenny, S.T., M.AIT.</u> Tanggal: _____

